

本类文档用于整理收集全网优秀的技术文章，开源项目，经典论文，面试题，行业分享等资料，整理成册，免费分享给相关技术的开发人员

涉及到技术点c/c++/历年【字节、百度、京东、腾讯、阿里等一线互联网大厂】（校招、社招）面试题及答案

文章素材来源于微信公众号/知乎/csdn/头条/掘金/51cto/博客园等技术网站的优秀文章，版权属于原作者（文章标注以及原文链接）

如有侵权，联系wangbojing@0voice.com 或留下的qq群联系，第一时间删除。由衷感谢各位优秀文章的作者，在互联网分享自己思想

C/C++ 常见1000道面试题

1.变量的声明和定义有什么区别

为变量分配地址和存储空间的称为定义，不分配地址的称为声明。一个变量可以在多个地方声明，但是只在一个地方定义。加入 extern 修饰的是变量的声明，说明此变量将在文件以外或在文件后面部分定义。说明：很多时候一个变量，只是声明不分配内存空间，直到具体使用时才初始化，分配内存空间，如外部变量。

2.写出 bool、int、float、指针变量与“零值”比较的 if 语句

```
//bool 型数据:
if( flag )
{
    A;
}
else
{
    B;
}

//int 型数据:
if( 0 != flag )
{
    A;
}
else {
    B;
}

//指针型:
if( NULL == flag )
{
    A;
}
else {
    B;
}

//float 型数据:
if ( ( flag >= -NORM ) && ( flag <= NORM ) )
{
```

```
A;  
}
```

注意：应特别注意在 int、指针型变量和“零值”比较的时候，把“零值”放在左边，这样当把“==”误写成“=”时，编译器可以报错，否则这种逻辑错误不容易发现，并且可能导致很严重的后果。

3.sizeof 和 strlen 的区别

sizeof 和 strlen 有以下区别：

1 sizeof 是一个操作符，strlen 是库函数。

2 sizeof 的参数可以是数据的类型，也可以是变量，而 strlen 只能以结尾为'\0'的字符串作参数。

3 编译器在编译时就计算出了 sizeof 的结果。而 strlen 函数必须在运行时才能计算出来。并且 sizeof 计算的是数据类型占内存的大小，而 strlen 计算的是字符串实际的长度。

4 数组做 sizeof 的参数不退化，传递给 strlen 就退化为指针了。

注意：有些是操作符看起来像是函数，而有些函数名看起来又像操作符，这类容易混淆的名称一定要加以区分，否则遇到数组名这类特殊数据类型作参数时就很容易出错。最容易混淆为函数的操作符就是 sizeof。

4.C 语言的关键字 static 和 C++ 的关键字 static 有什么区别

在 C 中 static 用来修饰局部静态变量和外部静态变量、函数。而 C++中除了上述功能外，还用来定义类的成员变量和函数。即静态成员和静态成员函数。

注意：编程时 static 的记忆性，和全局性的特点可以让在不同时期调用的函数进行通信，传递信息，而 C++的静态成员则可以在多个对象实例间进行通信，传递信息。

5.C 中的 malloc 和 C + + 中的 new 有什么区别

malloc 和 new 有以下不同：

(1) new、delete 是操作符，可以重载，只能在 C++中使用。

(2) malloc、free 是函数，可以覆盖，C、C++中都可以使用。

(3) new 可以调用对象的构造函数，对应的 delete 调用相应的析构函数。

(4) malloc 仅仅分配内存，free 仅仅回收内存，并不执行构造和析构函数

(5) new、delete 返回的是某种数据类型指针，malloc、free 返回的是 void 指针。

注意：malloc 申请的内存空间要用 free 释放，而 new 申请的内存空间要用 delete 释放，不要混用。因为两者实现的机理不同。

6.写一个“标准”宏 MIN

```
#define min(a,b)((a)<=(b)?(a):(b))
```

注意：在调用时一定要注意这个宏定义的副作用，如下调用：

```
((++*p)<=(x)?(++*p):(x))
```

p 指针就自加了两次，违背了 MIN 的本意。

7.一个指针可以是 volatile 吗

可以，因为指针和普通变量一样，有时也有变化程序的不可控性。常见例：子中断服务子程序修改一个指向一个 buffer 的指针时，必须用 volatile 来修饰这个指针。

说明：指针是一种普通的变量，从访问上没有什么不同于其他变量的特性。其保存的数值是个整型数据，和整型变量不同的是，这个整型数据指向的是一段内存地址。

8.a 和&a 有什么区别

请写出以下代码的打印结果，主要目的是考察 a 和&a 的区别。

```
#include<stdio.h>
void main( void )
{
    int a[5]={1,2,3,4,5};
    int *ptr=(int *)(&a+1);
    printf("%d,%d",*(a+1),*(ptr-1));
    return;
}
```

输出结果：2，5。

注意：数组名 a 可以作数组的首地址，而&a 是数组的指针。思考，将原式的 int *ptr=(int *)(&a+1); 改为 int *ptr=(int *)(&a);时输出结果将是什么呢？

9.简述 C、C++ 程序编译的内存分配情况

C、C++ 中内存分配方式可以分为三种：

(1) 从静态存储区域分配：

内存存在程序编译时就已经分配好，这块内存存在程序的整个运行期间都存在。速度快、不容易出错，因为有系统会善后。例如全局变量，static 变量等。

(2) 在栈上分配：

在执行函数时，函数内局部变量的存储单元都在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。

(3) 从堆上分配：

即动态内存分配。程序在运行的时候用 malloc 或 new 申请任意大小的内存，程序员自己负责在何时用 free 或 delete 释放内存。动态内存的生存期由程序员决定，使用非常灵活。如果在堆上分配了空间，就有责任回收它，否则运行的程序会出现内存泄漏，另外频繁地分配和释放不同大小的堆空间将会产生堆内存碎片。

一个 C、C++ 程序编译时内存分为 5 大存储区：堆区、栈区、全局区、文字常量区、程序代码区。

10.简述 strcpy、sprintf 与 memcpy 的区别

三者主要有以下不同之处：

(1) 操作对象不同，strcpy 的两个操作对象均为字符串，sprintf 的操作源对象可以是多种数据类型，目的操作对象是字符串，memcpy 的两个对象就是两个任意可操作的内存地址，并不限于何种数据类型。

(2) 执行效率不同，memcpy 最高，strcpy 次之，sprintf 的效率最低。

(3) 实现功能不同，strcpy 主要实现字符串变量间的拷贝，sprintf 主要实现其他数据类型格式到字符串的转化，memcpy 主要是内存块间的拷贝。

说明：strcpy、sprintf 与 memcpy 都可以实现拷贝的功能，但是针对的对象不同，根据实际需求，来选择合适的函数实现拷贝功能。

面试题 11：设置地址为 0x67a9 的整型变量的值为 0xaa66

```
int *ptr;  
ptr = (int *)0x67a9;  
*ptr = 0xaa66;
```

说明：这道题就是强制类型转换的典型例子，无论在什么平台地址长度和整型数据的长度是一样的，即一个整型数据可以强制转换成地址指针类型，只要有意义即可。

12.面向对象的三大特征

面向对象的三大特征是封装性、继承性和多态性：

封装性：将客观事物抽象成类，每个类对自身的数据和方法实行 protection (private , protected , public)。

继承性：广义的继承有三种实现形式：实现继承（使用基类的属性和方法而无需额外编码的能力）、可视继承(子窗体使用父窗体的外观和实现代码)、接口继承(仅使用属性和方法,实现滞后到子类实现)。

多态性：是将父类对象设置成为和一个或多个它的子对象相等的技术。用子类对象给父类对象赋值之后，父类对象就可以根据当前赋值给它的子对象的特性以不同的方式运作。这部分需要熟练掌握原理虚函数，了解一些概念（静态多态、动态多态）等，面试时经常会问。

说明：面向对象的三个特征是实现面向对象技术的关键，每一个特征的相关技术都非常的复杂，程序员应该多看、多练。

13.C++的空类有哪些成员函数

缺省构造函数。

缺省拷贝构造函数。

缺省析构函数。

缺省赋值运算符。

缺省取址运算符。

缺省取址运算符 const。

注意：有些书上只是简单的介绍了前四个函数。没有提及后面这两个函数。但后面这两个函数也是空类的默认函数。另外需要注意的是，只有当实际使用这些函数的时候，编译器才会去定义它们。

14.谈谈你对拷贝构造函数和赋值运算符的认识

拷贝构造函数和赋值运算符重载有以下两个不同之处：

- (1) 拷贝构造函数生成新的类对象，而赋值运算符不能。
- (2) 由于拷贝构造函数是直接构造一个新的类对象，所以在初始化这个对象之前不用检验源对象是否和新建对象相同。而赋值运算符则需要这个操作，另外赋值运算中如果原来的对象中有内存分配要先把内存释放掉

注意：当有类中有指针类型的成员变量时，一定要重写拷贝构造函数和赋值运算符，不要使用默认的。

15.用 C++设计一个不能被继承的类

```
template <typename T> class A  
{  
    friend T; private:  
    A() {}  
    ~A() {}  
};  
  
class B : virtual public A<B>  
{  
public:
```

```
B() {}  
~B() {}  
};  
class C : virtual public B  
{  
public:  
    C() {}  
    ~C() {}  
};  
void main( void )  
{  
    B b;  //C c;  
    return;  
}
```

注意：构造函数是继承实现的关键，每次子类对象构造时，首先调用的是父类的构造函数，然后才是自己的。

注意：构造函数是继承实现的关键，每次子类对象构造时，首先调用的是父类的构造函数，然后才是自己的。

16.访问基类的私有虚函数

写出以下程序的输出结果：

```
#include <iostream.h>  
class A  
{  
    virtual void g()  
    {  
        cout << "A::g" << endl;  
    }  
private:  
    virtual void f()  
    {  
        cout << "A::f" << endl;  
    }  
};  
class B : public A  
{  
    void g()  
    {  
        cout << "B::g" << endl;  
    }  
    virtual void h()  
    {  
        cout << "B::h" << endl;  
    }  
};  
typedef void( *Fun )( void ); void main()  
{  
    B b;  
    Fun pFun;  
    for(int i = 0 ; i < 3; i++)  
    {
```

```
pFun = ( Fun )((int*)0)((int*)0)(&b) + 1),  
pFun();  
  
}  
  
}
```

输出结果：

B::g

A::f

B::h

注意：本题主要考察了面试者对虚函数的理解程度。一个对虚函数不了解的人很难正确的做出本题。在学习面向对象的多态性时一定要深刻理解虚函数表的工作原理。

17.简述类成员函数的重写、重载和隐藏的区别

(1) 重写和重载主要有以下几点不同。

范围的区别：被重写的和重写的函数在两个类中，而重载和被重载的函数在同一个类中。

参数的区别：被重写函数和重写函数的参数列表一定相同，而被重载函数和重载函数的参数列表一定不同。

virtual 的区别：重写的基类中被重写的函数必须要有 virtual 修饰，而重载函数和被重载函数可以被 virtual 修饰，也可以没有。

(2) 隐藏和重写、重载有以下几点不同。

与重载的范围不同：和重写一样，隐藏函数和被隐藏函数不在同一个类中。

参数的区别：隐藏函数和被隐藏的函数的参数列表可以相同，也可不同，但是函数名肯定要相同。当参数不相同时，无论基类中的参数是否被 virtual 修饰，基类的函数都是被隐藏，而不是被重写。

说明：虽然重载和覆盖都是实现多态的基础，但是两者实现的技术完全不相同，达到的目的也是完全不同的，覆盖是动态态绑定的多态，而重载是静态绑定的多态。

18.简述多态实现的原理

编译器发现一个类中有虚函数，便会立即为此类生成虚函数表 vtable。虚函数表的各表项为指向对应虚函数的指针。编译器还会在此类中隐含插入一个指针 vptra（对 vc 编译器来说，它插在类的第一个位置上）指向虚函数表。调用此类的构造函数时，在类的构造函数中，编译器会隐含执行 vptra 与 vtable 的关联代码，将 vptra 指向对应的 vtable，将类与此类的 vtable 联系了起来。另外在调用类的构造函数时，指向基础类的指针此时已经变成指向具体的类的 this 指针，这样依靠此 this 指针即可得到正确的 vtable，。

如此才能真正与函数体进行连接，这就是动态联编，实现多态的基本原理。

注意：一定要区分虚函数，纯虚函数、虚拟继承的关系和区别。牢记虚函数实现原理，因为多态 C++ 面试的重要考点之一，而虚函数是实现多态的基础。

19.链表和数组有什么区别

数组和链表有以下几点不同：

(1) 存储形式：数组是一块连续的空间，声明时就要确定长度。链表是一块可不连续的动态空间，长度可变，每个结点要保存相邻结点指针。

(2) 数据查找：数组的线性查找速度快，查找操作直接使用偏移地址。链表需要按顺序检索结点，效率低。

(3) 数据插入或删除：链表可以快速插入和删除结点，而数组则可能需要大量数据移动。

(4) 越界问题：链表不存在越界问题，数组有越界问题。

说明：在选择数组或链表数据结构时，一定要根据实际需要进行选择。数组便于查询，链表便于插入删除。数组节省空间但是长度固定，链表虽然变长但是占了更多的存储空间。

20.怎样把一个单链表反序

(1) 反转一个链表。循环算法。

```
List reverse(List n)
{
    if(!n) //判断链表是否为空，为空即退出。
    {
        return n;
    }
    list cur = n.next; //保存头结点的下个结点
    list pre = n;
    list tmp; //保存头结点
    pre.next = null; //头结点的指针指空，转换后变尾结点
    while ( NULL != cur.next ) //循环直到 cur.next 为空
    {
        tmp = cur;
    }
    tmp.next = pre;
    pre = tmp;
    cur = cur.next;

    return tmp; //f 返回头指针
}
```

(2) 反转一个链表。递归算法。

```
List *reverse( List *oldList, List *newHead = NULL )
{
    List *next = oldList-> next; //记录上次翻转后的链表
    oldList-> next = newHead; //将当前结点插入到翻转后链表的开头
    newHead = oldList; //递归处理剩余的链表
    return ( next==NULL )? newHead: reverse( t, newHead );
}
```

说明：循环算法就是移动过程，比较好理解和想到。递归算法的设计虽有一点难度，但是理解了循环算法，再设计递归算法就简单多了。

21.简述队列和栈的异同

队列和栈都是线性存储结构，但是两者的插入和删除数据的操作不同，队列是“先进先出”，栈是“后进先出”。

注意：区别栈区和堆区。堆区的存取是“顺序随意”，而栈区是“后进先出”。栈由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。堆一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。分配方式类似于链表。

它与本题中的堆和栈是两回事。堆栈只是一种数据结构，而堆区和栈区是程序的不同内存存储区域。

22.能否用两个栈实现一个队列的功能

```
//结点结构体：
typedef struct node
```

```
{
    int data;
    node *next;
}node, *LinkStack;
//创建空栈:
LinkStack CreateNULLStack(LinkStack &S)
{
    S = (LinkStack)malloc(sizeof(node)); //申请新结点
    if (NULL == S)
    {
        printf("Fail to malloc a new node.\n")
        return NULL;
    }
    S->data = 0; //初始化新结点
    S->next = NULL;
    return S;
}
//栈的插入函数:
LinkStack Push(LinkStack &S, int data)
{
    if (NULL == S) //检验栈
    {
        printf("There no node in stack!");
        return NULL;
    }
    LinkStack p = NULL;
    p = (LinkStack)malloc(sizeof(node)); //申请新结点
    if (NULL == p)
    {
        printf("Fail to malloc a new node.\n");
        return S;
    }
    if (NULL == S->next)
    {
        p->next = NULL;
    }
    else
    {
        p->next = S->next;
    }
    p->data = data; //初始化新结点
    S->next = p; //插入新结点
    return S;
}
//出栈函数:
node Pop(LinkStack &S)
{
    node temp;
    temp.data = 0;
    temp.next = NULL;
    if (NULL == S) //检验栈
    {
        printf("There no node in stack!");
        return temp;
    }
}
```



```

temp = *S;
10
if (S->next == NULL)
{
    printf("The stack is NULL,can't pop!\n");
    return temp;
}
LinkStack p = S->next; //节点出栈
S->next = S->next->next;
temp = *p;
free(p);
p = NULL;
return temp;
}
//双栈实现队列的入队函数：
LinkStack StackToQueuePush(LinkStack &S, int data)
{
    node n;
    LinkStack S1 = NULL;
    CreateNULLStack(S1); //创建空栈
    while (NULL != S->next) //S 出栈入 S1
    {
        n = Pop(S);
        Push(S1, n.data);
    }
    Push(S1, data); //新结点入栈
    while (NULL != S1->next) //S1 出栈入 S
    {
        n = Pop(S1);
        Push(S, n.data);
    }
}

```

说明：用两个栈能够实现一个队列的功能，那用两个队列能否实现一个队列的功能呢？结果是否定的，因为栈是先进后出，将两个栈连在一起，就是先进先出。而队列是现先进先出，无论多少个连在一起都是先进先出，而无法实现先进后出。

23.计算一颗二叉树的深度

深度的计算函数：

```

int depth(BiTree T)
{
    if(!T) return 0; //判断当前结点是否为叶子结点
    int d1= depth(T->lchild); //求当前结点的左孩子树的深度
    int d2= depth(T->rchild); //求当前结点的右孩子树的深度

    return (d1>d2?d1:d2)+1;
}

```

注意：根据二叉树的结构特点，很多算法都可以用递归算法来实现。

24.编码实现直接插入排序

直接插入排序编程实现如下：

```
#include<iostream.h>
void main( void )
{
    int ARRAY[10] = { 0, 6, 3, 2, 7, 5, 4, 9, 1, 8 };
    int i,j;
    for( i = 0; i < 10; i++)
    {
        cout<<ARRAY[i]<<" ";
    }
    cout<<endl;
    for( i = 2; i <= 10; i++ )                //将 ARRAY[2],...,ARRAY[n]依次按序插入
    {
        if(ARRAY[i] < ARRAY[i-1])            //如果 ARRAY[i]大于一切有序的数值,
                                                //ARRAY[i]将保持原位不动
        {
            ARRAY[0] = ARRAY[i]; //将 ARRAY[0]看做是哨兵,是 ARRAY[i]的副本    j = i - 1;

            do{                                //从右向左在有序区 ARRAY[1..i-1]中
//查找 ARRAY[i]的插入位置

                ARRAY[j+1] = ARRAY[j]; //将数值大于 ARRAY[i]记录后移    j-- ;
            }while( ARRAY[j] < ARRAY[i] );
            ARRAY[j+1]=ARRAY[0];    //ARRAY[i]插入到正确的位置上
        }
    }
    for( i = 0; i < 10; i++)
    {
        cout<<ARRAY[i]<<" ";
    }
    cout<<endl;
}
```

注意：所有为简化边界条件而引入的附加结点（元素）均可称为哨兵。引入哨兵后使得查找循环条件的时间大约减少了一半，对于记录数较大的文件节约的时间就相当可观。类似于排序这样使用频率非常高的算法，要尽可能地减少其运行时间。所以不能把上述算法中的哨兵视为雕虫小技。

注意：所有为简化边界条件而引入的附加结点（元素）均可称为哨兵。引入哨兵后使得查找循环条件的时间大约减少了一半，对于记录数较大的文件节约的时间就相当可观。类似于排序这样使用频率非常高的算法，要尽可能地减少其运行时间。所以不能把上述算法中的哨兵视为雕虫小技。

25.编码实现冒泡排序

冒泡排序编程实现如下：

```
#include <stdio.h>
#define LEN 10                //数组长度

void main( void )
{
    int ARRAY[10] = { 0, 6, 3, 2, 7, 5, 4, 9, 1, 8 };    //待排序数组
    printf( "\n" );
    for( int a = 0; a < LEN; a++ )                //打印数组内容
    {
        printf( "%d ", ARRAY[a] );
    }
}
```

```
}
int i = 0; int j = 0;
bool isChange;                                //设定交换标志

for( i = 1; i < LEN; i++ )
{
    isChange = 0;                                //最多做 LEN-1 趟排序
    for( j = LEN-1; j >= i; j-- )                //本趟排序开始前,交换标志应为假
                                                //对当前无序区 ARRAY[i..LEN]自下向上扫描
    {
        if( ARRAY[j+1] < ARRAY[j] )
        {
            //交换记录
            ARRAY[0] = ARRAY[j+1];                //ARRAY[0]不是哨兵,仅做暂存单元
            ARRAY[j+1] = ARRAY[j];
            ARRAY[j] = ARRAY[0];
            isChange = 1;                        //发生了交换,故将交换标志置为真
        }
    }
}
printf( "\n" );
for( a = 0; a < LEN; a++)                      //打印本次排序后数组内容
{
    printf( "%d ", ARRAY[a] );
}
if( !isChange )
{
    break;
} //本趟排序未发生交换,提前终止算法
printf( "\n" ); return;
}
```

26.编码实现直接选择排序

面试题 26：编码实现直接选择排序

```
#include "stdio.h"
#define LEN 9
void main( void )
{
    int ARRAY[LEN]={ 5, 6, 8, 2, 4, 1, 9, 3, 7 };           //待序数组

    printf("Before sorted:\n");
    for( int m = 0; m < LEN; m++ )                        //打印排序前数组
    {
        printf( "%d  ", ARRAY[m] );
    }
    for (int i = 1; i <= LEN - 1; i++)                     //选择排序
    {
        int t = i - 1;
        int temp = 0;
        for (int j = i; j < LEN; j++)
        {
            if (ARRAY[j] < ARRAY[t])
            {
                t = j;
            }
        }
        if (t != (i - 1))
        {
            temp = ARRAY[i - 1];
            ARRAY[i - 1] = ARRAY[t];
            ARRAY[t] = temp;
        }
    }
    printf( "\n" );
    printf("After sorted:\n");
    for( i = 0; i < LEN; i++ )                             //打印排序后数组
    {
        printf( "%d  ", ARRAY[i] );
    }
    printf( "\n" );
}
```

<https://blog.csdn.net/SeatonRayAlen>

注意：在直接选择排序中，具有相同键码的对象可能会颠倒次序，因而直接选择排序算法是一种不稳定的排序方法。在本例中只是列举了简单的整形数组排序，肯定不会有什么问题。但是在复杂的数据元素序列组合中，只是根据单一的某一个键值排序，直接选择排序则不保证其稳定性，这是直接选择排序的一个弱点。

27.编程实现堆排序

堆排序编程实现：

```
void createHeep(int ARRAY[], int sPoint, int Len) //生成大根堆
{
    while ((2 * sPoint + 1) < Len)
    {
        int mPoint = 2 * sPoint + 1;
        if ((2 * sPoint + 2) < Len)
        {
            if (ARRAY[2 * sPoint + 1] < ARRAY[2 * sPoint + 2])
            {
                mPoint = 2 * sPoint + 2;
            }
        }
    }
}
```

```

    }
    if (ARRAY[sPoint] < ARRAY[mPoint]) //堆被破坏，需要重新调整
    {
        int tmpData = ARRAY[sPoint]; //交换 sPoint 与 mPoint 的数据
        ARRAY[sPoint] = ARRAY[mPoint];
        ARRAY[mPoint] = tmpData;
        sPoint = mPoint;
    }
    else
    {
        break; //堆未破坏，不再需要调整
    }
}
return;
}
void heapSort(int ARRAY[], int Len) //堆排序
{
    int i = 0;
    for (i = (Len / 2 - 1); i >= 0; i--) //将 Hr[0, Lenght-1]建成大根堆
    {
        createHeap(ARRAY, i, Len);
    }
    for (i = Len - 1; i > 0; i--)
    {
        int tmpData = ARRAY[0]; //与最后一个记录交换
        ARRAY[0] = ARRAY[i];
        ARRAY[i] = tmpData;
        createHeap(ARRAY, 0, i); //将 H.r[0..i]重新调整为大根堆
    }
    return;
}
int main(void)
{
    int ARRAY[] = { 5, 4, 7, 3, 9, 1, 6, 8, 2 };
    printf("Before sorted:\n"); //打印排序前数组内容
    for (int i = 0; i < 9; i++)
    {
        printf("%d ", ARRAY[i]);
    }
    printf("\n");
    heapSort(ARRAY, 9); //堆排序
    printf("After sorted:\n"); //打印排序后数组内容
    for (i = 0; i < 9; i++)
    {
        printf("%d ", ARRAY[i]);
    }
    printf("\n");
}

```

说明：堆排序，虽然实现复杂，但是非常的实用。另外读者可是自己设计实现小堆排序的算法。虽然和大堆排序的实现过程相似，但是却可以加深对堆排序的记忆和理解。

28.编程实现基数排序

```
#include <stdio.h>
```

Linux高性能服务器开发网网：<https://linuxcpp.0voice.com/>

```
#include <malloc.h>
#define LEN 8
typedef struct node //队列结点
{
    int data;
    struct node * next;
}node, *QueueNode;
typedef struct Queue //队列
{
    QueueNode front;
    QueueNode rear;
}Queue, *QueueLink;
QueueLink CreateNullQueue(QueueLink &Q) //创建空队列
{
    Q = NULL;
    Q = (QueueLink)malloc(sizeof(Queue));
    if (NULL == Q)
    {
        printf("Fail to malloc null queue!\n");
        return NULL;
    }
    Q->front = (QueueNode)malloc(sizeof(node));
    Q->rear = (QueueNode)malloc(sizeof(node));
    if (NULL == Q->front || NULL == Q->rear)
    {
        printf("Fail to malloc a new queue's front or rear!\n");
        return NULL;
    }
    Q->rear = NULL;
    Q->front->next = Q->rear;
    return Q;
}
int lenData(node data[], int len) //计算队列中各结点的数据的最大位数
{
    int m = 0;
    int temp = 0;
    int d;
    for (int i = 0; i < len; i++)
    {
        d = data[i].data;
        while (d > 0)
        {
            d /= 10;
            temp++;
        }
        if (temp > m)
        {
            m = temp;
        }
        temp = 0;
    }
    return m;
}
QueueLink Push(QueueLink &Q, node node) //将数据压入队列
{

```



```

QueueNode p1, p;
p = (QueueNode)malloc(sizeof(node));
if (NULL == p)
{
    printf("Fail to malloc a new node!\n");
    return NULL;
}
p1 = Q->front;
while (p1->next != NULL)
{
    p1 = p1->next;
}
p->data = node.data;
p1->next = p;
p->next = Q->rear;
return NULL;
}
node Pop(QueueLink &Q) //数据出队列
{
    node temp;
    temp.data = 0;
    temp.next = NULL;
    QueueNode p;
    p = Q->front->next;
    if (p != Q->rear)
    {
        temp = *p;
        Q->front->next = p->next;
        free(p);
        p = NULL;
    }
    return temp;
}
int IsEmpty(QueueLink Q)
{
    if (Q->front->next == Q->rear)
    {
        return 0;
    }
    return 1;
}
int main(void)
{
    int i = 0;
    int Max = 0; //记录结点中数据的最大位数
    int d = 10;
    int power = 1;
    int k = 0;
    node Array[LEN] = { { 450, NULL }, { 32, NULL }, { 781, NULL }, { 57, NULL },
组 { 145, NULL }, { 613, NULL }, { 401, NULL }, { 594, NULL } };
    //队列结点数
    QueueLink Queue[10];
    for (i = 0; i < 10; i++)
    {

```

```

CreateNullQueue(Queue[1]); //初始化队列数组
}
for (i = 0; i < LEN; i++)
{
    printf("%d ", Array[i].data);
}
printf("\n");
Max = lenData(Array, LEN); //计算数组中关键字的最大位数
printf("%d\n", Max);
for (int j = 0; j < Max; j++) //按位排序
{
    if (j == 0) power = 1;
    else power = power * d;
    for (i = 0; i < LEN; i++)
    {
        k = Array[i].data / power - (Array[i].data / (power * d)) * d;
        Push(Queue[k], Array[i]);
    }
    for (int l = 0, k = 0; l < d; l++) //排序后出队列重入数组
    {
        while (!IsEmpty(Queue[l]))
        {
            Array[k++] = Pop(Queue[l]);
        }
    }
    for (int t = 0; t < LEN; t++)
    {
        printf("%d ", Array[t].data);
    }
    printf("\n");
}
return 0;
}

```

说明：队列为基数排序的实现提供了很大的方便，适当的数据机构可以减少算法的复杂度，让更多的算法实现更容易。

29.谈谈你对编程规范的理解或认识

编程规范可总结为：程序的可行性、可读性、可移植性以及可测试性。

说明：这是编程规范的总纲目，面试者不一定要去背诵上面给出的那几个例子，应该去理解这几个例子说明的问题，想一想，自己如何解决可行性、可读性、可移植性以及可测试性这几个问题，结合以上几个例子和自己平时的编程习惯来回答这个问题。

30.short i = 0; i = i + 1L ; 这两句有错吗

代码一是错的，代码二是正确的。

说明：在数据安全的情况下大类型的数据向小类型的数据转换一定要显示的强制类型转换。

31.&&和&、||和|有什么区别

(1) &和|对操作数进行求值运算，&&和||只是判断逻辑关系。(2) &&和||在判断左侧操作数就能确定结果的情况下就不再对右侧操作数求值。

注意：在编程的时候有些时候将&&或||替换成&或|没有出错，但是其逻辑是错误的，可能会导致不可预想的后果（比如当两个操作数一个是1另一个是2时）。

32.C++的引用和 C 语言的指针有什么区别

指针和引用主要有以下区别：

(1) 引用必须被初始化，但是不分配存储空间。指针不声明时初始化，在初始化的时候需要分配存储空间。

(2) 引用初始化以后不能被改变，指针可以改变所指的对象。

(3) 不存在指向空值的引用，但是存在指向空值的指针。

注意：引用作为函数参数时，会引发一定的问题，因为让引用作参数，目的就是想改变这个引用所指向地址的内容，而函数调用时传入的是实参，看不出函数的参数是正常变量，还是引用，因此可能会引发错误。所以使用时一定要小心谨慎。

33.在二元树中找出和为某一值的所有路径

输入一个整数和一棵二元树。从树的根结点开始往下访问，一直到叶结点所经过的所有结点形成一条路径。打印出和与输入整数相等的所有路径。例如，输入整数9和如下二元树：

则打印出两条路径：3，6和3，2，4。

【答案】

```
typedef struct path
{
    BiTNode* tree;                //结点数据成员
    struct path* next;
}PATH,*pPath;                    //结点指针成员
//初始化树的结点栈:
void init_path( pPath* L )
{
    *L = ( pPath )malloc( sizeof( PATH ) );
    ( *L )->next = NULL;
} //创建空树
//树结点入栈函数:
void push_path(pPath H, pBTNode T)
{
    pPath p = H->next;
    pPath q = H;
    while( NULL != p )
    {
        q = p;
        p = p->next;
    }
    p = ( pPath )malloc( sizeof( PATH ) ); //申请新结点
    p->next = NULL; //初始化新结点
    p->tree = T;
    q->next = p; //新结点入栈
} //树结点打印函数:
void print_path( pPath L )
{
    pPath p = L->next;
```

```

while( NULL != p )
{
    printf("%d, ", p->tree->data);    p = p->next;
}

//打印当前栈中所有数据

///树结点出栈函数:

void pop_path( pPath H )
{
    pPath p = H->next;
    pPath q = H;
    if( NULL == p )                //检验当前栈是否为空
    {
        printf("Stack is null!\n");
        return;
    }
    p = p->next;
    while( NULL != p )            //出栈
    {
        q = q->next;
        p = p->next;
    }
    free( q->next );                //释放出栈结点空间
    q->next = NULL;
}

//判断结点是否为叶子结点:
int IsLeaf(pBTree T)
{
    return ( T->lchild == NULL )&&( T->rchild==NULL );
}

//查找符合条件的路径:
int find_path(pBTree T, int sum, pPath L)
{
    push_path( L, T); record += T->data;
    if( ( record == sum ) && ( IsLeaf( T ) ) )    //打印符合条件的当前路径
    {
        print_path( L );
        printf( "\n" );
    }
    if( T->lchild != NULL )                //递归查找当前节点的左孩子
    {
        find_path( T->lchild, sum, L);
    }
    if( T->rchild != NULL )                //递归查找当前节点的右孩子
    {
        find_path( T->rchild, sum, L);
    }
    record -= T->data;    pop_path(L);    return 0;
}

```

注意：数据结构一定要活学活用，例如本题，把所有的结点都压入栈，而不符合条件的结点弹出栈，很容易实现了有效路径的查找。虽然用链表也可以实现，但是用栈更利于理解这个问题，即适当的数据结构为更好的算法设计提供了有利的条件。

注意：数据结构一定要活学活用，例如本题，把所有的结点都压入栈，而不符合条件的结点弹出栈，很容易实现了有效路径的查找。虽然用链表也可以实现，但是用栈更利于理解这个问题，即适当的数据结构为更好的算法设计提供了有利的条件。

34.写一个“标准”宏 MIN

写一个“标准”宏 MIN，这个宏输入两个参数并且返回较小的一个。

【答案】

```
#define min(a,b)((a)<=(b)?(a):(b))
```

注意：在调用时一定要注意这个宏定义的副作用，如下调用：

```
((++p)<=(x)?(++p):(x))
```

p 指针就自加了两次，违背了 MIN 的本意。

35.typedef 和 define 有什么区别

(1) 用法不同：typedef 用来定义一种数据类型的别名，增强程序的可读性。define 主要用来定义常量，以及书写复杂使用频繁的宏。

(2) 执行时间不同：typedef 是编译过程的一部分，有类型检查的功能。define 是宏定义，是预编译的部分，其发生在编译之前，只是简单的进行字符串的替换，不进行类型的检查。

(3) 作用域不同：typedef 有作用域限定。define 不受作用域约束，只要是在 define 声明后的引用都是正确的。

(4) 对指针的操作不同：typedef 和 define 定义的指针时有很大的区别。

注意：typedef 定义是语句，因为句尾要加上分号。而 define 不是语句，千万不能在句尾加分号。

36.关键字 const 是什么

const 用来定义一个只读的变量或对象。主要优点：便于类型检查、同宏定义一样可以方便地进行参数的修改和调整、节省空间，避免不必要的内存分配、可为函数重载提供参考。

说明：const 修饰函数参数，是一种编程规范的要求，便于阅读，一看即知这个参数不能被改变，实现时不易出错。const 修饰成员函数不可修改成员变量。

37.static 有什么作用

static 在 C 中主要用于定义全局静态变量、定义局部静态变量、定义静态函数。在 C++ 中新增了两种作用：定义静态数据成员、静态函数成员。

注意：因为 static 定义的变量分配在静态区，所以其定义的变量的默认值为 0，普通变量的默认值为随机数，在定义指针变量时要特别注意。

38.extern 有什么作用

extern 标识的变量或者函数声明其定义在别的文件中，提示编译器遇到此变量和函数时在其它模块中寻找其定义。

39.流操作符重载为什么返回引用

在程序中，流操作符 >> 和 << 经常连续使用。因此这两个操作符的返回值应该是一个仍旧支持这两个操作符的流引用。其他的数据类型都无法做到这一点。

注意：除了在赋值操作符和流操作符之外的其他的一些操作符中，如 +、-、*、/ 等却千万不能返回引用。因为这四个操作符的对象都是右值，因此，它们必须构造一个对象作为返回值。

40.简述指针常量与常量指针区别

指针常量是指定义了一个指针，这个指针的值只能在定义时初始化，其他地方不能改变。常量指针是指定义了一个指针，这个指针指向一个只读的对象，不能通过常量指针来改变这个对象的值。

指针常量强调的是指针的不可改变性，而常量指针强调的是指针对其所指对象的不可改变性。

注意：无论是指针常量还是常量指针，其最大的用途就是作为函数的形式参数，保证实参在被调用函数中的不可改变特性。

41.数组名和指针的区别

请写出以下代码的打印结果：

```
#include <iostream.h>
#include <string.h>
void main(void)
{
    char str[13]="Hello world!";
    char *pstr="Hello world!";
    cout<<sizeof(str)<<endl;
    cout<<sizeof(pstr)<<endl;
    cout<<strlen(str)<<endl;
    cout<<strlen(pstr)<<endl;
    return;
}
```

【答案】

打印结果：

13

4

12 12

注意：一定要记得数组名并不是真正意义上的指针，它的内涵要比指针丰富的多。但是当数组名当做参数传递给函数后，其失去原来的含义，变作普通的指针。另外要注意 sizeof 不是函数，只是操作符。

42.如何避免“野指针”

“野指针”产生原因及解决办法如下：

（1）指针变量声明时没有被初始化。解决办法：指针声明时初始化，可以是具体的地址值，也可让它指向 NULL。

（2）指针 p 被 free 或者 delete 之后，没有置为 NULL。解决办法：指针指向的内存空间被释放后指针应该指向 NULL。

（3）指针操作超越了变量的作用范围。解决办法：在变量的作用域结束前释放掉变量的地址空间并且让指针指向 NULL。

注意：“野指针”的解决方法也是编程规范的基本原则，平时使用指针时一定要避免产生“野指针”，在使用指针前一定要检验指针的合法性。

43.常引用有什么作用

常引用的引入主要是为了避免使用变量的引用时，在不知情的情况下改变变量的值。常引用主要用于定义一个普通变量的只读属性的别名、作为函数的传入形参，避免实参在调用函数中被意外的改变。

说明：很多情况下，需要用常引用做形参，被引用对象等效于常对象，不能在函数中改变实参的值，这样的好处是有较高的易读性和较小的出错率。

44.编码实现字符串转化为数字

编码实现函数 `atoi()`，设计一个程序，把一个字符串转化为一个整型数值。例如数字：“5486321”，转化成字符：5486321。

【答案】

```
int myAtoi(const char * str)
{
    int num = 0; //保存转换后的数值
    int isNegative = 0; //记录字符串中是否有负号
    int n = 0;
    char *p = str;
    if (p == NULL) //判断指针的合法性
    {
        return -1;
    }
    while (*p++ != '\0') //计算数字字符串度
    {
        n++;
    }
    p = str;
    if (p[0] == '-') //判断数组是否有负号
    {
        isNegative = 1;
    }
    char temp = '0';
    for (int i = 0; i < n; i++)
    {
        char temp = *p++;
        if (temp > '9' || temp < '0') //滤除非数字字符
        {
            continue;
        }
        if (num != 0 || temp != '0') //滤除字符串开始的 0 字符
        {
            temp -= 0x30; //将数字字符转换为数值
            num += temp *int(pow(10, n - 1 - i));
        }
    }
    if (isNegative) //如果字符串中有负号，将数值取反
    {
        return (0 - num);
    }
    else
    {
        return num; //返回转换后的数值
    }
}
```

注意：此段代码只是实现了十进制字符串到数字的转化，读者可以自己实现 2 进制，8 进制，10 进制，16 进制的转化。

45.简述 `strcpy`、`sprintf` 与 `memcpy` 的区别

(1) 操作对象不同，strcpy 的两个操作对象均为字符串，sprintf 的操作源对象可以是多种数据类型，目的操作对象是字符串，memcpy 的两个对象就是两个任意可操作的内存地址，并不限于何种数据类型。

(2) 执行效率不同，memcpy 最高，strcpy 次之，sprintf 的效率最低。

(3) 实现功能不同，strcpy 主要实现字符串变量间的拷贝，sprintf 主要实现其他数据类型格式到字符串的转化，memcpy 主要是内存块间的拷贝。

说明：strcpy、sprintf 与 memcpy 都可以实现拷贝的功能，但是针对的对象不同，根据实际需求，来选择合适的函数实现拷贝功能。

46.用 C 编写一个死循环程序

```
while(1)
```

```
{}
```

说明：很多种途径都可实现同一种功能，但是不同的方法时间和空间占用度不同，特别是对于嵌入式软件，处理器速度比较慢，存储空间较小，所以时间和空间优势是选择各种方法的首要考虑条件。

47.编码实现某一变量某位清 0 或置 1

给定一个整型变量 a，写两段代码，第一个设置 a 的 bit 3，第二个清 a 的 bit 3，在以上两个操作中，要保持其他位不变。

【答案】

```
#define BIT3 (0x1 << 3 )
static int a;
//设置 a 的 bit 3:
void set_bit3( void )
{
    a |= BIT3;
} //将 a 第 3 位置 1
//清 a 的 bit 3
void set_bit3( void )
{
    a &= ~BIT3;
} //将 a 第 3 位清零
```

说明：在置或清变量或寄存器的某一位时，一定要注意不要影响其他位。所以用加减法是很难实现的。

48.评论下面这个中断函数

中断是嵌入式系统中重要的组成部分，这导致了很多编译开发商提供一种扩展——让标准 C 支持中断。具体代表事实是，产生了一个新的关键字 **interrupt**。下面的代码就使用了 interrupt 关键字去定义一个中断服务子程序(ISR)，请评论以下这段代码。

```
__interrupt double compute_area (double radius)
{
    double area = PI * radius * radius; printf(" Area = %f", area); return area;
}
```

【答案】

这段中断服务程序主要有以下四个问题：

- (1) ISR 不能返回一个值。
- (2) ISR 不能传递参数。
- (3) 在 ISR 中做浮点运算是明智的。
- (4) printf()经常有重入和性能上的问题。

注意：本题的第三个和第四个问题虽不是考察的重点，但是如果能提到这两点可给面试官留下一个好印象。

49.构造函数能否为虚函数

构造函数不能是虚函数。而且不能在构造函数中调用虚函数，因为那样实际执行的是父类的对应函数，因为自己还没有构造好。析构函数可以是虚函数，而且，在一个复杂类结构中，这往往是必须的。析构函数也可以是纯虚函数，但纯虚析构函数必须有定义体，因为析构函数的调用是在子类中隐含的。说明：虚函数的动态绑定特性是实现重载的关键技术，动态绑定根据实际的调用情况查询相应类的虚函数表，调用相应的虚函数。

50.谈谈你对面向对象的认识

面向对象可以理解成对待每一个问题，都是首先要确定这个问题由几个部分组成，而每一个部分其实就是一个对象。然后再分别设计这些对象，最后得到整个程序。传统的程序设计多是基于功能的思想来进行考虑和设计的，而面向对象的程序设计则是基于对象的角度来考虑问题。这样做能够使得程序更加的简洁清晰。

说明：编程中接触最多的“面向对象编程技术”仅仅是面向对象技术中的一个组成部分。发挥面向对象技术的优势是一个综合的技术问题，不仅需要面向对象的分析，设计和编程技术，而且需要借助必要的建模和开发工具。

51.const、static作用。

52.c++面向对象三大特征及对他们的理解，引出多态实现原理、动态绑定、菱形继承。

53.虚析构的必要性，引出内存泄漏，虚函数和普通成员函数的储存位置，虚函数表、虚函数表指针

54.malloc、free和new、delete区别，引出malloc申请大内存、malloc申请空间失败怎么办

55.stl熟悉吗，vector、map、list、hashMap，vector底层，map引出红黑树。优先队列用过吗，使用的场景。无锁队列听说过吗，原理是什么（比较并交换）

56.实现擅长的排序，说出原理（快排、堆排）

57.四种cast，智能指针

58.tcp和udp区别

59.进程和线程区别

60.指针和引用作用以及区别

61.c++11用过哪些特性，auto作为返回值和模板一起怎么用，函数指针能和auto混用吗

62.boost用过哪些类，thread、asio、signal、bind、function

63.单例、工厂模式、代理、适配器、模板，使用场景

64.QT信号槽实现机制，QT内存管理，MFC消息机制

65.进程间通信。会选一个详细问

66.多线程，锁和信号量，互斥和同步

67.动态库和静态库的区别

//auto作为返回值和模板一起怎么用，函数指针能和auto混用吗

```
#include <iostream>

using namespace std;

template <typename T,typename U>
auto add(T t,U u) -> decltype(t+u)
{
    return t+u;
}

template <typename T,typename U>
auto sub(T t,U u) -> decltype(t-u)
{
    return t-u;
}

template <typename T,typename U>
auto pro(T t,U u) -> decltype(t*u)
{
    return t*u;
}

template <typename T,typename U>
auto div(T t,U u) -> decltype(t/u)
{
    try
    {
        return t/u;
    }
    catch(...)
    {
        exit(0);
    }
}
```

```
}  
  
}  
  
int main()  
{  
    int x = 520;  
    double y= 13.14;  
    auto z = add(x,y);  
    cout<<z<<endl;  
  
    //auto(*funp[4])(int ,double) = {add,sub,pro,div}; //error  
    double (*funp[4])(int ,double) = {add,sub,pro,div};  
    for(unsigned char i=0;i<4;i++)  
    {  
        cout<<funp[i](x,y)<<endl;  
    }  
    return 0;  
}
```

68.提高c++性能，你用过哪些方式去提升

（构造、析构、返回值优化、临时对象（使用operator=()消除临时对象）、内联（内联技巧、条件内联、递归内联、静态局部变量内联）、内存池、使用函数对象不使用函数指针、编码（编译器优化、预先计算）、设计（延迟计算、高效数据结构）、系统体系结构（寄存器、缓存、上下文切换））

69.编译原理，尝试自己写过语言或语言编译器

70.泛型模板实用度高

71.对多种计算机语言熟悉

72.Git项目了解多少

73.针对网络框架（DPDK）、逆向工程（汇编）、分布式集群（docker、k8s、redis等）、CPU计算（nvidia cuda）、图像识别（opencv、opengl、tensorflow等）、AI等有研究

版权声明：本文为CSDN博主「bostonAlen」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/BostonRayAlen/article/details/93041395>

74.引用和指针的区别？

- 1)指针是一个实体，需要分配内存空间。引用只是变量的别名，不需要分配内存空间。
- 2)引用在定义的时候必须进行初始化，并且不能够改变。指针在定义的时候不一定要初始化，并且指向的空间可变。（注：不能有引用的值不能为NULL）
- 3)有多级指针，但是没有多级引用，只能有一级引用。
- 4)指针和引用的自增运算结果不一样。（指针是指向下一个空间，引用时引用的变量值加1）

- 5)sizeof 引用得到的是所指向的变量（对象）的大小，而sizeof 指针得到的是指针本身的大小。
- 6)引用访问一个变量是直接访问，而指针访问一个变量是间接访问。
- 7)使用指针前最好做类型检查，防止野指针的出现；
- 8)引用底层是通过指针实现的；
- 9)作为参数时也相同，传指针的实质是传值，传递的值是指针的地址；传引用的实质是传地址，传递的是变量的地址。

75.从汇编层去解释一下引用

```

1.9:          int x = 1;
2.00401048    mov         dword ptr [ebp-4],1
3.10:          int &b = x;
4.0040104F    lea         eax,[ebp-4]
5.00401052    mov         dword ptr [ebp-8],eax

```

x的地址为ebp-4，b的地址为ebp-8，因为栈内的变量内存是从高往低进行分配的。所以b的地址比x的低。lea eax,[ebp-4] 这条语句将x的地址ebp-4放入eax寄存器mov dword ptr [ebp-8],eax 这条语句将eax的值放入b的地址ebp-8中上面两条汇编的作用即：将x的地址存入变量b中，这不和将某个变量的地址存入指针变量是一样的吗？所以从汇编层次来看，的确引用是通过指针来实现的。

76.C++中的指针参数传递和引用参数传递

- 1)指针参数传递本质上是值传递，它所传递的是一个地址值。值传递过程中，被调函数的形式参数作为被调函数的局部变量处理，会在栈中开辟内存空间以存放由主调函数传递进来的实参值，从而形成了实参的一个副本（替身）。值传递的特点是，被调函数对形式参数的任何操作都是作为局部变量进行的，不会影响主调函数的实参变量的值（形参指针变了，实参指针不会变）。
- 2)引用参数传递过程中，被调函数的形式参数也作为局部变量在栈中开辟了内存空间，但是这时存放的是由主调函数放进来的实参变量的地址。被调函数对形参（本体）的任何操作都被处理成间接寻址，即通过栈中存放的地址访问主调函数中的实参变量（根据别名找到主调函数中的本体）。因此，被调函数对形参的任何操作都会影响主调函数中的实参变量。
- 3)引用传递和指针传递是不同的，虽然他们都是在被调函数栈空间上的一个局部变量，但是任何对于引用参数的处理都会通过一个间接寻址的方式操作到主调函数中的相关变量。而对于指针传递的参数，如果改变被调函数中的指针地址，它将应用不到主调函数的相关变量。如果想通过指针参数传递来改变主调函数中的相关变量（地址），那就得使用指向指针的指针或者指针引用。
- 4)从编译的角度来讲，程序在编译时分别将指针和引用添加到符号表上，符号表中记录的是变量名及变量所对应地址。指针变量在符号表上对应的地址值为指针变量的地址值，而引用在符号表上对应的地址值为引用对象的地址值（与实参名字不同，地址相同）。符号表生成之后就不会再改，因此指针可以改变其指向的对象（指针变量中的值可以改），而引用对象则不能修改。

77.形参与实参的区别？

- 1)形参变量只有在被调用时才分配内存单元，在调用结束时，即刻释放所分配的内存单元。因此，形参只有在函数内部有效。函数调用结束返回主调函数后则不能再使用该形参变量。
- 2)实参可以是常量、变量、表达式、函数等，无论实参是何种类型的量，在进行函数调用时，它们都必须具有确定的值，以便把这些值传送给形参。因此应预先用赋值，输入等办法使实参获得确定值，会产生一个临时变量。
- 3)实参和形参在数量上，类型上，顺序上应严格一致，否则会发生“类型不匹配”的错误。
- 4)函数调用中发生的数据传送是单向的。即只能把实参的值传送给形参，而不能把形参的值反向地传送给实参。因此在函数调用过程中，形参的值发生改变，而实参中的值不会变化。
- 5)当形参和实参不是指针类型时，在该函数运行时，形参和实参是不同的变量，他们在内存中位于不同的位置，形参将实参的内容复制一份，在该函数运行结束的时候形参被释放，而实参内容不会改变。

- 1)值传递：有一个形参向函数所属的栈拷贝数据的过程，如果值传递的对象是类对象 或是大的结构体对象，将耗费一定的时间和空间。（传值）
- 2)指针传递：同样有一个形参向函数所属的栈拷贝数据的过程，但拷贝的数据是一个固定为4字节的地址。（传值，传递的是地址值）
- 3)引用传递：同样有上述的数据拷贝过程，但它是针对地址的，相当于为该数据所在的地址起了一个别名。（传地址）
- 4)效率上讲，指针传递和引用传递比值传递效率高。一般主张使用引用传递，代码逻辑上更加紧凑、清晰。

78.static的用法和作用？

- 1.先来介绍它的第一条也是最重要的一条：隐藏。（static函数，static变量均可）
当同时编译多个文件时，所有未加static前缀的全局变量和函数都具有全局可见性。
 - 2.static的第二个作用是保持变量内容的持久。（static变量中的记忆功能和全局生存期）存储在静态数据区的变量会在程序刚开始运行时就完成初始化，也是唯一的一次初始化。共有两种变量存储在静态存储区：全局变量和static变量，只不过和全局变量比起来，static可以控制变量的可见范围，说到底static还是用来隐藏的。
 - 3.static的第三个作用是默认初始化为0（static变量）
其实全局变量也具备这一属性，因为全局变量也存储在静态数据区。在静态数据区，内存中所有的字节默认值都是0x00，某些时候这一特点可以减少程序员的工作量。
 - 4.static的第四个作用：C++中的类成员声明static
 - 1)函数体内static变量的作用范围为该函数体，不同于auto变量，该变量的内存只被分配一次，因此其值在下次调用时仍维持上次的值；
 - 2)在模块内的static全局变量可以被模块内所用函数访问，但不能被模块外其它函数访问；
 - 3)在模块内的static函数只可被这一模块内的其它函数调用，这个函数的使用范围被限制在声明它的模块内；
 - 4)在类中的static成员变量属于整个类所拥有，对类的所有对象只有一份拷贝；
 - 5)在类中的static成员函数属于整个类所拥有，这个函数不接收this指针，因而只能访问类的static成员变量。
- 类内：
- 6)static类对象必须要在类外进行初始化，static修饰的变量先于对象存在，所以static修饰的变量要在类外初始化；
 - 7)由于static修饰的类成员属于类，不属于对象，因此static类成员函数是没有this指针的，this指针是指向本对象的指针。正因为没有this指针，所以static类成员函数不能访问非static的类成员，只能访问static修饰的类成员；
 - 8)static成员函数不能被virtual修饰，static成员不属于任何对象或实例，所以加上virtual没有任何实际意义；静态成员函数没有this指针，虚函数的实现是为每一个对象分配一个vptr指针，而vptr是通过this指针调用的，所以不能为virtual；虚函数的调用关系，this->vptr->ctable->virtual function

79.静态变量什么时候初始化

- 1)初始化只有一次，但是可以多次赋值，在主程序之前，编译器已经为其分配好了内存。
- 2)静态局部变量和全局变量一样，数据都存放在全局区域，所以在主程序之前，编译器已经为其分配好了内存，但在C和C++中静态局部变量的初始化节点又有点不太一样。在C中，初始化发生在代码执行之前，编译阶段分配好内存之后，就会进行初始化，所以我们看到在C语言中无法使用变量对静态局部变量进行初始化，在程序运行结束，变量所处的全局内存会被全部回收。
- 3)而在C++中，初始化时在执行相关代码时才会进行初始化，主要是由于C++引入对象后，要进行初始化必须执行相应构造函数和析构函数，在构造函数或析构函数中经常会需要进行某些程序中需要进行的特定操作，并非简单地分配内存。所以C++标准定为全局或静态对象是有首次用到时才会进行构造，并通过atexit()来管理。在程序结束，按照构造顺序反方向进行逐个析构。所以在C++中是可以使用变量对静态局部变量进行初始化的。

80.const?

- 1)阻止一个变量被改变，可以使用const关键字。在定义该const变量时，通常需要对它进行初始化，因为以后就没有机会再去改变它了；
- 2)对指针来说，可以指定指针本身为const，也可以指定指针所指的数据为const，或二者同时指定为const；
- 3)在一个函数声明中，const可以修饰形参，表明它是一个输入参数，在函数内部不能改变其值；
- 4)对于类的成员函数，若指定其为const类型，则表明其是一个常函数，不能修改类的成员变量，类的常对象只能访问类的常成员函数；
- 5)对于类的成员函数，有时候必须指定其返回值为const类型，以使得其返回值不为“左值”。
- 6)const成员函数可以访问非const对象的非const数据成员、const数据成员，也可以访问const对象内的所有数据成员；
- 7)非const成员函数可以访问非const对象的非const数据成员、const数据成员，但不可以访问const对象的任意数据成员；
- 8)一个没有明确声明为const的成员函数被看作是将要修改对象中数据成员的函数，而且编译器不允许它作为一个const对象所调用。因此const对象只能调用const成员函数。
- 9)const类型变量可以通过类型转换符const_cast将const类型转换为非const类型；
- 10)const类型变量必须定义的时候进行初始化，因此也导致如果类的成员变量有const类型的变量，那么该变量必须在类的初始化列表中进行初始化；
- 11)对于函数值传递的情况，因为参数传递是通过复制实参创建一个临时变量传递进函数的，函数内只能改变临时变量，但无法改变实参。则这个时候无论加不加const对实参不会产生任何影响。但是在引用或指针传递函数调用中，因为传进去的是一个引用或指针，这样函数内部可以改变引用或指针所指向的变量，这时const才是实实在在地保护了实参所指向的变量。因为在编译阶段编译器对调用函数的选择是根据实参进行的，所以，只有引用传递和指针传递可以用是否加const来重载。一个拥有顶层const的形参无法和另一个没有顶层const的形参区分开来。

81.const成员函数的理解和应用？

- ①const Stock & Stock::topval (②const Stock & s) ③const
- ①处const：确保返回的Stock对象在以后的使用中不能被修改
- ②处const：确保此方法不修改传递的参数 S
- ③处const：保证此方法不修改调用它的对象，const对象只能调用const成员函数,不能调用非const函数

82.指针和const的用法

- 1)当const修饰指针时，由于const的位置不同，它的修饰对象会有所不同。
- 2)int const p2中const修饰p2的值,所以理解为p2的值不可以改变，即p2只能指向固定的一个变量地址，但可以通过p2读写这个变量的值。顶层指针表示指针本身是一个常量
- 3)int const p1或者const int p1两种情况中const修饰p1，所以理解为p1的值不可以改变，即不可以给*p1赋值改变p1指向变量的值，但可以通过给p赋值不同的地址改变这个指针指向。底层指针表示指针所指向的变量是一个常量。
- 4)int const *const p;

83.mutable

- 1)如果需要在const成员方法中修改一个成员变量的值，那么需要将这个成员变量修饰为mutable。即用mutable修饰的成员变量不受const成员方法的限制;
- 2)可以认为mutable的变量是类的辅助状态，但是只是起到类的一些方面表述的功能，修改他的内容我们可以认为对象的状态本身并没有改变的。实际上由于const_cast的存在，这个概念很多时候用处不是很到了。

84.extern用法？

1)extern修饰变量的声明

如果文件a.c需要引用b.c中变量int v，就可以在a.c中声明extern int v，然后就可以引用变量v。

2)extern修饰函数的声明

如果文件a.c需要引用b.c中的函数，比如在b.c中原型是int fun(int mu)，那么就可以在a.c中声明extern int fun (int mu)，然后就能使用fun来做任何事情。就像变量的声明一样，extern int fun (int mu)可以放在a.c中任何地方，而不一定非要放在a.c的文件作用域的范围中。

3)extern修饰符可用于指示C或者C++函数的调用规范。

比如在C++中调用C库函数，就需要在C++程序中用extern “C”声明要引用的函数。这是给链接器用的，告诉链接器在链接的时候用C函数规范来链接。主要原因是C++和C程序编译完成后在目标代码中命名规则不同。

85.int转字符串字符串转int？

strcat,strcpy,strncpy,memset,memcpy的内部实现？

c++11标准增加了全局函数std::to_string

可以使用std::stoi/stol/stoll等等函数

strcpy拥有返回值，有时候函数原本不需要返回值，但为了增加灵活性如支持链式表达，

86.深拷贝与浅拷贝？

1)浅复制——只是拷贝了基本类型的数据，而引用类型数据，复制后也是会发生引用，我们把这种拷贝叫做“（浅复制）浅拷贝”，换句话说，浅复制仅仅是指向被复制的内存地址，如果原地址中对象被改变了，那么浅复制出来的对象也会相应改变。

深复制——在计算机中开辟了一块新的内存地址用于存放复制的对象。

2)在某些状况下，类内成员变量需要动态开辟堆内存，如果实行浅拷贝，也就是把对象里的值完全复制给另一个对象，如A=B。这时，如果B中有一个成员变量指针已经申请了内存，那A中的那个成员变量也指向同一块内存。这就出现了问题：当B把内存释放了（如：析构），这时A内的指针就是野指针了，出现运行错误。

87.C++模板是什么，底层怎么实现的？

1)编译器并不是把函数模板处理成能够处理任意类的函数；编译器从函数模板通过具体类型产生不同的函数；编译器会对函数模板进行两次编译：在声明的地方对模板代码本身进行编译，在调用的地方对参数替换后的代码进行编译。

2)这是因为函数模板要被实例化后才能成为真正的函数，在使用函数模板的源文件中包含函数模板的头文件，如果该头文件中只有声明，没有定义，那编译器无法实例化该模板，最终导致链接错误。

88.C语言struct和C++struct区别

1)C语言中：struct是用户自定义数据类型（UDT）；C++中struct是抽象数据类型（ADT），支持成员函数的定义，（C++中的struct能继承，能实现多态）。

2)C中struct是没有权限的设置，且struct中只能是一些变量的集合体，可以封装数据却不可以隐藏数据，而且成员不可以是函数。

3)C++中，struct的成员默认访问说明符为public（为了与C兼容），class中的默认访问限定符为private，struct增加了访问权限，且可以和类一样有成员函数。

4)struct作为类的一种特例是用来自定义数据结构的。一个结构标记声明后，在C中必须在结构标记前加上struct，才能做结构类型名

89.虚函数可以声明为inline吗？

1)虚函数用于实现运行时的多态，或者称为晚绑定或动态绑定。而内联函数用于提高效率。内联函数的原理是，在编译期间，对调用内联函数的地方的代码替换成函数代码。内联函数对于程序中需要频繁使用 and 调用的小函数非常有用。

2)虚函数要求在运行时进行类型确定，而内联函数要求在编译期完成相关的函数替换；

90.类成员初始化方式？构造函数的执行顺序？为什么用成员初始化列表会快一些？

1)赋值初始化，通过在函数体内进行赋值初始化；列表初始化，在冒号后使用初始化列表进行初始化。这两种方式的主要区别在于：

对于在函数体中初始化,是在所有的数据成员被分配内存空间后才进行的。

列表初始化是给数据成员分配内存空间时就进行初始化,就是说分配一个数据成员只要冒号后有此数据成员的赋值表达式(此表达式必须是括号赋值表达式),那么分配了内存空间后在进入函数体之前给数据成员赋值,就是说初始化这个数据成员此时函数体还未执行。

2)一个派生类构造函数的执行顺序如下：

①虚拟基类的构造函数（多个虚拟基类则按照继承的顺序执行构造函数）。

②基类的构造函数（多个普通基类也按照继承的顺序执行构造函数）。

③类类型的成员对象的构造函数（按照初始化顺序）

④派生类自己的构造函数。

3)方法一是在构造函数当中做赋值的操作，而方法二是做纯粹的初始化操作。我们都知道，C++的赋值操作是会产生临时对象的。临时对象的出现会降低程序的效率。

91.成员列表初始化？

1)必须使用成员初始化的四种情况

①当初始化一个引用成员时；

②当初始化一个常量成员时；

③当调用一个基类的构造函数，而它拥有一组参数时；

④当调用一个成员类的构造函数，而它拥有一组参数时；

2)成员初始化列表做了什么

①编译器会——操作初始化列表，以适当的顺序在构造函数之内安插初始化操作，并且在任何显示用户代码之前；

②list中的项目顺序是由类中的成员声明顺序决定的，不是由初始化列表的顺序决定的；

92.构造函数为什么不能为虚函数？析构函数为什么要虚函数？

从存储空间角度，虚函数相应一个指向vtable虚函数表的指针，这大家都知道，但是这个指向vtable的指针事实上是存储在对象的内存空间的。问题出来了，假设构造函数是虚的，就须要通过vtable来调用，但是对象还没有实例化，也就是内存空间还没有，怎么找vtable呢？所以构造函数不能是虚函数。

从使用角度，虚函数主要用于在信息不全的情况下，能使重载的函数得到相应的调用。构造函数本身就是初始化实例，那使用虚函数也没有实际意义呀。所以构造函数没有必要是虚函数。虚函数的作用在于通过父类的指针或者引用来调用它的时候可以变成调用子类的那个成员函数。而构造函数是在创建对象时自己主动调用的，不可能通过父类的指针或者引用去调用，因此也就规定构造函数不能是虚函数。构造函数不须要是虚函数，也不同意是虚函数，由于创建一个对象时我们总是要明白指定对象的类型，虽然我们可能通过实验室的基类的指针或引用去访问它但析构却不一定，我们往往通过基类的指针来销毁对象。这时候假设析构函数不是虚函数，就不能正确识别对象类型从而不能正确调用析构函数。

从实现上看，vbt在构造函数调用后才建立，因而构造函数不可能成为虚函数从实际含义上看，在调用

构造函数时还不能确定对象的真实类型（由于子类会调父类的构造函数）；并且构造函数的作用是提供初始化，在对象生命周期仅仅运行一次，不是对象的动态行为，也没有必要成为虚函数。

当一个构造函数被调用时，它做的首要的事情之中的一个是初始化它的VPTR。因此，它仅仅能知道它是“当前”类的，而全然忽视这个对象后面是否还有继承者。当编译器为这个构造函数产生代码时，它是为这个类的构造函数产生代码——既不是为基类，也不是为它的派生类（由于类不知道谁继承它）。所以它使用的VPTR必须是对于这个类的VTABLE。并且，仅仅要它是最后的构造函数调用，那么在这个对象的生命期内，VPTR将保持被初始化为指向这个VTABLE，但假设接着另一个更晚派生的构造函数被调用，这个构造函数又将设置VPTR指向它的VTABLE，等直到最后的构造函数结束。VPTR的状态是由被最后调用的构造函数确定的。这就是为什么构造函数调用是从基类到更加派生类顺序的还有一个理由。可是，当这一系列构造函数调用正发生时，每一个构造函数都已经设置VPTR指向它自己的VTABLE。假设函数调用使用虚机制，它将仅仅产生通过它自己的VTABLE的调用，而不是最后的VTABLE（全部构造函数被调用后才会有最后的VTABLE）。

因为构造函数本来就是为了明确初始化对象成员才产生的，然而virtual function主要是为了再不完全了解细节的情况下也能正确处理对象。另外，virtual函数是在不同类型的对象产生不同的动作，现在对象还没有产生，如何使用virtual函数来完成你想完成的动作。

直接的讲，C++中基类采用virtual虚析构函数是为了防止内存泄漏。具体地说，如果派生类中申请了内存空间，并在其析构函数中对这些内存空间进行释放。假设基类中采用的是非虚析构函数，当删除基类指针指向的派生类对象时就不会触发动态绑定，因而只会调用基类的析构函数，而不会调用派生类的析构函数。那么在这种情况下，派生类中申请的空间就得不到释放从而产生内存泄漏。所以，为了防止这种情况的发生，C++中基类的析构函数应采用virtual虚析构函数。

93.析构函数的作用，如何起作用？

1)构造函数只是起初始化值的作用，但实例化一个对象的时候，可以通过实例去传递参数，从主函数传递到其他的函数里面，这样就使其他的函数里面有值了。规则，只要你一实例化对象，系统自动回调用一个构造函数，就是你写，编译器也自动调用一次。

2)析构函数与构造函数的作用相反，用于撤销对象的一些特殊任务处理，可以是释放对象分配的内存空间；特点：析构函数与构造函数同名，但该函数前面加~。析构函数没有参数，也没有返回值，而且不能重载，在一个类中只能有一个析构函数。当撤销对象时，编译器也会自动调用析构函数。每一个类必须有一个析构函数，用户可以自定义析构函数，也可以是编译器自动生成默认的析构函数。一般析构函数定义为类的公有成员。

94.构造函数和析构函数可以调用虚函数吗，为什么？

1)在C++中，提倡不在构造函数和析构函数中调用虚函数；

2)构造函数和析构函数调用虚函数时都不使用动态联编，如果在构造函数或析构函数中调用虚函数，则运行的是为构造函数或析构函数自身类型定义的版本；

3)因为父类对象会在子类之前进行构造，此时子类部分的数据成员还未初始化，因此调用子类的虚函数时不安全的，故而C++不会进行动态联编；

4)析构函数是用来销毁一个对象的，在销毁一个对象时，先调用子类的析构函数，然后再调用基类的析构函数。所以在调用基类的析构函数时，派生类对象的数据成员已经销毁，这个时候再调用子类的虚函数没有任何意义。

95.构造函数的执行顺序？析构函数的执行顺序？构造函数内部干了啥？拷贝构造干了啥？

1)构造函数顺序

①基类构造函数。如果有多个基类，则构造函数的调用顺序是某类在类派生表中出现的顺序，而不是它们在成员初始化表中的顺序。

②成员类对象构造函数。如果有多个成员类对象则构造函数的调用顺序是对象在类中被声明的顺序，而不是它们出现在成员初始化表中的顺序。

③派生类构造函数。

2)析构函数顺序

- ①调用派生类的析构函数；
- ②调用成员类对象的析构函数；
- ③调用基类的析构函数。

96.虚析构函数的作用，父类的析构函数是否要设置为虚函数？

1)C++中基类采用virtual虚析构函数是为了防止内存泄漏。具体地说，如果派生类中申请了内存空间，并在其析构函数中对这些内存空间进行释放。假设基类中采用的是非虚析构函数，当删除基类指针指向的派生类对象时就不会触发动态绑定，因而只会调用基类的析构函数，而不会调用派生类的析构函数。那么在这种情况下，派生类中申请的空间就得不到释放从而产生内存泄漏。所以，为了防止这种情况的发生，C++中基类的析构函数应采用virtual虚析构函数。

2)纯虚析构函数一定得定义，因为每一个派生类析构函数会被编译器加以扩张，以静态调用的方式调用其每一个虚基类以及上一层基类的析构函数。因此，缺乏任何一个基类析构函数的定义，就会导致链接失败。因此，最好不要把虚析构函数定义为纯虚析构函数。

24.构造函数析构函数可以调用虚函数吗？

- 1)在构造函数和析构函数中最好不要调用虚函数；
- 2)构造函数或者析构函数调用虚函数并不会发挥虚函数动态绑定的特性，跟普通函数没区别；
- 3)即使构造函数或者析构函数如果能成功调用虚函数，程序的运行结果也是不可控的。

97.构造函数析构函数可否抛出异常

C++只会析构已经完成的对象，对象只有在其构造函数执行完毕才算是完全构造妥当。在构造函数中发生异常，控制权转出构造函数之外。因此，在对象b的构造函数中发生异常，对象b的析构函数不会被调用。因此会造成内存泄漏。

2)用auto_ptr对象来取代指针类成员，便对构造函数做了强化，免除了抛出异常时发生资源泄漏的危机，不再需要在析构函数中手动释放资源；

3)如果控制权基于异常的因素离开析构函数，而此时正有另一个异常处于作用状态，C++会调用terminate函数让程序结束；

4)如果异常从析构函数抛出，而且没有在当地进行捕捉，那个析构函数便是执行不全的。如果析构函数执行不全，就是没有完成他应该执行的每一件事情。

98.类如何实现只能静态分配和只能动态分配

1)前者是把new、delete运算符重载为private属性。后者是把构造、析构函数设为protected属性，再用子类来动态创建

2)建立类的对象有两种方式：

①静态建立，静态建立一个类对象，就是由编译器为对象在栈空间中分配内存；

②动态建立，A *p = new A();动态建立一个类对象，就是使用new运算符为对象在堆空间中分配内存。这个过程分为两步，第一步执行operator new()函数，在堆中搜索一块内存并进行分配；第二步调用类构造函数构造对象；

3)只有使用new运算符，对象才会被建立在堆上，因此只要限制new运算符就可以实现类对象只能建立在栈上。可以将new运算符设为私有。

99.如果想将某个类用作基类，为什么该类必须定义而非声明？

1)派生类中包含并且可以使用它从基类继承而来的成员，为了使用这些成员，派生类必须知道他们是什么。

100.什么情况会自动生成默认构造函数？

1)带有默认构造函数的类成员对象，如果一个类没有任何构造函数，但它含有一个成员对象，而后者有默认构造函数，那么编译器就为该类合成出一个默认构造函数。不过这个合成操作只有在构造函数真正被需要的时候才会发生；如果一个类A含有多个成员类对象的话，那么类A的每一个构造函数必须调用每一个成员对象的默认构造函数而且必须按照类对象在类A中的声明顺序进行；

2)带有默认构造函数的基类，如果一个没有任务构造函数的派生类派生自一个带有默认构造函数基类，那么该派生类会合成一个构造函数调用上一层基类的默认构造函数；

3)带有一个虚函数的类

4)带有一个虚基类的类

5)合成的默认构造函数中，只有基类子对象和成员类对象会被初始化。所有其他的非静态数据成员都不会被初始化。

101.什么是类的继承？

1)类与类之间的关系

has-A包含关系，用以描述一个类由多个部件类构成，实现has-A关系用类的成员属性表示，即一个类的成员属性是另一个已经定义好的类；

use-A，一个类使用另一个类，通过类之间的成员函数相互联系，定义友元或者通过传递参数的方式来实现；

is-A，继承关系，关系具有传递性；

2)继承的相关概念

所谓的继承就是一个类继承了另一个类的属性和方法，这个新的类包含了上一个类的属性和方法，被称为子类或者派生类，被继承的类称为父类或者基类；

3)继承的特点

子类拥有父类的所有属性和方法，子类可以拥有父类没有的属性和方法，子类对象可以当做父类对象使用；

4)继承中的访问控制

public、protected、private

5)继承中的构造和析构函数

6)继承中的兼容性原则

102.什么是组合？

1)一个类里面的数据成员是另一个类的对象，即内嵌其他类的对象作为自己的成员；创建组合类的对象：首先创建各个内嵌对象，难点在于构造函数的设计。创建对象时既要和基本类型的成员进行初始化，又要对内嵌对象进行初始化。

2)创建组合类对象，构造函数的执行顺序：先调用内嵌对象的构造函数，然后按照内嵌对象成员在组合类中的定义顺序，与组合类构造函数的初始化列表顺序无关。然后执行组合类构造函数的函数体，析构函数调用顺序相反。

103.抽象基类为什么不能创建对象？

抽象类是一种特殊的类，它是为了抽象和设计的目的为建立的，它处于继承层次结构的较上层。

(1)抽象类的定义：

称带有纯虚函数的类为抽象类。

(2)抽象类的作用：

抽象类的主要作用是将有关的操作作为结果接口组织在一个继承层次结构中，由它来为派生类提供一个

公共的根，派生类将具体实现在其基类中作为接口的操作。所以派生类实际上刻画了一组子类的操作接口的通用语义，这些语义也传给子类，子类可以具体实现这些语义，也可以再将这些语义传给自己的子类。

(3) 使用抽象类时注意：

抽象类只能作为基类来使用，其纯虚函数的实现由派生类给出。如果派生类中没有重新定义纯虚函数，而只是继承基类的纯虚函数，则这个派生类仍然还是一个抽象类。如果派生类中给出了基类纯虚函数的实现，则该派生类就不再是抽象类了，它是一个可以建立对象的具体类。抽象类是不能定义对象的。一个纯虚函数不需要（但是可以）被定义。

一、纯虚函数定义

纯虚函数是一种特殊的虚函数，它的一般格式如下：

```
class <类名>
{
    virtual <类型><函数名>(<参数表>)=0;
    ...
};
```

在许多情况下，在基类中不能对虚函数给出有意义的实现，而把它声明为纯虚函数，它的实现留给该基类的派生类去做。这就是纯虚函数的作用。

纯虚函数可以让类先具有一个操作名称，而没有操作内容，让派生类在继承时再去具体地给出定义。凡是含有纯虚函数的类叫做抽象类。这种类不能声明对象，只是作为基类为派生类服务。除非在派生类中完全实现基类中所有的纯虚函数，否则，派生类也变成了抽象类，不能实例化对象。

二、纯虚函数引入原因

1、为了方便使用多态特性，我们常常需要在基类中定义虚拟函数。

2、在很多情况下，基类本身生成对象是不合情理的。例如，动物作为一个基类可以派生出老虎、孔雀等子类，但动物本身生成对象明显不合常理。

为了解决上述问题，引入了纯虚函数的概念，将函数定义为纯虚函数（方法：virtual Return Type Function()= 0;）。若要使派生类为非抽象类，则编译器要求在派生类中，必须对纯虚函数予以重载以实现多态性。同时含有纯虚函数的类称为抽象类，它不能生成对象。这样就很好地解决了上述两个问题。例如，绘画程序中，shape作为一个基类可以派生出圆形、矩形、正方形、梯形等，如果我要求面积总和的话，那么会可以使用一个 shape * 的数组，只要依次调用派生类的area()函数了。如果不用接口就没法定义成数组，因为既可以是circle,也可以是square,而且以后还可能加上rectangle，等等。

三、相似概念

1、多态性

指相同对象收到不同消息或不同对象收到相同消息时产生不同的实现动作。C++支持两种多态性：编译时多态性，运行时多态性。

a.编译时多态性：通过重载函数实现

b.运行时多态性：通过虚函数实现。

2、虚函数

虚函数是在基类中被声明为virtual，并在派生类中重新定义的成员函数，可实现成员函数的动态重载。

3、抽象类

包含纯虚函数的类称为抽象类。由于抽象类包含了没有定义的纯虚函数，所以不能定义抽象类的对象。

104.类什么时候会析构？

1)对象生命周期结束，被销毁时；

2)delete指向对象的指针时，或delete指向对象的基类类型指针，而其基类虚函数是虚函数时；

3)对象i是对象o的成员，o的析构函数被调用时，对象i的析构函数也被调用。

105.为什么友元函数必须在类内部声明？

1)因为编译器必须能够读取这个结构的声明以理解这个数据类型的大、行为等方面的所有规则。有一条规则在任何关系中都很重要，那就是谁可以访问我的私有部分。

106.介绍一下C++里面的多态？

(1) 静态多态（重载，模板）

是在编译的时候，就确定调用函数的类型。

(2) 动态多态（覆盖，虚函数实现）

在运行的时候，才确定调用的是哪个函数，动态绑定。运行基类指针指向派生类的对象，并调用派生类的函数。

虚函数实现原理：虚函数表和虚函数指针。

纯虚函数：virtual int fun() = 0;

函数的运行版本由实参决定，在运行时选择函数的版本，所以动态绑定又称为运行时绑定。

当编译器遇到一个模板定义时，它并不生成代码。只有当实例化出模板的一个特定版本时，编译器才会生成代码。

107.用C语言实现C++的继承

```
#include <iostream>
using namespace std;

//C++中的继承与多态
struct A
{
    virtual void fun()    //C++中的多态:通过虚函数实现
    {
        cout<<"A:fun()"<<endl;
    }

    int a;
};

struct B:public A        //C++中的继承:B类公有继承A类
{
    virtual void fun()    //C++中的多态:通过虚函数实现（子类的关键字virtual可加可不加）
    {
        cout<<"B:fun()"<<endl;
    }

    int b;
};

//C语言模拟C++的继承与多态

typedef void (*FUN)();    //定义一个函数指针来实现对成员函数的继承

struct _A                //父类
{
    FUN _fun;              //由于C语言中结构体不能包含函数，故只能用函数指针在外面实现
```



```
int _a;

};

struct _B          //子类
{
    _A _a_;        //在子类中定义一个基类的对象即可实现对父类的继承
    int _b;
};

void _fA()          //父类的同名函数
{
    printf("_A:_fun()\n");
}

void _fB()          //子类的同名函数
{
    printf("_B:_fun()\n");
}

void Test()
{
    //测试C++中的继承与多态
    A a;            //定义一个父类对象a
    B b;            //定义一个子类对象b

    A* p1 = &a;      //定义一个父类指针指向父类的对象
    p1->fun();        //调用父类的同名函数
    p1 = &b;         //让父类指针指向子类的对象
    p1->fun();        //调用子类的同名函数

    //C语言模拟继承与多态的测试
    _A _a;          //定义一个父类对象_a
    _B _b;          //定义一个子类对象_b
    _a._fun = _fA;   //父类的对象调用父类的同名函数
    _b._a._fun = _fB; //子类的对象调用子类的同名函数

    _A* p2 = &_a;    //定义一个父类指针指向父类的对象
    p2->_fun();       //调用父类的同名函数
    p2 = (_A*)&_b;    //让父类指针指向子类的对象,由于类型不匹配所以要进行强转
    p2->_fun();       //调用子类的同名函数
}
```

108.继承机制中对象之间如何转换？指针和引用之间如何转换？

1)向上类型转换

将派生类指针或引用转换为基类的指针或引用被称为向上类型转换，向上类型转换会自动进行，而且向上类型转换是安全的。

2)向下类型转换

将基类指针或引用转换为派生类指针或引用被称为向下类型转换，向下类型转换不会自动进行，因为一个基类对应几个派生类，所以向下类型转换时不知道对应哪个派生类，所以在向下类型转换时必须加动态类型识别技术。RTTI技术，用dynamic_cast进行向下类型转换。

109.组合与继承优缺点？

一：继承

继承是Is a 的关系，比如说Student继承Person,则说明Student is a Person。继承的优点是子类可以重写父类的方法来方便地实现对父类的扩展。

继承的缺点有以下几点：

- ①：父类的内部细节对子类是可见的。
- ②：子类从父类继承的方法在编译时就确定下来了，所以无法在运行期间改变从父类继承的方法的行为。
- ③：如果对父类的方法做了修改的话（比如增加了一个参数），则子类的方法必须做出相应的修改。所以说子类与父类是一种高耦合，违背了面向对象思想。

二：组合

组合也就是设计类的时候把要组合的类的对象加入到该类中作为自己的成员变量。

组合的优点：

- ①：当前对象只能通过所包含的那个对象去调用其方法，所以所包含的对象的内部细节对当前对象时不可见的。
- ②：当前对象与包含的对象是一个低耦合关系，如果修改包含对象的类中代码不需要修改当前对象类的代码。
- ③：当前对象可以在运行时动态的绑定所包含的对象。可以通过set方法给所包含对象赋值。

组合的缺点：①：容易产生过多的对象。②：为了能组合多个对象，必须仔细对接口进行定义。

110.左值右值

1)在C++11中所有的值必属于左值、右值两者之一，右值又可以细分为纯右值、将亡值。在C++11中可以取地址的、有名字的就是左值，反之，不能取地址的、没有名字的就是右值（将亡值或纯右值）。举个例子，`int a = b+c`, `a` 就是左值，其有变量名为`a`，通过`&a`可以获取该变量的地址；表达式`b+c`、函数`int func()`的返回值是右值，在其被赋值给某一变量前，我们不能通过变量名找到它，`&(b+c)`这样的操作则不会通过编译。

2)C++11对C++98中的右值进行了扩充。在C++11中右值又分为纯右值（`prvalue`, `Pure Rvalue`）和将亡值（`xvalue`, `Expiring Value`）。其中纯右值的概念等同于我们在C++98标准中右值的概念，指的是临时变量和不跟对象关联的字面量值；将亡值则是C++11新增的跟右值引用相关的表达式，这样表达式通常是将要被移动的对象（移为他用），比如返回右值引用`T&&`的函数返回值、`std::move`的返回值，或者转换为`T&&`的类型转换函数的返回值。将亡值可以理解为通过“盗取”其他变量内存空间的方式获取到的值。在确保其他变量不再被使用、或即将被销毁时，通过“盗取”的方式可以避免内存空间的释放和分配，能够延长变量值的生命期。

3)左值引用就是对一个左值进行引用的类型。右值引用就是对一个右值进行引用的类型，事实上，由于右值通常不具有名字，我们也只能通过引用的方式找到它的存在。右值引用和左值引用都是属于引用类型。无论是声明一个左值引用还是右值引用，都必须立即进行初始化。而其原因可以理解为是引用类型本身自己并不拥有所绑定对象的内存，只是该对象的一个别名。左值引用是具名变量值的别名，而右值引用则是不具名（匿名）变量的别名。左值引用通常也不能绑定到右值，但常量左值引用是个“万能”的引用类型。它可以接受非常量左值、常量左值、右值对其进行初始化。不过常量左值所引用的右值在它的“余生”中只能是只读的。相对地，非常量左值只能接受非常量左值对其进行初始化。

4)右值引用通常不能绑定到任何的左值，要想绑定一个左值到右值引用，通常需要`std::move`将左值强制转换为右值。

111.移动构造函数

1)我们用对象`a`初始化对象`b`，后对象`a`我们就不在使用了，但是对象`a`的空间还在呀（在析构之前），既然拷贝构造函数，实际上就是把`a`对象的内容复制一份到`b`中，那么为什么我们不能直接使用`a`的空间呢？这样就避免了新的空间的分配，大大降低了构造的成本。这就是移动构造函数设计的初衷；

2)拷贝构造函数中，对于指针，我们一定要采用深层复制，而移动构造函数中，对于指针，我们采用浅

层复制。浅层复制之所以危险，是因为两个指针共同指向一片内存空间，若第一个指针将其释放，另一个指针的指向就不合法了。所以我们只要避免第一个指针释放空间就可以了。避免的方法就是将第一个指针（比如a->value）置为NULL，这样在调用析构函数的时候，由于有判断是否为NULL的语句，所以析构a的时候并不会回收a->value指向的空间；

3)移动构造函数的参数和拷贝构造函数不同，拷贝构造函数的参数是一个左值引用，但是移动构造函数的初值是一个右值引用。意味着，移动构造函数的参数是一个右值或者将亡值的引用。也就是说，只引用一个右值，或者将亡值初始化另一个对象的时候，才会调用移动构造函数。而那个move语句，就是将一个左值变成一个将亡值。

112.C语言的编译链接过程？

源代码 - ->预处理 - ->编译 - ->优化 - ->汇编 - ->链接->可执行文件

1)预处理

读取c源程序，对其中的伪指令（以#开头的指令）和特殊符号进行处理。包括宏定义替换、条件编译指令、头文件包含指令、特殊符号。预编译程序所完成的基本上是对源程序的“替代”工作。经过此种替代，生成一个没有宏定义、没有条件编译指令、没有特殊符号的输出文件。i)预处理后的c文件，ii)预处理后的C++文件。

2)编译阶段

编译程序所要作的工作就是通过词法分析和语法分析，在确认所有的指令都符合语法规则之后，将其翻译成等价的中间代码表示或汇编代码。.s文件

3)汇编过程

汇编过程实际上指把汇编语言代码翻译成目标机器指令的过程。对于被翻译系统处理的每一个C语言源程序，都将最终经过这一处理而得到相应的目标文件。目标文件中所存放的也就是与源程序等效的目标的机器语言代码。.o目标文件

4)链接阶段

链接程序的主要工作就是将有关的目标文件彼此相连接，也即将在一个文件中引用的符号同该符号在另外一个文件中的定义连接起来，使得所有的这些目标文件成为一个能够诶操作系统装入执行的统一整体。

113.vector与list的区别与应用？怎么找某vector或者list的倒数第二个元素

1)vector数据结构

vector和数组类似，拥有一段连续的内存空间，并且起始地址不变。因此能高效的进行随机存取，时间复杂度为 $O(1)$ ；但因为内存空间是连续的，所以在进行插入和删除操作时，会造成内存块的拷贝，时间复杂度为 $O(n)$ 。另外，当数组中内存空间不够时，会重新申请一块内存空间并进行内存拷贝。连续存储结构：vector是可以实现动态增长的对象数组，支持对数组高效率的访问和在数组尾端的删除和插入操作，在中间和头部删除和插入相对不易，需要挪动大量的数据。它与数组最大的区别就是vector不需程序员自己去考虑容量问题，库里面本身已经实现了容量的动态增长，而数组需要程序员手动写入扩容函数进行扩容。

2)list数据结构

list是由双向链表实现的，因此内存空间是不连续的。只能通过指针访问数据，所以list的随机存取非常没有效率，时间复杂度为 $O(n)$ ；但由于链表的特点，能高效地进行插入和删除。非连续存储结构：list是一个双链表结构，支持对链表的双向遍历。每个节点包括三个信息：元素本身，指向前一个元素的节点（prev）和指向下一个元素的节点（next）。因此list可以高效率的对数据元素任意位置进行访问和插入删除等操作。由于涉及对额外指针的维护，所以开销比较大。

区别：

vector的随机访问效率高，但在插入和删除时（不包括尾部）需要挪动数据，不易操作。list的访问要遍历整个链表，它的随机访问效率低。但对数据的插入和删除操作等都比较方便，改变指针的指向即可。

list是单向的，vector是双向的。vector中的迭代器在使用后就失效了，而list的迭代器在使用之后还可以

继续使用。

3)

```
int mySize = vec.size();vec.at(mySize -2);
```

list不提供随机访问，所以不能用下标直接访问到某个位置的元素，要访问list里的元素只能遍历，不过你要是只需要访问list的最后N个元素的话，可以用反向迭代器来遍历：

114.STL vector的实现，删除其中的元素，迭代器如何变化？为什么是两倍扩容？释放空间？

size()函数返回的是已用空间大小，capacity()返回的是总空间大小，capacity()-size()则是剩余的可用空间大小。当size()和capacity()相等，说明vector目前的空间已被用完，如果再添加新元素，则会引起vector空间的动态增长。

由于动态增长会引起重新分配内存空间、拷贝原空间、释放原空间，这些过程会降低程序效率。因此，可以使用reserve(n)预先分配一块较大的指定大小的内存空间，这样当指定大小的内存空间未使用完时，是不会重新分配内存空间的，这样便提升了效率。只有当n>capacity()时，调用reserve(n)才会改变vector容量。

resize()成员函数只改变元素的数目，不改变vector的容量。

- 空的vector对象，size()和capacity()都为0
- 当空间大小不足时，新分配的空间大小为原空间大小的2倍。
- 使用reserve()预先分配一块内存后，在空间未满的情况下，不会引起重新分配，从而提升了效率。
- 当reserve()分配的空间比原空间小时，是不会引起重新分配的。
- resize()函数只改变容器的元素数目，未改变容器大小。
- 用reserve(size_type)只是扩大capacity值，这些内存空间可能还是“野”的，如果此时使用“[]”来访问，则可能会越界。而resize(size_type new_size)会真正使容器具有new_size个对象。

- 1.不同的编译器，vector有不同的扩容大小。在vs下是1.5倍，在GCC下是2倍；
- 2.空间和时间的权衡。简单来说，空间分配的多，平摊时间复杂度低，但浪费空间也多。
- 3.使用k=2增长因子的问题在于，每次扩展的新尺寸必然刚好大于之前分配的总和，也就是说，之前分配的内存空间不可能被使用。这样对内存不友好。最好把增长因子设为(1,2)
- 4.对比可以发现采用采用成倍方式扩容，可以保证常数的时间复杂度，而增加指定大小的容量只能达到O(n)的时间复杂度，因此，使用成倍的方式扩容。

如何释放空间：

由于vector的内存占用空间只增不减，比如你首先分配了10,000个字节，然后erase掉后面9,999个，留下一个有效元素，但是内存占用仍为10,000个。所有内存空间是在vector析构时候才能被系统回收。empty()用来检测容器是否为空的，clear()可以清空所有元素。但是即使clear()，vector所占用的内存空间依然如故，无法保证内存的回收。

如果需要空间动态缩小，可以考虑使用deque。如果vector，可以用swap()来帮助你释放内存。

```
vector(Vec).swap(Vec);
```

将Vec的内存空洞清除；

```
vector().swap(Vec);
```

清空Vec的内存；

115.容器内部删除一个元素

1)顺序容器

erase迭代器不仅使所指向被删除的迭代器失效，而且使被删元素之后的所有迭代器失效(list除外)，所以不能使用erase(it++)的方式，但是erase的返回值是下一个有效迭代器；

```
It = c.erase(it);
```

2)关联容器

erase迭代器只是被删除元素的迭代器失效，但是返回值是void，所以要采用erase(it++)的方式删除迭代器；

```
c.erase(it++)
```

116.STL迭代器如何实现

- 1.迭代器是一种抽象的设计理念，通过迭代器可以在不了解容器内部原理的情况下遍历容器，除此之外，STL中迭代器一个最重要的作用就是作为容器与STL算法的粘合剂。
- 2.迭代器的作用就是提供一个遍历容器内部所有元素的接口，因此迭代器内部必须保存一个与容器相关的指针，然后重载各种运算操作来遍历，其中最重要的是*运算符与->运算符，以及++、-等可能需要重载的运算符重载。这和C++中的智能指针很像，智能指针也是将一个指针封装，然后通过引用计数或是其他方法完成自动释放内存的功能。
- 3.最常用的迭代器的相应型别有五种：value type、difference type、pointer、reference、iterator catagoly;

117.set与hash_set的区别

- 1.set底层是以RB-Tree实现，hash_set底层是以hash_table实现的；
- 2.RB-Tree有自动排序功能，而hash_table不具有自动排序功能；
- 3.set和hash_set元素的键值就是实值；
- 4.hash_table有一些无法处理的型别；

118.hashmap与map的区别

- 1.底层实现不同；
- 2.map具有自动排序的功能，hash_map不具有自动排序的功能；
- 3.hashtable有一些无法处理的型别；

119.map、set是怎么实现的，红黑树是怎么能够同时实现这两种容器？为什么使用红黑树？

- 1)他们的底层都是以红黑树的结构实现，因此插入删除等操作都在O(logn)时间内完成，因此可以完成高效的插入删除；
- 2)在这里我们定义了一个模版参数，如果它是key那么它就是set，如果它是map，那么它就是map；底层是红黑树，实现map的红黑树的节点数据类型是key+value，而实现set的节点数据类型是value
- 3)因为map和set要求是自动排序的，红黑树能够实现这一功能，而且时间复杂度比较低。

120.如何在共享内存上使用stl标准库？

1)想像一下把STL容器，例如map, vector, list等等，放入共享内存中，IPC一旦有了这些强大的通用数据结构做辅助，无疑进程间通信的能力一下子强了很多。我们没必要再为共享内存设计其他额外的数据结构，另外，STL的高度可扩展性将为IPC所驱使。STL容器被良好的封装，默认情况下有它们自己的内存管理方案。当一个元素被插入到一个STL列表(list)中时，列表容器自动为其分配内存，保存数据。考虑到要将STL容器放到共享内存中，而容器却自己在堆上分配内存。一个最笨拙的办法是在堆上构造STL容器，然后把容器复制到共享内存，并且确保所有容器的内部分配的内存指向共享内存中的相应区域，这基本是个不可能完成的任务。

2)假设进程A在共享内存中放入了数个容器，进程B如何找到这些容器呢？一个方法就是进程A把容器放在共享内存中的确定地址上（fixed offsets），则进程B可以从该已知地址上获取容器。另外一个改进点的办法是，进程A先在共享内存某块确定地址上放置一个map容器，然后进程A再创建其他容器，然后给其取个名字和地址一并保存到这个map容器里。进程B知道如何获取该保存了地址映射的map容器，然后同样再根据名字取得其他容器的地址。

121.map插入方式有几种？

- 1)用insert函数插入pair数据，
`mapStudent.insert(pair<int, string>(1, "student_one"));`
- 2)用insert函数插入value_type数据
`mapStudent.insert(map<int, string>::value_type (1, "student_one"));`
- 3)在insert函数中使用make_pair()函数
`mapStudent.insert(make_pair(1, "student_one"));`
- 4)用数组方式插入数据
`mapStudent[1] = "student_one";`

122.STL中unordered_map(hash_map)和map的区别，hash_map如何解决冲突以及扩容

- 1)unordered_map和map类似，都是存储的key-value的值，可以通过key快速索引到value。不同的是unordered_map不会根据key的大小进行排序，
- 2)存储时是根据key的hash值判断元素是否相同，即unordered_map内部元素是无序的，而map中的元素是按照二叉搜索树存储，进行中序遍历会得到有序遍历。
- 3)所以使用时map的key需要定义operator<。而unordered_map需要定义hash_value函数并且重载operator==。但是很多系统内置的数据类型都自带这些，
- 4)那么如果是自定义类型，那么就需要自己重载operator<或者hash_value()了。
- 5)如果需要内部元素自动排序，使用map，不需要排序使用unordered_map
- 6)unordered_map的底层实现是hash_table;
- 7)hash_map底层使用的是hash_table，而hash_table使用的开链法进行冲突避免，所有hash_map采用开链法进行冲突解决。
- 8)什么时候扩容：当向容器添加元素的时候，会判断当前容器的元素个数，如果大于等于阈值—即当前数组的长度乘以加载因子的值的时候，就要自动扩容啦。
- 9)扩容(resize)就是重新计算容量，向HashMap对象里不停的添加元素，而HashMap对象内部的数组无法装载更多的元素时，对象就需要扩大数组的长度，以便能装入更多的元素。

123.vector越界访问下标，map越界访问下标？vector删除元素时会不会释放空间？

- 1)通过下标访问vector中的元素时不会做边界检查，即便下标越界。也就是说，下标与first迭代器相加的结果超过了finish迭代器的位置，程序也不会报错，而是返回这个地址中存储的值。如果想在访问vector中的元素时首先进行边界检查，可以使用vector中的at函数。通过使用at函数不但可以通过下标访问vector中的元素，而且在at函数内部会对下标进行边界检查。
- 2)map的下标运算符[]的作用是：将key作为下标去执行查找，并返回相应的值；如果不存在这个key，就将一个具有该key和value的某人值插入这个map。
- 3)erase()函数，只能删除内容，不能改变容量大小；erase成员函数，它删除了itVect迭代器指向的元素，并且返回要被删除的itVect之后的迭代器，迭代器相当于一个智能指针；clear()函数，只能清空内容，不能改变容量大小；如果要想在删除内容的同时释放内存，那么你可以选择deque容器。

124.map[]与find的区别？

- 1)map的下标运算符[]的作用是：将关键码作为下标去执行查找，并返回对应的值；如果不存在这个关键码，就将一个具有该关键码和值类型的默认值的项插入这个map。
- 2)map的find函数：用关键码执行查找，找到了返回该位置的迭代器；如果不存在这个关键码，就返回尾迭代器。

125.STL中list与queue之间的区别

- 1)list不再能够像vector一样以普通指针作为迭代器，因为其节点不保证在存储空间中连续存在；
- 2)list插入操作和结合才做都不会造成原有的list迭代器失效；
- 3)list不仅是一个双向链表，而且还是一个环状双向链表，所以它只需要一个指针；
- 4)list不像vector那样有可能在空间不足时做重新配置、数据移动的操作，所以插入前的所有迭代器在插入操作之后都仍然有效；
- 5)deque是一种双向开口的连续线性空间，所谓双向开口，意思是在可以在头尾两端分别做元素的插入和删除操作；可以在头尾两端分别做元素的插入和删除操作；
- 6)deque和vector最大的差异，一在于deque允许常数时间内对起头端进行元素的插入或移除操作，二在于deque没有所谓容量概念，因为它是动态地以分段连续空间组合而成，随时可以增加一段新的空间并链接起来，deque没有所谓的空间保留功能。

126.STL中的allocator,deallocator

- 1)第一级配置器直接使用malloc()、free()和realloc()，第二级配置器视情况采用不同的策略：当配置区块超过128bytes时，视之为足够大，便调用第一级配置器；当配置器区块小于128bytes时，为了降低额外负担，使用复杂的内存池整理方式，而不再用一级配置器；
- 2)第二级配置器主动将任何小额区块的内存需求量上调至8的倍数，并维护16个free-list，各自管理大小为8~128bytes的小额区块；
- 3)空间配置函数allocate()，首先判断区块大小，大于128就直接调用第一级配置器，小于128时就检查对应的free-list。如果free-list之内有可用区块，就直接拿来用，如果没有可用区块，就将区块大小调整至8的倍数，然后调用refill()，为free-list重新分配空间；
- 4)空间释放函数deallocate()，该函数首先判断区块大小，大于128bytes时，直接调用一级配置器，小于128bytes就找到对应的free-list然后释放内存。

127.STL中hash_map扩容发生什么？

- 1)hash table表格内的元素称为桶 (bucket),而由桶所链接的元素称为节点 (node),其中存入桶元素的容器为stl本身很重要的一种序列式容器——vector容器。之所以选择vector为存放桶元素的基础容器，主要是因为vector容器本身具有动态扩容能力，无需人工干预。
- 2)向前操作：首先尝试从目前所指的节点出发，前进一个位置 (节点)，由于节点被安置于list内，所以利用节点的next指针即可轻易完成前进操作，如果目前正巧是list的尾端，就跳至下一个bucket身上，那正是指向下一个list的头部节点。

128.map如何创建？

- 1.vector 底层数据结构为数组，支持快速随机访问
- 2.list 底层数据结构为双向链表，支持快速增删
- 3.deque 底层数据结构为一个中央控制器和多个缓冲区，详细见STL源码剖析P146，支持首尾 (中间不能) 快速增删，也支持随机访问
deque是一个双端队列(double-ended queue)，也是在堆中保存内容的.它的保存形式如下：
[堆1] --> [堆2] --> [堆3] --> ...
每个堆保存好几个元素,然后堆和堆之间有指针指向,看起来像是list和vector的结合品.
- 4.stack 底层一般用list或deque实现，封闭头部即可，不用vector的原因应该是容量大小有限制，扩容耗时
- 5.queue 底层一般用list或deque实现，封闭头部即可，不用vector的原因应该是容量大小有限制，扩容耗时 (stack和queue其实是适配器,而不叫容器，因为是对容器的再封装)
- 6.priority_queue 的底层数据结构一般为vector为底层容器，堆heap为处理规则来管理底层容器实现
- 7.set 底层数据结构为红黑树，有序，不重复
- 8.multiset 底层数据结构为红黑树，有序，可重复

- 9.map 底层数据结构为红黑树，有序，不重复
- 10.multimap 底层数据结构为红黑树，有序，可重复
- 11.hash_set 底层数据结构为hash表，无序，不重复
- 12.hash_multiset 底层数据结构为hash表，无序，可重复
- 13.hash_map 底层数据结构为hash表，无序，不重复
- 14.hash_multimap 底层数据结构为hash表，无序，可重复

129.vector的增加删除都是怎么做的？为什么是1.5倍？

- 1)新增元素：vector通过一个连续的数组存放元素，如果集合已满，在新增数据的时候，就要分配一块更大的内存，将原来的数据复制过来，释放之前的内存，在插入新增的元素；
- 2)对vector的任何操作，一旦引起空间重新配置，指向原vector的所有迭代器就都失效了；
- 3)初始时刻vector的capacity为0，塞入第一个元素后capacity增加为1；
- 4)不同的编译器实现的扩容方式不一样，VS2015中以1.5倍扩容，GCC以2倍扩容。

对比可以发现采用采用成倍方式扩容，可以保证常数的时间复杂度，而增加指定大小的容量只能达到O(n)的时间复杂度，因此，使用成倍的方式扩容。

- 1)考虑可能产生的堆空间浪费，成倍增长倍数不能太大，使用较为广泛的扩容方式有两种，以2二倍的方式扩容，或者以1.5倍的方式扩容。
- 2)以2倍的方式扩容，导致下一次申请的内存必然大于之前分配内存的总和，导致之前分配的内存不能再被使用，所以最好倍增长因子设置为(1,2)之间；
- 3)向量容器vector的成员函数pop_back()可以删除最后一个元素。
- 4)而函数erase()可以删除由一个iterator指出的元素，也可以删除一个指定范围的元素。
- 5)还可以采用通用算法remove()来删除vector容器中的元素。
- 6)不同的是：采用remove一般情况下不会改变容器的大小，而pop_back()与erase()等成员函数会改变容器的大小。

130.函数指针？

- 1)什么是函数指针？

函数指针指向的是特殊的数据类型，函数的类型是由其返回的数据类型和其参数列表共同决定的，而函数的名称则不是其类型的一部分。

一个具体函数的名字，如果后面不跟调用符号(即括号)，则该名字就是该函数的指针(注意：大部分情况下，可以这么认为，但这种说法并不很严格)。

- 2)函数指针的声明方法

int (pf)(const int&, const int&); (1)

上面的pf就是一个函数指针，指向所有返回类型为int，并带有两个const int&参数的函数。注意pf两边的括号是必须的，否则上面的定义就变成了：

int *pf(const int&, const int&); (2)

而这声明了一个函数pf，其返回类型为int *，带有两个const int&参数。

- 3)为什么有函数指针

函数与数据项相似，函数也有地址。我们希望在同一个函数中通过使用相同的形参在不同的时间使用产生不同的效果。

- 4)一个函数名就是一个指针，它指向函数的代码。一个函数地址是该函数的进入点，也就是调用函数的地址。函数的调用可以通过函数名，也可以通过指向函数的指针来调用。函数指针还允许将函数作为变元传递给其他函数；

- 5)两种方法赋值：

指针名 = 函数名； 指针名 = &函数名

131.说说你对c和c++的看法，c和c++的区别？

- 1) 第一点就应该想到C是面向过程的语言，而C++是面向对象的语言，一般简历上第一条都是熟悉C/C++基本语法，了解C++面向对象思想，那么，请问什么是面向对象？
- 2) C和C++动态管理内存的方法不一样，C是使用malloc/free函数，而C++除此之外还有new/delete关键字；（关于malloc/free与new/delete的不同又可以说一大堆，最后的扩展1部分列出十大区别）；
- 3) 接下来就不得不谈到C中的struct和C++的类，C++的类是C所没有的，但是C中的struct是可以在C++中正常使用的，并且C++对struct进行了进一步的扩展，使struct在C++中可以和class一样当做类使用，而唯一和class不同的地方在于struct的成员默认访问修饰符是public，而class默认的是private；
- 4) C++支持函数重载，而C不支持函数重载，而C++支持重载的依仗就在于C++的名字修饰与C不同，例如在C++中函数int fun(int,int)经过名字修饰之后变为_fun_int_int，而C是_fun，一般是这样的，所以C++才会支持不同的参数调用不同的函数；
- 5) C++中有引用，而C没有；这样就不得不提一下引用和指针的区别（文后扩展2）；
- 6) 当然还有C++全部变量的默认链接属性是外链接，而C是内连接；
- 7) C中用const修饰的变量不可以用在定义数组时的大小，但是C++用const修饰的变量可以（如果不进行&解引用的操作的话，是存放在符号表的，不开辟内存）；
- 8) 当然还有局部变量的声明规则不同，多态，C++特有输入输出流之类的，很多，下面就不再列出来了；“

132.c/c++的内存分配，详细说一下栈、堆、静态存储区？

- 1、栈区（stack）— 由编译器自动分配释放，存放函数的参数值，局部变量的值等，其操作方式类似于数据结构中的栈。
- 2、堆区（heap）— 一般由程序员分配释放，若程序员不释放，程序结束时可能由OS（操作系统）回收。注意它与数据结构中的堆是两回事，分配方式倒是类似于链表。
- 3、全局区（静态区）（static）— 全局变量和静态变量的存储是放在一块的，初始化的全局变量和静态变量在一块区域，未初始化的全局变量和未初始化的静态变量在相邻的另一块区域。程序结束后由系统释放。
- 4、文字常量区 — 常量字符串就是放在这里的。程序结束后由系统释放。
- 5、程序代码区 — 存放函数体的二进制代码。

133.堆与栈的区别？

- 1) 管理方式：对于栈来讲，是由编译器自动管理，无需我们手工控制；对于堆来说，释放工作由程序员控制，容易产生memory leak。
- 2) 空间大小：一般来讲在32位系统下，堆内存可以达到4G的空间，从这个角度来看堆内存几乎是没有什么限制的。但是对于栈来讲，一般都是有一定的空间大小的，例如，在VC6下面，默认的栈空间大小是1M（好像是，记不清楚了）。当然，我们可以修改：打开工程，依次操作菜单如下：Project->Setting->Link，在Category中选中Output，然后在Reserve中设定堆栈的最大值和commit。注意：reserve最小值为4Byte；commit是保留在虚拟内存的页文件里面，它设置的较大值会使栈开辟较大的值，可能增加内存的开销和启动时间。
- 3) 碎片问题：对于堆来讲，频繁的new/delete势必会造成内存空间的不连续，从而造成大量的碎片，使程序效率降低。对于栈来讲，则不会存在这个问题，因为栈是先进后出的队列，他们是如此的——对应，以至于永远都不可能有一个内存块从栈中间弹出，在他弹出之前，在他上面的后进的栈内容已经被弹出，详细的可以参考数据结构，这里我们就不再——讨论了。
- 4) 生长方向：对于堆来讲，生长方向是向上的，也就是向着内存地址增加的方向；对于栈来讲，它的生长方向是向下的，是向着内存地址减小的方向增长。
- 5) 分配方式：堆都是动态分配的，没有静态分配的堆。栈有2种分配方式：静态分配和动态分配。静态分配是编译器完成的，比如局部变量的分配。动态分配由alloca函数进行分配，但是栈的动态分配和堆是不同的，它的动态分配是由编译器进行释放，无需我们手工实现。
- 6) 分配效率：栈是机器系统提供的数据结构，计算机会在底层对栈提供支持：分配专门的寄存器存放栈的地址，压栈出栈都有专门的指令执行，这就决定了栈的效率比较高。堆则是C/C++函数库提供的，它的机制是很复杂的，例如为了分配一块内存，库函数会按照一定的算法（具体的算法可以参考数据结构/

操作系统)在堆内存中搜索可用的足够大小的空间,如果没有足够大小的空间(可能是由于内存碎片太多),就有可能调用系统功能去增加程序数据段的内存空间,这样就有机会分到足够大小的内存,然后进行返回。显然,堆的效率比栈要低得多。

134.野指针是什么?如何检测内存泄漏?

1)野指针:指向内存被释放的内存或者没有访问权限的内存的指针。

2)“野指针”的成因主要有3种:

①指针变量没有被初始化。任何指针变量刚被创建时不会自动成为NULL指针,它的缺省值是随机的,它会乱指一气。所以,指针变量在创建的同时应当被初始化,要么将指针设置为NULL,要么让它指向合法的内存。例如

```
char *p = NULL;
```

```
char *str = new char(100);
```

②指针p被free或者delete之后,没有置为NULL;

③指针操作超越了变量的作用范围。

3)如何避免野指针:

①对指针进行初始化

①将指针初始化为NULL。

```
char *p = NULL;
```

②用malloc分配内存

```
char *p = (char *)malloc(sizeof(char));
```

③用已有合法的可访问的内存地址对指针初始化

```
char num[30] = {0};
```

```
char *p = num;
```

②指针用完释放内存,将指针赋NULL。

```
delete p;
```

```
p = NULL;
```

135.悬空指针和野指针有什么区别?

1)野指针:野指针指,访问一个已删除或访问受限的内存区域的指针,野指针不能判断是否为NULL来避免。指针没有初始化,释放后没有置空,越界

2)悬空指针:一个指针的指向对象已被删除,那么就成了悬空指针。野指针是那些未初始化的指针。

136.内存泄漏

1)内存泄漏

内存泄漏是指由于疏忽或错误造成了程序未能释放掉不再使用的内存的情况。内存泄漏并非指内存存在物理上消失,而是应用程序分配某段内存后,由于设计错误,失去了对该段内存的控制;

2)后果

只发生一次小的内存泄漏可能不被注意,但泄漏大量内存的程序将会出现各种证照:性能下降到内存逐渐用完,导致另一个程序失败;

3)如何排除

使用工具软件BoundsChecker, BoundsChecker是一个运行时错误检测工具,它主要定位程序运行时期发生的各种错误;

调试运行DEBUG版程序,运用以下技术: CRT(C run-time libraries)、运行时函数调用堆栈、内存泄漏时提示的内存分配序号(集成开发环境OUTPUT窗口),综合分析内存泄漏的原因,排除内存泄漏。

4)解决方法

智能指针。

5)检查、定位内存泄漏

检查方法：在main函数最后面一行，加上一句CrtDumpMemoryLeaks()。调试程序，自然关闭程序让其退出，查看输出：

输出这样的格式{453}normal block at 0x02432CA8,868 bytes long

被{}包围的453就是我们需要的内存泄漏定位值，868 bytes long就是说这个地方有868比特内存没有释放。

定位代码位置

在main函数第一行加上CrtSetBreakAlloc(453);意思就是在申请453这块内存的位置中断。然后调试程序，程序中断了，查看调用堆栈。加上头文件#include <crtdbg.h>

137.new和malloc的区别？

- 1、new/delete是C++关键字，需要编译器支持。malloc/free是库函数，需要头文件支持；
- 2、使用new操作符申请内存分配时无须指定内存块的大小，编译器会根据类型信息自行计算。而malloc则需要显式地指出所需内存的尺寸。
- 3、new操作符内存分配成功时，返回的是对象类型的指针，类型严格与对象匹配，无须进行类型转换，故new是符合类型安全性的操作符。而malloc内存分配成功则是返回void *，需要通过强制类型转换将void*指针转换成我们需要的类型。
- 4、new内存分配失败时，会抛出bad_alloc异常。malloc分配内存失败时返回NULL。
- 5、new会先调用operator new函数，申请足够的内存（通常底层使用malloc实现）。然后调用类型的构造函数，初始化成员变量，最后返回自定义类型指针。delete先调用析构函数，然后调用operator delete函数释放内存（通常底层使用free实现）。malloc/free是库函数，只能动态的申请和释放内存，无法强制要求其做自定义类型对象构造和析构工作。

138.delete p;与delete[]p，allocator

- 1、动态数组管理new一个数组时，[]中必须是一个整数，但是不一定是常量整数，普通数组必须是一个常量整数；
- 2、new动态数组返回的并不是数组类型，而是一个元素类型的指针；
- 3、delete[]时，数组中的元素按逆序的顺序进行销毁；
- 4、new在内存分配上面有一些局限性，new的机制是将内存分配和对象构造组合在一起，同样的，delete也是将对象析构和内存释放组合在一起的。allocator将这两部分分开进行，allocator申请一部分内存，不进行初始化对象，只有当需要的时候才进行初始化操作。

139.new和delete的实现原理，delete是如何知道释放内存的大小的额？

- 1、new简单类型直接调用operator new分配内存；而对于复杂结构，先调用operator new分配内存，然后在分配的内存上调用构造函数；对于简单类型，new[]计算好大小后调用operator new；对于复杂数据结构，new[]先调用operator new[]分配内存，然后在p的前四个字节写入数组大小n，然后调用n次构造函数，针对复杂类型，new[]会额外存储数组大小；
 - ①new表达式调用一个名为operator new(operator new[])函数，分配一块足够大的、原始的、未命名的内存空间；
 - ②编译器运行相应的构造函数以构造这些对象，并为其传入初始值；
 - ③对象被分配了空间并构造完成，返回一个指向该对象的指针。
- 2、delete简单数据类型默认只是调用free函数；复杂数据类型先调用析构函数再调用operator delete；针对简单类型，delete和delete[]等同。假设指针p指向new[]分配的内存。因为要4字节存储数组大小，实际分配的内存地址为[p-4]，系统记录的也是这个地址。delete[]实际释放的就是p-4指向的内存。而delete会直接释放p指向的内存，这个内存根本没有被系统记录，所以会崩溃。
- 3、需要在 new [] 一个对象数组时，需要保存数组的维度，C++ 的做法是在分配数组空间时多分配了 4 个字节的大小，专门保存数组的大小，在 delete [] 时就可以取出这个保存的数，就知道了需要调用析构函数多少次了。

140.malloc申请的存储空间能用delete释放吗

不能，malloc/free主要为了兼容C，new和delete完全可以取代malloc/free的。malloc/free的操作对象都是必须明确大小的。而且不能用在动态类上。new和delete会自动进行类型检查和大小，malloc/free不能执行构造函数与析构函数，所以动态对象它是不行的。当然从理论上说使用malloc申请的内存是可以通过delete释放的。不过一般不这样写的。而且也不能保证每个C++的运行时都能正常。

141.malloc与free的实现原理？

- 1、在标准C库中，提供了malloc/free函数分配释放内存，这两个函数底层是由brk、mmap、munmap这些系统调用实现的；
- 2、brk是将数据段(.data)的最高地址指针edata往高地址推，mmap是在进程的虚拟地址空间中（堆和栈中间，称为文件映射区域的地方）找一块空闲的虚拟内存。这两种方式分配的都是虚拟内存，没有分配物理内存。在第一次访问已分配的虚拟地址空间的时候，发生缺页中断，操作系统负责分配物理内存，然后建立虚拟内存和物理内存之间的映射关系；
- 3、malloc小于128k的内存，使用brk分配内存，将edata往高地址推；malloc大于128k的内存，使用mmap分配内存，在堆和栈之间找一块空闲内存分配；brk分配的内存需要等到高地址内存释放以后才能释放，而mmap分配的内存可以单独释放。当最高地址空间的空闲内存超过128K（可由M_TRIM_THRESHOLD选项调节）时，执行内存紧缩操作（trim）。在上一个步骤free的时候，发现最高地址空闲内存超过128K，于是内存紧缩。
- 4、malloc是从堆里面申请内存，也就是说函数返回的指针是指向堆里面的一块内存。操作系统中有一个记录空闲内存地址的链表。当操作系统收到程序的申请时，就会遍历该链表，然后就寻找第一个空间大于所申请空间的堆结点，然后就将该结点从空闲结点链表中删除，并将该结点的空间分配给程序。

142.malloc、realloc、calloc的区别

1)malloc函数

```
void* malloc(unsigned int num_size);
```

int p = malloc(20*sizeof(int));申请20个int类型的空间；

2)calloc函数

```
void* calloc(size_t n,size_t size);
```

```
int *p = calloc(20, sizeof(int));
```

省去了人为空间计算；malloc申请的空间的值是随机初始化的，calloc申请的空间的值是初始化为0的；

3)realloc函数

```
void realloc(void *p, size_t new_size);
```

给动态分配的空间分配额外的空间，用于扩充容量。

143.stdcall和cdecl的区别？

1)stdcall

stdcall是函数恢复堆栈，只有在函数代码的结尾出现一次恢复堆栈的代码；在编译时就规定了参数个数，无法实现不定个数的参数调用；

2)cdecl

cdecl是调用者恢复堆栈，假设有100个函数调用函数a，那么内存中就有100端恢复堆栈的代码；可以不定参数个数；每一个调用它的函数都包含清空堆栈的代码，所以产生的可执行文件大小会比调用__stdcall函数大。

144.使用智能指针管理内存资源，RAII

1)RAII全称是“Resource Acquisition Is Initialization”，直译过来是“资源获取即初始化”，也就是说在构造函数中申请分配资源，在析构函数中释放资源。因为C++的语言机制保证了，当一个对象创建的时候，自动调用构造函数，当对象超出作用域的时候会自动调用析构函数。所以，在RAII的指导下，我们应该使用类来管理资源，将资源和对象的生命周期绑定。

2)智能指针（`std::shared_ptr`和`std::weak_ptr`）即RAII最具代表的实现，使用智能指针，可以实现自动的内存管理，再也不需要担心忘记delete造成的内存泄漏。毫不夸张的来讲，有了智能指针，代码中几乎不需要再出现delete了。

145.手写实现智能指针类

1)智能指针是一个数据类型，一般用模板实现，模拟指针行为的同时还提供自动垃圾回收机制。它会自动记录`SmartPointer<T*>`对象的引用计数，一旦T类型对象的引用计数为0，就释放该对象。除了指针对象外，我们还需要一个引用计数的指针设定对象的值，并将引用计数计为1，需要一个构造函数。新增对象还需要一个构造函数，析构函数负责引用计数减少和释放内存。通过覆写赋值运算符，才能将一个旧的智能指针赋值给另一个指针，同时旧的引用计数减1，新的引用计数加1

2)一个构造函数、拷贝构造函数、复制构造函数、析构函数、移走函数；

146.内存对齐？位域？

- 1、 分配内存的顺序是按照声明的顺序。
- 2、 每个变量相对于起始位置的偏移量必须是该变量类型大小的整数倍，不是整数倍空出内存，直到偏移量是整数倍为止。
- 3、 最后整个结构体的大小必须是里面变量类型最大值的整数倍。

添加了`#pragma pack(n)`后规则就变成了下面这样：

- 1、 偏移量要是n和当前变量大小中较小值的整数倍
- 2、 整体大小要是n和最大变量大小中较小值的整数倍
- 3、 n值必须为1,2,4,8...，为其他值时就按照默认的分配规则

147.结构体变量比较是否相等

1)重载了“==”操作符

```
struct foo {
    int a;
    int b;
    bool operator(const foo& rhs) // 操作运算符重载
    {
        return( a == rhs.a) && (b == rhs.b);
    }
};
```

2)元素的话，一个个比；

3)指针直接比较，如果保存的是同一个实例地址，则(`p1==p2`)为真；

148.位运算

若一个数m满足 $m = 2^n$ ，那么 $k \% m = k \& (m-1)$

149.为什么内存对齐

1、平台原因(移植原因)

- 1)不是所有的硬件平台都能访问任意地址上的任意数据的；
- 2)某些硬件平台只能在某些地址处取某些特定类型的数据，否则抛出硬件异常

2、性能原因：

- 1)数据结构(尤其是栈)应该尽可能地在自然边界上对齐。
- 2)原因在于，为了访问未对齐的内存，处理器需要作两次内存访问；而对齐的内存访问仅需要一次访问。

150.函数调用过程栈的变化，返回值和参数变量哪个先入栈？

- 1、调用者函数把被调函数所需要的参数按照与被调函数的形参顺序相反的顺序压入栈中,即:从右向左依次把被调函数所需要的参数压入栈;
- 2、调用者函数使用call指令调用被调函数,并把call指令的下一条指令的地址当成返回地址压入栈中(这个压栈操作隐含在call指令中);
- 3、在被调函数中,被调函数会先保存调用者函数的栈底地址(push ebp),然后再保存调用者函数的栈顶地址,即:当前被调函数的栈底地址(mov ebp,esp);
- 4、在被调函数中,从ebp的位置处开始存放被调函数中的局部变量和临时变量,并且这些变量的地址按照定义时的顺序依次减小,即:这些变量的地址是按照栈的延伸方向排列的,先定义的变量先入栈,后定义的变量后入栈;

151.怎样判断两个浮点数是否相等？

对两个浮点数判断大小和是否相等不能直接用==来判断，会出错！明明相等的两个数比较反而是不相等！对于两个浮点数比较只能通过相减并与预先设定的精度比较，记得要取绝对值！浮点数与0的比较也应该注意。与浮点数的表示方式有关。

152.宏定义一个取两个数中较大值的功能

```
#define MAX ( x,y ) ((x>y)?x:y)
```

153.define、const、typedef、inline使用方法？

一、const与#define的区别：

- 1)const定义的常量是变量带类型，而#define定义的只是个常数不带类型；
- 2)define只在预处理阶段起作用，简单的文本替换，而const在编译、链接过程中起作用；
- 3)define只是简单的字符串替换没有类型检查。而const是有数据类型的，是要进行判断的，可以避免一些低级错误；
- 4)define预处理后，占用代码段空间，const占用数据段空间；
- 5)const不能重定义，而define可以通过#undef取消某个符号的定义，进行重定义；
- 6)define独特功能，比如可以用来防止文件重复引用。

二、#define和别名typedef的区别

- 1)执行时间不同，typedef在编译阶段有效，typedef有类型检查的功能；#define是宏定义，发生在预处理阶段，不进行类型检查；
- 2)功能差异，typedef用来定义类型的别名，定义与平台无关的数据类型，与struct的结合使用等。
#define不只是可以为类型取别名，还可以定义常量、变量、编译开关等。

3)作用域不同，#define没有作用域的限制，只要是之前预定义过的宏，在以后的程序中都可以使用。而typedef有自己的作用域。

三、define与inline的区别

- 1)#define是关键字，inline是函数；

2)宏定义在预处理阶段进行文本替换，inline函数在编译阶段进行替换；

3)inline函数有类型检查，相比宏定义比较安全；

154.printf实现原理？

在C/C++中，对函数参数的扫描是从后向前的。C/C++的函数参数是通过压入堆栈的方式来给函数传参数的（堆栈是一种先进后出的数据结构），最先压入的参数最后出来，在计算机的内存中，数据有2块，一块是堆，一块是栈（函数参数及局部变量在这里），而栈是从内存的高地址向低地址生长的，控制生长的就是堆栈指针了，最先压入的参数是在最上面，就是说在所有参数的最后面，最后压入的参数在最下面，结构上看起来是第一个，所以最后压入的参数总是能够被函数找到，因为它就在堆栈指针的上方。printf的第一个被找到的参数就是那个字符指针，就是被双引号括起来的那一部分，函数通过判断字符串里控制参数的个数来判断参数个数及数据类型，通过这些就可算出数据需要的堆栈指针的偏移量了，下面给出printf("%d,%d",a,b);（其中a、b都是int型的）的汇编代码。

155.#include 的顺序以及尖括号和双引号的区别

表示编译器只在系统默认目录或尖括号内的工作目录下搜索头文件，并不去用户的工作目录下寻找，所以一般尖括号用于包含标准库文件；

表示编译器先在用户的工作目录下搜索头文件，如果搜索不到则到系统默认目录下去寻找，所以双引号一般用于包含用户自己编写的头文件。

156.lambda函数

1)利用lambda表达式可以编写内嵌的匿名函数，用以替换独立函数或者函数对象；

2)每当你定义一个lambda表达式后，编译器会自动生成一个匿名类（这个类当然重载了()运算符），我们称为闭包类型（closure type）。那么在运行时，这个lambda表达式就会返回一个匿名的闭包实例，其实一个右值。所以，我们上面的lambda表达式的结果就是一个个闭包。闭包的一个强大之处是其可以通过传值或者引用的方式捕捉其封装作用域内的变量，前面的方括号就是用来定义捕捉模式以及变量，我们又将其称为lambda捕捉块。

3)lambda表达式的语法定义如下：

[capture] (parameters) mutable ->return-type {statement};

4)lambda必须使用尾置返回来指定返回类型，可以忽略参数列表和返回值，但必须永远包含捕获列表和函数体；

157.hello world 程序开始到打印到屏幕上的全过程？

1.用户告诉操作系统执行HelloWorld程序（通过键盘输入等）

2．操作系统：找到helloworld程序的相关信息，检查其类型是否是可执行文件；并通过程序首部信息，确定代码和数据在可执行文件中的位置并计算出对应的磁盘块地址。

3．操作系统：创建一个新进程，将HelloWorld可执行文件映射到该进程结构，表示由该进程执行helloworld程序。

4．操作系统：为helloworld程序设置cpu上下文环境，并跳到程序开始处。

5．执行helloworld程序的第一条指令，发生缺页异常

6．操作系统：分配一页物理内存，并将代码从磁盘读入内存，然后继续执行helloworld程序

7．helloworld程序执行puts函数（系统调用），在显示器上写一字符串

8．操作系统：找到要将字符串送往的显示设备，通常设备是由一个进程控制的，所以，操作系统将要写的字符串送给该进程

9．操作系统：控制设备的进程告诉设备的窗口系统，它要显示该字符串，窗口系统确定这是一个合法的操作，然后将字符串转换成像素，将像素写入设备的存储映像区

10．视频硬件将像素转换成显示器可接收和一组控制数据信号

11. 显示器解释信号，激发液晶屏

12. OK，我们在屏幕上看到了HelloWorld

158.模板类和模板函数的区别是什么？

函数模板的实例化是由编译程序在处理函数调用时自动完成的，而类模板的实例化必须由程序员在程序中显式地指定。即函数模板允许隐式调用和显式调用而类模板只能显示调用。在使用时类模板必须加，而函数模板不必

159.为什么模板类一般都是放在一个h文件中

1)模板定义很特殊。由template<...>处理的任何东西都意味着编译器在当时不为它分配存储空间，它一直处于等待状态直到被一个模板实例告知。在编译器和连接器的某一处，有一机制能去掉指定模板的多重定义。所以为了容易使用，几乎总是在头文件中放置全部的模板声明和定义。

2)在分离式编译的环境下，编译器编译某一个.cpp文件时并不知道另一个.cpp文件的存在，也不会去查找（当遇到未决符号时它会寄希望于连接器）。这种模式在没有模板的情况下运行良好，但遇到模板时就傻眼了，因为模板仅在需要的时候才会实例化出来，所以，当编译器只看到模板的声明时，它不能实例化该模板，只能创建一个具有外部连接的符号并期待连接器能够将符号的地址决议出来。然而当实现该模板的.cpp文件中没有用到模板的实例时，编译器懒得去实例化，所以，整个工程的.obj中就找不到一行模板实例的二进制代码，于是连接器也黔驴技穷了。

160.C++中类成员的访问权限和继承权限问题。

1)三种访问权限

①public:用该关键字修饰的成员表示公有成员，该成员不仅可以在类内可以被访问，在类外也是可以访问的，是类对外提供的可访问接口；

② private:用该关键字修饰的成员表示私有成员，该成员仅在类内可以被访问，在类体外是隐藏状态；

③ protected:用该关键字修饰的成员表示保护成员，保护成员在类体外同样是隐藏状态，但是对于该类的派生类来说，相当于公有成员，在派生类中可以被访问。

2)三种继承方式

①若继承方式是public，基类成员在派生类中的访问权限保持不变，也就是说，基类中的成员访问权限，在派生类中仍然保持原来的访问权限；

② 若继承方式是private，基类所有成员在派生类中的访问权限都会变为私有(private)权限；

③若继承方式是protected，基类的共有成员和保护成员在派生类中的访问权限都会变为保护(protected)权限，私有成员在派生类中的访问权限仍然是私有(private)权限。

161.cout和printf有什么区别？

cout<<是一个函数，cout<<后可以跟不同的类型是因为cout<<已存在针对各种类型数据的重载，所以会自动识别数据的类型。输出过程会首先将输出字符放入缓冲区，然后输出到屏幕。

cout是有缓冲输出：

```
cout << "abc" << endl;
```

或cout << "abc\n";cout << flush; 这两个才是一样的。

endl相当于输出回车后，再强迫缓冲输出。

flush立即强迫缓冲输出。

printf是无缓冲输出。有输出时立即输出

162.重载运算符？

- 1、我们只能重载已有的运算符，而无权发明新的运算符；对于一个重载的运算符，其优先级和结合律与内置类型一致才可以；不能改变运算符操作数个数；
- 2、`sizeof typeid` *不能重载；
- 3、两种重载方式，成员运算符和非成员运算符，成员运算符比非成员运算符少一个参数；下标运算符、箭头运算符必须是成员运算符；
- 4、引入运算符重载，是为了实现类的多态性；
- 5、当重载的运算符是成员函数时，`this`绑定到左侧运算符对象。成员运算符函数的参数数量比运算符对象的数量少一个；至少含有一个类类型的参数；
- 6、从参数的个数推断到底定义的是哪种运算符，当运算符既是一元运算符又是二元运算符（`+`，`-`，`,`，`&`）；
- 7、下标运算符必须是成员函数，下标运算符通常以所访问元素的引用作为返回值，同时最好定义下标运算符的常量版本和非常量版本；
- 8、箭头运算符必须是类的成员，解引用通常也是类的成员；重载的箭头运算符必须返回类的指针；

163.函数重载函数匹配原则

- 1)名字查找
- 2)确定候选函数
- 3)寻找最佳匹配

164.定义和声明的区别

- 1.如果是指变量的声明和定义

从编译原理上来说，声明是仅仅告诉编译器，有个某类型的变量会被使用，但是编译器并不会为它分配任何内存。而定义就是分配了内存。

- 2.如果是指函数的声明和定义

声明：一般在头文件里，对编译器说：这里我有一个函数叫`function()` 让编译器知道这个函数的存在。

定义：一般在源文件里，具体就是函数的实现过程 写明函数体。

165.C++类型转换有四种

- 1)`static_cast`能进行基础类型之间的转换，也是最常看到的类型转换。它主要有如下几种用法：

- 1.用于类层次结构中父类和子类之间指针或引用的转换。进行上行转换（把子类的指针或引用转换成父类表示）是安全的；
- 2.进行下行转换（把父类指针或引用转换成子类指针或引用）时，由于没有动态类型检查，所以是不安全的；
- 3.用于基本数据类型之间的转换，如把`int`转换成`char`，把`int`转换成`enum`。这种转换的安全性也要开发人员来保证。
- 4.把`void`指针转换成目标类型的指针（不安全！！）
- 5.把任何类型的表达式转换成`void`类型。

2)`const_cast`运算符用来修改类型的`const`或`volatile`属性。除了去掉`const`或`volatile`修饰之外，`type_id`和`expression`得到的类型是一样的。但需要特别注意的是`const_cast`不是用于去除变量的常量性，而是去除指向常数对象的指针或引用的常量性，其去除常量性的对象必须为指针或引用。

3)`reinterpret_cast`它可以把一个指针转换成一个整数，也可以把一个整数转换成一个指针（先把一个指针转换成一个整数，在把该整数转换成原类型的指针，还可以得到原先的指针值）。

4)`dynamic_cast` 主要用在继承体系中的安全向下转型。它能安全地将指向基类的指针转型为指向子类的指针或引用，并获知转型动作成功与否。转型失败会返回`null`（转型对象为指针时）或抛出异常`bad_cast`（转型对象为引用时）。`dynamic_cast` 会动用运行时信息（RTTI）来进行类型安全检查，因此`dynamic_cast` 存在一定的效率损失。当使用`dynamic_cast`时，该类型必须含有虚函数，这是因为`dynamic_cast`使用了存储在VTABLE中的信息来判断实际的类型，RTTI运行时类型识别用于判断类型。

typeid表达式的形式是typeid(e)，typeid操作的结果是一个常量对象的引用，该对象的类型是type_info或type_info的派生。

166.全局变量和static变量的区别

1、全局变量（外部变量）的说明之前再冠以static就构成了静态的全局变量。全局变量本身就是静态存储方式，静态全局变量当然也是静态存储方式。

这两者在存储方式上并无不同。这两者的区别在于非静态全局变量的作用域是整个源程序，当一个源程序由多个原文件组成时，非静态的全局变量在各个源文件中都是有效的。而静态全局变量则限制了其作用域，即只在定义该变量的源文件内有效，在同一源程序的其它源文件中不能使用它。由于静态全局变量的作用域限于一个源文件内，只能为该源文件内的函数公用，因此可以避免在其他源文件中引起错误。static全局变量与普通的全局变量的区别是static全局变量只初始化一次，防止在其他文件单元被引用。

2.static函数与普通函数有什么区别？

static函数与普通的函数作用域不同。尽在本文件中。只在当前源文件中使用的函数应该说明为内部函数（static），内部函数应该在当前源文件中说明和定义。对于可在当前源文件以外使用的函数应该在一个头文件中说明，要使用这些函数的源文件要包含这个头文件。

static函数与普通函数最主要区别是static函数在内存中只有一份，普通静态函数在每个被调用中维持一份拷贝程序的局部变量存在于（堆栈）中，全局变量存在于（静态区）中，动态申请数据存在于（堆）

167.静态成员与普通成员的区别

1)生命周期

静态成员变量从类被加载开始到类被卸载，一直存在；

普通成员变量只有在类创建对象后才开始存在，对象结束，它的生命期结束；

2)共享方式

静态成员变量是全类共享；普通成员变量是每个对象单独享用的；

3)定义位置

普通成员变量存储在栈或堆中，而静态成员变量存储在静态全局区；

4)初始化位置

普通成员变量在类中初始化；静态成员变量在类外初始化；

5)默认实参

可以使用静态成员变量作为默认实参，

168.说一下理解 ifdef endif

1)一般情况下，源程序中所有的行都参加编译。但是有时希望对其中一部分内容只在满足一定条件才进行编译，也就是对一部分内容指定编译的条件，这就是“条件编译”。有时，希望当满足某条件时对一组语句进行编译，而当条件不满足时则编译另一组语句。

2)条件编译命令最常见的形式为：

#ifdef 标识符

程序段1

#else

程序段2

#endif

它的作用是：当标识符已经被定义过(一般是用#define命令定义)，则对程序段1进行编译，否则编译程序段2。

其中#else部分也可以没有，即：

#ifdef

程序段1

#endif

3)在一个大的软件工程里面，可能会有多个文件同时包含一个头文件，当这些文件编译链接成一个可执行文件上时，就会出现大量“重定义”错误。在头文件中使用#define、#ifndef、#ifdef、#endif能避免头文件重定义。

169.隐式转换，如何消除隐式转换？

1.C++的基本类型中并非完全的对立，部分数据类型之间是可以进行隐式转换的。所谓隐式转换，是指不需要用户干预，编译器私下进行的类型转换行为。很多时候用户可能都不知道进行了哪些转换

2. C++面向对象的多态特性，就是通过父类的类型实现对子类的封装。通过隐式转换，你可以直接将一个子类的对象使用父类的类型进行返回。在比如，数值和布尔类型的转换，整数和浮点数的转换等。某些方面来说，隐式转换给C++程序开发者带来了不小的便捷。C++是一门强类型语言，类型的检查是非常严格的。

3.基本数据类型 基本数据类型的转换以取值范围的作为转换基础（保证精度不丢失）。隐式转换发生在从小->大的转换中。比如从char转换为int。从int->long。自定义对象 子类对象可以隐式的转换为父类对象。

4.C++中提供了explicit关键字，在构造函数声明的时候加上explicit关键字，能够禁止隐式转换。

5.如果构造函数只接受一个参数，则它实际上定义了转换为此类类型的隐式转换机制。可以通过将构造函数声明为explicit加以制止隐式类型转换，关键字explicit只对一个实参的构造函数有效，需要多个实参的构造函数不能用于执行隐式转换，所以无需将这些构造函数指定为explicit。

170.虚函数的内存结构，那菱形继承的虚函数内存结构呢

171.多继承的优缺点，作为一个开发者怎么看待多继承

1)C++允许为一个派生类指定多个基类，这样的继承结构被称做多重继承。

2)多重继承的优点很明显，就是对象可以调用多个基类中的接口；

3)如果派生类所继承的多个基类有相同的基类，而派生类对象需要调用这个祖先类的接口方法，就会容易出现二义性

4)加上全局符确定调用哪一份拷贝。比如pa.Author::eat()调用属于Author的拷贝。

5)使用虚拟继承，使得多重继承类Programmer_Author只拥有Person类的一份拷贝。

172.迭代器++it,it++哪个好，为什么

1)前置返回一个引用，后置返回一个对象

// ++i实现代码为：

```
int& operator++()
{
    *this += 1;
    return *this;
}
```

2)前置不会产生临时对象，后置必须产生临时对象，临时对象会导致效率降低

//i++实现代码为：

```
int operator++(int)
{
    int temp = *this;
    ++*this;
    return temp;
}
```

101.C++如何处理多个异常的？

1)C++中的异常情况：

语法错误（编译错误）：比如变量未定义、括号不匹配、关键字拼写错误等等编译器在编译时能发现的错误，这类错误可以及时被编译器发现，而且可以及时知道出错的位置及原因，方便改正。

运行时错误：比如数组下标越界、系统内存不足等等。这类错误不易被程序员发现，它能够通过编译且能进入运行，但运行时会出错，导致程序崩溃。为了有效处理程序运行时错误，C++中引入异常处理机制来解决此问题。

2)C++异常处理机制：

异常处理基本思想：执行一个函数的过程中发现异常，可以不用在本函数内立即进行处理，而是抛出该异常，让函数的调用者直接或间接处理这个问题。

C++异常处理机制由3个模块组成：try(检查)、throw(抛出)、catch(捕获)

抛出异常的语句格式为：throw 表达式；如果try块中程序段发现了异常则抛出异常。

```
try
{
    可能抛出异常的语句；（检查）
}
catch（类型名[形参名]）//捕获特定类型的异常
{
    //处理1；
}
catch（类型名[形参名]）//捕获特定类型的异常
{
    //处理2；
}
catch（...）//捕获所有类型的异常
{
}
```

173.模板和实现可不可以不写在一个文件里面？为什么？

因为在编译时模板并不能生成真正的二进制代码，而是在编译调用模板类或函数的CPP文件时才会去找对应的模板声明和实现，在这种情况下编译器是不知道实现模板类或函数的CPP文件的存在，所以它只能找到模板类或函数的声明而找不到实现，而只好创建一个符号寄希望于链接程序找地址。但模板类或函数的实现并不能被编译成二进制代码，结果链接程序找不到地址只好报错了。

《C++编程思想》第15章(第300页)说明了原因：模板定义很特殊。由template<...>处理的任何东西都意味着编译器在当时不为它分配存储空间，它一直处于等待状态直到被一个模板实例告知。在编译器和连接器的某一处，有一机制能去掉指定模板的多重定义。所以为了容易使用，几乎总是在头文件中放置全部的模板声明和定义。

174.在成员函数中调用delete this会出现什么问题？对象还可以使用吗？

1.在类对象的内存空间中，只有数据成员和虚函数表指针，并不包含代码内容，类的成员函数单独放在代码段中。在调用成员函数时，隐含传递一个this指针，让成员函数知道当前是哪个对象在调用它。当调用delete this时，类对象的内存空间被释放。在delete this之后进行的其他任何函数调用，只要不涉及到this指针的内容，都能够正常运行。一旦涉及到this指针，如操作数据成员，调用虚函数等，就会出现不可预期的问题。

2.为什么是不可预期的问题？

delete this之后不是释放了类对象的内存空间了么，那么这段内存应该已经还给系统，不再属于这个进程。照这个逻辑来看，应该发生指针错误，无访问权限之类的令系统崩溃的问题才对啊？这个问题牵涉

到操作系统的内存管理策略。delete this释放了类对象的内存空间，但是内存空间却并不是马上被回收到系统中，可能是缓冲或者其他什么原因，导致这段内存空间暂时并没有被系统收回。此时这段内存是可以访问的，你可以加上100，加上200，但是其中的值却是不确定的。当你获取数据成员，可能得到的是串很长的未初始化的随机数；访问虚函数表，指针无效的可能性非常高，造成系统崩溃。

3.如果在类的析构函数中调用delete this，会发生什么？

会导致堆栈溢出。原因很简单，delete的本质是“为将被释放的内存调用一个或多个析构函数，然后，释放内存”。显然，delete this会去调用本对象的析构函数，而析构函数中又调用delete this，形成无限递归，造成堆栈溢出，系统崩溃。

175.智能指针的作用；

1)C++11中引入了智能指针的概念，方便管理堆内存。使用普通指针，容易造成堆内存泄露（忘记释放），二次释放，程序发生异常时内存泄露等问题等，使用智能指针能更好的管理堆内存。

2)智能指针在C++11版本之后提供，包含在头文件中，shared_ptr、unique_ptr、weak_ptr。

shared_ptr多个指针指向相同的对象。shared_ptr使用引用计数，每一个shared_ptr的拷贝都指向相同的内存。每使用他一次，内部的引用计数加1，每析构一次，内部的引用计数减1，减为0时，自动删除所指向的堆内存。shared_ptr内部的引用计数是线程安全的，但是对象的读取需要加锁。

3)初始化。智能指针是个模板类，可以指定类型，传入指针通过构造函数初始化。也可以使用make_shared函数初始化。不能将指针直接赋值给一个智能指针，一个是类，一个是指针。例如std::shared_ptr p4 = new int(1);的写法是错误的

拷贝和赋值。拷贝使得对象的引用计数增加1，赋值使得原对象引用计数减1，当计数为0时，自动释放内存。后来指向的对象引用计数加1，指向后来的对象

4)unique_ptr“唯一”拥有其所指对象，同一时刻只能有一个unique_ptr指向给定对象（通过禁止拷贝语义、只有移动语义来实现）。相比与原始指针unique_ptr用于其RAII的特性，使得在出现异常的情况下，动态资源能得到释放。unique_ptr指针本身的生命周期：从unique_ptr指针创建时开始，直到离开作用域。离开作用域时，若其指向对象，则将其所指对象销毁(默认使用delete操作符，用户可指定其他操作)。unique_ptr指针与其所指对象的关系：在智能指针生命周期内，可以改变智能指针所指对象，如创建智能指针时通过构造函数指定、通过reset方法重新指定、通过release方法释放所有权、通过移动语义转移所有权。

5)智能指针类将一个计数器与类指向的对象相关联，引用计数跟踪该类有多少个对象共享同一指针。每次创建类的新对象时，初始化指针并将引用计数置为1；当对象作为另一对象的副本而创建时，拷贝构造函数拷贝指针并增加与之相应的引用计数；对一个对象进行赋值时，赋值操作符减少左操作数所指对象的引用计数（如果引用计数为减至0，则删除对象），并增加右操作数所指对象的引用计数；调用析构函数时，构造函数减少引用计数（如果引用计数减至0，则删除基础对象）。

6)weak_ptr 是一种不控制对象生命周期的智能指针，它指向一个 shared_ptr 管理的对象。进行该对象的内存管理的是那个强引用的 shared_ptr。weak_ptr只是提供了对管理对象的一个访问手段。weak_ptr 设计的目的是为配合 shared_ptr 而引入的一种智能指针来协助 shared_ptr 工作，它只可以从一个 shared_ptr 或另一个 weak_ptr 对象构造，它的构造和析构不会引起引用计数的增加或减少。

176.auto_ptr作用

1)auto_ptr的出现，主要是为了解决“有异常抛出时发生内存泄漏”的问题；抛出异常，将导致指针p所指向的空间得不到释放而导致内存泄漏；

2)auto_ptr构造时取得某个对象的控制权，在析构时释放该对象。我们实际上是创建一个auto_ptr类型的局部对象，该局部对象析构时，会将自身所拥有的指针空间释放，所以不会有内存泄漏；

3)auto_ptr的构造函数是explicit，阻止了一般指针隐式转换为 auto_ptr的构造，所以不能直接将一般类型的指针赋值给auto_ptr类型的对象，必须用auto_ptr的构造函数创建对象；

4)由于auto_ptr对象析构时会删除它所拥有的指针，所以使用时避免多个auto_ptr对象管理同一个指针；

5)Auto_ptr内部实现，析构函数中删除对象用的是delete而不是delete[]，所以auto_ptr不能管理数组；

6)auto_ptr支持所拥有的指针类型之间的隐式类型转换。

7)可以通过和->运算符对auto_ptr所有用的指针进行提领操作；

8)T get(),获得auto_ptr所拥有的指针；T* release(), 释放auto_ptr的所有权，并将所有用的指针返回。

177.class、union、struct的区别

1)C语言中，struct只是一个聚合数据类型，没有权限设置，无法添加成员函数，无法实现面向对象编程，且如果没有typedef结构名，声明结构变量必须添加关键字struct。

2)C++中，struct功能大大扩展，可以有权限设置（默认权限为public），可以像class一样有成员函数，继承（默认public继承），可以实现面对对象编程，允许在声明结构变量时省略关键字struct。

3)C与C++中的union:一种数据格式，能够存储不同的数据类型，但只能同时存储其中的一种类型。C++ union结构式一种特殊的类。它能够包含访问权限、成员变量、成员函数（可以包含构造函数和析构函数）。它不能包含虚函数和静态数据变量。它也不能被用作其他类的基类，它本身也不能有从某个基类派生而来。Union中得默认访问权限是public。union类型是共享内存的，以size最大的结构作为自己的大小。每个数据成员在内存中的起始地址是相同的。

4)在C/C++程序的编写中，当多个基本数据类型或复合数据结构要占用同一片内存时，我们要使用联合体；当多种类型，多个对象，多个事物只取其一（我们姑且通俗地称其为“n 选1”），我们也可以使用联合体来发挥其长处。在某一时刻，一个union中只能有一个值是有效的。union的一个用法就是可以用来测试CPU是大端模式还是小端模式：

178.动态联编与静态联编

1)在C++中，联编是指一个计算机程序的不同部分彼此关联的过程。按照联编所进行的阶段不同，可以分为静态联编和动态联编；

2)静态联编是指联编工作在编译阶段完成的，这种联编过程是在程序运行之前完成的，又称为早期联编。要实现静态联编，在编译阶段就必须确定程序中的操作调用（如函数调用）与执行该操作代码间的关系，确定这种关系称为束定，在编译时的束定称为静态束定。静态联编对函数的选择是基于指向对象的指针或者引用的类型。其优点是效率高，但灵活性差。

3)动态联编是指联编在程序运行时动态地进行，根据当时的情况来确定调用哪个同名函数，实际上是在运行时虚函数的实现。这种联编又称为晚期联编，或动态束定。动态联编对成员函数的选择是基于对象的类型，针对不同的对象类型将做出不同的编译结果。C++中一般情况下的联编是静态联编，但是当涉及到多态性和虚函数时应该使用动态联编。动态联编的优点是灵活性强，但效率低。动态联编规定，只能通过指向基类的指针或基类对象的引用来调用虚函数，其格式为：指向基类的指针变量名->虚函数名（实参表）或基类对象的引用名.虚函数名（实参表）

4)实现动态联编三个条件：

必须把动态联编的行为定义为类的虚函数；

类之间应满足子类型关系，通常表现为一个类从另一个类公有派生而来；

必须先使用基类指针指向子类型的对象，然后直接或间接使用基类指针调用虚函数；

179.动态编译与静态编译

1)静态编译，编译器在编译可执行文件时，把需要用到的对应动态链接库中的部分提取出来，连接到可执行文件中，使可执行文件在运行时不需要依赖于动态链接库；

2)动态编译的可执行文件需要附带一个动态链接库，在执行时，需要调用其对应动态链接库的命令。所以其优点一方面是缩小了执行文件本身的体积，另一方面是加快了编译速度，节省了系统资源。缺点是哪怕是很简单的程序，只用到了链接库的一两条命令，也需要附带一个相对庞大的链接库；二是如果其他计算机上没有安装对应的运行库，则用动态编译的可执行文件就不能运行。

180.动态链接和静态链接区别

1)静态链接库就是把(lib)文件中用到的函数代码直接链接进目标程序，程序运行的时候不再需要其它的库文件；动态链接就是把调用的函数所在文件模块（DLL）和调用函数在文件中的位置等信息链接进目标程序，程序运行的时候再从DLL中寻找相应函数代码，因此需要相应DLL文件的支持。

2)静态链接库与动态链接库都是共享代码的方式，如果采用静态链接库，则无论你愿不愿意，lib 中的指令都全部被直接包含在最终生成的 EXE 文件中了。但是若使用 DLL，该 DLL 不必被包含在最终 EXE 文件中，EXE 文件执行时可以“动态”地引用和卸载这个与 EXE 独立的 DLL 文件。静态链接库和动态链接库的另外一个区别在于静态链接库中不能再包含其他的动态链接库或者静态库，而在动态链接库中还可以再包含其他的动态或静态链接库。

3)动态库就是在需要调用其中的函数时，根据函数映射表找到该函数然后调入堆栈执行。如果在当前工程中有多处对dll文件中同一个函数的调用，那么执行时，这个函数只会留下一份拷贝。但是如果有多处对lib文件中同一个函数的调用，那么执行时，该函数将在当前程序的执行空间里留下多份拷贝，而且是一处调用就产生一份拷贝。

181.在不使用额外空间的情况下，交换两个数？

1)算术

```
x = x + y;
```

```
y = x - y;
```

```
x = x - y;
```

2)异或

```
x = x^y;// 只能对int,char...
```

```
y = x^y;
```

```
x = x^y;
```

```
x ^= y ^= x;
```

182.strcpy和memcpy的区别

1、复制的内容不同。strcpy只能复制字符串，而memcpy可以复制任意内容，例如字符数组、整型、结构体、类等。

2、复制的方法不同。strcpy不需要指定长度，它遇到被复制字符串的串结束符“\0”才结束，所以容易溢出。memcpy则是根据其第3个参数决定复制的长度。

3、用途不同。通常在复制字符串时用strcpy，而需要复制其他类型数据时则一般用memcpy

183.执行int main(int argc, char *argv[])时的内存结构

参数的含义是程序在命令行下运行的时候，需要输入argc 个参数，每个参数是以char 类型输入的，依次存在数组里面，数组是 argv[]，所有的参数在指针

char * 指向的内存中，数组的中元素的个数为 argc 个，第一个参数为程序的名称。

184.volatile关键字的作用？

volatile 关键字是一种类型修饰符，用它声明的类型变量表示可以被某些编译器未知的因素更改，比如：操作系统、硬件或者其它线程等。遇到这个关键字声明的变量，编译器对访问该变量的代码就不再进行优化，从而可以提供对特殊地址的稳定访问。声明时语法：int volatile vInt; 当要求使用 volatile 声明的变量的值的时候，系统总是重新从它所在的内存读取数据，即使它前面的指令刚刚从该处读取过数据。而且读取的数据立刻被保存。

volatile用在如下的几个地方：

中断服务程序中修改的供其它程序检测的变量需要加volatile；

多任务环境下各任务间共享的标志应该加volatile；

存储器映射的硬件寄存器通常也要加volatile说明，因为每次对它的读写都可能由不同意义；

185.讲讲大端小端，如何检测（三种方法）

大端模式：是指数据的高字节保存在内存的低地址中，而数据的低字节保存在内存的高地址端。

小端模式，是指数据的高字节保存在内存的高地址中，低位字节保存在内存的低地址端。

1)直接读取存放在内存中的十六进制数值，取低位进行值判断

```
int a = 0x12345678;
```

```
int *c = &a;
```

```
c[0] == 0x12 大端模式
```

```
c[0] == 0x78 小端模式
```

2)用共同体来进行判断

union共同体所有数据成员是共享一段内存的，后写入的成员数据将覆盖之前的成员数据，成员数据都有相同的首地址。Union的大小为最大数据成员的大小。

union的成员数据共用内存，并且首地址都是低地址首字节。Int i = 1时：大端存储1放在最高位，小端存储1放在最低位。当读取char ch时，是最低地址首字节，大小端会显示不同的值。

```
union w w p;
```

```
{ p.i = 1;
```

```
int i; if(ch == 1)
```

```
char ch;
```

```
};
```

186.查看内存的方法

1.首先打开vs编译器，创建好项目，并且将代码写进去，这里就不贴代码了，你可以随便的写个做个测试;

2.调试的时候做好相应的断点，然后点击开始调试;

3.程序调试之后会在你设置断点的地方暂停，然后选择调试->窗口->内存，就打开了内存数据查看的窗口了。

187.空类会默认添加哪些东西？怎么写？

1)Empty(); // 缺省构造函数//

2)Empty(const Empty&); // 拷贝构造函数//

3)~Empty(); // 析构函数//

4)Empty& operator=(const Empty&); // 赋值运算符//

188.标准库是什么？

1)C++ 标准库可以分为两部分：

标准函数库：这个库是由通用的、独立的、不属于任何类的函数组成的。函数库继承自 C 语言。

面向对象类库：这个库是类及其相关函数的集合。

2)输入/输出 I/O、字符串和字符处理、数学、时间、日期和本地化、动态分配、其他、宽字符函数

3)标准的 C++ I/O 类、String 类、数值类、STL 容器类、STL 算法、STL 函数对象、STL 迭代器、STL 分配器、本地化库、异常处理类、杂项支持库

118.const char* 与string之间的关系，传递参数问题？

1)string 是c++标准库里面其中一个，封装了对字符串的操作，实际操作过程我们可以用const char*给string类初始化

2)三者的转化关系如下所示：

```
a)string转const char*  
string s = "abc";  
const char* c_s = s.c_str();
```

```

b) const char* 转 string, 直接赋值即可
const char* c_s = "abc";
string s(c_s);
c) string 转 char*
string s = "abc";
char* c;
const int len = s.length();
c = new char[len+1];
strcpy(c, s.c_str());
d) char* 转 string
char* c = "abc";
string s(c);
e) const char* 转 char*
const char* cpc = "abc";
char* pc = new char[strlen(cpc)+1];
strcpy(pc, cpc);
f) char* 转 const char*, 直接赋值即可
char* pc = "abc";
const char* cpc = pc;

```

189.new、delete、operator new、operator delete、placement new、placement delete

1)new operator

new operator 完成了两件事情：用于申请内存和初始化对象。

例如：string* ps = new string("abc");

2)operator new

operator new 类似于C语言中的malloc，只是负责申请内存。

例如：void* buffer = operator new(sizeof(string)); 注意这里new前要有个operator。

3)placement new

用于在给定的内存中初始化对象。

例如：void* buffer = operator new(sizeof(string)); buffer = new(buffer) string("abc"); 调用了 placement new，在buffer所指向的内存中创建了一个string类型的对象并且初始值为"abc"。

4)因此可以看出：new operator 可以分解operator new 和 placement new两个动作，是 operator new 和 placement new 的结合。与new对应的delete没有 placement delete 语法，它只有两种，分别是delete operator 和 operator delete。delete operator 和 new operator 对应，完成析构对象和释放内存的操作。而 operator delete 只是用于内存的释放，与C语言中的free相似。

190.为什么拷贝构造函数必须传引用不能传值？

拷贝构造函数的作用就是用来复制对象的，在使用这个对象的实例来初始化这个对象的一个新的实例。

参数传递过程到底发生了什么？

将地址传递和值传递统一起来，归根结底还是传递的是"值"(地址也是值，只不过通过它可以找到另一个值)！

i)值传递:

对于内置数据类型的传递时，直接赋值拷贝给形参(注意形参是函数内局部变量)；

对于类类型的传递时，需要首先调用该类的拷贝构造函数来初始化形参(局部对象)；如void

foo(class_type obj_local){}, 如果调用foo(obj); 首先class_type obj_local(obj), 这样就定义了局部变量 obj_local供函数内部使用

ii)引用传递:

无论对内置类型还是类类型，传递引用或指针最终都是传递的地址值！而地址总是指针类型(属于简单类型)，显然参数传递时，按简单类型的赋值拷贝，而不会有拷贝构造函数的调用(对于类类型)。

上述1) 2)回答了为什么拷贝构造函数使用值传递会产生无限递归调用，内存溢出。

拷贝构造函数用来初始化一个非引用类类型对象，如果用传值的方式进行传参数，那么构造实参需要调用拷贝构造函数，而拷贝构造函数需要传递实参，所以会一直递归。

191.空类的大小是多少？为什么？

1)C++空类的大小不为0，不同编译器设置不一样，vs设置为1；

2)C++标准指出，不允许一个对象（当然包括类对象）的大小为0，不同的对象不能具有相同的地址；

3)带有虚函数的C++类大小不为1，因为每一个对象会有一个vptr指向虚函数表，具体大小根据指针大小确定；

4)C++中要求对于类的每个实例都必须有独一无二的地址,那么编译器自动为空类分配一个字节大小，这样便保证了每个实例均有独一无二的内存地址。

192.你什么情况用指针当参数，什么时候用引用，为什么？

1)使用引用参数的主要原因有两个：

程序员能修改调用函数中的数据对象

通过传递引用而不是整个数据-对象，可以提高程序的运行速度

2)一般的原则：

对于使用引用的值而不做修改的函数：

如果数据对象很小，如内置数据类型或者小型结构，则按照值传递；

如果数据对象是数组，则使用指针（唯一的选择），并且指针声明为指向const的指针；

如果数据对象是较大的结构，则使用const指针或者引用，已提高程序的效率。这样可以节省结构所需的时间和空间；

如果数据对象是类对象，则使用const引用（传递类对象参数的标准方式是按照引用传递）；

3)对于修改函数中数据的函数：

如果数据是内置数据类型，则使用指针

如果数据对象是数组，则只能使用指针

如果数据对象是结构，则使用引用或者指针

如果数据是类对象，则使用引用

193.大内存申请时候选用哪种？C++变量存在哪？变量的大小存在哪？符号表存在哪？

1.大内存申请时，采用堆申请空间，用new申请；

2.不同的变量存储在不同的地方，局部变量、全局变量、静态变量；

3.C++对变量名不作存储，在汇编以后不会出现变量名，变量名作用只是用于方便编译成汇编代码，是给编译器看的，是方便人阅读的

194.为什么会有大端小端，htol这一类函数的作用

1)这是因为在计算机系统中，我们是以字节为单位的，每个地址单元都对应着一个字节，一个字节为8bit。但是在C语言中除了8bit的char之外，还有16bit的short型，32bit的long型（要看具体的编译器），另外，对于位数大于8位的处理器，例如16位或者32位的处理器，由于寄存器宽度大于一个字节，那么必然存在着一个如何将多个字节安排的问题。因此就导致了大端存储模式和小端存储模式。例如一个16bit的short型x，在内存中的地址为0x0010，x的值为0x1122，那么0x11为高字节，0x22为低字节。对于大端模式，就将0x11放在低地址中，即0x0010中，0x22放在高地址中，即0x0011中。小端模式，刚好相反。我们常用的X86结构是小端模式，而KEIL C51则为大端模式。很多的ARM，DSP都为小端模式。有些ARM处理器还可以由硬件来选择是大端模式还是小端模式。

195.静态函数能定义为虚函数吗？常函数？

1、static成员不属于任何类对象或类实例，所以即使给此函数加上virtual也是没有任何意义的。2、静态与非静态成员函数之间有一个主要的区别。那就是静态成员函数没有this指针。虚函数依靠vptr和vtable来处理。vptr是一个指针，在类的构造函数中创建生成，并且只能用this指针来访问它，因为它是类的一个成员，并且vptr指向保存虚函数地址的vtable.对于静态成员函数，它没有this指针，所以无法访问vptr. 这就是为何static函数不能为virtual.虚函数的调用关系：this -> vptr -> vtable -> virtual function

196.this指针调用成员变量时，堆栈会发生什么变化？

当在类的非静态成员函数访问类的非静态成员时，编译器会自动将对象的地址传给作为隐含参数传递给函数，这个隐含参数就是this指针。即使你并没有写this指针，编译器在链接时也会加上this的，对各成员的访问都是通过this的。例如你建立了类的多个对象时，在调用类的成员函数时，你并不知道具体是哪个对象在调用，此时你可以通过查看this指针来查看具体是哪个对象在调用。This指针首先入栈，然后成员函数的参数从右向左进行入栈，最后函数返回地址入栈。

197.静态绑定和动态绑定的介绍

- 1)对象的静态类型：对象在声明时采用的类型。是在编译期确定的。
- 2)对象的动态类型：目前所指对象的类型。是在运行期决定的。对象的动态类型可以更改，但是静态类型无法更改。
- 3)静态绑定：绑定的是对象的静态类型，某特性（比如函数）依赖于对象的静态类型，发生在编译期。
- 4)动态绑定：绑定的是对象的动态类型，某特性（比如函数）依赖于对象的动态类型，发生在运行期。

198.设计一个类计算子类的个数

- 1.为类设计一个static静态变量count作为计数器；
- 2.类定义结束后初始化count;
- 3.在构造函数中对count进行+1;
- 4.设计拷贝构造函数，在进行拷贝构造函数中进行count +1，操作；
- 5.设计复制构造函数，在进行复制函数中对count+1操作；
- 6.在析构函数中对count进行-1；

199.怎么快速定位错误出现的地方

- 1.如果是简单的错误，可以直接双击错误列表里的错误项或者生成输出的错误信息中带行号的地方就可以让编辑窗口定位到错误的位置上。
- 2.对于复杂的模板错误，最好使用生成输出窗口。多数情况下出发错误的位置是最靠后的引用位置。如果这样确定不了错误，就需要先把自己写的代码里的引用位置找出来，然后逐个分析了。

200.虚函数的代价？

- 1)带有虚函数的类，每一个类会产生一个虚函数表，用来存储指向虚成员函数的指针，增大类；
- 2)带有虚函数的类的每一个对象，都会有有一个指向虚表的指针，会增加对象的空间大小；
- 3)不能再是内联的函数，因为内联函数在编译阶段进行替代，而虚函数表示等待，在运行阶段才能确定到底是采用哪种函数，虚函数不能是内联函数。

201.类对象的大小

- 1)类的非静态成员变量大小，静态成员不占据类的空间，成员函数也不占据类的空间大小；
- 2)内存对齐另外分配的空间大小，类内的数据也是需要内存对齐操作的；
- 3)虚函数的话，会在类对象插入vptr指针，加上指针大小；
- 4)当该该是某类的派生类，那么派生类继承的基类部分的数据成员也会存在在派生类中的空间中，也会对派生类进行扩展。

202.移动构造函数

- 1)有时候我们会遇到这样一种情况，我们用对象a初始化对象b后对象a我们就不在使用了，但是对象a的空间还在呀（在析构之前），既然拷贝构造函数，实际上就是把a对象的内容复制一份到b中，那么为什么我们不能直接使用a的空间呢？这样就避免了新的空间的分配，大大降低了构造的成本。这就是移动构造函数设计的初衷；
- 2)拷贝构造函数中，对于指针，我们一定要采用深层复制，而移动构造函数中，对于指针，我们采用浅层复制；
- 3)C++引入了移动构造函数，专门处理这种，用a初始化b后，就将a析构的情况；
- 4)与拷贝类似，移动也使用一个对象的值设置另一个对象的值。但是，又与拷贝不同的是，移动实现的是对象值真实的转移（源对象到目的对象）：源对象将丢失其内容，其内容将被目的对象占有。移动操作发生的时候，是当移动值的对象是未命名的对象的时候。这里未命名的对象就是那些临时变量，甚至都不会有名称。典型的未命名对象就是函数的返回值或者类型转换的对象。使用临时对象的值初始化另一个对象值，不会要求对对象的复制：因为临时对象不会有其它使用，因而，它的值可以被移动到目的对象。做到这些，就要使用移动构造函数和移动赋值：当使用一个临时变量对象进行构造初始化的时候，调用移动构造函数。类似的，使用未命名的变量的值赋给一个对象时，调用移动赋值操作；
- 5)

```
Example6 (Example6&& x) : ptr(x.ptr)
{
    x.ptr = nullptr;
}
// move assignment
Example6& operator= (Example6&& x)
{
    delete ptr;
    ptr = x.ptr;
    x.ptr=nullptr;
    return *this;
}
```

203.何时需要合成构造函数

- 1)如果一个类没有任何构造函数，但他含有一个成员对象，该成员对象含有默认构造函数，那么编译器就为该合成一个默认构造函数，因为不合成一个默认构造函数那么该成员对象的构造函数不能调用；
- 2)没有任何构造函数的类派生自一个带有默认构造函数的基类，那么需要为该派生类合成一个构造函数，只有这样基类的构造函数才能被调用；
- 3)带有虚函数的类，虚函数的引入需要进入虚表，指向虚表的指针，该指针是在构造函数中初始化的，所以没有构造函数的话该指针无法被初始化；
- 4)带有一个虚基类的类

- 1)并不是任何没有构造函数的类都会合成一个构造函数
- 2)编译器合成出来的构造函数并不会显示设定类内的每一个成员变量

204.何时需要合成复制构造函数

有三种情况会以一个对象的内容作为另一个对象的初值：

- 1) 对一个对象做显示的初始化操作，`X xx = x;`
- 2) 当对象被当做参数交给某个函数时；
- 3) 当函数传回一个类对象时；

- 1) 如果一个类没有拷贝构造函数，但是含有一个类类型的成员变量，该类型含有拷贝构造函数，此时编译器会为该类合成一个拷贝构造函数；
- 2) 如果一个类没有拷贝构造函数，但是该类继承自含有拷贝构造函数的基类，此时编译器会为该类合成一个拷贝构造函数；
- 3) 如果一个类没有拷贝构造函数，但是该类声明或继承了虚函数，此时编译器会为该类合成一个拷贝构造函数；
- 4) 如果一个类没有拷贝构造函数，但是该类含有虚基类，此时编译器会为该类合成一个拷贝构造函数；

205.何时需要成员初始化列表？过程是什么？

- 1) 当初始化一个引用成员变量时；
- 2) 初始化一个const成员变量时；
- 3) 当调用一个基类的构造函数，而构造函数拥有一组参数时；
- 4) 当调用一个成员类的构造函数，而他拥有一组参数；
- 5) 编译器会——操作初始化列表，以适当顺序在构造函数之内安插初始化操作，并且在任何显示用户代码前。list中的项目顺序是由类中的成员声明顺序决定的，不是初始化列表中的排列顺序决定的。

206.程序员定义的析构函数被扩展的过程？

- 1) 析构函数函数体被执行；
- 2) 如果class拥有成员类对象，而后者拥有析构函数，那么它们会以其声明顺序的相反顺序被调用；
- 3) 如果对象有一个vptr，现在被重新定义
- 4) 如果有任何直接的上一层非虚基类拥有析构函数，则它们会以声明顺序被调用；
- 5) 如果任何虚基类拥有析构函数

207.构造函数的执行算法？

- 1) 在派生类构造函数中，所有的虚基类及上一层基类的构造函数调用；
- 2) 对象的vptr被初始化；
- 3) 如果有成员初始化列表，将在构造函数体内展开来，这必须在vptr被设定之后才做；
- 4) 执行程序员所提供的代码；

208.构造函数的扩展过程？

- 1) 记录在成员初始化列表中的数据成员初始化操作会被放在构造函数的函数体内，并与成员的声明顺序为顺序；
- 2) 如果一个成员并没有出现在成员初始化列表中，但它有一个默认构造函数，那么默认构造函数必须被调用；
- 3) 如果class有虚表，那么它必须被设定初值；
- 4) 所有上一层的基类构造函数必须被调用；
- 5) 所有虚基类的构造函数必须被调用。

209.哪些函数不能是虚函数

- 1)构造函数，构造函数初始化对象，派生类必须知道基类函数干了什么，才能进行构造；当有虚函数时，每一个类有一个虚表，每一个对象有一个虚表指针，虚表指针在构造函数中初始化；
- 2)内联函数，内联函数表示在编译阶段进行函数体的替换操作，而虚函数意味着在运行期间进行类型确定，所以内联函数不能是虚函数；
- 3)静态函数，静态函数不属于对象属于类，静态成员函数没有this指针，因此静态函数设置为虚函数没有任何意义。
- 4)友元函数，友元函数不属于类的成员函数，不能被继承。对于没有继承特性的函数没有虚函数的说法。
- 5)普通函数，普通函数不属于类的成员函数，不具有继承特性，因此普通函数没有虚函数。

210.sizeof 和strlen 的区别

- 1)strlen计算字符串的具体长度（只能是字符串），不包括字符串结束符。返回的是字符个数。
- 2)sizeof计算声明后所占的内存数（字节大小），不是实际长度。
- 3)sizeof是一个取字节运算符，而strlen是个函数。
- 4)sizeof的返回值=字符个数*字符所占的字节数，字符实际长度小于定义的长度，此时字符个数就等于定义的长度。若未给出定义的大小，分类讨论，对于字符串数组，字符大小等于实际的字符个数+1；对于整型数组，字符个数为实际的字符个数。字符串每个字符占1个字节，整型数据每个字符占的字节数需根据系统的位数类确定，32位占4个字节。
- 5)sizeof可以用类型做参数，strlen只能用char做参数，且必须以'\0'结尾，sizeof还可以用函数做参数；
- 6)数组做sizeof的参数不退化，传递给strlen就退化为指针；

211.简述strcpy、sprintf与memcpy的区别

- 1)操作对象不同
 - ①strcpy的两个操作对象均为字符串
 - ②sprintf的操作源对象可以是多种数据类型，目的操作对象是字符串
 - ③memcpy的两个对象就是两个任意可操作的内存地址，并不限于何种数据类型。
- 2)执行效率不同
memcpy最高，strcpy次之，sprintf的效率最低。
- 3)实现功能不同
 - ①strcpy主要实现字符串变量间的拷贝
 - ②sprintf主要实现其他数据类型格式到字符串的转化
 - ③memcpy主要是内存块间的拷贝。

212.编码实现某一变量某位清0或置1

```
#define BIT3 (0x1 << 3) static int a;
//设置a的bit 3:
void set_bit3( void )
{
    a |= BIT3; //将a第3位置1
}
//清a的bit 3
void set_bit3( void )
{
    a &= ~BIT3; //将a第3位清零
}
```

213.将“引用”作为函数参数有哪些特点？

- 1)传递引用给函数与传递指针的效果是一样的。这时，被调函数的形参就成为原来主调函数中的实参变量或对象的一个别名来使用，所以在被调函数中对形参变量的操作就是对其相应的目标对象（在主调函数中）的操作。
- 2)使用引用传递函数的参数，在内存中并没有产生实参的副本，它是直接对实参操作；而使用一般变量传递函数的参数，当发生函数调用时，需要给形参分配存储单元，形参变量是实参变量的副本；如果传递的是对象，还将调用拷贝构造函数。因此，当参数传递的数据较大时，用引用比用一般变量传递参数的效率和所占空间都好。
- 3)使用指针作为函数的参数虽然也能达到与使用引用的效果，但是，在被调函数中同样要给形参分配存储单元，且需要重复使用"*指针变量名"的形式进行运算，这很容易产生错误且程序的阅读性较差；另一方面，在主调函数的调用点处，必须用变量的地址作为实参。而引用更容易使用，更清晰。

214.分别写出BOOL,int,float,指针类型的变量a与“零”的比较语句。

```
BOOL : if ( !a ) or if(a)
int : if ( a == 0)
float : const EXPRESSION EXP = 0.000001
if ( a < EXP && a > -EXP)
pointer : if ( a != NULL) or if(a == NULL)
```

无论是float还是double类型的变量，都有精度限制。所以一定要避免将浮点变量用“==”或“!=”与数字比较，应该设法转化成“>”或“<”形式。

215.局部变量全局变量的问题？

- 1)局部会屏蔽全局。要用全局变量，需要使用"::"局部变量可以与全局变量同名，在函数内引用这个变量时，会用到同名的局部变量，而不会用到全局变量。对于有些编译器而言，在同一个函数内可以定义多个同名的局部变量，比如在两个循环体内都定义一个同名的局部变量，而那个局部变量的作用域就在那个循环体内。
- 2)如何引用一个已经定义过的全局变量，可以用引用头文件的方式，也可以用extern关键字，如果用引用头文件方式来引用某个在头文件中声明的全局变理，假定你将那个变写错了，那么在编译期间会报错，如果你用extern方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在连接期间报错。
- 3)全局变量可不可以定义在可被多个.C文件包含的头文件中，在不同的C文件中以static形式来声明同名全局变量。可以在不同的C文件中声明同名的全局变量，前提是其中只能有一个C文件中对此变量赋初值，此时连接不会出错

216.数组和指针的区别？

- 1)数组在内存中是连续存放的，开辟一块连续的内存空间；数组所占存储空间：sizeof（数组名）；数组大小：sizeof(数组名)/sizeof(数组元素数据类型)；
- 2)用运算符sizeof 可以计算出数组的容量（字节数）。sizeof s,p 为指针得到的是一个指针变量的字节数，而不是p 所指的内存容量。
- 3)编译器为了简化对数组的支持，实际上是利用指针实现了对数组的支持。具体来说，就是将表达式中的数组元素引用转换为指针加偏移量的引用。
- 4)在向函数传递参数的时候，如果实参是一个数组，那用于接受的形参为对应的指针。也就是传递过去是数组的首地址而不是整个数组，能够提高效率；
- 5)在使用下标的时候，两者的用法相同，都是原地址加上下标值，不过数组的原地址就是数组首元素的地址是固定的，指针的原地址就不是固定的。

217.C++如何阻止一个类被实例化？一般在什么时候将构造函数声明为private？

- 1)将类定义为抽象基类或者将构造函数声明为private；
- 2)不允许类外部创建类对象，只能在类内部创建对象

218.如何禁止自动生成拷贝构造函数？

- 1)为了阻止编译器默认生成拷贝构造函数和拷贝赋值函数，我们需要手动去重写这两个函数，某些情况下，为了避免调用拷贝构造函数和拷贝赋值函数，我们需要将他们设置成private，防止被调用。
- 2)类的成员函数和friend函数还是可以调用private函数，如果这个private函数只声明不定义，则会产生一个连接错误；
- 3)针对上述两种情况，我们可以定一个base类，在base类中将拷贝构造函数和拷贝赋值函数设置成private,那么派生类中编译器将不会自动生成这两个函数，且由于base类中该函数是私有的，因此，派生类将阻止编译器执行相关的操作。

219.assert与NDEBUG

- 1)assert宏的原型定义在<assert.h>中，其作用是如果它的条件返回错误，则终止程序执行，原型定义：
`#include <assert.h>`
`void assert( int expression );`
assert的作用是现计算表达式 expression，如果其值为假（即为0），那么它先向stderr打印一条出错信息，然后通过调用 abort 来终止程序运行。如果表达式为真，assert什么也不做。
- 2)NDEBUG宏是Standard C中定义的宏，专门用来控制assert()的行为。如果定义了这个宏，则assert不会起作用。定义NDEBUG能避免检查各种条件所需的运行时开销，当然此时根本就不会执行运行时检查。
- 3)C Standard中规定了assert以宏来实现。<assert.h>被设计来可以被多次包含，其中一上来就undef assert，然后由NDEBUG宏来决定其行为。

220.Debug和release的区别

- 1)调试版本，包含调试信息，所以容量比Release大很多，并且不进行任何优化（优化会使调试复杂化，因为源代码和生成的指令间关系会更复杂），便于程序员调试。Debug模式下生成两个文件，除了.exe或.dll文件外，还有一个.pdb文件，该文件记录了代码中断点等调试信息；
- 2)发布版本，不对源代码进行调试，编译时对应用程序的速度进行优化，使得程序在代码大小和运行速度上都是最优的。（调试信息可在单独的PDB文件中生成）。Release模式下生成一个文件.exe或.dll文件。
- 3)实际上，Debug 和 Release 并没有本质的界限，他们只是一组编译选项的集合，编译器只是按照预定的选项行动。事实上，我们甚至可以修改这些选项，从而得到优化过的调试版本或是带跟踪语句的发布版本。

221.main函数有没有返回值

- 1)程序运行过程入口点main函数，main（）函数返回值类型必须是int，这样返回值才能传递给程序激活者（如操作系统）表示程序正常退出。main（int args, char **argv）参数的传递。参数的处理，一般会调用getopt（）函数处理，但实践中，这仅仅是一部分，不会经常用到的技能点。

222.写一个比较大小的模板函数

```
#include<iostream>
using namespace std;
template<typename type1,typename type2>//函数模板
type1 Max(type1 a,type2 b)
{

return a > b ? a : b;
}
void main()
{

cout<<"Max = "<<Max(5.5,'a')<<endl;
}
```

223.c++怎么实现一个函数先于main函数运行

1)如果在main函数之前声明一个类的全局的对象。那么其执行顺序，根据全局对象的生存期和作用域，肯定先于main函数。

```
class simpleClass
{
public:
    simpleClass( )
    {
        cout << "simpleClass constructor.." << endl;
    }
};
simpleClass g_objectSimple;           //step1全局对象
int _tmain(int argc, _TCHAR* argv[]) //step3
{
    return 0;
}
```

2)定义在main()函数之前的全局对象、静态对象的构造函数在main()函数之前执行。

3)Main函数执行之前，主要就是初始化系统相关资源；

①设置栈指针

②初始化static静态和global全局变量，即data段的内容

③将未初始化部分的全局变量赋初值：数值型short, int, long等为0, bool为FALSE, 指针为NULL, 等等，即.bss段的内容

④全局对象初始化，在main之前调用构造函数

⑤将main函数的参数，argc, argv等传递给main函数，然后才真正运行main函数

4)Main函数执行之后

①全局对象的析构函数会在main函数之后执行；

②可以用_onexit 注册一个函数，它会在main 之后执行;

224.虚函数与纯虚函数的区别在于

1)纯虚函数只有定义没有实现，虚函数既有定义又有实现；

2)含有纯虚函数的类不能定义对象，含有虚函数的类能定义对象；

225.智能指针怎么用？智能指针出现循环引用怎么解决？

1)shared_ptr

调用一个名为make_shared的标准库函数，shared_ptr p = make_shared(42);通常用auto更方便，
auto p = ...;shared_ptr p2(new int(2));

每个shared_ptr都有一个关联的计数器，通常称为引用计数，一旦一个shared_ptr的计数器变为0，它就会自动释放自己所管理的对象；shared_ptr的析构函数就会递减它所指的对象的引用计数。如果引用计数变为0，shared_ptr的析构函数就会销毁对象，并释放它占用的内存。

2)unique_ptr

一个unique_ptr拥有它所指向的对象。某个时刻只能有一个unique_ptr指向一个给定对象。当unique_ptr被销毁时，它所指向的对象也被销毁。

3)weak_ptr

weak_ptr是一种不控制所指向对象生存期的智能指针，它指向由一个shared_ptr管理的对象，将一个weak_ptr绑定到一个shared_ptr不会改变引用计数，一旦最后一个指向对象的shared_ptr被销毁，对象就会被释放，即使有weak_ptr指向对象，对象还是会被释放。

4)弱指针用于专门解决shared_ptr循环引用的问题，weak_ptr不会修改引用计数，即其存在与否并不影响对象的引用计数器。循环引用就是：两个对象互相使用一个shared_ptr成员变量指向对方。弱引用并不对对象的内存进行管理，在功能上类似于普通指针，然而一个比较大的区别是，弱引用能检测到所管理的对象是否已经被释放，从而避免访问非法内存。

226.strcpy函数和strncpy函数的区别？哪个函数更安全？

1)函数原型

char* strcpy(char* strDest, const char* strSrc)

char* strncpy(char* strDest, const char* strSrc, int pos)

2)strcpy函数: 如果参数 dest 所指的内存空间不够大，可能会造成缓冲溢出(buffer Overflow)的错误情况，在编写程序时请特别留意，或者用strncpy()来取代。

strncpy函数：用来复制源字符串的前n个字符，src 和 dest 所指的内存区域不能重叠，且 dest 必须有足够的空间放置n个字符。

3)如果目标长>指定长>源长，则将源长全部拷贝到目标长，自动加上'\0'

如果指定长<源长，则将源长中按指定长度拷贝到目标字符串，不包括'\0'

如果指定长>目标长，运行时错误；

227.为什么要用static_cast转换而不用c语言中的转换？

1)更加安全；

2)更直接明显，能够一眼看出是什么类型转换为什么类型，容易找出程序中的错误；可清楚地辨别代码中每个显式的强制转；可读性更好，能体现程序员的意图

228.成员函数里memset(this,0,sizeof(*this))会发生什么

1)有时候类里面定义了很多int,char,struct等c语言里的那些类型的变量，我习惯在构造函数中将它们初始化为0，但是一句句的写太麻烦，所以直接就memset(this, 0, sizeof *this);将整个对象的内存全部置为0。对于这种情形可以很好的工作，但是下面几种情形是不可以这么使用的；

2)类含有虚函数表：这么做会破坏虚函数表，后续对虚函数的调用都将出现异常；

3)类中含有C++类型的对象：例如，类中定义了一个list的对象，由于在构造函数体的代码执行之前就对list对象完成了初始化，假设list在它的构造函数里分配了内存，那么我们这么一做就破坏了list对象的内存。

229.方法调用的原理（栈，汇编）

1)机器用栈来传递过程参数、存储返回信息、保存寄存器用于以后恢复，以及本地存储。而为单个过程分配的那部分栈称为帧栈；帧栈可以认为是程序栈的一段，它有两个端点，一个标识起始地址，一个标识着结束地址，两个指针结束地址指针esp，开始地址指针ebp；

2)由一系列栈帧构成，这些栈帧对应一个过程，而且每一个栈指针+4的位置存储函数返回地址；每一个栈帧都建立在调用者的下方，当被调用者执行完毕时，这一段栈帧会被释放。由于栈帧是向地址递减的方向延伸，因此如果我们将栈指针减去一定的值，就相当于给栈帧分配了一定空间的内存。如果将栈指针加上一定的值，也就是向上移动，那么就相当于压缩了栈帧的长度，也就是说内存被释放了。

3)过程实现

①备份原来的帧指针，调整当前的栈帧指针到栈指针位置；

②建立起来的栈帧就是为被调用者准备的，当被调用者使用栈帧时，需要给临时变量分配预留内存；

③使用建立好的栈帧，比如读取和写入，一般使用mov，push以及pop指令等等。

④恢复被调用者寄存器当中的值，这一过程其实是从栈帧中将备份的值再恢复到寄存器，不过此时这些值可能已经不在栈顶了

⑤恢复被调用者寄存器当中的值，这一过程其实是从栈帧中将备份的值再恢复到寄存器，不过此时这些值可能已经不在栈顶了。

⑥释放被调用者的栈帧，释放就意味着将栈指针加大，而具体的做法一般是直接将栈指针指向帧指针，因此会采用类似下面的汇编代码处理。

⑦恢复调用者的栈帧，恢复其实就是调整栈帧两端，使得当前栈帧的区域又回到了原始的位置。

⑧弹出返回地址，跳出当前过程，继续执行调用者的代码。

4)过程调用和返回指令

①call指令

②leave指令

③ret指令

230.MFC消息处理如何封装的？

231.回调函数的作用

1)当发生某种事件时，系统或其他函数将会自动调用你定义的一段函数；

2)回调函数就相当于一个中断处理函数，由系统在符合你设定的条件时自动调用。为此，你需要做三件事：1，声明；2，定义；3，设置触发条件，就是在你的函数中把你的回调函数名称转化为地址作为一个参数，以便于系统调用；

3)回调函数就是一个通过函数指针调用的函数。如果你把函数的指针（地址）作为参数传递给另一个函数，当这个指针被用为调用它所指向的函数时，我们就说这是回调函数；

4)因为可以把调用者与被调用者分开。调用者不关心谁是被调用者，所有它需知道的，只是存在一个具有某种特定原型、某些限制条件（如返回值为int）的被调用函数。

232.随机数的生成

1)#include<time.h> srand((unsigned)time(NULL)); cout<<(rand()%(b-a))+a;

2)由于rand()的内部实现是用线性同余法做的，所以生成的并不是真正的随机数，而是在一定范围内可视为随机的伪随机数。

3)种子写为srand(time(0))代表着获取系统时间，电脑右下角的时间，每一秒后系统时间的改变，数字序列的改变得到的数字不同，这才得带不同的数字，形成了真随机数，即使是真随机数，也是有规律可循。

版权声明：本文为CSDN博主「绝代风华i」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_42581177/article/details/125831432

233.变量的声明和定义有什么区别？

变量的定义为变量分配地址和存储空间，变量的声明不会分配地址，一个变量可以在多个地方声明，但是只能在一个地方定义，加入extern关键字修饰的是变量的声明，说明此变量将在文件以外或者文件后面部分定义。

说明：很多时候一个变量，只是声明不分配内存空间，直到具体使用时才初始化，分配内存空间，如外部变量。



```
int main()
{
    extern int A;
    //这是个声明而不是定义，声明A是一个已经定义了的外部变量

    dosth();//执行函数
}

int A; //定义A变量
```



234.请简述#ifdef、#else、#endif、和#ifndef的作用？

- 利用#ifdef、#endif将某程序功能模块包括进去，以向特定用户提供该功能，在不需要时，用户可以轻易将其屏蔽；

```
#ifdef MATH
#include "math.c"
#endif
```

- 在子程序上加上标记，以便于追踪和调试；

```
#ifdef DEBUG
printf("Indebugging.....!\r\n");
#endif
```

- 应对硬件的限制，由于一些具体应用环境的硬件不一样，限于条件，本地缺乏某种设备，只能绕过硬件，直接写出预期结果。

注意：虽然不用条件编译命令而直接用if语句也能到达要求，但是这样做的话，目标程序长（所有的语句都需要编译），运行时间长（在程序运行时会对if语句进行测试），采用条件编译，可以减少被编译的语句，减少目标程序的长度，减少程序的运行时间。

235.请写出int、bool、float、指针变量与"零值"比较的if语句？



```
//int类型与零值比较
if (n == 0)
if (n != 0)

//bool类型与零值比较
if (flag)    //表示flag为真
if (!flag)   //表示flag为假

//float类型与零值比较
const float EPSINON = 0.00001;
if ((x >= -EPSINON) && (x <= EPSINON))    //其中EPSINON是允许的误差

//指针变量与零值比较
if (p == NULL)
if (p != NULL)
```



236.结构体是否可以直接赋值？

声明时可以直接初始化，同一结构体的不同对象之间也可以直接赋值，但是当结构体中含有指针成员时一定要小心。

注意：当有多个指针指向同一段内存时，某个指针释放这段内存可能会导致其它指针的非法操作，因此在释放前一定要确保其它指针不再使用这段内存空间。

237.sizeof和strlen的区别？

- sizeof是一个操作符，strlen是库函数；
- sizeof的参数可以是数据的类型，也可以是变量，而strlen只能以结尾为'\0'的字符串作为参数；
- 编译器在编译时就计算出了sizeof的结果，而strlen函数必须在程序运行时才能计算出来，并且sizeof计算的是数据类型占内存的大小，而strlen计算的是字符串实际占用的内存大小；
- 数组做sizeof的参数不退化，数组传递给strlen就退化为指针了。

238.C语言和C++语言中的关键字static有什么区别？

在C语言中static用来修饰局部静态变量和外部静态变量、函数，而C++中的static关键字除了具有上述功能外，还能用来定义类的成员变量和函数，也就是静态成员和静态成员函数。

注意：编程时static的记忆性和全局性特点可以让在不同时期调用的函数进行通信、传递信息，而C++的静态成员则可以在多个对象实例间进行通信、传递信息。

239.C语言的malloc和C++中的new有什么区别？

- new和delete是操作符，可以重载，只能在C++中使用；

- malloc和free是函数，可以覆盖，C和C++都能使用；
- new可以调用对象的构造函数，对应的delete调用对象的析构函数；
- malloc仅仅分配内存，free则是回收内存，并不会执行构造函数和析构函数；
- new、delete返回的是某种数据类型指针，malloc、free返回的是void指针。

注意：malloc申请的内存空间要使用free进行释放，而new申请的内存空间要使用delete释放，不能够混用。

240.请写一个标准宏MIN？

```
#define min(a, b) ((a) <= (b) ? (a) : (b))
```

241.++i和i++的区别？

++i先自增1，再返回，i++先返回i，再自增1。

242.关键字volatile有什么作用？

- 状态寄存器一类的并行设备硬件寄存器；
- 中断服务子程序会访问到的非自动变量；
- 多线程间被几个任务访问共享变量。

注意：虽然volatile在嵌入式方面应用比较多，但是在PC软件的多线程中，volatile修饰的临界变量也是非常实用的。

243.一个参数可以既是const又是volatile吗？

一个参数既可以是const又可以是volatile的，当使用const和volatile同时修饰变量时，表示这个变量在程序内部是只读的，不能改变的，只在程序外部条件变化下改变，并且编译器不会优化这个变量，每次使用这个变量时，都会小心地去内存读取这个变量的值，而不是去寄存器读取它的备份。

注意：在这一定要注意const的意思，const只是不允许程序中的代码改变某一变量，其在程序编译器发挥作用，它并没有实际地禁止某段内存的读写特性。

244.*a和&a有什么区别？

&a：含义就是取变量a的地址

*a：使用在不同的地方，含义也不一样

- 在声明语句中，*a只说明a是一个指针变量，例如int *a；
- 在其它语句中，a前面没有操作数并且a是一个指针时，a代表指针a指向的内存地址存放的数据，如b=*a；
- a前面有操作数并且a是一个普通变量时，a代表乘以a，如c=b*a。

245.用C语言编写一个死循环程序？



```
while (1)
{
    dosth();
}

for (;;)
{
    dosth();
}
```



注意：很多种途径都可以实现同一种功能，但是不同的方式，时间和空间占用度不同，特别是对于嵌入式软件，处理器速度比较慢，存储空间有限，所以时间和空间优势是选择各种方法的首要考虑条件。

246.结构体内存对齐问题？

<https://www.cnblogs.com/Cqlismy/p/11440057.html>

247.全局变量和局部变量有什么区别？是怎么实现的？操作系统和编译器是怎么知道的？

- 全局变量是整个程序都可以访问的变量，生存期是从整个程序运行道结束，在程序结束时，全局变量所占用的内存会被释放；
- 局部变量存在于模块（子程序、函数）中，只有所在的模块可以访问，其它模块不能访问，模块结束（函数调用完成），局部变量所占用的内存被释放；
- 操作系统和编译器，可能是通过内存分配的位置来知道的，全局变量会被分配在全局数据段，并且在程序开始运行的时候被加载，局部变量则是在堆栈中进行分配的。

248.请简述C/C++程序编译的内存分配情况？

- 从静态存储区域分配：内存在程序编译时就已经分配好，这块内存在程序的整个运行期间都存在，速度快，不容易出错，例如：全局变量、static变量，常量字符串等；
- 从栈上分配：在执行函数时，函数内部局部变量的存储单元都在栈上创建，函数执行结束时，这些存储单元自动释放，栈内存分配运算内置于处理器的指令中，效率很高，但是分配的内存容量有限；
- 从堆上分配：也就是动态内存分配，程序在运行时使用malloc或者new申请任意大小的内存，程序员自己负责何时使用free或者delete进行内存释放，动态内存的生存期是由程序员决定的，使用非常灵活，如果在堆上分配了内存空间，就有责任回收它，否则运行的程序会出现内存泄漏，另外频繁地分配和释放不同大小的堆空间将会产生堆内碎块。
- 一个C/C++程序编译时内存分为5大存储区，分别是栈区、堆区、全局区、文字常量区、程序代码区。

249.请简述strcpy、sprintf和memcpy的区别？

- 操作对象不同，strcpy的两个操作对象均为字符串，sprintf的操作源对象可以是多种数据类型，目的操作对象是字符串，memcpy的两个对象就是任意可操作的内存地址，并不限于何种数据类型；
- 执行效率不同，memcpy的效率最高，strcpy次之，sprintf的效率最低；
- 实现功能不同，strcpy主要实现字符串变量间的拷贝，sprintf主要实现其它数据类型格式化到字符串，memcpy主要用于内存块间的拷贝；

注意：strcpy、sprintf和memcpy都可以实现拷贝的功能，但是针对的对象不同，需要根据实际需求，来选择合适的函数区实现拷贝功能。

250.请解释((void ())0)()的含义？

- void (*0)()：是一个返回值为void，参数为空的函数指针0；
- (void (*))0：把0转变成一个返回值为void，参数为空的函数指针；
- (void ())0：在上句的基础上加*表示整个是一个返回值为void，无参数，并且起始地址为0的函数的名字；
- ((void (*))0)()：上句的函数名所对应的函数的调用。

251.C语言的指针和引用和C++的有什么区别？

- 指针有自己的一块空间，而引用只是一个别名；
- 使用sizeof看一个指针的大小是4字节，而引用则是被引用对象的大小；
- 作为参数传递时，指针需要被解引用才可以对对象进行操作，而直接对引用的修改都会改变引用所指向的对象；
- 可以有const指针，但是没有const引用；
- 指针在使用中可以指向其它对象，但是引用只能是一个对象的引用，不能被改变；
- 指针可以有多级指针，而引用只有一级；
- 指针和引用使用++运算符的意义不一样；
- 如果返回动态内存分配的对象或者内存，必须使用指针，引用可能会引起内存泄漏。

252.typedef和define有什么区别？

- 用法不同：typedef用来定义一种数据类型的别名，增强程序的可读性，define主要用来定义常量以及书写复杂使用频繁的宏；
- 执行时间不同：typedef是编译过程的一部分，有类型检查的功能，define是宏定义，是预编译的部分，其发生在编译之前，只是简单的进行字符串替换，不会进行类型的检查；
- 作用域不同：typedef有作用域限定，define不受作用域限定，只要是在define声明后的引用，都是正确的；
- 对指针的操作不同：typedef和define定义指针时有很大区别。

注意：typedef定义是语句，句尾需要加上分号，而define不是语句，不能在句尾加上分号。

253.指针常量和常量指针有什么区别？

指针常量是指定义了一个指针，这个指针的值只能在定义时初始化，其它地方不能改变。常量指针是指定义了一个指针，这个指针指向一个只读的对象，不能通过常量指针来改变这个对象的值。指针常量强调的是指针的不可变性，而常量指针强调的是指针对其所指对象的不可变性。

注意：无论是指针常量还是常量指针，其最大的用途就是作为函数的形式参数，保证实参在被调用函数中的不可改变特性。

254.请简述队列和栈的异同？

队列和栈都是线性存储结构，但是两者的插入和删除数据的操作不同，队列是"先进先出"，栈是"先进后出"。

注意：区别栈区和堆区，堆区的存取是"顺序随意"，而栈区是"先进后出"，栈是由编译器自动分配释放，存放函数的参数值，局部变量的值等，其操作方式类似于数据结构中的栈，堆一般是由程序员分配释放，若程序员不释放，程序结束时可能由OS回收，分配方式类似于链表，它与本题中的堆和栈是两回事，堆栈只是一种数据结构，而堆区和栈区是程序的不同内存存储区域。

255.如何设置地址为0x67a9的整型变量的值为0xaa66？

```
int *ptr;
ptr = (int *)0x67a9;
*ptr = 0xaa66;
```

注意：这道题就是强制类型转换的典型例子，无论在什么平台，地址长度和整型数据的长度是一样的，既一个整型数据可以强制转换成地址指针类型，只要有意义即可。

256.请编程实现字符串转换为数字？

编码实现函数atoi()，设计一个程序，把一个字符串转化为一个整型数值，例如：字符串"5486321"，转化成整型5486321。



```
int myatoi(const char *str)
{
    int num = 0; //保存转换后的数值
    int isNegative = 0; //记录字符串中是否有负号
    int n = 0;
    char *p = str;

    if (p == NULL) //判断指针的合法性
        return -1;

    while (*p++ != '\0') //计算字符串长度
        n++;

    p = str;
    if (p[0] == '-') //判断数组是否有负号
        isNegative = 1;
```

```
for (int i = 0; i < n; i++)
{
    char temp = *p++;
    if (temp > '9' || temp < '0') //滤除非数字字符
        continue;

    if (num != 0 || temp != '0') //滤除字符串开始的'0'字符
    {
        temp -= 0x30; //将数字字转换为数值
        num += temp * int(pow(10, n - 1 - i));
    }
}

if (isNegative)
    return (0 - num);
else
    return num;
}
```



257.C语言的结构体和C++的有什么区别？

- C语言的结构体中是不能有函数成员的，而C++的类可以有；
- C语言的结构体中数据成员是没有private、public和protected访问限定的，而C++的类成员有这些访问限定；
- C语言的结构体是没有继承关系的，而C++的类却有丰富的继承关系。

注意：虽然C的结构体和C++的类有很大的相似度，但是类是实现面向对象的基础，而结构体只可以简单地理解为类的前身。

258.简述指针常量与常量指针的区别？

- 指针常量是指定义了一个指针，这个指针的值只能在定义时初始化，其它地方不能改变，常量指针是指定义了一个指针，这个指针指向一个只读的对象，不能通过常量指针来改变这个对象的值；
- 指针常量调调的是指针的不可变性，而常量指针强调的是指针对其所值对象的不可改变性。

注意：无论是指针常量还是常量指针，其最大的用途就是作为函数的形式参数，保证实参在被调用函数中的不可改变特性。

259.如何避免"野指针"？

- 指针变量声明时没有被初始化，解决办法为，指针声明时初始化，初始值可以是具体的地址值，也可以让它指向NULL；
- 指针p被free或者delete之后，没有设置为NULL，解决办法为，指针指向的内存空间被释放后指针应该指向NULL；
- 指针操作超越了变量的作用范围，解决办法为，在变量的作用域结束前释放掉变量的地址空间，并且让指针指向NULL。

260.句柄和指针的区别和联系是什么？

句柄和指针其实是两个截然不同的概念，Windows系统用句柄标记系统资源，隐藏系统的信息，只要知道有这个东西，然后去调用就行了，是32bit的uint。指针则标记某个内存地址，两者是不同的概念。

261.new/delete与malloc/free的区别是什么？

- new能自动计算需要分配的内存空间，而malloc需要手工计算字节数；

```
int *p = new int[2];  
int *q = (int *)malloc(2 * sizeof(int));
```

- new与delete直接带具体类型的指针，malloc与free返回void类型的指针；
- new类型是安全的，而malloc不是，例如，`int * p = new float[2];`就会报错，而`int *p = malloc(2 * sizeof(float));`编译时编译器无法指出错误来；
- new一般分为两步，new操作和构造，new操作对应与malloc，但new操作可以重载，可以自定义内存分配策略，步做内存分配，甚至分配到非内存设备上，而malloc步行；
- new调用构造函数，malloc不会调用构造函数，delete调用析构函数，而free不会调用析构函数；
- malloc/free需要库文件stdlib.h支持，而new/delete则不需要。

注意：delete和free被调用后，内存不会立即回收，指针也不会指向空，delete或free仅仅是告诉操作系统，这一块内存被释放了，可以用作其它用途，但是由于没有重新对这块内存进行写操作，所以内存中的变量数值并没有发生变化，出现野指针的情况，因此，释放完内存后，应该将该指针指向NULL。

262.请说一说extern "C"？

extern "C"的主要作用就是为了能够正确实现C++代码调用其它C语言代码，加上extern "C"后，会指示编译器这部分代码按C语言的方式进行编译，而不是C++。由于C++支持函数重载，因此编译器编译函数的过程中会将函数的参数类型也加到编译后的代码中，而不仅仅是函数名，而C语言不支持函数重载，因此编译C语言代码的函数时，不会带上函数的参数类型，一般只包括函数名。

该功能十分有用处，因为在C++出现以前，很多代码都是C语言编写的，而且很底层的库也是C语言写的，为了更好地支持原来的C代码和已经写好的C语言库，需要在C++中尽可能地支持C，而extern "C"就是其中的一个策略。

- C++代码调用C语言代码；
- 在C++的头文件中使用；
- 在多个人的协同开发时，可能有人擅长C语言，有的人擅长C++，在这样的情况下也会用到。

263.请说一说C++中struct和class的区别是什么？

在C++中，class和struct做类型定义时只有两点区别：

- 默认的继承权限不同，class默认继承权限是private继承，而struct默认是public继承；
- class还可用于定义模板参数，像typename，但是关键字struct不能定义模板参数。

C++之所以保留struct关键字，原因是：

- 保证与C语言的向下兼容性，C++必须提供一个struct；
- C++中的struct定义必须百分百地保证与C语言中的struct的向下兼容性，把C++中的最基本的对象单元规定为class而不是struct，就是为了避免各种兼容性要求的限制；
- 对struct定义的拓展性使C语言的代码能够更容易地移植到C++中。

原文作者：[liangliangge](#)

原文链接：<https://www.cnblogs.com/liangliangge/p/14397918.html>

264.new、delete、malloc、free关系

delete会调用对象的析构函数,和new对应free只会释放内存，new调用构造函数。malloc与free是C++/C语言的标准库函数，new/delete是C++的运算符。它们都可用于申请动态内存和释放内存。对于非内部数据类型的对象而言，光用malloc/free无法满足动态对象的要求。对象在创建的同时要自动执行构造函数，对象在消亡之前要自动执行析构函数。由于malloc/free是库函数而不是运算符，不在编译器控制权限之内，不能够把执行构造函数和析构函数的任务强加于malloc/free。因此C++语言需要一个能完成动态内存分配和初始化工作的运算符new，以及一个能完成清理与释放内存工作的运算符delete。注意new/delete不是库函数。

265.delete与 delete []区别

delete只会调用一次析构函数，而delete[]会调用每一个成员的析构函数。在More Effective C++中有更为详细的解释：“当delete操作符用于数组时，它为每个数组元素调用析构函数，然后调用operator delete来释放内存。”delete与new配套，delete []与new []配套

```
MemTest *mTest1=new MemTest[10];
```

```
MemTest *mTest2=new MemTest;
```

```
Int *pInt1=new int [10];
```

```
Int *pInt2=new int;
```

```
delete[]pInt1; //-1-
```

```
delete[]pInt2; //-2-
```

```
delete[]mTest1; //-3-
```

```
delete[]mTest2; //-4-
```

在-4-处报错。

这就说明：对于内建简单数据类型，delete和delete[]功能是相同的。对于自定义的复杂数据类型，delete和delete[]不能互用。delete[]删除一个数组，delete删除一个指针。简单来说，用new分配的内存用delete删除；用new[]分配的内存用delete[]删除。delete[]会调用数组元素的析构函数。内部数据类型没有析构函数，所以问题不大。如果你在用delete时没用括号，delete就会认为指向的是单个对象，否则，它就会认为指向的是一个数组。

266.C++有哪些性质（面向对象特点）

封装，继承和多态。

267.子类析构时要调用父类的析构函数吗？

析构函数调用的次序是先派生类的析构后基类的析构，也就是说在基类的析构调用的时候，派生类的信息已经全部销毁了。定义一个对象时先调用基类的构造函数、然后调用派生类的构造函数；析构的时候恰好相反：先调用派生类的析构函数、然后调用基类的析构函数。

268.多态，虚函数，纯虚函数

多态：是对于不同对象接收相同消息时产生不同的动作。C++的多态性具体体现在运行和编译两个方面：在程序运行时的多态性通过继承和虚函数来体现；

在程序编译时多态性体现在函数和运算符的重载上；

虚函数：在基类中冠以关键字 `virtual` 的成员函数。它提供了一种接口界面。允许在派生类中对基类的虚函数重新定义。

纯虚函数的作用：在基类中为其派生类保留一个函数的名字，以便派生类根据需要对它进行定义。作为接口而存在 纯虚函数不具备函数的功能，一般不能直接被调用。

从基类继承来的纯虚函数，在派生类中仍是虚函数。如果一个类中至少有一个纯虚函数，那么这个类被称为抽象类（`abstract class`）。

抽象类中不仅包括纯虚函数，也可包括虚函数。抽象类必须用作派生其他类的基类，而不能用于直接创建对象实例。但仍可使用指向抽象类的指针支持运行时多态性。

269.求下面函数的返回值（微软）

```
int func(x)
{
    int countx = 0;

    while(x)
    {
        countx ++;

        x = x&(x-1);
    }

    return countx;
}
```

假定 $x = 9999$ 。答案：8

思路：将 x 转化为2进制，看含有的1的个数。

270.什么是“引用”？申明和使用“引用”要注意哪些问题？

答：引用就是某个目标变量的“别名”(alias)，对应用的操作与对变量直接操作效果完全相同。申明一个引用的时候，切记要对其进行初始化。引用声明完毕后，相当于目标变量名有两个名称，即该目标原名称和引用名，不能再把该引用名作为其他变量名的别名。声明一个引用，不是新定义了一个变量，它只表示该引用名是目标变量名的一个别名，它本身不是一种数据类型，因此引用本身不占存储单元，系统也

271.将“引用”作为函数参数有哪些特点？

(1) 传递引用给函数与传递指针的效果是一样的。这时，被调函数的形参就成为原来主调函数中的实参变量或对象的一个别名来使用，所以在被调函数中对形参变量的操作就是对其相应的目标对象（在主调函数中）的操作。

(2) 使用引用传递函数的参数，在内存中并没有产生实参的副本，它是直接对实参操作；而使用一般变量传递函数的参数，当发生函数调用时，需要给形参分配存储单元，形参变量是实参变量的副本；如果传递的是对象，还将调用拷贝构造函数。因此，当参数传递的数据较大时，用引用比用一般变量传递参数的效率和所占空间都好。

(3) 使用指针作为函数的参数虽然也能达到与使用引用的效果，但是，在被调函数中同样要给形参分配存储单元，且需要重复使用“*指针变量名”的形式进行运算，这很容易产生错误且程序的阅读性较差；另一方面，在主调函数的调用点处，必须用变量的地址作为实参。而引用更容易使用，更清晰。

272.在什么时候需要使用“常引用”？

如果既要利用引用提高程序的效率，又要保护传递给函数的数据不在函数中被改变，就应使用常引用。

常引用声明方式：const 类型标识符 &引用名=目标变量名；

例1

```
int a;
```

```
const int &ra=a;
```

```
ra=1; //错误
```

```
a=1; //正确
```

例2

```
string foo( );
```

```
void bar(string & s);
```

那么下面的表达式将是非法的：

```
bar(foo( ));
```

```
bar("hello world");
```

原因在于foo()和"hello world"串都会产生一个临时对象，而在C++中，这些临时对象都是const类型的。因此上面的表达式就是试图将一个const类型的对象转换为非const类型，这是非法的。引用型参数应该在能被定义为const的情况下，尽量定义为const。

10.将“引用”作为函数返回值类型的格式、好处和需要遵守的规则？

格式：类型标识符 &函数名（形参列表及类型说明）{ //函数体 }

好处：在内存中不产生被返回值的副本；（注意：正是因为这点原因，所以返回一个局部变量的引用是不可取的。因为随着该局部变量生存期的结束，相应的引用也会失效，产生runtime error!

注意事项：

(1) 不能返回局部变量的引用。这条可以参照Effective C++[1]的Item 31。主要原因是局部变量会在函数返回后被销毁，因此被返回的引用就成为了“无所指”的引用，程序会进入未知状态。

(2) 不能返回函数内部new分配的内存的引用。这条可以参照Effective C++[1]的Item 31。虽然不存在局部变量的被动销毁问题，可对于这种情况（返回函数内部new分配内存的引用），又面临其它尴尬局面。例如，被函数返回的引用只是作为一个临时变量出现，而没有被赋予一个实际的变量，那么这个引用所指向的空间（由new分配）就无法释放，造成memory leak。

(3) 可以返回类成员的引用，但最好是const。这条原则可以参照Effective C++[1]的Item 30。主要原因是当对象的属性是与某种业务规则（business rule）相关联的时候，其赋值常常与某些其它属性或者对象的状态有关，因此有必要将赋值操作封装在一个业务规则当中。如果其它对象可以获得该属性的非常量引用（或指针），那么对该属性的单纯赋值就会破坏业务规则的完整性。

(4) 流操作符重载返回值申明为“引用”的作用：

流操作符<<和>>，这两个操作符常常希望被连续使用，例如：cout << "hello" << endl; 因此这两个操作符的返回值应该是一个仍然支持这两个操作符的流引用。可选的其它方案包括：返回一个流对象和返回一个流对象指针。但是对于返回一个流对象，程序必须重新（拷贝）构造一个新的流对象，也就是说，连续的两个<<操作符实际上是针对不同对象的！这无法让人接受。对于返回一个流指针则不能连续使用<<操作符。因此，返回一个流对象引用是唯一选择。这个唯一选择很关键，它说明了引用的重要性以及无可替代性，也许这就是C++语言中引入引用这个概念的原因吧。

赋值操作符=。这个操作符象流操作符一样，是可以连续使用的，例如：x = j = 10;或者(x=10)=100;赋值操作符的返回值必须是一个左值，以便可以被继续赋值。因此引用成了这个操作符的唯一返回值选择。

```
#include<iostream.h>

int &put(int n);

int vals[10];

int error=-1;

void main()

{

    put(0)=10; //以put(0)函数值作为左值，等价于vals[0]=10;

    put(9)=20; //以put(9)函数值作为左值，等价于vals[9]=20;

    cout<<vals[0];

    cout<<vals[9];

}

int &put(int n)

{

    if (n>=0 && n<=9 ) return vals[n];

    else { cout<<"subscript error"; return error; }

}
```

(5) 在另外的一些操作符中，却千万不能返回引用：+、*、/、% 四则运算符。它们不能返回引用，Effective C++[1]的Item 23详细的讨论了这个问题。主要原因是这四个操作符没有side effect，因此，它们必须构造一个对象作为返回值，可选的方案包括：返回一个对象、返回一个局部变量的引用，返回一个new分配的对象的引用、返回一个静态对象引用。根据前面提到的引用作为返回值的三个规则，2、3两个方案都被否决了。静态对象的引用又因为((a+b) == (c+d))会永远为true而导致错误。所以可选的只剩下返回一个对象了。

273.结构与联合有和区别？

(1). 结构和联合都是由多个不同的数据类型成员组成,但在任何同一时刻,联合中只存放了一个被选中的成员(所有成员共用一块地址空间),而结构的所有成员都存在(不同成员的存放地址不同)。

(2). 对于联合的不同成员赋值,将会对其它成员重写,原来成员的值就不存在了,而对于结构的不同成员赋值是互不影响的。

274.试写出程序结果：

```
int a=4;

int &f(int x)

{ a=a+x;

return a;

}

int main(void)

{ int t=5;

cout<<f(t)<<endl; a = 9

f(t)=20; a = 20

cout<<f(t)<<endl; t = 5,a = 20 a = 25

t=f(t); a = 30 t = 30

cout<<f(t)<<endl; } t = 60

}
```

275.重载 (overload)和重写(overried , 有的书也叫做“覆盖”) 的区别？

常考的题目。从定义上来说：

重载：是指允许存在多个同名函数，而这些函数的参数表不同（或许参数个数不同，或许参数类型不同，或许两者都不同）。

重写：是指子类重新定义父类虚函数的方法。

从实现原理上来说：

重载：编译器根据函数不同的参数表，对同名函数的名称做修饰，然后这些同名函数就成了不同的函数（至少对于编译器来说是这样的）。如，有两个同名函数：function func(p:integer):integer;和function func(p:string):integer;。那么编译器做过修饰后的函数名称可能是这样的：int_func、str_func。对于这两个函数的调用，在编译期间就已经确定了，是静态的。也就是说，它们的地址在编译期就绑定了（早绑定），因此，重载和多态无关！

重写：和多态真正相关。当子类重新定义了父类的虚函数后，父类指针根据赋给它的不同的子类指针，动态的调用属于子类的该函数，这样的函数调用在编译期间是无法确定的（调用的子类的虚函数的地址无法给出）。因此，这样的函数地址是在运行期绑定的（晚绑定）。

276.有哪几种情况只能用initialization list 而不能assignment?

答案：当类中含有const、reference 成员变量；基类的构造函数都需要初始化表。

277. C++是不是类型安全的？

答案：不是。两个不同类型的指针之间可以强制转换（用reinterpret cast）。C#是类型安全的。

278. main 函数执行以前，还会执行什么代码？

答案：全局对象的构造函数会在main 函数之前执行。

279. 描述内存分配方式以及它们的区别？

- 1) 从静态存储区域分配。内存存在程序编译的时候就已经分配好，这块内存存在程序的整个运行期间都存在。例如全局变量，static 变量。
- 2) 在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集。
- 3) 从堆上分配，亦称动态内存分配。程序在运行的时候用malloc 或new 申请任意多少的内存，程序员自己负责在何时用free 或delete 释放内存。动态内存的生存期由程序员决定，使用非常灵活，但问题也最多。

280.分别写出BOOL,int,float,指针类型的变量a 与“零”的比较语句。

答案：

BOOL : if (!a) or if(a)

int : if (a == 0)

float : const EXPRESSION EXP = 0.000001

if (a < EXP && a > -EXP)

pointer : if (a != NULL) or if(a == NULL)

281.请说出const与#define 相比，有何优点？

答案：

const作用：定义常量、修饰函数参数、修饰函数返回值三个作用。被Const修饰的东西都受到强制保护，可以预防意外的变动，能提高程序的健壮性。

- 1) const 常量有数据类型，而宏常量没有数据类型。编译器可以对前者进行类型安全检查。而对后者只进行字符替换，没有类型安全检查，并且在字符替换可能会产生意料不到的错误。
- 2) 有些集成化的调试工具可以对const 常量进行调试，但是不能对宏常量进行调试。

282.简述数组与指针的区别？

数组要么在静态存储区被创建（如全局数组），要么在栈上被创建。指针可以随时指向任意类型的内存块。

(1)修改内容上的差别

```
char a[] = "hello";  
  
a[0] = 'X';  
  
char *p = "world"; // 注意p 指向常量字符串  
  
p[0] = 'X'; // 编译器不能发现该错误，运行时错误
```

(2) 用运算符sizeof 可以计算出数组的容量（字节数）。sizeof(p),p 为指针得到的是一个指针变量的字节数，而不是p 所指的内存容量。C++/C 语言没有办法知道指针所指的内存容量，除非在申请内存时记住它。注意当数组作为函数的参数进行传递时，该数组自动退化为同类型的指针。

```
char a[] = "hello world";  
  
char *p = a;  
  
cout<< sizeof(a) << endl; // 12 字节  
  
cout<< sizeof(p) << endl; // 4 字节  
  
计算数组和指针的内存容量  
  
void Func(char a[100])  
{  
  
cout<< sizeof(a) << endl; // 4 字节而不是100 字节  
  
}
```

283. int (*s[10])(int) 表示的是什么？

int (*s[10])(int) 函数指针数组，每个指针指向一个int func(int param)的函数。

284. 栈内存与文字常量区

```
char str1[] = "abc";  
  
char str2[] = "abc";  
  
const char str3[] = "abc";  
  
const char str4[] = "abc";  
  
const char *str5 = "abc";  
  
const char *str6 = "abc";  
  
char *str7 = "abc";  
  
char *str8 = "abc";  
  
cout << ( str1 == str2 ) << endl; //0 分别指向各自的栈内存  
  
cout << ( str3 == str4 ) << endl; //0 分别指向各自的栈内存  
  
cout << ( str5 == str6 ) << endl; //1 指向文字常量区地址相同  
  
cout << ( str7 == str8 ) << endl; //1 指向文字常量区地址相同  
  
结果是：0 0 1 1
```


解答：str1,str2,str3,str4是数组变量，它们有各自的内存空间；而str5,str6,str7,str8是指针，它们指向相同的常量区域。

285.将程序跳转到指定内存地址

要对绝对地址0x100000赋值，我们可以用(unsigned int*)0x100000 = 1234;那么要是想让程序跳转到绝对地址是0x100000去执行，应该怎么做？

```
((void *) (0x100000))();
```

首先要将0x100000强制转换成函数指针,即:

```
(void (*)( ))0x100000
```

然后再调用它:

```
((void (*)( ))0x100000)();
```

用typedef可以看得更直观些:

```
typedef void(*)() voidFuncPtr;
```

```
*((voidFuncPtr)0x100000)();
```

286.int id[sizeof(unsigned long)];这个对吗？为什么？

答案:正确 这个 sizeof是编译时运算符，编译时就确定了,可以看成和机器有关的常量。

287.引用与指针有什么区别？

【参考答案】

- \1) 引用必须被初始化，指针不必。
- \2) 引用初始化以后不能被改变，指针可以改变所指的对象。
- \3) 不存在指向空值的引用，但是存在指向空值的指针。

288.const 与 #define 的比较，const有什么优点？

【参考答案】

(1) const 常量有数据类型，而宏常量没有数据类型。编译器可以对前者进行类型安全检查。而对后者只进行字符替换，没有类型安全检查，并且在字符替换可能会产生意料不到的错误（边际效应）。

(2) 有些集成化的调试工具可以对 const 常量进行调试，但是不能对宏常量进行调试。

27题：复杂声明

```
void * ( * (*fp1)(int))[10];
```

```
float ((fp2)(int,int,int))(int);
```

```
int (* (*fp3)())10;
```

分别表示什么意思？

【标准答案】

1. void * (* (fp1)(int))[10]; fp1是一个指针，指向一个函数，这个函数的参数为int型，函数的返回值是一个指针，这个指针指向一个数组，这个数组有10个元素，每个元素是一个void型指针。

2.float ((fp2)(int,int,int))(int); fp2是一个指针，指向一个函数，这个函数的参数为3个int型，函数的返回值是一个指针，这个指针指向一个函数，这个函数的参数为int型，函数的返回值是float型。

<http://3.int> (* (* fp3()))10; fp3是一个指针，指向一个函数，这个函数的参数为空，函数的返回值是一个指针，这个指针指向一个数组，这个数组有10个元素，每个元素是一个指针，指向一个函数，这个函数的参数为空，函数的返回值是int型。

289.内存的分配方式有几种？

【参考答案】

一、从静态存储区域分配。内存存在程序编译的时候就已经分配好，这块内存存在程序的整个运行期间都存在。例如全局变量。

二、在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。

三、从堆上分配，亦称动态内存分配。程序在运行的时候用malloc或new申请任意多少的内存，程序员自己负责在何时用free或delete释放内存。动态内存的生存期由我们决定，使用非常灵活，但问题也最多。

290.基类的析构函数不是虚函数，会带来什么问题？

【参考答案】派生类的析构函数用不上，会造成资源的泄漏。

291.全局变量和局部变量有什么区别？是怎么实现的？操作系统和编译器是怎么知道的？

【参考答案】

生命周期不同：

全局变量随主程序创建和创建，随主程序销毁而销毁；局部变量在局部函数内部，甚至局部循环体等内部存在，退出就不存在；

使用方式不同：通过声明后全局变量程序的各个部分都可以用到；局部变量只能在局部使用；分配在栈区。

操作系统和编译器通过内存分配的位置来知道的，全局变量分配在全局数据段并且在程序开始运行的时候被加载。局部变量则分配在堆栈里面。

原文作者：[Joey Seung](#)

原文链接：https://zhuanlan.zhihu.com/p/75347892?utm_source=wechat_session

292. const关键字（反义词mutable）

（1）定义时必须初始化

（2）指针可以是const指针，也可以是指向const对象的指针

（3）定义为const的形参，在函数体内不能被修改

（4）后面加Const，表示该成员函数不会修改类的成员变量。本质是修饰隐藏的*this指针。加const的成员函数可以被const或非const对象调用，但是普通成员函数（无const修饰）只能被普通对象（无const修饰）调用。

(6) Const修饰成员变量时，不能在声明时初始化，必须在构造函数的列表里初始化

293. static关键字

(1) 在函数中，一个static的变量在此函数被调用过程中维持其值不变

(2) 在模块中（不在函数中），一个static变量可以被模块中所有函数访问，但不可以被模块外的其他函数访问。

(3) 在模块内，一个static的函数只可以被这一模块内的其他函数调用。

(4) 类中的static成员变量属于整个类，不能在类内进行定义，只能在类的作用域中进行定义。

(5) 类中的static成员函数属于整个类，不包含this指针，只能调用static成员函数。

(6) static全局变量只能在本文件中使用，限制了它的作用域；而普通全局变量可以在其他文件中使用。

(7) static局部变量必须初始化，普通局部变量不需要；前者所在的函数被多次调用时，依据上一次的计算结果进行计算，而后者所在的函数被调用时，还是原来的值。虽然静态局部变量在函数调用结束后仍然存在，但其他函数不能引用它。

(8) static函数限定在本文件中使用，虽然其他文件可以知道它的存在，但不能使用；而普通函数默认是extern的，其他文件也可以使用。Static函数有两个好处：一是其他文件可以定义相同名字的函数，不会冲突；二是静态函数不能为其他函数使用。

294. extern关键字

(1) extern C，表示该段代码以C语言进行编译。

(2) extern 放在变量或函数前，说明该变量或函数定义在别的文件中，提示编译器去其他模块中找定义，相当于前向声明。

295. 指针和引用的区别

(1) 引用是直接访问，指针是间接访问。

(2) 引用是变量的别名，本身不单独分配自己的内存空间，而指针有自己的内存空间

(3) 引用绑定内存空间（必须赋初值），是一个变量别名不能更改绑定，可以改变对象的值。

总的来说：引用既具有指针的效率，又具有变量使用的方便性和直观性

296.explicit是干什么用的？

声明为explicit的构造函数不能在隐式转换中使用。可以阻止不应该允许的经过[转换构造函数]进行的隐式转换的发生。

297.inline的用法

<https://www.cnblogs.com/fnlingnzb-learner/p/6423917.html>

298. new/delete与malloc/free之间的区别？

(1) malloc/free是C/C++语言的标准库函数，new/delete是C++的运算符

(2) new能够自动分配空间大小，malloc传入参数。

(3) new/delete能对对象进行构造和析构函数的调用进而对内存进行更加详细的工作，而malloc/free不能。

既然new/delete的功能完全覆盖了malloc/free，为什么C++还保留malloc/free呢？因为C++程序经常要调用C函数，而C程序只能用malloc/free管理动态内存。

299. 浅拷贝与深拷贝？为什么要使用深拷贝？

(1) 浅拷贝 `char * arr[] = "hello"; char * a = arr;`浅拷贝只是对指针的拷贝，拷贝后两个指针指向同一内存空间。

(2) 深拷贝 `char * arr[] = "hello"; char * a = new char[]; a = arr;`深拷贝不但对指针进行拷贝，而且对指针指向的内容进行拷贝，经过深拷贝后的指针是指向两个不同地址的指针。

浅拷贝会出现的问题：(1) 浅拷贝只是拷贝了指针，使得两个指针指向同一地址，这样在对象结束调用析构函数时，会造成同一份资源析构两次，即delete同一块内存两次，造成程序崩溃；(2) 浅拷贝使得两个指针指向同一个地址，任何一方的改动都会影响另一方；(3) 同一个空间，第二次释放失败，导致无法操作该空间，造成内存泄漏。

300. 深入谈谈堆和栈？

(1) 分配和管理方式不同：

堆是动态分配的，其空间的分配和释放都由程序员控制。

栈由编译器自动管理。栈有两种分配方式：静态分配和动态分配。静态分配由编译器完成，比如局部变量的分配。动态分配由alloca()函数进行分配，但是栈的动态分配和堆是不同的，它的动态分配是由编译器进行释放，无须手工控制。

(2) 产生碎片不同

对堆来说，频繁的new/delete或者malloc/free势必会造成内存空间的不连续，造成大量的碎片，使程序效率降低。

对栈而言，则不存在碎片问题，因为栈是先进后出的队列，永远不可能有一个内存块从栈中间弹出。

(3) 生长方向不同

堆是向着内存地址增加的方向增长的，从内存的低地址向高地址方向增长。

栈是向着内存地址减小的方向增长，由内存的高地址向低地址方向增长。

301. 内存的静态分配和动态分配的区别？

(1) 时间不同。静态分配发生在程序编译和连接时。动态分配则发生在程序调入和执行时。

(2) 空间不同。堆都是动态分配的，没有静态分配的堆。栈有2种分配方式：静态分配和动态分配。静态分配是编译器完成的，比如局部变量的分配。alloca，可以从栈里动态分配内存，不用担心内存泄露问题，当函数返回时，通过alloca申请的内存就会被自动释放掉。

302. 实现string类

```

class ` `string` `{ ` `public` `: ` `String() ` `//初始化` `: _pstr(` `new`
` `char` `[1])` `{ ` `} ` `String(` `const` ` `char` ` * pstr ); ` `//普通构造函数` `:
_pstr(` `new` ` `char` `[` `strlen` `(pstr) + 1]())` `{ ` ` ` `strcpy` `_` `( _pstr,pstr);` `{ `
` `String(` `const` ` `String & rhs); ` `//复制构造函数` `: _pstr(` `new`
` `char` `[` `strlen` `(pstr) + 1]())` `{ ` ` ` `strcpy` `_` `( _pstr, rhs.pstr);` `{ `
` `String(String && rhs); ` `//移动构造函数,右值引用` `: _pstr(rhs._pstr)` `{ `
rhs.pstr = NULL;` `{ ` `} ` `String & operator=(` `const` ` `String & rhs) ` `//重载复制运算符
函数` `{ ` ` ` `if` `( ` `this` ` != & rhs) ` ` { ` ` ` `delete` ` `[] _pstr;`
_pstr = ` `new` ` `char` `[` `strlen` `(rhs._pstr) + 1](); ` ` ` `strcpy` `_` `( _pstr,
rhs._pstr);` `{ ` ` ` `return` ` `*` ` `this` `;` `{ ` `} ` `String & operator=(String &&
rhs) ` `//移动赋值运算符函数` `{ ` ` ` `if` `( ` `this` ` != &rhs) ` ` { ` ` ` `delete`
`[] _pstr;` ` ` ` _pstr = rhs._pstr;` ` ` ` rhs._pstr = NULL;` ` ` ` } `
` `return` ` `*` ` `this` `;` `{ ` `} ` `~String() ` `{ ` ` ` `delete` ` `[] _pstr;` `{ ` `} `
` `friend` ` `std::ostream &operator<<(` `std::ostream & os, ` `const` ` `String &
rhs);` `private` `: ` ` ` `char` ` * _pstr;` `{ ` `} ` `std::ostream &
operator<<(` `std::ostream & os, ` `const` ` `String & rhs) ` `{ ` ` ` `os << rhs._pstr;`
` `return` ` `os;` `{ ` `} `

```

303. 什么是继承？什么是多态？

(1)

(2) C++中多态机制主要体现在两个方面，一个是函数的重载，一个是接口的重写。接口多态指的是“一个接口多种形态”。每一个对象内部都有一个虚表指针，该虚表指针被初始化为本类的虚表。所以在程序中，不管你的对象类型如何转换，但该对象内部的虚表指针是固定的，所以呢，才能实现动态的对象函数调用，这就是C++多态性实现的原理。

多态的基础是继承，需要虚函数的支持，简单的多态是很简单的。子类继承父类大部分的资源，不能继承的有构造函数，析构函数，拷贝构造函数，operator=函数，友元函数等等

多态作用：

隐藏实现细节，代码能够模块化；2. 接口重用：为了类在继承和派生的时候正确调用。

多态的两个必要条件：

1. 一个基类的指针或者引用指向派生类的对象；2. 虚函数

304. 什么是静态关联？什么是动态关联？

静态关联是程序在编译阶段就能确定实际执行动作，程序运行时才能确定执行的动作叫动态关联。

305. 虚函数是如何实现的？

编译时若基类中有虚函数，编译器为该的类创建一个一维数组的虚表，存放是每个虚函数的地址。基类和派生类都包含虚函数时，这两个类都建立一个虚表。构造函数中进行虚表的创建和虚表指针的初始化。在构造子类对象时，要先调用父类的构造函数，初始化父类对象的虚表指针，该虚表指针指向父类的虚表。执行子类的构造函数时，子类对象的虚表指针被初始化，指向自身的虚表。每一个类都有虚表。虚表可以继承，如果子类没有重写虚函数，那么子类虚表中仍然会有该函数的地址，只不过这个地址指向的是基类的虚函数实现。派生类的虚表中虚函数地址的排列顺序和基类的虚表中虚函数地址排列顺序相同。当用一个指针/引用调用一个函数的时候，被调用的函数是取决于这个指针/引用的类型。即如果这个指针/引用是基类对象的指针/引用就调用基类的方法；如果指针/引用是派生类对象的指针/引用就调用派生类的方法，当然如果派生类中没有此方法，就会向上到基类里面去寻找相应的方法。这些调

用在编译阶段就确定了。当涉及到多态性的时候，采用了虚函数和动态绑定，此时的调用就不会在编译时候确定而是在运行时确定。不在单独考虑指针/引用的类型而是看指针/引用的对象的类型来判断函数的调用，根据对象中虚指针指向的虚表中的函数的地址来确定调用哪个函数。

306. 虚函数与纯虚函数的区别？含有纯虚函数的类叫什么？

- (1) 虚函数与纯虚函数都可以在子类中重写。
- (2) 纯虚函数只有定义，没有实现；虚函数既要有定义，也要有实现的代码。
- (3) 纯虚函数 `virtual void print() = 0`；虚函数 `virtual void print() { XXX }`；
- (4) 包含纯虚函数的类叫抽象类，该类不可以创建对象；而含有虚函数的类可以创建对象。

307. 多重继承如何解决？

虚拟继承解决了多重继承的问题。如：A是基类，B、C继承自A，D多重继承自B和C，那么D访问A中的变量时，就会出现二义性错误。如果类B和类C虚拟继承自A，那么类D只会有A的一个对象，这样就解决了二义性问题。或者用成员限定符解决二义性。

308. 派生类与虚函数概述

- (1) 派生类继承的函数不能定义为虚函数。虚函数是希望派生类重新定义。如果派生类没有重新定义某个虚函数，则在调用的时候会使用基类中定义的版本。
- (2) 派生类中函数的声明必须与基类中定义的方式完全匹配。
- (3) 基类中声明为虚函数，则派生类也为虚函数。

309. 为什么析构函数要定义为虚函数？哪些函数不能是虚函数？

- (1) 如果析构函数不是虚函数，那么释放内存时候，编译器会使用静态联编，认为p就是一个基类指针，调用基类析构函数，这样子类对象的内存没有释放，造成内存泄漏。定义成虚函数以后，就会动态联编，先调用子类析构函数，再基类。
- (2) 1) 普通函数只能重载，不能被重写，因此编译器会在编译时绑定函数。
2) 构造函数是知道全部信息才能创建对象，然而虚函数允许只知道部分信息。
3) 内联函数在编译时被展开，虚函数在运行时才能动态绑定函数。
4) 友元函数 因为不可以被继承。
5) 静态成员函数 只有一个实体，不能被继承。父类和子类共有。

310. 析构函数可以抛出异常吗？为什么不能抛出异常？除了资源泄露，还有其他需考虑的因素吗？

C++标准指明析构函数不能、也不应该抛出异常。C++异常处理模型最大的特点和优势就是对C++中的面向对象提供了最强大的无缝支持。那么如果对象在运行期间出现了异常，C++异常处理模型有责任清除那些由于出现异常所导致的已经失效了的对象(也即对象超出了它原来的作用域)，并释放对象原来所分配的资源，这就是调用这些对象的析构函数来完成释放资源的任务，所以从这个意义上说，析构函数已经变成了异常处理的一部分。

- 1) 如果析构函数抛出异常，则异常点之后的程序不会执行，如果析构函数在异常点之后执行了某些必要的动作比如释放某些资源，则这些动作不会执行，会造成诸如资源泄漏的问题。

2) 通常异常发生时, c++的机制会调用已经构造对象的析构函数来释放资源, 此时若析构函数本身也抛出异常, 则前一个异常尚未处理, 又有新的异常, 会造成程序崩溃的问题。

311.动态链接库的两种使用方法及特点？

1). 载入时动态链接, 模块非常明确调用某个导出函数, 使得他们就像本地函数一样。这需要链接时链接那些函数所在DLL的导入库, 导入库向系统提供了载入DLL时所需的信息及DLL函数定位。

2)运行时动态链接。

312. STL各类容器（3个顺序+4个关联+1个无序关联）的实现原理及使用情形

(1) vector：可变数组大小。支持快速随机访问。在尾部之外的位置插入或删除元素可能很慢。

(2) deque：双端队列。支持快速随机访问。在头尾插入或删除速度很快。

(3) list：双向链表。只支持双向顺序访问。在list的任何位置进行插入或删除操作速度都很快。

(4) set/multiset：只有键值, 可以把set当做集合使用。multiset可以存放相同的元素, set只能存放不同的元素。

(5) map/multimap：键值对, 每一个元素都是pair, pair的第一个元素是关键字, 第二个元素是值。这两者的区别就在于multimap可以存放多个相同的关键字, map则不可以。

(3)与(5)的底层实现都是红黑树, 动态平衡二叉树。插入和删除等操作的时间复杂度是 $O(\log n)$

(6)中的底层实现是哈希函数。

(6) unordered_map 映射

unordered_multimap 多重映射

unordered_set 集合

unordered_multiset 多重集合

313.什么是STL？

六大组件：容器、迭代器、适配器、算法、函数对象、配置器（透明）

(1) 容器（略，自己看）

(2) 迭代器：随机访问迭代器（Random Access Iterator）

双向迭代器（Bidirectional Iterator）

前向迭代器（Forward Iterator）

输入迭代器（Input Iterator）

输出迭代器（Output Iterator）

(3) 适配器就是Interface(接口), 对容器、迭代器和算法进行包装, 但其实质还是容器、迭代器和算法, 只是不依赖于具体的标准容器、迭代器和算法类型, 容器适配器可以理解为容器的模板, 迭代器适配器可理解为迭代器的模板, 算法适配器可理解为算法的模板。

常见的容器适配器有：stack、queue、priority_queue（不支持迭代器访问）

前面简要提到了适配器的概念, 适配器相当于提供了一个接口, 使得某些不适用于特定对象的方法可以被该对象所用, 适配器形象的功能图解如图所示, 图中, 容器或函数对象无法直接应用于算法, 因此, 必须有一种中间过渡机制来实现两者的匹配, 这就是适配器, 本质上, 适配器是使一事物的行为类似于另一事物的行为的一种机制。

(4) STL将算法库分为4组，前3个在algorithm头文件中描述，而第4个在numeric头文件中描述：

非修改式序列操作：不改变容器的内容，如find()、for_each()等。

修改式序列操作：可以修改容器中的内容，如transform()、random_shuffle()、copy等。

排序和相关操作：包括各种排序函数等，如sort()等。

通用数字运算：计算两个容器的内部乘积等。

(5) 函数对象是可以以函数方式与()结合使用的任意对象，包括：(functor-仿函数)

函数名；指向函数的指针；重载了()操作符的类对象（即定义了函数operator()()的类）。

(6) 一级配置器和二级配置器

空间配置器，就是用来配置、管理和释放空间的，给所有的容器包括算法提供生存空间。

作用：

(1) 提高代码复用率，功能模块化。

(2) 减少内存碎片问题。

(3) 提高内存分配的效率。

(4) 有内存不足时的应对措施。

(5) 隐藏实际中对存储空间的分配及释放细节，确保所有被分配的存储空间都最终获得释放。

(5) 考虑多线程状态。

考虑到小型区块可能导致的内存碎片问题，设置了两级空间配置器。分别为：一级空间配置器、二级空间配置器。当区块大于128字节，调用一级空间配置器；小于等于128字节，为了降低额外开销，用底层较复杂的二级空间配置器。

一级空间配置器

用malloc()、free()、realloc()等C函数执行内存配置、释放、重配置操作，并实现出类似的C++new_hanle的机制

二级空间配置器

SGI二级空间配置器的原理是：当区块小于128字节，则以内存池（memory pool）管理，回收时管理一个用户归还的空间，类似于哈希桶。每次配置一块内存，并维护对应的自由链表（free_list）。为了方便管理，SGI二级配置器会对齐到8个字节。（例：需要30字节的内存，自动调整到32字节）。维护16个free_lists，各自管理大小分别为

8,16,24,32,40,48,56,64,72,80,88,96,104,112,120,128字节。

314. 什么是智能指针？底层实现？

(1) C++11中引入了智能指针的概念，方便管理堆内存。使用普通指针，容易造成堆内存泄露（忘记释放），二次释放，程序发生异常时内存泄露等问题等，使用智能指针能更好的管理堆内存。

(2) 理解智能指针需要从下面三个层次：

从较浅的层面看，智能指针是利用了一种叫做RAII（资源获取即初始化）的技术对普通的指针进行封装，这使得智能指针实质是一个对象，行为表现的却像一个指针。

智能指针的作用是防止忘记调用delete释放内存和程序异常的进入catch块忘记释放内存。另外指针的释放时机也是非常考究的，多次释放同一个指针会造成程序崩溃，这些都可以通过智能指针来解决。

智能指针还有一个作用是把值语义转换成引用语义。

(3) 智能指针#include, unique_ptr, shared_ptr, weak_ptr (弱引用智能指针)。

(4) unique_ptr“唯一”拥有其所指对象，同一时刻只能有一个unique_ptr指向给定对象(通过禁止拷贝语义、只有移动语义来实现)。相比与原始指针unique_ptr用于其RAII的特性，使得在出现异常的情况下，动态资源能得到释放。unique_ptr指针本身的生命周期：从unique_ptr指针创建时开始，直到离开作用域。离开作用域时，若其指向对象，则将其所指对象销毁(默认使用delete操作符，用户可指定其他操作)。

(5) shared_ptr多个指针指向相同的对象。shared_ptr使用引用计数，每一个shared_ptr的拷贝都指向相同的内存。每使用他一次，内部的引用计数加1，每析构一次，内部的引用计数减1，减为0时，自动删除所指向的堆内存。shared_ptr内部的引用计数是线程安全的，但是对象的读取需要加锁。

(6) weak_ptr是为了配合shared_ptr而引入的一种智能指针，因为它不具有普通指针的行为，没有重载operator*和->，它的最大作用在于协助shared_ptr工作，像旁观者那样观测资源的使用情况。

315. 多进程与多线程之间的区别？（最好要了解透彻）

- 1) 进程数据是分开的:共享复杂，需要用IPC，同步简单；多线程共享进程数据：共享简单，同步复杂
- 2) 进程创建销毁、切换复杂，速度慢；线程创建销毁、切换简单，速度快
- 3) 进程占用内存多，CPU利用率低；线程占用内存少，CPU利用率高
- 4) 进程编程简单，调试简单；线程编程复杂，调试复杂
- 5) 进程间不会相互影响；线程一个线程挂掉将导致整个进程挂掉
- 6) 进程适应于多核、多机分布；线程适用于多核

线程所私有的：

线程id、寄存器的值、栈、线程的优先级和调度策略、线程的私有数据、信号屏蔽字、errno变量、

316. 什么是进程池和线程池？

在面向对象程序编程中，对象的创建与析构都是一个较为复杂的过程，较费时间，所以为了提高程序的运行效率尽可能减少创建和销毁对象的次数，特别是一些很耗资源的对象创建和销毁。

所以我们可以创建一个进程池(线程池)，预先放一些进程(线程)进去，要用的时候就直接调用，用完之后再吧进程归还给进程池，省下创建删除进程的时间，不过当然就需要额外的开销了。

利用线程池与进程池可以使管理进程与线程的工作交给系统管理，不需要程序员对里面的线程、进程进行管理。

以进程池为例

进程池是由服务器预先创建的一组子进程，这些子进程的数目在3~10个之间(当然这只是典型情况)。线程池中的线程数量应该和CPU数量差不多。

进程池中的所有子进程都运行着相同的代码，并具有相同的属性，比如优先级、PGID等。

当有新的任务来到时，主进程将通过某种方式选择进程池中的某一个子进程来为之服务。相比于动态创建子进程，选择一个已经存在的子进程的代价显得小得多。至于主进程选择哪个子进程来为新任务服务，则有两种方法：

主进程使用某种算法来主动选择子进程。最简单、最常用的算法是随机算法和Round Robin(轮流算法)。

主进程和所有子进程通过一个共享的工作队列来同步，子进程都睡眠在该工作队列上。当有新的任务到来时，主进程将任务添加到工作队列中。这将唤醒正在等待任务的子进程，不过只有一个子进程将获得新任务的“接管权”，它可以从工作队列中取出任务并执行之，而其他子进程将继续睡眠在工作队列上。

当选择好子进程后，主进程还需要使用某种通知机制来告诉目标子进程有新任务需要处理，并传递必要的数据。最简单的方式是，在父进程和子进程之间预先建立好一条管道，然后通过管道来实现所有的进程间通信。在父线程和子线程之间传递数据就要简单得多，因为我们可以把这些数据定义为全局，那么它们本身就是被所有线程共享的。

317.进程间的通信方式有哪些？如何实现的？

信号和信号量是不同的，它们虽然都可以用来同步和互斥，但是信号是使用信号处理器来进行的，信号量是使用P,V操作来实现的。

消息队列是比较高级的一种进程间通信方式，因为它真的是可以在进程间传送message，传送普通字符串也可以。

一个消息队列可以被多个进程所共享（IPC(Inter-Process Communication,进程间通信)就是在这个基础上进行的）；如果一个进程消息太多，一个消息队列放不下，也可以用多于一个的消息队列（不管管理可能会比较复杂）。共享消息队列的进程所发送的消息除了message本身外还有一个标志，这个标志可以指明该消息将由哪个进程或者哪类进程接受。每一个共享消息队列的进程针对这个队列也有自己的标志，可以用来申明自己的身份。

共享内存(shared memory)：共享内存就是映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但多个进程都可以访问。共享内存是最快的IPC方式，它是针对其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制，如信号量，配合使用，来实现进程间的同步和通信。

套接字(socket)：套接口也是一种进程间通信机制，与其他通信机制不同的是，它可用于不同机器间的进程通信

318. 简述linux中的同步与异步机制？

同步：

所谓同步，就是在发出一个功能调用时，在没有得到结果之前，该调用就不返回。也就是必须一件一件事做，等前一件做完了才能做下一件事。

例如普通B/S模式（同步）：提交请求->等待服务器处理->处理完毕返回 这个期间客户端浏览器不能干任何事

异步：

异步的概念和同步相对。当一个异步过程调用发出后，调用者不能立刻得到结果。实际处理这个调用的部件在完成后，通过状态、通知和回调来通知调用者。

例如 ajax请求（异步）：请求通过事件触发->服务器处理（这是浏览器仍然可以作其他事情）->处理完毕

319.简述阻塞与非阻塞？

阻塞：

阻塞调用是指调用结果返回之前，当前线程会被挂起（线程进入非可执行状态，在这个状态下，cpu不会给线程分配时间片，即线程暂停运行）。函数只有在得到结果之后才会返回。

有人也许会把阻塞调用和同步调用等同起来，实际上他是不同的。对于同步调用来说，很多时候当前线程还是激活的，只是从逻辑上当前函数没有返回，它还会抢占cpu去执行其他逻辑，也会主动检测io是否准备好。

非阻塞

非阻塞和阻塞的概念相对应，指在不能立刻得到结果之前，该函数不会阻塞当前线程，而会立刻返回。

再简单点理解就是：

1. 同步，就是我调用一个功能，该功能没有结束前，我死等结果。
2. 异步，就是我调用一个功能，不需要知道该功能结果，该功能有结果后通知我（回调通知）
3. 阻塞，就是调用我（函数），我（函数）没有接收完数据或者没有得到结果之前，我不会返回。
4. 非阻塞，就是调用我（函数），我（函数）立即返回，通过select通知调用者

同步IO和异步IO的区别就在于：数据拷贝的时候进程是否阻塞

阻塞IO和非阻塞IO的区别就在于：应用程序的调用是否立即返回

综上可知，同步和异步,阻塞和非阻塞,有些混用,其实它们完全不是一回事,而且它们修饰的对象也不相同。

320.简述Linux中的5种I/O模式？

1.阻塞I/O（blocking I/O）

2.非阻塞I/O（nonblocking I/O）

3. I/O复用(select 和poll，还有epoll)（I/O multiplexing）！！！！（必须搞懂，超终极容易遇到）
4. 信号驱动I/O（signal driven I/O (SIGIO)）
5. 异步I/O（asynchronous I/O (the POSIX aio_functions)）

其中前4种都是同步，最后一种才是异步。

详情见：<https://www.cnblogs.com/chaser24/p/6112071.html>

321. 什么是死锁？四个死锁的条件？避免死锁的方法？

死锁是指两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法推进下去。

产生原因：竞争资源，和进程推进顺序非法

四个条件：

- 1) 互斥条件：指进程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。如果此时还有其它进程请求资源，则请求者只能等待，直至占有资源的进程用毕释放。
- 2) 请求和保持条件：指进程已经保持至少一个资源，但又提出了新的资源请求，而该资源已被其它进程占有，此时请求进程阻塞，但又对自己已获得的其它资源保持不放。
- 3) 不剥夺条件：指进程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。
- 4) 环路等待条件：指在发生死锁时，必然存在一个进程——资源的环形链，即进程集合{P0，P1，P2，…，Pn}中的P0正在等待一个P1占用的资源；P1正在等待P2占用的资源，……，Pn正在等待已被P0占用的资源。

四种解决办法：预防（破坏死锁发四种发生条件中的一个或多个）、避免（银行家算法：如果一个进程增加的资源请求会导致死锁，则不允许此分配，记住当时算的那张矩阵图）、检测与解除

322. Linux的任务调度机制是什么？

[Linux](#) 分实时进程和普通进程，实时进程应该先于普通进程而运行。实时进程：

- 1) FIFO(先来先服务调度)
- 2) RR (时间片轮转调度)。

每个进程有两个优先级（动态优先级和实时优先级），实时优先级就是用来衡量实时进程是否值得运行的。非实时进程有两种优先级，一种是静态优先级，另一种是动态优先级。实时进程又增加了第三种优先级，实时优先级。优先级越高，得到CPU时间的机会也就越大。

323.标准库函数与系统调用的区别？

系统调用：是操作系统为用户态运行的进程和硬件设备(如CPU、磁盘、打印机等)进行交互提供的一组接口，即就是设置在应用程序和硬件设备之间的一个接口层。linux内核是单内核，结构紧凑，执行速度快，各个模块之间是直接调用的关系。linux系统上到下依次是用户进程->linux内核->硬件。其中系统调用接口是位于Linux内核中的，整个linux系统从上到下可以是：用户进程->系统调用接口->linux内核子系统->硬件，也就是说Linux内核包括了系统调用接口和内核子系统两部分；或者从下到上可以是：物理硬件->OS内核->OS服务->应用程序，操作系统起到“承上启下”作用，向下管理物理硬件，向上为操作系统服务和应用程序提供接口，这里的接口就是系统调用了。

库函数：把函数放到库里。是把一些常用到的函数编完放到一个lib文件里，供别人用。别人用的时候把它所在的文件名用#include<>加到里面就可以了。一类是c语言标准规定的库函数，一类是编译器特定的库函数。

系统调用是为了方便使用操作系统的接口，而库函数则是为了人们编程的方便。

324. 分别简述三次握手与四次挥手的过程？

三次握手：C----->SYN K

S----->ACK K+1 SYN J

C----->ACK J+1

DONE!

client 的 connect 引起3次握手

server 在socket，bind，listen后，阻塞在accept，三次握手完成后，accept返回一个fd，

325. tcp和udp之间的区别？

- 1) 基于连接与无连接
- 2) 对系统资源的要求（TCP较多，UDP少）
- 3) UDP程序结构较简单
- 4) 流模式与数据报模式
- 5) TCP保证数据正确性，UDP可能丢包，TCP保证数据顺序，UDP不保证
- 6) TCP有拥塞控制和流量控制，UDP没有

TCP提供的是面向连接、可靠的字节流服务。当客户和服务器彼此交换数据前，必须先在双方之间建立一个TCP连接，之后才能传输数据。TCP提供超时重发，丢弃重复数据，检验数据，流量控制等功能，保证数据能从一端传到另一端。

是一个简单的面向数据报的传输层协议。UDP不提供可靠性，它只是把应用程序传给IP层的数据报发送出去，但是并不能保证它们能到达目的地。由于UDP在传输数据报前不用在客户和服务器之间建立一个连接，且没有超时重发等机制，故而传输速度很快

326. select、poll、epoll之间的区别？

<https://www.cnblogs.com/Anker/p/3265058.html> (参考阅读)

327. epoll有哪些触发模式？

(必须非常详尽的解释水平触发和边缘触发的区别，以及边缘触发在编程中要做哪些更多的确认)

注意：**epoll必须深入理解，必须要张口就来，必须随心所欲说出来。**

epoll有EPOLLLT和EPOLLET两种触发模式，LT是默认的模式，ET是“高速”模式。LT模式下，只要这个fd还有数据可读，每次epoll_wait都会返回它的事件，提醒用户程序去操作，而在ET（边缘触发）模式中，它只会提示一次，直到下次再有数据流入之前都不会再提示了，无论fd中是否还有数据可读。所以在ET模式下，read一个fd的时候一定要把它的buffer读光，也就是说一直读到read的返回值小于请求值。

也就是说在LT模式的情况下一定要确认收发的数据包的buffer是不是足够大如果收发数据包大小大于buffer的大小的时候就可能会出现数据丢失的情况。

328. 若是有大规模的数据连接，并发模型如何设计？

Epoll+线程池（epoll可以采用libevent处理）

原文作者：[CTHON](#)

原文链接：<https://www.cnblogs.com/cthon/p/9614736.html>

329.C和C++的区别

- 1) C是面向过程的语言，是一个结构化的语言，考虑如何通过一个过程对输入进行处理得到输出；C++是面向对象的语言，主要特征是“封装、继承和多态”。封装隐藏了实现细节，使得代码模块化；派生类可以继承父类的数据和方法，扩展了已经存在的模块，实现了代码重用；多态则是“一个接口，多种实现”，通过派生类重写父类的虚函数，实现了接口的重用。
- 2) C和C++动态管理内存的方法不一样，C是使用malloc/free，而C++除此之外还有new/delete关键字。
- 3) C++中有引用，C中不存在引用的概念

330.C++中指针和引用的区别

- 1) 指针是一个新的变量，存储了另一个变量的地址，我们可以通过访问这个地址来修改另一个变量；引用只是一个别名，还是变量本身，对引用的任何操作就是对变量本身进行操作，以达到修改变量的目的
- 2) 引用只有一级，而指针可以有多个
- 3) 指针传参的时候，还是值传递，指针本身的值不可以修改，需要通过解引用才能对指向的对象进行操作

331.结构体struct和共同体union（联合）的区别

结构体：将不同类型的数据组合成一个整体，是自定义类型

共同体：不同类型的几个变量共同占用一段内存

1) 结构体中的每个成员都有自己独立的地址，它们是同时存在的；

共同体中的所有成员占用同一段内存，它们不能同时存在；

2) sizeof(struct)是内存对齐后所有成员长度的总和，sizeof(union)是内存对齐后最长数据成员的长度、

结构体为什么要内存对齐呢？

1.平台原因（移植原因）：不是所有的硬件平台都能访问任意地址上的任意数据，某些硬件平台只能在某些地址处取某些特定类型的数据，否则抛出硬件异常

2.硬件原因：经过内存对齐之后，CPU的内存访问速度大大提升。

332.#define和const的区别

1) #define定义的常量没有类型，所给出的是一个立即数；const定义的常量有类型名字，存放在静态区域

2) 处理阶段不同，#define定义的宏变量在预处理时进行替换，可能有多个拷贝，const所定义的变量在编译时确定其值，只有一个拷贝。

3) #define定义的常量是不可以用指针去指向，const定义的常量可以用指针去指向该常量的地址

4) #define可以定义简单的函数，const不可以定义函数

333.重载overload，覆盖（重写）override，隐藏（重定义）overwrite，这三者之间的区别

1) overload，将语义相近的几个函数用同一个名字表示，但是参数列表（参数的类型，个数，顺序不同）不同，这就是函数重载，返回值类型可以不同

特征：相同范围（同一个类中）、函数名字相同、参数不同、virtual关键字可有可无

2) override，派生类覆盖基类的虚函数，实现接口的重用，返回值类型必须相同

特征：不同范围（基类和派生类）、函数名字相同、参数相同、基类中必须有virtual关键字（必须是虚函数）

3) overwrite，派生类屏蔽了其同名的基类函数，返回值类型可以不同

特征：不同范围（基类和派生类）、函数名字相同、参数不同或者参数相同且无virtual关键字

334.new、delete、malloc、free之间的关系

new/delete,malloc/free都是动态分配内存的方式

1) malloc对开辟的空间大小严格指定，而new只需要对象名

2) new为对象分配空间时，调用对象的构造函数，delete调用对象的析构函数

既然有了malloc/free，C++中为什么还需要new/delete呢？

运算符是语言自身的特性，有固定的语义，编译器知道意味着什么，由编译器解释语义，生成相应的代码。

库函数是依赖于库的，一定程度上独立于语言的。编译器不关心库函数的作用，只保证编译，调用函数参数和返回值符合语法，生成call函数的代码。

malloc/free是库函数，new/delete是C++运算符。对于非内部数据类型而言，光用malloc/free无法满足动态对象都要求。new/delete是运算符，编译器保证调用构造和析构函数对对象进行初始化/析构。但是库函数malloc/free是库函数，不会执行构造/析构。

335.delete和delete[]的区别

delete只会调用一次析构函数，而delete[]会调用每个成员的析构函数

用new分配的内存用delete释放，用new[]分配的内存用delete[]释放

一.构造函数

构造函数是和类名相同的一个函数，它的作用是实现对象的初始化。当对象被创建时，构造函数自动被调用。

特点：

没有类型

没有返回值（也不用写void）

名字与类名相同

可重载！

作用：完成类的对象的初始化

Cdate d; //定义对象d

注意：当对象d被创建时，会自动调用构造函数 d.Cdate()。

当类中未定义构造函数时，编译器会自动假设存在以下两个默认构造函数：(此构造函数什么都不做，就是个形式)。如果作者自己定义了构造函数，则默认的构造函数不会存在。

```
//默认构造函数一
```

```
Cdate::Cdate()
```

```
{
```

```
}
```

```
//默认构造函数二
```

```
Cdate::Cdate(const Cdate& a)
```

```
{
```

```
}
```

三.析构函数

我们已经知道构造函数是在创建对象时，对其进行初始化。而析构函数与其相反，是在对象被删除前象由系统自动执行它做清理工作。

作为一个类，可能有多多个对象，每个对象生命结束时都要调用析构函数，且每个对象调用一次。

特点：

无类型

无返回值

名字与类名相同

不带参数，不可重载，析构函数只有一个！

析构函数前“~” (取反符，表示逆构造函数)

作用：在对象被删除前做清理工作。

注意：对象的析构函数在对象被销毁前被调用，对象何时销毁也与其作用域相关。

例如，全局对象是在程序运行结束时销毁；

自动对象是在离开其作用域时销毁；

而动态对象是在使用delete运算符时销毁。

析构函数特别适用于当一个对象被动态分配内存空间，而在对象被销毁前希望释放它所占用的内存空间的时候。我们不会忽略初始化的重要性，却常常忽略清除的重要性，然而对销毁变量的内存清理是非常重要的。

例如，我们在堆中申请了一些内存，如果没有用完就释放，会造成内存泄露，会导致应用程序运行效率降低，甚至崩溃，不可掉以轻心。

而在c++中提供有析构函数，可以保证对象清除工作自动执行。

析构与构造的调用次序相反，即最先构造的最后被析构，最后构造的最先被析构。

336.虚函数、纯虚函数

虚函数：虚函数是C++中用于实现多态(polymorphism)的机制。核心理念就是通过基类访问派生类定义的函数,是C++中多态性的一个重要体现。利用基类指针访问派生类中的虚函数，这种情况下采用的是动态绑定技术。

纯虚函数：纯虚函数是在基类中声明的虚函数，它在基类中没有定义，但要求任何派生类都要定义自己的实现方法。在基类中实现纯虚函数的方法是在函数原型后加“=0”。纯虚函数不能实例化对象。

抽象类的介绍

抽象类是一种特殊的类，它是为了抽象和设计的目的为建立的，它处于继承层次结构的较上层。

(1) 抽象类的定义：称带有纯虚函数的类为抽象类。

(2) 抽象类的作用：抽象类的主要作用是将有关的操作作为结果接口组织在一个继承层次结构中，由它来为派生类提供一个公共的根，派生类将具体实现在其基类中作为接口的操作。所以派生类实际上刻画了一组子类的操作接口的通用语义，这些语义也传给子类，子类可以具体实现这些语义，也可以再将这些语义传给自己的子类。

(3) 使用抽象类时注意：

抽象类只能作为基类来使用，其纯虚函数的实现由派生类给出。如果派生类中没有重新定义纯虚函数，而只是继承基类的纯虚函数，则这个派生类仍然还是一个抽象类。如果派生类中给出了基类纯虚函数的实现，则该派生类就不再是抽象类了，它是一个可以建立对象的具体类。

抽象类是不能定义对象的。

总结：

1、纯虚函数声明如下：virtual void fuction1()=0; 纯虚函数一定没有定义，纯虚函数用来规范派生类的行为，即接口。包含纯虚函数的类是抽象类，抽象类不能定义实例，但可以声明指向实现该抽象类的具体类的指针或引用。

2、虚函数声明如下：virtual Return Type FunctionName(Parameter) 虚函数必须实现，如果不实现，编译器将报错，错误提示为：

3、对于虚函数来说，父类和子类都有各自的版本。由多态方式调用的时候动态绑定。

4、实现了纯虚函数的子类，该纯虚函数在子类中就编程了虚函数，子类的子类即孙子类可以覆盖该虚函数，由多态方式调用的时候动态绑定。

5、虚函数是C++中用于实现多态(polymorphism)的机制。核心理念就是通过基类访问派生类定义的函数。

6、在有动态分配堆上内存的时候，析构函数必须是虚函数，但没有必要是纯虚的。

纯虚函数的引入，是出于两个目的：

1、为了安全，因为避免任何需要明确但是因为不小心而导致的未知的结果，提醒子类去做应做的实现。

2、为了效率，不是程序执行的效率，而是为了编码的效率。

动态绑定：基类指针是调用派生类的中的成员函数还是调用基类中的成员函数要到程序运行时确定。主要看此时基类指针所指向的对象。这里要涉及一些很重要的概念，也是我最近看完Effective C++才明白的东西，记录下来。这些概念就是静态类型和动态类型，静态绑定和动态绑定。静态绑定和动态绑定。静态绑定是说前期绑定。所谓对象的静态类型，就是它在程序中被声明的时候采用的类型。考虑下面的class继承体系：

```
class Shape{

    virtual void draw(color = Red) const=0;

    ...

    ...

};

class Rectangle:public Shape{

    virtual void draw(color = Red) const;

    ...

    ...

};

class Circle:public Shape

{

    virtual void draw(color = Red) const;
```

```
...
```

```
...
```

```
};
```

现在考虑以下这些指针：

```
Shape* ps;//静态类型为Shape*
```

```
Shape*pc =new Circle;//静态类型Shape*
```

```
Shape*pr = new Rectangle;//静态类型Shape
```

在本例中，ps,pc,pr都被声明为Shape类型的，所以它们的静态类型都是Shape。注意：无论它们真正指向什么，它们的静态类型都是Shape。所谓的对象的动态类型是指“当前所指对象的类型”。也就是说，动态类型可以表现出一个对象将会有有什么行为。根据上面的例子，pc的动态类型是Circle，pr的动态类型是Rectangle*。ps没有动态类型，因为它没有指向任何对象。动态类型一如其名所示，可以在执行过程中改变（通常是经过赋值运算）：

```
ps=pc; \\ps的动态类型如今是Circle*
```

```
ps=pr; \\ps的动态类型如今是Rectangle*
```

Virtual函数系动态绑定而来，意思是调用一个virtual函数的时候，究竟调用的是哪一个函数代码，取决于发出调用的那个对象的动态类型。

```
ps->draw(); \\调用的是Rectangle::draw(Red)
```

337.STL库用过吗？常见的STL容器有哪些？算法用过几个？

STL包括两部分内容：容器和算法

容器即存放数据的地方，比如array, vector，分为两类，序列式容器和关联式容器

序列式容器，其中的元素不一定有序，但是都可以被排序，比如vector,list,queue,stack，heap, priority-queue, slist

关联式容器，内部结构是一个平衡二叉树，每个元素都有一个键值和一个实值，比如map, set, hashtable, hash_set

算法有排序，复制等，以及各个容器特定的算法

迭代器是STL的精髓，迭代器提供了一种方法，使得它能够按照顺序访问某个容器所含的各个元素，但无需暴露该容器的内部结构，它将容器和算法分开，让二者独立设计。

Vector是顺序容器，是一个动态数组，支持随机存取、插入、删除、查找等操作，在内存中是一块连续的空间。在原有空间不够情况下自动分配空间，增加为原来的两倍。vector随机存取效率高，但是在vector插入元素，需要移动的数目多，效率低下。

注意：vector动态增加大小时，并不是在原空间之后持续新空间（因为无法保证原空间之后尚有可供配置的空间），而是以原大小的两倍另外配置一块较大的空间，然后将原内容拷贝过来，然后才开始在原内容之后构造新元素，并释放原空间。因此，对vector的任何操作，一旦引起空间重新配置，指向原vector的所有迭代器就都失效了。

二叉查找树

要想了解二叉查找树，我们首先看下二叉查找树有哪些特性呢？

- 1，左子树上所有的节点的值均小于或等于他的根节点的值
- 2，右子数上所有的节点的值均大于或等于他的根节点的值
- 3，左右子树也一定分别为二叉排序树

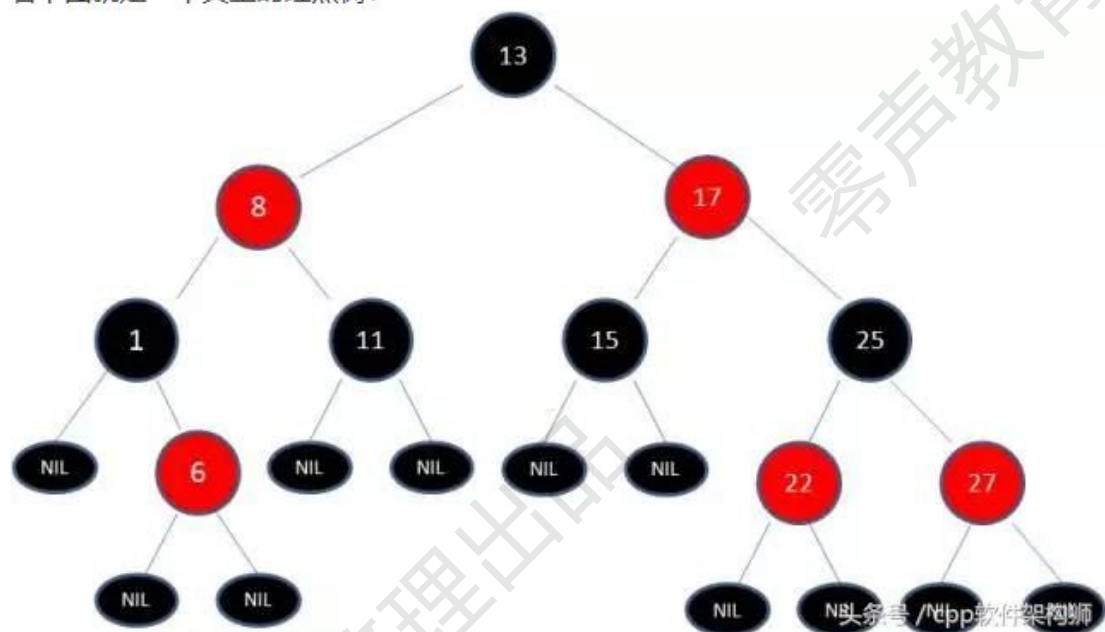
红黑树

红黑树就是一种平衡的二叉查找树，说他平衡的意思是他不会变成“瘸子”，左腿特别长或者右腿特别长。除了符合二叉查找树的特性之外，还具体下列的特性：

1. 节点是红色或者黑色
2. 根节点是黑色
3. 每个叶子的节点都是黑色的空节点（NULL）
4. 每个红色节点的两个子节点都是黑色的。
5. 从任意节点到其每个叶子的所有路径都包含相同的黑色节点。

看下图就是一个典型的红黑树：

看下图就是一个典型的红黑树：



很多童鞋又会惊讶了，天啊这个条条框框也太多了吧。没错，正式因为这些规则，才能保证红黑树的自平衡。最长路径不超过最短路径的2倍。

当插入和删除节点，就会对平衡造成破坏，这时候需要对树进行调整，从而重新达到平衡。那什么情况下会破坏红黑树的规则呢？

有两种方式：变色和旋转。

338.const知道吗？解释一下其作用

const修饰类的成员变量，表示常量不可能被修改

const修饰类的成员函数，表示该函数不会修改类中的数据成员，不会调用其他非const的成员函数

const函数只能调用const函数，非const函数可以调用const函数

339.虚函数是怎么实现的

每一个含有虚函数的类都至少有一个与之对应的虚函数表，其中存放着该类所有虚函数对应的函数指针（地址），

类的示例对象不包含虚函数表，只有虚指针；

派生类会生成一个兼容基类的虚函数表。

340.堆和栈的区别

1) 栈 stack 存放函数的参数值、局部变量，由编译器自动分配释放

堆heap，是由new分配的内存块，由应用程序控制，需要程序员手动利用delete释放，如果没有，程序结束后，操作系统自动回收

2) 因为堆的分配需要使用频繁的新/new/delete，造成内存空间的不连续，会有大量的碎片

3) 对于堆来讲, 生长方向是向上的, 也就是向着内存地址增加的方向; 对于栈来讲, 它的生长方式是向下的, 是向着内存地址减小的方向增长。

C++内存区域分为5个区域。分别是堆, 栈, 自由存储区, 全局/静态存储区和常量存储区。

栈: 由编译器在需要的时候分配, 在不需要的时候自动清除的变量存储区。里面通常是局部变量, 函数参数等。

堆: 由new分配的内存块, 他们的释放编译器不去管, 由我们的应用程序去控制, 一般一个new对应一个delete。如果程序员没有释放掉, 那么在程序结束后, 操作系统会自动回收。

自由存储区: 由malloc等分配的内存块, 和堆十分相似, 不过它使用free来结束自己的生命。

全局/静态存储区: 全局变量和静态变量被分配到同一块内存中, 在以前的c语言中。全局变量又分为初始化的和未初始化的, 在c++里面没有这个区分了, 他们共同占用同一块内存。

常量存储区: 这是一块比较特殊的存储区, 里面存放的是常量, 不允许修改。

C++内存区域中堆和栈的区别:

管理方式不同: 栈是由编译器自动管理, 无需我们手工控制; 对于堆来说, 释放由程序员完成, 容易产生内存泄漏。

空间大小不同: 一般来讲, 在32为系统下面, 堆内存可达到4G的空间, 从这个角度来看堆内存几乎是没有什么限制的。但是对于栈来讲, 一般都是有一定空间大小的, 例如, 在vc6下面, 默认的栈大小好像是1M。当然, 也可以自己修改: 打开工程。 project-->setting-->link, 在category中选中output, 然后再reserve中设定堆栈的最大值和 commit。

能否产生碎片: 对于堆来讲, 频繁的new/delete势必会造成内存空间的不连续, 从而造成大量的碎片, 使程序效率降低。对于栈来讲, 则不会存在这个问题。

生长方向不同: 对于堆来讲, 生长方向是向上的, 也就是向着内存地址增加的方向; 对于栈来讲, 它的生长方式是向下的, 是向着内存地址减小的方向增长。

分配方式不同: 堆都是动态分配的; 栈有静态和动态两种分配方式。静态分配由编译器完成, 比如局部变量的分配。动态分配由malloca函数进行、但栈的动态分配和堆是不同的, 它的动态分配由编译器进行释放, 无需我们手工实现。

分配效率不同: 栈是机器系统提供的数据结构, 计算机会在底层对栈提供支持: 分配专门的寄存器存放栈的地址, 压栈出栈都有专门的指令执行, 这就决定了栈的效率比较高。堆则是c/c++库函数提供的, 机制很复杂。库函数会按照一定的算法进行分配。显然, 堆的效率比栈要低得多。

进程内存中的映像, 主要有代码区, 堆(动态存储区, new/delete的动态数据), 栈, 静态存储区

341.关键字static的作用

1) 函数体内: static 修饰的局部变量作用范围为该函数体, 不同于auto变量, 其内存只被分配一次, 因此其值在下次调用的时候维持了上次的值

2) 模块内: static修饰全局变量或全局函数, 可以被模块内的所有函数访问, 但是不能被模块外的其他函数访问, 使用范围限制在声明它的模块内

3) 类中: 修饰成员变量, 表示该变量属于整个类所有, 对类的所有对象只有一份拷贝

4) 类中: 修饰成员函数, 表示该函数属于整个类所有, 不接受this指针, 只能访问类中的static成员变量

注意和const的区别!!! const强调值不能被修改, 而static强调唯一的拷贝, 对所有类的对象

342.STL中map和set的原理（关联式容器）

map和set的底层实现主要通过红黑树来实现

343.#include<file.h> #include "file.h" 的区别

前者是从标准库路径寻找

后者是从当前工作路径

344.什么是内存泄漏？面对内存泄漏和指针越界，你有哪些方法？

动态分配内存所开辟的空间，在使用完毕后未手动释放，导致一直占据该内存，即为内存泄漏。

方法：malloc/free要配套，对指针赋值的时候应该注意被赋值的指针是否需要释放；使用的时候记得指针的长度，防止越界

345.定义和声明的区别

声明是告诉编译器变量的类型和名字，不会为变量分配空间

定义需要分配空间，同一个变量可以被声明多次，但是只能被定义一次

346.C++文件编译与执行的四个阶段

1) 预处理：根据文件中的预处理指令来修改源文件的内容

2) 编译：编译成汇编代码

3) 汇编：把汇编代码翻译成目标机器指令

4) 链接：链接目标代码生成可执行程序

18、STL中的vector的实现，是怎么扩容的？

vector使用的注意点及其原因，频繁对vector调用push_back()对性能的影响和原因。

vector就是一个动态增长的数组，里面有一个指针指向一片连续的空间，当空间装不下的时候，会申请一片更大的空间，将原来的数据拷贝过去，并释放原来的旧空间。当删除的时候空间并不会被释放，只是清空了里面的数据。对比array是静态空间一旦配置了就不能改变大小。

vector的动态增加大小的时候，并不是在原有的空间上持续新的空间（无法保证原空间的后面还有可供配置的空间），而是以原大小的两倍另外配置一块较大的空间，然后将原内容拷贝过来，并释放原空间。在VS下是1.5倍扩容，在GCC下是2倍扩容。

347.STL中unordered_map和map的区别

map是STL中的一个关联容器，提供键值对的数据管理。底层通过红黑树来实现，实际上是二叉排序树和非严格意义上的二叉平衡树。所以在map内部所有的数据都是有序的，且map的查询、插入、删除操作的时间复杂度都是 $O(\log N)$ 。

unordered_map和map类似，都是存储key-value对，可以通过key快速索引到value，不同的是unordered_map不会根据key进行排序。unordered_map底层是一个防冗余的哈希表，存储时根据key的hash值判断元素是否相同，即unordered_map内部是无序的。

348.C++的内存管理

栈：存放函数的参数和局部变量，编译器自动分配和释放

堆：new关键字动态分配的内存，由程序员手动进行释放，否则程序结束后，由操作系统自动进行回收

自由存储区：由malloc分配的内存，和堆十分相似，由对应的free进行释放

全局/静态存储区：存放全局变量和静态变量

常量区：存放常量，不允许被修改

349.构造函数为什么一般不定义为虚函数？而析构造函数一般写成虚函数的原因？

1、构造函数不能声明为虚函数

1) 因为创建一个对象时需要确定对象的类型，而虚函数是在运行时确定其类型的。而在构造一个对象时，由于对象还未创建成功，编译器无法知道对象的实际类型，是类本身还是类的派生类等等

2) 虚函数的调用需要虚函数表指针，而该指针存放在对象的内存空间中；若构造函数声明为虚函数，那么由于对象还未创建，还没有内存空间，更没有虚函数表地址用来调用虚函数即构造函数了

2、析构造函数最好声明为虚函数

首先析构造函数可以为虚函数，当析构一个指向派生类的基类指针时，最好将基类的析构造函数声明为虚函数，否则可以存在内存泄露的问题。

如果析构造函数不被声明成虚函数，则编译器实施静态绑定，在删除指向派生类的基类指针时，只会调用基类的析构造函数而不调用派生类析构造函数，这样就会造成派生类对象析构不完全。

子类析构时，要调用父类的析构造函数吗？

析构造函数调用的次序时先派生类后基类的。和构造函数的执行顺序相反。并且析构造函数要是virtual的，否则如果用父类的指针指向子类对象的时候，析构造函数静态绑定，不会调用子类的析构。

不用显式调用，会自动调用

350.静态绑定和动态绑定的介绍

静态绑定和动态绑定是C++多态性的一种特性

1) 对象的静态类型和动态类型

静态类型：对象在声明时采用的类型，在编译时确定

动态类型：当前对象所指的类型，在运行期决定，对象的动态类型可变，静态类型无法更改

2) 静态绑定和动态绑定

静态绑定：绑定的是对象的静态类型，函数依赖于对象的静态类型，在编译期确定

动态绑定：绑定的是对象的动态类型，函数依赖于对象的动态类型，在运行期确定

只有虚函数才使用的是动态绑定，其他的全部是静态绑定

351.引用是否能实现动态绑定，为什么引用可以实现

可以。因为引用（或指针）既可以指向基类对象也可以指向派生类对象，这一事实是动态绑定的关键。用引用（或指针）调用的虚函数在运行时确定，被调用的函数是引用（或指针）所指的对象的实际类型所定义的。

352.深拷贝和浅拷贝的区别

深拷贝和浅拷贝可以简单的理解为：如果一个类拥有资源，当这个类的对象发生复制过程的时候，如果资源重新分配了就是深拷贝；反之没有重新分配资源，就是浅拷贝。

353.什么情况下会调用拷贝构造函数（三种情况）

系统自动生成的构造函数：普通构造函数和拷贝构造函数（在没有定义对应的构造函数的时候）

生成一个实例化的对象会调用一次普通构造函数，而用一个对象去实例化一个新的对象所调用的就是拷贝构造函数

调用拷贝构造函数的情形：

- 1) 用类的一个对象去初始化另一个对象的时候
- 2) 当函数的参数是类的对象时，就是值传递的时候，如果是引用传递则不会调用
- 3) 当函数的返回值是类的对象或者引用的时候

354.C++的四种强制转换

类型转化机制可以分为隐式类型转换和显示类型转化（强制类型转换）

(new-type) expression

new-type (expression)

隐式类型转换比较常见，在混合类型表达式中经常发生；四种强制类型转换操作符：

static_cast、dynamic_cast、const_cast、reinterpret_cast

- 1) static_cast：编译时期的静态类型检查

static_cast < type-id > (expression)

该运算符把expression转换成type-id类型，在编译时使用类型信息执行转换，在转换时执行必要的检测（指针越界、类型检查），其操作数相对是安全的

- 2) dynamic_cast：运行时的检查

用于在集成体系中进行安全的向下转换downcast，即基类指针/引用->派生类指针/引用

dynamic_cast是4个转换中唯一的RTTI操作符，提供运行时类型检查。

dynamic_cast如果不能转换返回NULL

dynamic_cast转为引用类型的时候转型失败会抛bad_cast

源类中必须要有虚函数，保证多态，才能使用dynamic_cast(expression)

- 3) const_cast

去除const常量属性，使其可以修改；volatile属性的转换

- 4) reinterpret_cast

通常为了将一种数据类型转换成另一种数据类型

355.调试程序的方法

windows下直接使用vs的debug功能

linux下直接使用gdb，我们可以在其过程中给程序添加断点，监视等辅助手段，监控其行为是否与我们设计相符

356.extern"C"作用

extern "C"的主要作用就是为了能够正确实现C++代码调用其他C语言代码。加上extern "C"后，会指示编译器这部分代码按C语言的进行编译，而不是C++的。

357.typdef和define区别

#define是预处理命令，在预处理是执行简单的替换，不做正确性的检查

typedef是在编译时处理的，它是在自己的作用域内给已经存在的类型一个别名

```
typedef (int*) pINT;
```

```
#define pINT2 int*
```

效果相同？实则不同！实践中见差别：pINT a,b;的效果同int *a; int *b;表示定义了两个整型指针变量。而pINT2 a,b;的效果同int *a, b;表示定义了一个整型指针变量a和整型变量b。

358.引用作为函数参数以及返回值的好处

对比值传递，引用传参的好处：

- 1) 在函数内部可以对此参数进行修改
- 2) 提高函数调用和运行的效率（所以没有了传值和生成副本的时间和空间消耗）

值传递：

形参是实参的拷贝，改变形参的值并不会影响外部实参的值。从被调用函数的角度来说，值传递是单向的（实参→形参），参数的值只能传入，

不能传出。当函数内部需要修改参数，并且不希望这个改变影响调用者时，采用值传递。

指针传递：

形参为指向实参地址的指针，当对形参的指向操作时，就相当于对实参本身进行的操作

引用传递：

形参相当于是实参的“别名”，对形参的操作其实就是对实参的操作，在引用传递过程中，被调函数的形式参数虽然也作为局部变量在栈中开辟了内存空间，但是这时存放的是由主调函数放进来的实参变量的地址。被调函数对形参的任何操作都被处理成间接寻址，即通过栈中存放的地址访问主调函数中的实参变量。正因为如此，被调函数对形参做的任何操作都影响了主调函数中的实参变量。

用引用作为返回值最大的好处就是在内存中不产生被返回值的副本。

但是有以下的限制：

- 1) 不能返回局部变量的引用。因为函数返回以后局部变量就会被销毁
- 2) 不能返回函数内部new分配的内存的引用。虽然不存在局部变量的被动销毁问题，可对于这种情况（返回函数内部new分配内存的引用），又面临其它尴尬局面。例如，被函数返回的引用只是作为一个临时变量出现，而没有被赋予一个实际的变量，那么这个引用所指向的空间（由new分配）就无法释放，造成memory leak
- 3) 可以返回类成员的引用，但是最好是const。因为如果其他对象可以获得该属性的非常量的引用，那么对该属性的单纯赋值就会破坏业务规则的完整性。

359.纯虚函数

纯虚函数是只有声明没有实现的虚函数，是对子类的约束，是接口继承

包含纯虚函数的类是抽象类，它不能被实例化，只有实现了这个纯虚函数的子类才能生成对象

360.什么是野指针

野指针不是NULL指针，是未初始化或者未清零的指针，它指向的内存地址不是程序员所期望的，可能指向了受限的内存。

成因：

- 1) 指针变量没有被初始化
- 2) 指针指向的内存被释放了，但是指针没有置NULL
- 3) 指针超过了变量的作用范围，比如b[10]，指针b+11

361.线程安全和线程不安全

线程安全就是多线程访问时，采用了加锁机制，当一个线程访问该类的某个数据时，进行保护，其他线程不能进行访问直到该线程读取完，其他线程才可以使用，不会出现数据不一致或者数据污染。

线程不安全就是不提供数据访问保护，有可能多个线程先后更改数据所得到的数据就是脏数据。

362.C++中内存泄漏的几种情况

内存泄漏是指动态分配的堆内存由于某种原因程序未释放或无法释放，造成系统内存的浪费，导致程序运行速度减慢甚至系统崩溃等严重后果。

- 1) 类的构造函数和析构函数中new和delete没有配套
- 2) 在释放对象数组时没有使用delete[]，使用了delete
- 3) 没有将基类的析构函数定义为虚函数，当基类指针指向子类对象时，如果基类的析构函数不是virtual，那么子类的析构函数将不会被调用，子类的资源没有正确释放，因此造成内存泄露
- 4) 没有正确的清楚嵌套的对象指针

363.栈溢出的原因以及解决方法

栈溢出是指函数中的局部变量造成的溢出（注：函数中形参和函数中的局部变量存放在栈上）

栈的大小通常是1M-2M,所以栈溢出包含两种情况，一是分配的的大小超过栈的最大值，二是分配的的大小没有超过最大值，但是接收的buf比原buf小。

- 1) 函数调用层次过深,每调用一次,函数的参数、局部变量等信息就压一次栈
- 2) 局部变量体积太大。

解决办法大致说来也有两种：

- 1> 增加栈内存的数目；如果是不超过栈大小但是分配值小的，就增大分配的大小
- 2> 使用堆内存；具体实现由很多种方法可以直接把数组定义改成指针,然后动态申请内存;也可以把局部变量变成全局变量,一个偷懒的办法是直接在定义前边加个static,呵呵,直接变成静态变量(实质就是全局变量)

364.C++标准库vector以及迭代器

每种容器类型都定义了自己的迭代器类型，每种容器都定义了一对命名为begin和end的函数，用于返回迭代器。

迭代器是容器的精髓，它提供了一种方法使得它能够按照顺序访问某个容器所含的各个元素，但无需暴露该容器的内部结构，它将容器和算法分开，让二者独立设计。

365.C++中vector和list的区别

vector和数组类似，拥有一段连续的内存空间。vector申请的是一段连续的内存，当插入新的元素内存不够时，通常以2倍重新申请更大的一块内存，将原来的元素拷贝过去，释放旧空间。因为内存空间是连续的，所以在进行插入和删除操作时，会造成内存块的拷贝，时间复杂度为 $O(n)$ 。

list是由双向链表实现的，因此内存空间是不连续的。只能通过指针访问数据，所以list的随机存取非常没有效率，时间复杂度为 $O(n)$ ；但由于链表的特点，能高效地进行插入和删除。

vector拥有一段连续的内存空间，能很好的支持随机存取，因此vector::iterator支持“+”，“+=”，“<”等操作符。

list的内存空间可以是不连续，它不支持随机访问，因此list::iterator则不支持“+”、“+=”、“<”等

vector::iterator和list::iterator都重载了“++”运算符。

总之，如果需要高效的随机存取，而不在乎插入和删除的效率，使用vector；

如果需要大量的插入和删除，而不关心随机存取，则应使用list。

366.C++中的基本数据类型及派生类型

1) 整型 int

2) 浮点型 单精度float，双精度double

3) 字符型 char

4) 逻辑型 bool

5) 控制型 void

基本类型的字长及其取值范围可以放大和缩小，改变后的类型就叫做基本类型的派生类型。派生类型声明符由基本类型关键字char、int、float、double前面加上类型修饰符组成。

类型修饰符包括：

short 短类型，缩短字长

long 长类型，加长字长

signed 有符号类型，取值范围包括正负值

unsigned 无符号类型，取值范围只包括正值

367.友元函数和友元类

友元提供了不同类的成员函数之间、类的成员函数和一般函数之间进行数据共享的机制。

通过友元，另一个类中的成员函数可以访问类中的私有成员和保护成员。

友元的正确使用能提高程序的运行效率，但同时也破坏了类的封装性和数据的隐藏性，导致程序可维护性变差。

1) 友元函数

友元函数是可以访问类的私有成员的非成员函数。它是定义在类外的普通函数，不属于任何类，但是需要在类的定义中加以声明。

friend 类型 函数名(形式参数);

一个函数可以是多个类的友元函数，只需要在各个类中分别声明。

2) 友元类

友元类的所有成员函数都是另一个类的友元函数，都可以访问另一个类中的隐藏信息（包括私有成员和保护成员）。

friend class 类名;

使用友元类时注意：

(1) 友元关系不能被继承。

(2) 友元关系是单向的，不具有交换性。若类B是类A的友元，类A不一定是类B的友元，要看在类中是否有相应的声明。

(3) 友元关系不具有传递性。若类B是类A的友元，类C是B的友元，类C不一定是类A的友元，同样要看类中是否有相应的申明

368.c++函数库中一些实用的函数

1. __gcd(x, y)

求两个数的最大公约数，如__gcd(6, 8)就返回2。

2. reverse(a + 1, a + n + 1)

将数组中的元素反转。a 是数组名，n是长度，跟 sort 的用法一样。值得一提的是，对于字符型数组也同样适用。

3. unique(a + 1, a + n + 1)

去重函数。跟sort的用法一样。不过他返回的值是最后一个数的地址，所以要得到新的数组长度应该这么写：_n = unique(a + 1, a + n + 1) - a - 1.

4.lower_bound(a + 1, a + n + 1, x); upper_bound(a + 1, a + n + 1, x)

lower_bound是查找数组中第一个小于等于x的数，返回该地址，同理也是 pos = lower_bound(a + 1, a + n + 1, x) - a

upper_bound是查找第一个大于x的数，用法和lower_bound一样

复杂度是二分的复杂度，O(logn)。（其实就是代替了手写二分）

5.fill(a + 1, a + n + 1, x)

例如

int数组：fill(arr, arr + n, 要填入的内容);

vector也可以：fill(v.begin(), v.end(), 要填入的内容);

fill(vector.begin(), cnt, val); // 从当前起始点开始，将之后的cnt个元素赋值为val。

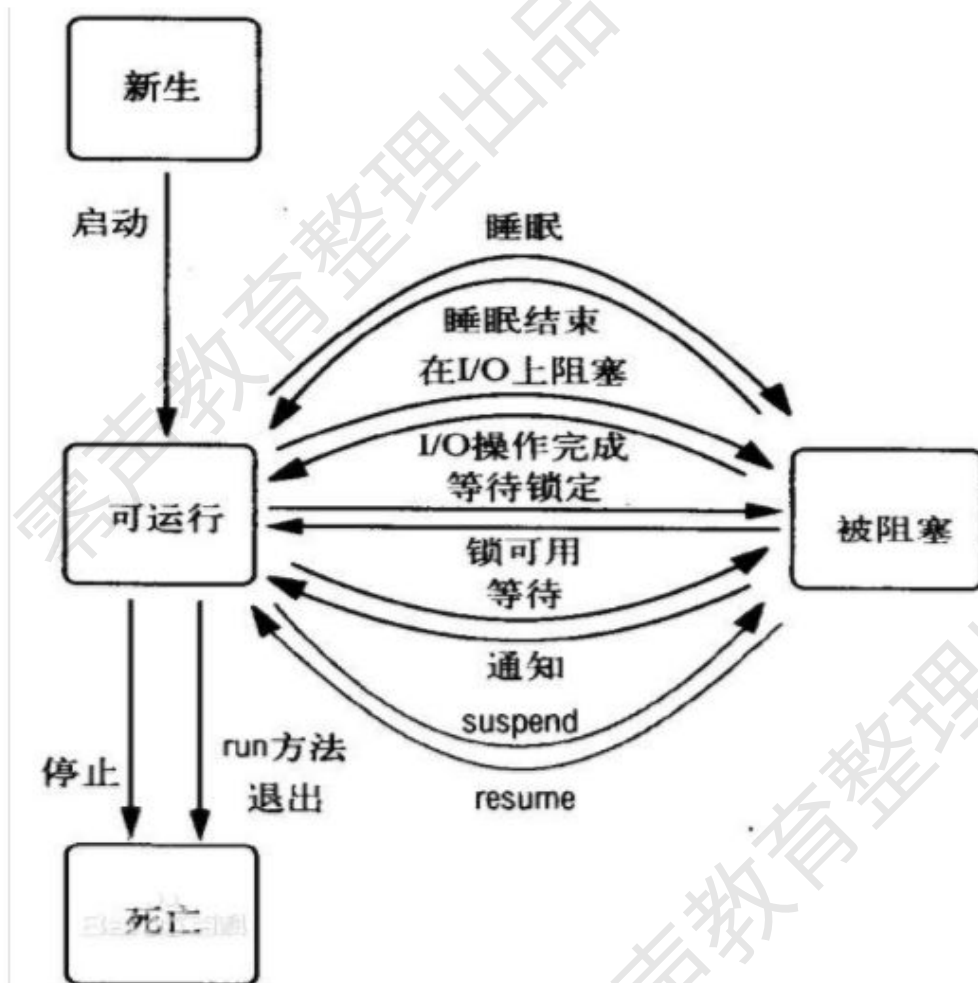
memset(arr, val, cnt); // 在头文件里。

将数组a中的每一个元素都赋成x，跟memset的区别是，memset函数按照字节填充，所以一般memset只能用来填充char型数组，（因为只有char型占一个字节）如果填充int型数组，除了0和-1，其他的不能。

369.线程的基本概念、线程的基本状态及状态之间的关系？

线程，有时称为轻量级进程，是CPU使用的基本单元；它由线程ID、程序计数器、寄存器集合和堆栈组成。它与属于同一进程的其他线程共享其代码段、数据段和其他操作系统资源（如打开文件和信号）。

线程有四种状态：新生状态、可运行状态、被阻塞状态、死亡状态。状态之间的转换如下图所示：



370.线程与进程的区别？

- 1、线程是进程的一部分，所以线程有的时候被称为是轻权进程或者轻量级进程。
- 2、一个没有线程的进程是可以被看作单线程的，如果一个进程内拥有多个进程，进程的执行过程不是一条线（线程）的，而是多条线（线程）共同完成的。
- 3、系统在运行的时候会为每个进程分配不同的内存区域，但是不会为线程分配内存（线程所使用的资源是它所属的进程的资源），线程组只能共享资源。那就是说，出了CPU之外（线程在运行的时候要占用CPU资源），计算机内部的软硬件资源的分配与线程无关，线程只能共享它所属进程的资源。

4、与进程的控制表PCB相似，线程也有自己的控制表TCB，但是TCB中所保存的线程状态比PCB表中少多了。

5、进程是系统所有资源分配时候的一个基本单位，拥有一个完整的虚拟空间地址，并不依赖线程而独立存在。

371.C++多线程有几种实现方法，都是什么？

```
#include  
#include <condition_variable>  
#include  
1 std::thread
```

关键点

a. C++ 11中创建线程非常简单，使用std::thread类就可以，thread类定义于thread头文件，构造thread对象时传入一个可调用对象作为参数（如果可调用对象有参数，把参数同时传入），这样构造完成后，新的线程马上被创建，同时执行该可调用对象；

b. 用std::thread默认的构造函数构造的对象不关联任何线程；判断一个thread对象是否关联某个线程，使用joinable()接口，如果返回true，表明该对象关联着某个线程（即使该线程已经执行结束）；

c. "joinable"的对象析构前，必须调用join()接口等待线程结束，或者调用detach()接口解除与线程的关联，否则会抛异常；

d. 正在执行的线程从关联的对象detach后会自主执行直至结束，对应的对象变成不关联任何线程的对象，joinable()将返回false

e. std::thread没有拷贝构造函数和拷贝赋值操作符，因此不支持复制操作（但是可以move），也就是说，没有两个std::thread对象会表示同一执行线程；

f. 容易知道，如下几种情况下，std::thread对象是不关联任何线程的（对这种对象调用join或detach接口会抛异常）：

默认构造的thread对象；

被移动后的thread对象；

detach 或 join 后的thread对象；

2 std::mutex (轻松实现互斥)

常做多线程编程的人一定对mutex（互斥）非常熟悉，C++ 11当然也支持mutex，通过mutex可以方便的对临界区域加锁，std::mutex类定义于mutex头文件，是用于保护共享数据避免从多个线程同时访问的同步原语。它提供了lock，try_lock,unlock等几个接口，功能如下：

调用方线程从成功调用lock()或try_lock()开始，到unlock()为止占有mutex对象

线程占有mutex时，所有其他线程若试图要求mutex的所有权，则将阻塞（对于lock的调用）或收到false返回值（对于try_lock）；

调用方线程在调用lock或try_lock前必须不占有mutex。

mutex和thread一样，不可复制（拷贝构造函数和拷贝赋值操作符都被删除），而且，mutex也不可移动；

备注

a.操作系统提供mutex可以设置属性，C++11根据mutex的属性提供四种的互斥量，分别是

std::mutex，最常用，普遍的互斥量（默认属性），

std::recursive_mutex，允许同一线程使用recursive_mutex多次加锁，然后使用相同次数的解锁操作解锁。mutex多次加锁会造成死锁

std::timed_mutex，在mutex上增加了时间的属性。增加了两个成员函数try_lock_for()，try_lock_until()，分别接收一个时间范围，再给定的时间内如果互斥量被锁主了，线程阻塞，超过时间，返回false。

std::recursive_timed_mutex，增加递归和时间属性

b.mutex成员函数加锁解锁

lock()，互斥量加锁，如果互斥量已被加锁，线程阻塞

bool try_lock()，尝试加锁，如果互斥量未被加锁，则执行加锁操作，返回true；如果互斥量已被加锁，返回false，线程不阻塞。

void unlock()，解锁互斥量

c. mutex RAII式的加锁解锁

std::lock_guard，管理mutex的类。对象构建时传入mutex，会自动对mutex加入，直到离开类的作用域，析构时完成解锁。RAII式的栈对象能保证在异常情形下mutex可以在lock_guard对象析构被解锁。

std::unique_lock与lock_guard功能类似，但是比lock_guard的功能更强大。比如std::unique_lock维护了互斥量的状态，可通过bool owns_lock()访问，当locked时返回true，否则返回false

3 std::lock_guard（有作用域的mutex，让程序更稳定，防止死锁）

很容易想到，mutex的lock和unlock必须成对调用，lock之后忘记调用unlock将是非常严重的错误，再次lock时会造成死锁。有时候一段程序中会有各种出口，如return，continue，break等等语句，在每个出口前记得unlock已经加锁的mutex是有一定负担的，而假如程序段中有抛异常的情况，就更为隐蔽棘手，C++ 11提供了更好的解决方案，对的，RAII，本系列文章多次提到RAII，想必大家应该不陌生。

类模板std::lock_guard是mutex封装器，通过便利的RAII机制在其作用域内占有mutex。

创建lock_guard对象时，它试图接收给定mutex的所有权。当程序流程离开创建lock_guard对象的作用域时，lock_guard对象被自动销毁并释放mutex，lock_guard类也是不可复制的。

一般，需要加锁的代码段，我们用{}括起来形成一个作用域，括号的开端创建lock_guard对象，把mutex对象作为参数传入lock_guard的构造函数即可，比如上面的例子加锁的部分，我们可以改写如下：

版权声明：本文为CSDN博主「Cpp编程小茶馆」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/xu_fu_yong/article/details/122948379

372.数据类型说明

数据类型	16位平台	32位平台	64位平台
char	1 字节	1 字节	1 字节
pointer	2 字节	4 字节	8 字节
short	2 字节	2 字节	2 字节
int	2 字节	4 字节	4 字节
float	4 字节	4 字节	4 字节
double	8 字节	8 字节	8 字节
long	4 字节	4 字节	8 字节
long long	8 字节	8 字节	8 字节

373.C和C++的区别

C是一个结构化语言。C程序的设计首要考虑的是如何通过一个过程，对输入进行运算处理得到输出。
C++在C的基础上增添类，首要考虑的是如何构造一个对象模型，所以C++是面向对象的语言。

374.封装、继承、多态

封装

封装是把过程和数据包围起来，对数据的访问只能通过已定义的函数或属性。把客观事物封装成抽象的类，并且类可以把自己的数据和方法只让可信的类或者对象操作，对不可信的进行信息隐藏。封装可以隐藏实现细节，使得代码模块化。

多态

多态指同一个实体同时具有多种形式。在基类的函数前加上virtual关键字，在派生类中重写该函数，运行时将会根据对象的实际类型来调用相应的函数。如果对象类型是派生类，就调用派生类的函数；如果对象类型是基类，就调用基类的函数。C++中，实现多态有以下方法：虚函数，抽象类，覆盖，模板，条件是要有重写，要有继承，父类指向子类。

继承

如果一个类别A“继承自”另一个类别B，就把这个A称为“B的子类”，而把B称为“A的父类”。继承可以使得子类具有父类的各种属性和方法，而不需要再次编写相同的代码。在令子类别继承父类别的同时，可以重新定义某些属性，并重写某些方法，即覆盖父类别的原有属性和方法，使其获得与父类别不同的功能，这就是多态。为子类别追加新的属性和方法也是常见的做法。

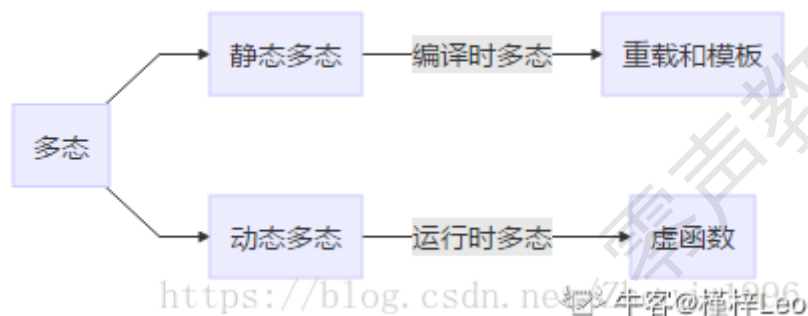
继承概念的实现方式有三类：实现继承、接口继承和可视继承。

实现继承是指使用基类的属性和方法而无需额外编码的能力；

接口继承是指仅使用属性和方法的名称、但是子类必须提供实现的能力；

可视继承是指子窗体（类）使用基窗体（类）的外观和实现代码的能力。

375.虚函数的作用及其实现原理



虚函数的作用：

虚函数实现了多态的机制。基类的函数前加上virtual关键字定义了虚函数，子类可以重写该函数，当子类重新定义了父类的虚函数后，父类指针根据赋给它的不同的子类指针，动态地调用属于子类的该函数，且这样的函数调用是无法在编译器期间确认的，而是在运行期确认，叫做动态绑定。只有指定为虚函数的成员函数才能进行动态绑定，成员函数默认为非虚函数，非虚函数不进行动态绑定；必须通过基类类型的指针或引用进行函数的调用。

举例：

从形状类Shape可以派生出圆Circle、三角形Triangle、矩形Rectangle、正方形Square等。每个类都有一个成员函数Draw，表示在屏幕上画出自己的形状。使用者使用Shape*调用Draw，那么要使画出的形状都正确，必须根据对象类型来确定使用哪个Draw来画。将Draw在Shape中定义为虚函数，并在每个派生类中重新定义该函数，那么当程序运行时，系统自动调用合适的Draw函数版本。

376.深拷贝和浅拷贝（值拷贝和位拷贝）

深拷贝指拷贝时对象资源重新分配，两个对象的资源内存不同，释放一个对象资源不会影响另一个。浅拷贝指两个对象均指向同一内存空间，释放一个对象的资源，另一个对象的资源也没了，造成野指针。

377.虚函数、纯虚函数怎么实现

用virtual关键字申明的函数叫做虚函数，虚函数肯定是类的成员函数；

存在虚函数的类都有一个一维的虚函数表叫做虚表，类的对象有一个指向虚表开始的虚指针。虚表是和类对应的，虚表指针是和对象对应的；

多态性是一个接口多种实现，是面向对象的核心，分为类的多态性和函数的多态性；

多态用虚函数来实现，结合动态绑定；

纯虚函数是虚函数再加上 = 0；

抽象类是指包括至少一个纯虚函数的类。纯虚函数:virtual void fun()=0;即抽象类！抽象基类不能定义对象。必须在子类实现这个函数，即先有名称，没有内容，在派生类实现内容。

378.为什么要有纯虚函数

为了方便使用多态特性，我们常常需要在基类中定义虚拟函数；

在很多情况下，基类本身生成对象是不合情理的。例如，动物作为一个基类可以派生出老虎、孔雀等子类，但动物本身生成对象明显不合常理；

为了解决上述问题，引入了纯虚函数的概念，将函数定义为纯虚函数（方法：virtual Return Type Function()= 0;），则编译器要求在派生类中必须予以重写以实现多态性。同时含有纯虚函数的类称为抽象类，它不能生成对象。这样就很好地解决了上述两个问题。

379.纯虚函数

纯虚函数没有函数体；

最后面的“=0”并不表示函数返回值为0，它只起形式上的作用，告诉编译系统“这是虚函数”；

这是一个声明语句，最后有分号。

380.为什么要有虚析构函数

C++中基类采用virtual虚析构函数是为了防止内存泄漏。具体地说，如果派生类中申请了内存空间，并在其析构函数中对这些内存空间进行释放。假设基类中采用的是非虚析构函数，当删除基类指针指向的派生类对象时就不会触发动态绑定，因而只会调用基类的析构函数，而不会调用派生类的析构函数。那么在这种情况下，派生类中申请的空间就得不到释放从而产生内存泄漏。所以，为了防止这种情况的发生，C++中基类的析构函数应采用virtual虚析构函数。

381.构造函数能不能是虚函数

不能。构造一个对象时，必须知道对象实际类型，而虚函数是在运行期间确定实际类型的。而在构造一个对象时，由于对象还未构造成功，编译器就无法知道对象的实际类型，是该类本身，还是派生类，还是其他。

虚函数的执行依赖于虚函数表，而虚函数表是在构造函数中进行初始化的，即初始化虚表指针（vptr），使得正确指向虚函数表。而在构造对象期间，虚函数表（vtable）还没有被初始化，将无法进行。

382.C++里面构造函数能有返回值吗？

构造函数没有返回值，他只是描述了类初始化的行为；但是new一个类实例是有返回值的，因为new返回的是类实例的指针。

383.构造函数和析构函数能被继承吗？

不能。不是所有的函数都能自动地从基类继承到派生类中的。构造函数和析构函数是用来处理对象的创建和析构的，它们只知道对在它们的特殊层次的对象做什么。

所以，在整个层次中的所有构造函数和析构函数都必须被调用，也就是说，构造函数和析构函数不能被继承。子类的构造函数会显示的调用父类的构造函数或隐式的调用父类的默认的构造函数进行父类部分的初始化。

析构函数也一样。它们都是每个类都有的东西，如果能被继承，那就没有办法初始化了。

384.C++中Overload、Overwrite及Override的区别

Overload(重载)：

在C++程序中，可以将语义、功能相似的几个函数用同一个名字表示，但参数或返回值不同（包括类型、顺序不同），即函数重载。

- (1) 相同的范围（在同一个类中）；
- (2) 函数名字相同；
- (3) 参数不同；
- (4) virtual 关键字可有可无。

Override(覆盖)：

指派生类函数覆盖基类函数，特征是：

- (1) 不同的范围（分别位于派生类与基类）；
- (2) 函数名字相同；
- (3) 参数相同；
- (4) 基类函数必须有virtual 关键字。

Overwrite(重写)：

是指派生类的函数屏蔽了与其同名的基类函数，规则如下：

(1) 如果派生类的函数与基类的函数同名，但是参数不同。此时，不论有无virtual关键字，基类的函数将被隐藏（注意别与重载混淆）。

(2) 如果派生类的函数与基类的函数同名，并且参数也相同，但是基类函数没有virtual关键字。此时，基类的函数被隐藏（注意别与覆盖混淆）。

385.一个空的class类里有什么

构造函数

拷贝构造函数

析构函数

赋值运算符重载

取地址操作符重载（this指针）

被const修饰的取地址操作符重载

386.C++中一个空类的大小为什么是1？

这就是实例化的原因（空类同样可以被实例化），每个实例在内存中都有一个独一无二的地址，为了达到这个目的，编译器往往会给一个空类隐含的加一个字节，这样空类在实例化后在内存得到了独一无二的地址，所以空类所占的内存大小是1个字节。

387.一个结构体中有一个int，一个char，一个static int，问这个结构体占多少内存？（涉及到内存对齐机制）

假定这个结构体所在的操作系统环境是64位，那么结构体中int占四字节，char内存对齐4字节，静态不计算，结构体一共占8字节内存。

特别注意的是：c结构体中不允许定义static变量；C++结构体中可以定义static变量，sizeof时不计算该变量，但需注意初始化格式。

相同类型的数据放在连续的内存。

388.结构体与联合体的区别

结构体struct：

各成员各自拥有自己的内存，各自使用互不干涉，同时存在的，遵循内存对齐原则。一个struct变量的总长度等于所有成员的长度之和。

联合体union：

各成员共用一块内存空间，并且同时只有一个成员可以得到这块内存的使用权(对该内存的读写)，各变量共用一个内存首地址。因而，联合体比结构体更节约内存。

389.函数与宏的差别

宏做的是简单的字符串替换，不会考虑数据类型；而函数是参数的传递，参数是有数据类型的。

宏的参数替换是不经计算而直接处理的，而函数调用是将实参的值传递给形参，既然说是值，自然是计算得来的。

宏占用的是编译的时间，而函数占用的是执行时的时间。

宏的参数是不占内存空间的，因为只做字符串的替换，形参作为函数的局部变量，是占用内存的。

函数的调用是需要付出一定的时空开销的，因为系统在调用函数时，要保留现场，然后转入被调用函数，执行完后返回主函数，再恢复现场，这些操作在宏中是没有的。

390.宏函数和inline函数的异同点

内联函数在编译时展开，而宏在预编译时展开；

在编译的时候，内联函数直接被嵌入到目标代码中去，而宏只是一个简单的文本替换；

内联函数可以进行诸如类型安全检查、语句是否正确等编译功能，宏不具有这样的功能；

宏不是函数，而inline是函数；

宏在定义时要小心处理宏参数，一般用括号括起来，否则容易出现二义性。而内联函数不会出现二义性；

inline可以不展开，宏一定要展开。因为inline指示对编译器来说，只是一个建议，编译器可以选择忽略该建议，不对该函数进行展开；

宏定义在形式上类似于一个函数，但在使用它时，仅仅只是做预处理器符号表中的简单替换，因此它不能进行参数有效性的检测，也就不能享受C++编译器严格类型检查的好处，另外它的返回值也不能被强制转换为可转换的合适的类型，这样，它的使用就存在着一系列的隐患和局限性。

391.define 和 typedef 区别

原理不同

#define是C语言中定义的语法，是预处理指令，在预处理时进行简单而机械的字符串替换，不作正确性检查，只有在编译已被展开的源程序时才会发现可能的错误并报错。

typedef是关键字，在编译时处理，有类型检查功能。它在自己的作用域内给一个已经存在的类型一个别名，但不能在一个函数定义里面使用typedef。用typedef定义数组、指针、结构等类型会带来很大的方便，不仅使程序书写简单，也使意义明确，增强可读性。

功能不同

typedef用来定义类型的别名，起到类型易于记忆的功能。另一个功能是定义机器无关的类型。如定义一个REAL的浮点类型，在目标机器上它可以获得最高的精度：typedef long double REAL；在不支持long double的机器上，看起来是这样的：typedef double REAL；在不支持double的机器上，是这样的：typedef float REAL

#define不只是可以为类型取别名，还可以定义常量、变量、编译开关等。

作用域不同

#define没有作用域的限制，只要是之前预定义过的宏，在以后的程序中都可以使用，而typedef有自己的作用域。

对指针的操作不同

```
#define INTPTR1 int*
```

```
typedef int* INTPTR2;
```

392.标准C++中的include "" 与<>的区别

#include<>直接从编译器自带的函数库中寻找文件

#include""是先从自定义的文件中找（通常是当前源文件的文件夹里面），如果找不到在从函数库中寻找文件，如果是自己写的头文件，建议使用#include""。

393.C++的内存管理机制

在C++中，内存分成5个区，他们分别是堆、栈、自由存储区、全局/静态存储区和常量存储区。

栈，在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。

堆，就是那些由new分配的内存块，他们的释放编译器不去管，由我们的应用程序去控制，一般一个new就要对应一个delete。如果程序员没有释放掉，那么在程序结束后，操作系统会自动回收。

自由存储区，就是那些由malloc等分配的内存块，他和堆是十分相似的，不过它是用free来结束自己的

生命的。

全局/静态存储区，全局变量和静态变量被分配到同一块内存中，在以前的C语言中，全局变量又分为初始化的和未初始化的，在C++里面没有这个区分了，他们共同占用同一块内存区。
常量存储区，这是一块比较特殊的存储区，他们里面存放的是常量，不允许修改。

394.C语言中的malloc/free和C++中的new/delete的区别和联系

相同点：它们都是对内存进行管理。

区别1：类型

malloc/free是函数，而new/delete是关键字、操作符

区别2：作用

malloc/free只是简单的进行内存的申请和释放；new/delete除了进行内存申请和释放，还会调用对象的构造函数和析构函数进行空间的初始化和清理

区别3：参数与返回值

malloc/free需要手动计算申请内存的空间大小，而且返回值是void*，需要自己转换成所需要的类型；new/delete可以自己计算类型的大小，返回为对应的类型指针。

395.迭代和递归区别

递归与迭代都是基于控制结构：迭代用重复结构，而递归用选择结构。

递归与迭代都涉及重复：迭代显式使用重复结构，而递归通过重复函数调用实现重复。

递归与迭代都涉及终止测试：迭代在循环条件失败时终止，递归在遇到基本情况时终止。

396.不可操作的操作符

作用域操作符：::

条件操作符：?:

点操作符：.

指向成员操作的指针操作符：->, .

预处理符号：#

397.C++关键字mutable作用

在C++中，mutable是为了突破const的限制而设置的。被mutable修饰的变量，将永远处于可变的状况，即使在一个const函数中，甚至结构体变量或者类对象为const，其mutable成员也可以被修改。mutable在类中只能修饰非静态数据成员。mutable数据成员的使用看上去像是骗术，因为它能够使const函数修改对象的数据成员。然而，明智地使用mutable关键字可以提高代码质量，因为它能够让你向用户隐藏实现细节，而无须使用不确定的东西。我们知道，如果类的成员函数不会改变对象的状态，那么这个成员函数一般会声明成const的。但是，有些时候，我们需要在const的函数里面修改一些跟类状态无关的数据成员，那么这个数据成员就应该被mutable来修饰。

398.引用与指针有什么区别？

- 1.引用必须被初始化，指针不必。
- 2.引用初始化以后不能被改变，指针可以改变所指的對象。
- 3.不存在指向空值的引用，但是存在指向空值的指针。
- 4.引用是变量的一个别名，内部实现是只读指针

399.什么是黑盒测试和白盒测试？

白盒测试：

是通过程序的源代码进行测试而不使用用户界面。这种类型的测试需要从代码句法发现内部代码在算法，溢出，路径，条件等等中的缺点或者错误，进而加以修正。

黑盒测试：

是通过使用整个软件或某种软件功能来严格地测试，而并没有通过检查程序的源代码或者很清楚地了解该软件的源代码程序具体是怎样设计的。测试人员通过输入他们的数据然后看输出的结果从而了解软件怎样工作。在测试时，把程序看作一个不能打开的黑盒子，在完全不考虑程序内部结构和内部特性的情况下，测试者在程序接口进行测试，它只检查程序是否能适当地接收和正确的输出。

400.你知道的类模版有哪些

vector 向量

string 字符串

list 列表

queue 队列

map 映射

set 集合

stack 栈

401.new可以搭配free吗，为什么？

可以，但不安全，通过 free 调用释放 new 申请的内存并不总是能正确的释放所有申请的内存。因为使用 free 方法释放内存时并不会调用实例的析构函数，此时如果实例中有动态申请的内存将因为析构函数没有被调用而没有得到释放，从而导致内存泄漏。而通常你不一定总能知道该类中是否使用了动态内存，因此最佳的做法是 new 与 delete 搭配使用。

402.怎么查看内存泄漏

第一：良好的编码习惯，尽量在涉及内存的程序段，检测出内存泄露。当程式稳定之后，再来检测内存泄露时，无疑增加了排除的困难和复杂度。使用了内存分配的函数，一旦使用完毕,要记得要使用其相应的函数释放掉。

第二：将分配的内存的指针以链表的形式自行管理，使用完毕之后从链表中删除，程序结束时可检查改链表。防止出现野指针。

第三：Boost 中的三种智能指针。

403.什么是内存溢出

内存溢出是指应用系统中存在无法回收的内存或使用的内存过多，最终使得程序运行要用到的内存大于虚拟机能提供的最大内存。引起内存溢出的原因有很多种，常见的有以下几种：

- 1.内存中加载的数据量过于庞大，如一次从数据库取出过多数据；
- 2.集合类中有对对象的引用，使用完后未清空，使得JVM不能回收；
- 3.代码中存在死循环或循环产生过多重复的对象实体；
- 4.使用的第三方软件中的BUG；
- 5.启动参数内存值设定的过小；

404.内存溢出的解决方案

第一步，修改JVM启动参数，直接增加内存。（-Xms，-Xmx参数一定不要忘记加。）

第二步，检查错误日志，查看“OutOfMemory”错误前是否有其它异常或错误。

第三步，对代码进行走查和分析，找出可能发生内存溢出的位置。重点排查以下几点：

1.检查对数据库查询中，是否有一次获得全部数据的查询。一般来说，如果一次取十万条记录到内存，就可能引起内存溢出。这个问题比较隐蔽，在上线前，数据库中数据较少，不容易出问题，上线后，数据库中数据多了，一次查询就有可能引起内存溢出。因此对于数据库查询尽量采用分页的方式查询。

2.检查代码中是否有死循环或递归调用。

3.检查是否有大循环重复产生新对象实体。

4.检查对数据库查询中，是否有一次获得全部数据的查询。一般来说，如果一次取十万条记录到内存，就可能引起内存溢出。这个问题比较隐蔽，在上线前，数据库中数据较少，不容易出问题，上线后，数据库中数据多了，一次查询就有可能引起内存溢出。因此对于数据库查询尽量采用分页的方式查询。

5.检查List、MAP等集合对象是否有使用完后，未清除的问题。List、MAP等集合对象会始终存有对对象的引用，使得这些对象不能被GC回收。

第四步，使用内存查看工具动态查看内存使用情况

405.函数指针与指针函数分别是什么

指针函数本质是一个函数，其返回值为指针。

```
int *fun(int x,int y);
```

函数指针本质是一个指针，其指向一个函数。

```
int (*fun)(int x,int y);
```

406.C++11新特性了解吗

1、新增容器std::array 保存在栈内存中，相比堆内存中的 std::vector，我们能够灵活的访问这里的元素，从而获得更高的性能。

2、auto 和 decltype 这两个关键字实现了类型推导

3、替代NULL的nullptr

4、三种智能指针帮助内存管理(说一下名称)：unique_ptr、shared_ptr 和 weak_ptr。

5、C++11 引入了基于范围的迭代写法，比如基于范围的for循环，用一个冒号就可实现遍历，我们拥有了能够写出像 Python一样简洁的循环语句。

407.接口和抽象类的区别

接口中所有的方法隐含的都是抽象的。而抽象类则可以同时包含抽象和非抽象的方法。

类可以实现很多个接口，但是只能继承一个抽象类。

类如果实现一个接口，它必须要实现接口声明的所有方法。但是，类可以不实现抽象类声明的所有方法，当然，在这种情况下，类也必须得声明成是抽象的。

抽象类可以在不提供接口方法实现的情况下实现接口。

Java接口中的成员函数默认是public的。抽象类的成员函数可以是private，protected或者是public。

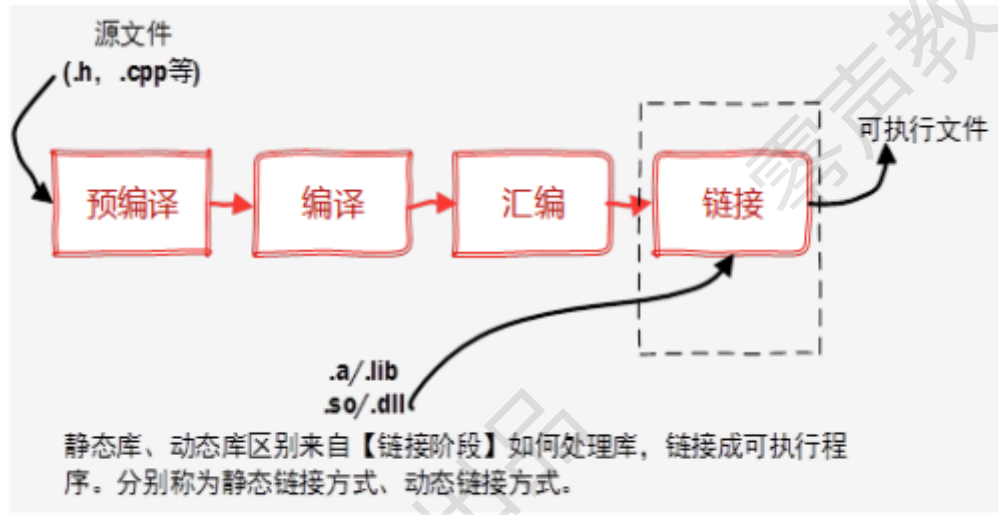
struct 和 class的区别

struct作为数据结构的实现体，它默认的数据访问控制是public的；而class作为对象的实现体，它默认的成员变量访问控制是private的。

408.预编译在做些什么事情？

预编译又称为预处理，是做些代码文本的替换工作。主要处理#开头的指令，比如拷贝#include包含的文件代码，#define宏定义的替换,条件编译等。就是为编译做预备工作的阶段。

409.可执行文件编译过程



图：编译过程

牛客@權梓Leo

410.动态库和静态库？

静态库特点总结如下：

静态库对函数库的链接是放在编译时期完成的。

程序在运行时与函数库再无瓜葛，移植方便。

浪费空间和资源，因为所有相关的目标文件与牵涉到的函数库被链接合成一个可执行文件。

文件后缀常为（.a、.lib）。

动态库特点总结如下：

动态库把对一些库函数的链接载入推迟到程序运行的时期。

可以实现进程之间的资源共享。

将一些程序升级变得简单，直接改动态库即可。

文件后缀常为（.so、.dll）。

411.堆和栈的区别，以及为什么栈效率高

堆是由低地址向高地址扩展；栈是由高地址向低地址扩展。

堆中的内存需要手动申请和手动释放；栈中内存是由OS自动申请和自动释放，存放着参数、局部变量等内存。

堆中频繁调用malloc和free,会产生内存碎片，降低程序效率；而栈由于其先进后出的特性，不会产生内存碎片。

堆的分配效率较低，而栈的分配效率较高。

栈的效率高的原因：

栈是操作系统提供的数据结构，计算机底层对栈提供了一系列支持：分配专门的寄存器存储栈的地址，压栈和入栈有专门的指令执行；而堆是由C/C++函数库提供的，机制复杂，需要一些列分配内存、合并内存和释放内存的算法，因此效率较低。

412.函数参数压栈方式为什么是从右到左的？

因为C++支持可变长函数参数。正是这个原因使得C语言函数参数入栈顺序为从右至左。具体原因：C方式参数入栈顺序（从右至左）的好处就是可以动态变化参数个数。C程序栈底为高地址，栈顶为低地址。

函数最左边确定的参数在栈上的位置必须是确定的，否则意味着已经确定的参数是不能定位和找到的，这样是无法保证函数正确执行的。衡量参数在栈上的位置，就是离开确切的函数调用点（call f）有多远。已经确定的参数，它在栈上的位置，不应该依赖参数的具体数量，因为参数的数量是未知的！所以只有确定的参数最后入栈才能保证它在栈中的位置是确定的。

413.C++中的智能指针

C++里面的四个智能指针: auto_ptr, shared_ptr, weak_ptr, unique_ptr 其中后三个是c++11支持，并且第一个已经被11弃用。

智能指针的作用：

管理一个指针，因为存在以下这种情况：申请的空间在函数结束时忘记释放，造成内存泄漏。使用智能指针可以很大程度上的避免这个问题，因为智能指针就是一个类，当超出了类的作用域是，类会自动调用析构函数，析构函数会自动释放资源。所以智能指针的作用原理就是在函数结束时自动释放内存空间，不需要手动释放内存空间。

1、auto_ptr（c++98的方案，cpp11已经抛弃）采用所有权模式。

缺点是：存在潜在的内存崩溃问题！

```
auto_ptr< string> p1 (new string ("I reigned lonely as a cloud."));
auto_ptr <string> p2;p2 = p1;
//auto_ptr不会报错.此时不会报错，p2剥夺了p1的所有权，但是当程序运行时访问p1将会报错。所以
auto_ptr的缺点是：存在潜在的内存崩溃问题！
```

2、unique_ptr（替换auto_ptr）unique_ptr实现独占式拥有或严格拥有概念，保证同一时间内只有一个智能指针可以指向该对象。它对于避免资源泄露(例如“以new创建对象后因为发生异常而忘记调用delete”)特别有用。采用所有权模式。

```
unique_ptr<string> p3 (new string ("auto"));
unique_ptr<string> p4;
p4 = p3;//此时会报错！！
//编译器认为p4=p3非法，避免了p3不再指向有效数据的问题。因此，unique_ptr比auto_ptr更安全。
```

//编译器认为p4=p3非法，避免了p3不再指向有效数据的问题。因此，unique_ptr比auto_ptr更安全。

3、shared_ptr

shared_ptr实现共享式拥有概念。多个智能指针可以指向相同对象，该对象和其相关资源会在“最后一个引用被销毁”时候释放。从名字share就可以看出了资源可以被多个指针共享，它使用计数机制来表明资源被几个指针共享。可以通过成员函数use_count()来查看资源的所有者个数。除了可以通过new来构造，还可以通过传入auto_ptr, unique_ptr, weak_ptr来构造。当我们调用release()时，当前指针会释放资源所有权，计数减一。当计数等于0时，资源会被释放。

shared_ptr是为了解决auto_ptr在对象所有权上的局限性(auto_ptr是独占的)，在使用引用计数的机制上提供了可以共享所有权的智能指针。

4、weak_ptr

weak_ptr是一种不控制对象生命周期的智能指针，它指向一个shared_ptr管理的对象。进行该对象的内存管理的是那个强引用的shared_ptr。weak_ptr只是提供了对管理对象的一个访问手段。weak_ptr设计的目的是为配合shared_ptr而引入的一种智能指针来协助shared_ptr工作，它只可以从一个shared_ptr或另一个weak_ptr对象构造，它的构造和析构不会引起引用记数的增加或减少。

weak_ptr是用来解决shared_ptr相互引用时的死锁问题，如果说两个shared_ptr相互引用，那么这两个指

针的引用计数永远不可能下降为0,资源永远不会释放。它是对对象的一种弱引用,不会增加对象的引用计数,和shared_ptr之间可以相互转化,shared_ptr可以直接赋值给它,它可以通过调用lock函数来获得shared_ptr。

414.基类里private成员函数可以声明为虚函数吗？

可以的,不过就和私有成员创建的本意有点相反,不建议实现。

415.函数A调用函数B的时候,有什么需要压栈？

函数的参数,压栈顺序是从右往左的。还有函数B内部定义的变量。

416.数组和指针区别？数组和链表呢？双向链表和单向链表？

数组和指针区别：

- 1、把数组作为参数传递的时候,会退化为指针
- 2、数组名可作为指针常量
- 3、数组是开辟一块连续的内存空间,数组本身的标示符代表整个数组,可以用sizeof取得真实的大小；指针则是只分配一个指针大小的内存,并可把它的值指向某个有效的内存空间

数组和链表区别：

不同：

链表是链式的存储结构；数组是顺序的存储结构。

链表通过指针来连接元素与元素,数组则是把所有元素按次序依次存储。

链表的插入删除元素相对数组较为简单,不需要移动元素,且较为容易实现长度扩充,但是寻找某个元素较为困难；数组寻找某个元素较为简单,但插入与删除比较复杂,由于最大长度需要再编程一开始时指定,故当达到最大长度时,扩充长度不如链表方便。

相同：

两种结构均可实现数据的顺序存储,构造出来的模型呈线性结构。

双向链表和单向链表区别：

单向链表包含两个域,一个是信息域,一个是指针域。也就是单向链表的节点被分成两部分,一部分是保存或显示关于节点的信息,第二部分存储下一个节点的地址,而最后一个节点则指向一个空值。

双向链表每个节点有2个链接,一个是指向前一个节点(当此链接为第一个链接时,指向的是空值或空列表),另一个则指向后一个节点(当此链接为最后一个链接时,指向的是空值或空列表)。意思就是说双向链表有2个指针,一个是指向前一个节点的指针,另一个则指向后一个节点的指针。

417.vector底层实现？

底层数据结构是一个动态数组。默认构造的大小是0,之后插入按照1 2 4 8 16 二倍扩容。注(GCC是二倍扩容,VS13是1.5倍扩容。原因可以考虑内存碎片和伙伴系统,内存的浪费)。扩容后是一片新的内存,需要把旧内存空间中的所有元素都拷贝进新内存空间中去,之后再在新内存空间中的原数据的后面继续进行插入构造新元素,并且同时释放旧内存空间,并且,由于vector空间的重新配置,导致旧vector的所有迭代器都失效了。

vector的初始的扩容方式代价太大,初始扩容效率低,需要频繁增长,不仅操作效率比较低,而且频繁的向操作系统申请内存容易造成过多的内存碎片,所以这个时候需要合理使用resize()和reserve()方法提高效率减少内存碎片的。

resize()：


```
void resize (size_type n);  
void resize (size_type n, value_type val);
```

- 1、resize方法被用来改变vector中元素的数量，我们可以说，resize方法改变了容器的大小，且创建了容器中的对象；
- 2、如果resize中所指定的n小于vector中当前的元素数量，则会删除vector中多于n的元素，使vector得大小变为n；
- 3、如果所指定的n大于vector中当前的元素数量，则会在vector当前的尾部插入适量的元素，使得vector的大小变为n，在这里，如果为resize方法指定了第二个参数，则会把后插入的元素值初始化为该指定值，如果没有为resize指定第二个参数，则用默认值填充新位置，一般为0；
- 4、如果resize所指定的n不仅大于vector中当前的元素数量，还大于vector当前的capacity容量值时，则会自动为vector重新分配存储空间；

reserve():避免了频繁的申请内存空间，造成过多内存碎片。

```
void reserve (size_type n);
```

- 1、reserve的作用是更改vector的容量，使vector至少可以容纳n个元素。
- 2、如果n大于vector当前的容量，reserve会对vector进行扩容。其他情况下都不会重新分配vector的存储空间。
- 3、reserve方法对于vector元素大小没有任何影响，不创建对象。

vector中数据的随机存取效率很高， $O(1)$ 的时间的复杂度，但是在vector中随机插入元素，需要移动的元素数量较多，效率比较低。

418.vector与list的区别？

vector拥有一段连续的内存空间，因此支持随机存取，如果需要高效的随即存取，而不在乎插入和删除的效率，使用vector。

vector和数组类似，它拥有一段连续的内存空间，并且起始地址不变，因此它能非常好的支持随机存取（使用[]操作符访问其中元素），但由于它的内存空间是连续的，所以在中间进行插入和删除会造成内存块的拷贝（复杂度是 $O(n)$ ），另外，当该数组后的内存空间不够时，需要重新申请一块足够大的内存并进行内存的拷贝。这些都影响了vector的效率。

list拥有一段不连续的内存空间，因此不支持随机存取，如果需要大量的插入和删除，而不关心随即存取，则应使用list。list是由数据结构中的双向链表实现的，因此它的内存空间可以是不连续的。因此只能通过指针来进行数据的访问，这个特点使得它的随机存取变的非常没有效率，需要遍历中间的元素，搜索复杂度 $O(n)$ ，因此它没有提供[]操作符的重载。但由于链表的特点，它可以以很好的效率支持任意地方的删除和插入。

版权声明：本文为CSDN博主「一骑走烟尘」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/zgcr654321/article/details/90574583>

419.变量的声明和定义有什么区别？

变量的定义为变量分配地址和内存空间，变量的声明不分配地址。一个变量可以在多个地方声明，但是只在一个地方定义。加入extern修饰的是变量的声明，说明此变量将在文件以外或在文件后面部分定义。

说明：很多时候一个变量，只是声明不分配内存空间，知道具体使用时才初始化，分配内存空间，如外部变量。

```
int main()
{
    extern int A;
    //这是个声明而不是定义，声明A是一个已经定义了的外部变量
    //注意：声明外部变量时可以把变量类型去掉如：extern A;
    dosth(); //执行函数
}
int A; //是定义，定义了A为整型的外部变量
```

420.简述#ifdef、#else、#endif和#ifndef的作用

利用#ifdef、#endif将某程序功能模块包括进去，以向特定用户提供该功能。在不需要时用户可轻易将其屏蔽。

```
#ifdef MATH
#include "math.c"
#endif
```

在子程序前加上标记，以便于追踪和调试。

```
#ifdef DEBUG
printf ("Indebugging.....!");
#endif
```

应对硬件的限制。由于一些具体应用环境的硬件不一样，限于条件，本地缺乏这种设备，只能绕过硬件，直接写出预期结果。

注意：虽然不用条件编译命令而直接用if语句也能达到要求，但那样做目标程序长（因为所有语句都编译），运行时间长（因为在程序运行时间对if语句进行测试）。而采用条件编译，可以减少被编译的语句，从而减少目标程序的长度，减少运行时间。

421.写出int、bool、float、指针变量与“零值”比较的if语句

```
//int与零值比较
if (n == 0)
if (n != 0)

//bool与零值比较
if (flag) //表示flag为真
if (!flag) //表示flag为假

//float与零值比较
const float EPSILON = 0.00001;
```

```
if((x>=-EPSILON)&&(x<=EPSILON))//其中EPSILON为允许的误差
```

```
//指针变量与零值比较
```

```
if(p==NULL)
```

```
if(p!=NULL)
```

422.结构体可以直接赋值吗？

声明时可以直接初始化，同一结构体的不同对象之间也可以直接赋值，但是当结构体中含有指针“成员”时一定要小心。

注意：当有多个指针指向同一段内存时，某个指针释放这段内存可能会导致其他指针的非法操作。因此在释放前一定要确保其他指针不再使用这段内存空间。

423.sizeof和strlen的区别

sizeof是一个操作符，strlen是库函数

sizeof的参数可以是数据的类型，也可以是变量，而strlen只能以结尾为'\0'的字符串做参数。

编译器在编译时就计算出了sizeof的结果，而strlen函数必须在运行时才能计算出来。并且sizeof计算的是数据类型占内存的大小，而strlen计算的是字符串实际的长度。

数组做sizeof的参数不退化，传递给strlen就退化为指针了。

424.c语言的关键字static和c++关键字static有什么区别？

在c中static用来修饰局部静态变量和外部静态变量、函数。而c++中除了上述功能外，还用来定义类的成员变量和函数。即静态成员和成员函数。

注意：编程时static的记忆性，和全局性的特点可以让在不同时期调用的函数进行通信，传递消息，而c++的静态成员则可以在多个对象实例间进行通信，传递消息。

425.c语言的malloc和c++中的new有什么区别？

new、delete是操作符，可以重载，只能在c++中使用

malloc、free是函数，可以覆盖，c、c++中都可以使用

new可以调用对象的构造函数，对应的delete调用相应的析构函数

malloc仅仅分配内存，free仅仅收回内存，并不执行构造和析构函数

new、delete返回的是某种数据类型指针，malloc、free返回的是void指针

注意：malloc申请的内存空间要用free释放，而new申请的内存空间要用delete释放，不要混用。

426.写一个“标准”宏MIN

```
#define min(a,b)((a)<=(b)?(a):(b))
```

427.++i和i++的区别

++i先自增1，再返回，i++先返回i，再自增1

428.volatile有什么作用

状态寄存器一类的并行设备硬件寄存器

一个中断服务子程序会访问到的非自动变量。

多线程被几个任务共享的变量

注意：虽然volatile在嵌入式方面应用比较多，但是在pc软件的多线程中，volatile修饰的临界变量也是非常实用的。

429.一个参数可以既是const又是volatile吗

可以，用const和volatile同时修饰变量，并且这个变量在程序内部是只读的，不能改变的，只在程序外部条件变化下改变，并且编译器不会优化这个变量。每次使用这个变量时，都要小心的去内存读取这个变量的值，而不是去寄存器读取他的备份。

注意：在此一定要注意const的意思，const只是不允许程序中的代码改变某一变量，其在编译期发挥作用，他并没有实际的禁止某段内存的读写特性。

430.a和&a有什么区别？

&a：其含义就是“变量a的地址”

*a：用在不同的地方，含义也不一样

在声明语句中，*a只说明a是一个指针变量，如int a；

在其他语句中，a前面没有操作数且a是一个指针时，a代表指针a指向的地址内存放的数据，如b=a；

*a前面有操作数且a是一个普通变量，a代表乘以a，如c=ba。

431.用c编写一个死循环程序

```
while ( 1)
```

```
{
```

注意：很多种途径都可以实现同一种功能，但是不同的方法时间和空间占用度不同，特别是对于嵌入式软件，处理器速度比较慢，存储空间较小，所以时间和空间优势是选择各种方法的首要考虑条件。

432.结构体内存对齐的问题

请写出以下代码的输出结果：

```
#include <iostream>
using namespace std;
struct s1
{
    int i : 8;
    char j : 4;
    int a : 4;
    double b;
};
struct s2
{
    int i : 8;
    char j : 4;
    double b;
    int a : 4;
};
struct s3
{
    int i;
    char j;
    double b;
    int a;
};
int main()
{
    cout<<sizeof(s1);
```

```
cout << sizeof(s2);  
cout << sizeof(s3);  
return 0;  
}
```

说明：结构体作为一种复合数据类型，其构成元素既可以是基本数据类型的变量，也可以是一些复合型类型数据。对此，编译器会自动进行成员变量的对齐以提高运算速率。默认情况下，按自然对齐条件分配空间。各个成员按照他们被声明的顺序在内存中顺序存储，第一个成员的地址和整个结构的地址相同，向结构体成员中size最大的成员对齐。

许多实际的计算机系统对基本类型数据在内存中存放的位置有限制，他们会要求这些数据的首地址是某个数k（通常为4或8）的倍数，而这个k则被称为该数据类型的对齐模数。

433.全局变量和局部变量有什么区别？是怎么实现的？操作系统和编译器是怎么知道的？

全局变量是整个程序都可访问的变量，谁都可以访问，生存期在整个程序从运行到结束（在程序结束时所占内存释放）；

而局部变量存在于模块（子程序、函数）中，只有所在模块可以访问，其他模块不可直接访问，模块结束（函数调用完毕），局部变量消失，所占据的内存释放；

操作系统和编译器，可能是通过内存分配的位置来知道的，全局变量分配在全局数据段并且在程序开始运行的时候被加载，局部变量则分配在堆栈里面。

434.简述c、c++程序编译的内存分配情况

从静态存储区域分配

内存存在程序编译时就已经分配好，这块内存存在程序的整个运行期间都存在。速度快、不容易出错，因为有系统会善后。例如全局变量，static变量、常量字符串等。

从栈上分配

在执行函数时，函数内局部变量的存储单元都在栈上创建，函数执行结束时这些存储单元自动被释放。

栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。大小为2M。

从堆上分配

即动态内存分配。程序在运行的时候用malloc和new申请任意大小的内存，程序员自己负责在何时用free和delete释放内存。动态内存的生存期由程序员决定，使用非常灵活。如果在堆上分配了空间，就有责任回收他，否则运行的程序会出现内存泄露，另外频繁的分配和释放不同大小的堆空间将会产生“堆内碎片”。

一个c、c++程序编译时内存分为5大存储区：堆区、栈区、全局区、文字常量区、程序代码块区。

435.简述strcpy、sprintf、memcpy的区别

操作对象不同，strcpy的两个操作对象均为字符串，sprintf的操作源对象可以是多种数据类型，目的操作对象是字符串，memcpy的两个对象就是就是两个任意可操作的内存地址，并不限于何种数据类型。

执行效率不同，memcpy最高，strcpy次之，sprintf的效率最低

实现功能不同，strcpy主要实现字符串变量间的拷贝，sprintf主要实现其他数据类型格式到字符串的转化，memcpy主要是内存块间的拷贝

注意：strcpy、sprintf与memcpy都可以实现拷贝的功能，但是针对的对象不同，根据实际需求，来选择合适的函数实现拷贝功能。

436.解析 (void()())0()的含义

`void ( 0 ) ( )` : 是一个返回值为`void`，参数为空的函数指针；

`( void ( ) ( ) ) 0` : 把0转换为一个返回值为`void`，参数为空的函数指针；

`( void ( ) ( ) ) 0 ( )` : 在上句的基础上加表示整个是一个返回值为`void`，无参数，并且起始地址为0的函数的名字。；

`(( void ( * ) ( ) ) 0 ) ( )` : 这就是上句的函数名所对应的函数的调用。

437.c语言的指针和引用和c++的有什么区别？

指针有自己的一块空间，而引用只是一个别名

使用`sizeof`看一个指针的大小是4，而引用则是被引用对象的大小；

作为参数传递时，指针需要被解引用才可以对对象进行操作，而直接对引用的修饰都会改变引用所指向的对象

可以有`const`指针，但是没有`const`引用

指针在使用中可以指向其他对象，但是引用只能是一个对象的引用，不能被改变；

指针可以有级指针，而引用止于一级；

指针和引用使用++运算符的意义不一样；

如果返回动态内存分配的对象或者内存，必须使用指针，引用可能会引起内存泄漏。

版权声明：本文为CSDN博主「young Danny」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_44886002/article/details/121055037

438.new与malloc的区别

`new`操作符从自由存储区上为对象动态分配内存空间，而`malloc`函数从堆上动态分配内存。

使用`new`操作符申请内存分配时无须指定内存块的大小，而`malloc`则需要显式地指出所需内存的尺寸。

```
1 int *p = new int;
2 delete p; //一定要配对使用new 和 delete
1 int *p = new int [10]; //对于数组，应使用另一种格式的delete来释放
2 delete [] p;
1 int *p = (int *)malloc(sizeof(int));
2 *p = 100;
3 free(p);
4 p = NULL;
5 return 0;
```

439.malloc/free 为什么还要 new/delete ？

`malloc` 与 `free` 是 C++/C 语言的标准库函数，`new/delete` 是 C++的运算符。它们都可用于申请动态内存和释放内存。对于非内部数据类型的对象而言，

光用 `maloc/free` 无法满足动态对象的要求。对象在创建的同时要自动执行构造函数，对象在消亡之前要自动执行析构函数。由于`malloc/free` 是库函数而不是运算符，

不在编译器控制权限之内，不能够把执行构造函数和析构函数的任务强加`malloc/free`。因此 C++语言需要一个能完成动态内存分配和初始化工作的运算符 `new`，

以及一个能完成清理与释放内存工作的运算符 `delete`。注意 `new/delete` 不是库函数。

440.delete与 delete []区别

delete只会调用一次析构函数，而delete[]会调用每一个数组元素的析构函数。

delete与new配套，delete []与new []配套。

441.在物理内存为1G的计算机中能否malloc(1.2G)？

在支持虚拟内存管理的操作系统上：当请求的内存大于128kb，malloc()会调用mmap()分配一起分配内存。

在不支持虚拟内存管理的实时操作系统上运行：这种情况下，在只有1G物理内存的计算机上malloc(1.2G)就会失败。

442.用C写个程序，如何判断一个操作系统是16位还是32位的？

定义一个指针p，打印出sizeof(p)，如果结果是4，则表示该操作系统是32位，打印结果是2，表示是16位。

443.解释下位域，为什么要用位域，位域的好处？

在计算机中，有些信息存储时并不需要占用一个完整的字节，而只需占用一个或几个二进制位。比如在存放一个只有0和1两种状态的开关量时，

用一位二进制位即可表示。因此，为了节省存储空间，C语言提供了一种称为“位域”的数据结构来充分利用存储空间。

444.位操作

位与符号是一个&，两个&&是逻辑与，位或符号是一个|，两个||是逻辑或。

C语言中按位取反是~，C语言中的逻辑取反是!。

左移位<<与右移位>>

对于无符号数，左移时右侧补0（相当于逻辑移位），右移时左侧补0（相当于逻辑移位）

对于有符号数，左移时右侧补0（叫算术移位，相当于逻辑移位），右移时左侧补符号位（如果正数就补0，负数就补1，叫算术移位）

嵌入式中研究的移位，以及使用的移位都是无符号数。

https://blog.csdn.net/weixin_49303682/article/details/118662325

445.在某工程中，要求设置一绝对地址为0x67a9的整型变量的值为0xaa66。编译器是一个纯粹的ANSI编译器。写代码去完成这一任务。


```
1 int *p;  
2 p = (int *)0x67a9;  
3 *p = 0xaa66;
```

446.给定一个整型变量a，写两段代码，第一个设置a的bit3，第二个清除a的bit，在以上两个操作中，要保持其它位不变。



```
1 #define BIT3 (0x1<<3)  
2 static int a;  
3 void set_bit3(void)  
4 {  
5     a |= BIT3;  
6 }  
7 void clear_bit3(void)  
8 {  
9     a &= ~BIT3;  
10 }
```



447.什么是右值引用，跟左值又有什么区别

左值：能对表达式取地址，一般指表达式结束后依然存在的持久对象。

右值：不能对表达式取地址，一般表达式结束后就不再存在的临时对象。

448.判断x=x+1,x+=1,x++哪个效率最高?为什么?

x++的效率最高，直接去出来+1后在放回原来的地址。

x=x+1最低, 因为执行步骤如下: <1>读取右x的地址; <2>x+1; <3>读取左x的地址, <4>将右值传给左边的x(编译器并不认为左右两边的x的地址相同)。

449.用变量a定义

一个整型数 int a;

一个指向整型数的指针 int *a;

一个指向指针的指针，它指向的指针式指向一个整型数 int **a;

一个有10个整型数的数组 int a[10];

一个有10个指针的数组，该指针是指向一个整型数 int *a[10];

一个指向有10个整型数数组的指针 int (a)[10];

一个指向函数的指针，该函数有一个整型数参数并返回一个整型数 int (a)(int);

一个有10个指针的数组，该指针指向一个函数，该函数有一个整型数参数并返回一个整型 int (a[10]) (int);

450.C语言是强类型的语言，这是什么意思？

C语言是一种强类型的程序设计语言，int x,y,z;在C程序中，每一个变量都必须声明其取值类型。

char 占8位1字节，short占16位2字节，int 占32位4字节，long 占32位4字节，float占32位4字节，double占64位8字节。

451.char 与 int之间的转换

从长字节数据类型转换为短字节数据类型，会产生截断：从4字节的int类型转换成1个字节的char类型，则取int数据的最低的一个字节。

从短字节类型转换为长字节类型：从char转换为int：则在前面的三个字节补符号位0x000000。

452.float（单精度浮点型）和double（双精度浮点型）

float:1bit(符号位)+8bit(指数位)+23（尾数位） $2^{23}=8388608$ ，共7位，意味着最多有7位有效数字。

double:1bit(符号位)+11bit（指数位）+52bit(尾数位) $2^{52}=4503599627370496$ 。一共16位，同理double的精度为15~16位。



```
1 #include <stdio.h>
2 int main(void)
3 {
4     float m = (float)3.14;
5     double n = 10.5;
6     printf("%f,%lf\n",m,n);
7     return 0;
8 }
```



输出：3.140000,10.500000 因为%lf只保留小数点后6位。

453.字符常量

char beta; beta='ab'; 是错误的，改正：char beta; beta='a';

char beta; beta="a"; 是错误的，改正：char beta; beta='a';

定义字符串数组：char a[] = "abcde"

非图形字符，如退格，换行，也可以表示成字符型常量。表示方法是使用转义字符"\";

已知char b[5],*p=b;，则正确的赋值语句是 C

下列可以正确表示字符型常量的是 D

A、297 B、"a" C、"\n" D、'\t'

454.写出bool、int、指针变量与“零值”比较的if语句

bool型数据：

```
if( flag ) { A; } else { B ; }
```

int型数据：

```
if( 0 != flag ) { A; } else { B ; }
```

指针型数：

```
if( NULL == flag ) { A; } else { B ; }
```

455.写出float x 与“零值”比较的if语句。

```
if(x>0.000001&& x<-0.000001)
```

456.区分 %d, %ld, %lld, %lf, %f

%d=int,

%ld=long,

%lld=long long;

在32位编译器上，int=long=32bit；long long=64bit。

输入时：

float 输入用 %f，小数点后6位。

double 输入用 %lf，小数点后6位。

457.输出数据问题

printf("%d\n",101010); 默认的方式是左对齐。

printf("%-10d\n",101010); 在打印数字宽度前面加一个“-”也是左对齐，10表示数字宽度为10，其他空格补齐。

printf("%10d",101010); 在%和d之间加上数字宽度，就可以右对齐了，10表示数字宽度为10，其他空格补齐。

```
1 double a = 326.3845;
2 printf("%10.3f\n", a);
1 结果是:
2      326.385（注意前面是有空格的）
1 printf("%04d", 3);
1 结果为:
2 0003
```

458.嵌入式系统中经常要用到无限循环，你怎么样用C编写死循环

```
1 while(1)
2 {
3 ;}
1 for(;;)
2 {
3 ;}
```

459.惰性计算方法



```
1 #include "stdio.h"
2 void main()
3 {
4     int n = 2, k = 0;
5     while(k++&& n++>2);
6     printf("%d %d\n", k, n);
7 }
```



1 输出的正确答案是1 2

刚开始运算的时候，k的值是0，n的值是2，并还没有进行自增1的操作，自增1要在下一次循环中。对于循环条件(k++&& n++>2)，

首先计算k，由于k的初值为0，k++是后坐，因此k++在参与比较的时候是0（假），比较以后k值增1（变为1），这时，系统不会再计算&&后面的表达式了（即

所谓的惰性计算方法），所以，n++>2没有参与运算，也就是说，循环语句执行之后，k的值是1，n的值是2。

460.变量的声明和定义有什么区别

为变量分配地址和存储空间的称为定义，不分配地址的称为声明。一个变量可以在多个地方声明，但是只在一个地方定义。

461.用预处理指令#define 声明一个常数，用以表明1年中有多少秒(忽略闰年问题)

```
#define SECONDS_PER_YEAR (60 * 60 * 24 * 365)UL
```

462.写一个“标准”宏MIN，这个宏输入两个参数并返回较小的一个

```
#define MIN(a, b) (a) <= (b) ? (a) : (b)
```

463.sizeof和strlen的区别

sizeof的参数可以是数据的类型，也可以是变量，而strlen只能以结尾为'\0'的字符串作参数。

sizeof计算的是数据类型占内存的大小，而strlen计算的是字符串实际的长度。

464.c语言中的static和C++中static的区别

static在C语言中具有：记忆性，隐藏性，初始化为0的特点。

而C++中除了上述功能外，还用来定义类的成员变量和函数。即静态成员和静态成员函数。

465.C++函数中值的传递方式有哪几种？

C++函数的三种传递方式为：值传递、指针传递和引用传递。

466.C++里面是不是所有的动作都是main()引起的？如果不是，请举例。

比如全局变量的初始化，就不是由main函数引起的。举例：***

```
1 class A{};
2 A a;          //a的构造函数限执行
3 int main() {}
```

467.谈谈对面向对象的认识

面向对象可以理解成对待每一个问题，都是首先要确定这个问题由几个部分组成，而每一个部分其实就是一个对象。

然后再分别设计这些对象，最后得到整个程序。传统的程序设计多是基于功能的思想来进行考虑和设计的，而面向对象的

468.谈谈你对编程规范的理解

编程规范可总结为：程序的可行性、可读性、可移植性以及可测试性。

说明：这是编程规范的总纲目，面试者不一定要去背诵上面给出的那几个例子，应该去理解这几个例子说明的问题，

想一想，自己如何解决可行性、可读性、可移植性以及可测试性这几个问题，结合以上几个例子和自己平时的编程习惯来回答这个问题。

469.面向对象的三大特性

面向对象的三大特征是封装性、继承性和多态性：

封装性：将客观事物抽象成类，每个类对自身的数据和方法实行protection (private , protected , public) 。

继承：继承主要实现重用代码，节省开发时间，子类可以继承父类的属性和方法。

多态性：是将父类对象设置成为和一个或更多它的子对象相等的技术。用子类对象给父类对象赋值之后，父类对象就

可以根据当前赋值给它的子对象的特性以不同的方式运作。

470.简述多态的原理

编译器发现一个类中有虚函数，便会立即为此类生成虚函数表 vtable。虚函数表的各表项为指向对应虚函数的指针。

编译器还会在此类中隐含插入一个指针vptr（对vc编译器来说，它插在类的第一个位置上）指向虚函数表。调用此类的构造函数时，

在类的构造函数中，编译器会隐含执行vptr与vtable的关联代码，将vptr指向对应的vtable，将类与此类的vtable联系了起来。

另外在调用类的构造函数时，指向基础类的指针此时已经变成指向具体的类的this指针，这样依靠此this指针即可得到正确的vtable，。

如此才能真正与函数体进行连接，这就是动态联编，实现多态的基本原理。

471.多态的作用？

主要是两个：（1）隐藏实现细节，使得代码能够模块化；扩展代码模块，实现代码重用；

（2）接口重用：为了类在继承和派生的时候，保证使用家族中任一类的实例的某一属性时的正确调用。

472.多态，虚函数，纯虚函数

多态：是对于不同对象接收相同消息时产生不同的动作。

虚函数：在基类中冠以关键字 virtual 的成员函数。它提供了一种接口界面。允许在派生类中对基类的虚函数重新定义。

纯虚函数的作用：在基类中为其派生类保留一个函数的名字，以便派生类根据需要对它进行定义。作为接口而存在，纯虚函数不具备函数的功能，一般不能直接被调用。

具有纯虚函数的类是抽象类（abstract class），不能声明对象，只能作为基类为派生类服务，除非派生类完全实现了基类的所有纯虚函数，否则派生类也成为抽象类，不能

声明对象。

473.重载（overload）、重写(override，有的书也叫做“覆盖”）、重定义（redefinition）的区别？

重载	同一名字子空间	是指允许存在多个同名函数，而这些函数的参数表不同。
重定义/隐藏	不同名字子空间	用于继承，派生类与基类的函数同名，屏蔽基类的函数
重写/覆盖	不同名字子空间	用于继承，子类重新定义父类虚函数的方法

474.所有的运算符都能重载吗？

不能被重载的运算符

在 C++ 运算符集合中，有一些运算符是不允许被重载的。这种限制是出于安全方面的考虑，可防止错误和混乱。

- （1）不能改变 C++ 内部数据类型（如 int, float 等）的运算符。
- （2）不能重载 '，'，因为 '，' 在类中对任何成员都有意义，已经成为标准用法。
- （3）不能重载目前 C++ 运算符集合中没有的符号，如 #, @, \$ 等。原因有两点，一是难以理解，二是难以确定优先级。
- （4）对已经存在的运算符进行重载时，不能改变优先级规则，否则将引起混乱。

475.用C++设计一个不能继承的类



```

1 template <typename T>
2
3 class A
4 {
5     friend T;
6 private:
7     A() {}
8     ~A() {}
9 };

```



```
10
11 class B : virtual public A<B>
12 {
13 public:
14     B() {}
15     ~B() {}
16 };
17
18 void main( void )
19 {
20     B b;
21     return;
22 }
```



476.构造函数能否为虚函数

构造函数不可以，析构函数可以。

因为对象中的虚函数表指针是在构造函数初始化列表阶段才初始化的。

如果它的构造函数是虚函数，那就要通过对象中的虚函数表指针来调用，而这个虚函数表是在构造函数初始化列表阶段才初始化的。

https://blog.csdn.net/qj_41830537/article/details/102626648

477.在C中用const 能定义真正意义上的常量吗？C++中的const呢？

const修饰的变量是只读的，本质上还是变量，与C语言不同，c++中的const不是只读变量。**C++中的const是个真正意义上的常量。

C语言中的const还是可变的：



```
1 int main()
2 {
3     const int c = 0;
4     int* p = (int*)&c;
5     printf("Begin...\n");
6     *p = 5;
7     printf("c = %d\n", c); //C语言编译：输出5，
8                             //说明c并不是真正意义上的常量
9     return 0;
10 }
```



由于c++中const常量的值在编译期就已经决定，下面的做法是OK的，但是c中是编译通不过的。



```
1 int main(void)
2 {
3     const int a = 8;
4     const int b = 2;
5     int array[a+b] = {0};
6     return 0;
7 }
```



478.宏和内联（inline）函数的比较？

- (1)内联（inline）函数是C++中引用的一个新的关键字；C++中推荐使用内联函数来替代宏代码片段；
 - (2)内联函数将函数体直接扩展到调用内联函数的地方，这样减少了参数压栈，跳转，返回等过程；
 - (3)由于内联发生在编译阶段，所以内联相较于宏，是有参数检查和返回值检查的，因此使用起来更为安全；
 - (4)内联函数不能过于复杂，最初C++限定不能有任何形式的循环，不能有过多的条件判断，不能对函数进行取地址操作等，
- 但是现在的编译器几乎没有什么限制，基本都可以实现内联。

479.typedef和define由什么区别

用法不同：typedef用来定义一种数据类型的别名，增强程序的可读性。define主要用来定义常量，以及书写复杂使用频繁的宏。

执行时间不同：typedef是编译过程的一部分，有类型检查的功能。define是宏定义，是预编译的部分，其发生在编译之前，只是简单的进行字符串的替换，不进行类型的检查。

https://blog.csdn.net/qq_45607873/article/details/123746610

480.strcat、strncat、strcpy哪些函数会导致内存溢出？如何改进？

strcpy函数会导致内存溢出。

strcpy拷贝函数不安全，他不做任何的检查措施，也不判断拷贝大小，不判断目的地址内存是否够用。

```
1 char *strcpy(char *strDest, const char *strSrc)
```

strncat()主要功能是在字符串的结尾追加n个字符。

```
1 char * strncat(char *dest, const char *src, size_t n);
```

strcat()函数主要用来将两个char类型连接。例如：

```
1 char d[20]="Golden";  
2 char s[20]="View";  
3 strcat(d,s); //打印d printf("%s",d);
```

输出 d 为 GoldenView （中间无空格）

****延伸****：

memcpy拷贝函数，它与strcpy的区别就是memcpy可以拷贝任意类型的数据，strcpy只能拷贝字符串类型。

memcpy 函数用于把资源内存（src所指向的内存区域）拷贝到目标内存（dest所指向的内存区域）；

有一个size变量控制拷贝的字节数；

函数原型：

```
1 void *memcpy(void *dest, void *src, unsigned int count);
```

481.简述队列和栈的异同

队列和栈都是线性存储结构，但是两者的插入和删除数据的操作不同，队列是“先进先出”，栈是“后进先出”。

482.堆和栈的区别？

堆存放动态分配的对象——即那些在程序运行时动态分配的对象，比如 malloc出来的对象，其生存期由程序控制，必须手动申请，手动释放。

栈用来保存定义在函数内的非static对象，如局部变量，仅在其定义的程序块运行时才存在，由编译器自动创建和销毁。

483.堆和自由存储区的区别？

总的来说，堆是C语言和操作系统的术语，是操作系统维护的一块动态分配内存；自由存储是C++中通过new与delete动态分配和释放对象的抽象概念。他们并不是完全一样。

从技术上来说，堆（heap）是C语言和操作系统的术语。堆是操作系统所维护的一块特殊内存，它提供了动态分配的功能，当运行程序调用malloc()时就会从中分配，

稍后调用free可把内存交还。而自由存储是C++中通过new和delete动态分配和释放对象的抽象概念，通过new来申请的内存区域可称为自由存储区。

484.什么是内存泄漏？面对内存泄漏有什么避免方法

用动态存储分配函数动态开辟的空间，在使用完毕后未释放，结果导致一直占据该内存单元即为内存泄露。

malloc的时候得确定在那里free. new的时候得确定delete。在C++中应该优先考虑使用智能指针。

485.链表和数组的区别

存储形式：数组是一块连续的空间，声明时就要确定长度。链表是一块可不连续的动态空间，长度可变，每个结点要保存相邻结点指针。

数据查找：数组的线性查找速度快，查找操作直接使用偏移地址。链表需要按顺序检索结点，效率低。

数据插入或删除：链表可以快速插入和删除结点，而数组则可能需要大量数据移动。

越界问题：链表不存在越界问题，数组有越界问题。

486.结构与联合有和区别

在使用结构变量时要先对其定义；结构名是结构的标识符不是变量名；结构是按变量名字来访问成员的；结构体指针采用“->”访问成员，结构体变量采用“.”访问。

结构和联合都是由多个不同的数据类型成员组成，但在任何同一时刻，联合中只存放了一个被选中的成员（所有成员共用一块地址空间），而结构的

所有成员都存在（不同成员的存放地址不同）。

对于联合的不同成员赋值，将会对其它成员重写，原来成员的值就不存在了，而对于结构的不同成员赋值是互不影响的。

487.什么是“引用”？申明和使用“引用”要注意哪些问题？

引用就是某个目标变量的“别名”(alias)，对应用的操作与对变量直接操作效果完全相同。

声明一个引用，不是新定义了一个变量，它只表示该引用名是目标变量名的一个别名，它本身不是一种数据类型，因此引用本身不占存储单元，系统也不给引用

分配存储单元。不能建立数组的引用。

488.将“引用”作为函数参数有哪些特点？

传递引用给函数与传递指针的效果是一样的。这时，被调函数的形参就成为原来主调函数中的实参变量或对象的一个别名来使用，所以在被调函数中对形参变量的

操作就是对其相应的目标对象（在主调函数中）的操作。

使用引用传递函数的参数，在内存中并没有产生实参的副本，它是直接对实参操作。

489.STL标准模板库

vector单端数组，当现有的内存空间不够装下数据时，首先配置一块新的空间，然后将旧空间的数据搬往新空间，再释放原来的空间。

deque双端数组，可以在头尾两端分别做元素的插入和删除操作，除非必要，应尽可能的选择使用vector而非deque，因为deque的迭代器比vector的迭代器要复杂的多。

list双向链表，相较于vector的连续线性空间，list就显得负责许多，它的好处是每次插入或者删除一个元素，就是配置或者释放一个元素的空间，适合频繁的不确实位置元素的移除插入。

set二叉树，map二叉树，底层实现都是红黑树，它可以在 $O(\log n)$ 时间内高效的做查找，插入和删除。

490.陷阱题



```
1 int main()
2 {
3     int j=2;
4     int i=1;
5     if(i = 1) j=3;
6     if(i = 2) j=5;
7     printf("%d",j);
8 }
9 输出为5；如果再加上if(i=3)j=6;则输出6。
```



```
1 int main(void)
2 {
3     unsigned int a = 6;
4     int b = -20;
5     char c;
6     (a+b>6)?(c=1):(c=0);
7 }则c=1，但a+b=-14；如果a为int类型则c=0。
8 原来有符号数和无符号数进行比较运算时（==，<，>，<=，>=），有符号数隐式转换成了无符号数（即底层的补码不变，但是此数从有符号数变成了无符号数），
9 比如上面 (a+b)>6这个比较运算，a+b=-14，-14的补码为1111111111110010。此数进行比较运算时，
10 被当成了无符号数，它远远大于6，所以得到上述结果。
```



```
1 main()
2 {
3     int a[5]={1,2,3,4,5};
4     int *ptr=(int *)(&a+1);//&a相当于变成了行指针，加1则变成了下一行首地址
5     printf("%d,%d",*(a+1),*(ptr-1));
6 }
7 *(a+1) 就是a[1]，*(ptr-1)就是a[4]，执行结果是2，5
```



```
1 int main()
2 {
3     char a;
4     char *str=&a;
5     strcpy(str,"hello");
6     printf(str);
7     return 0;
8 }
9 将字符串赋值给字符变量，内存不够，存在越界。
```



```
1 void test(void) {
2     char *p = null;
3     strcpy(p,"hello");
4     printf("%s\n",p);
5 }
6 段错误
7
8 野指针问题
```



```
1 wap( int* p1,int* p2 )
2 {
3     int *p;
4     *p = *p1;
5     *p1 = *p2;
6     *p2 = *p;
7 }
8 段错误，p没有空间，为其分配空间或者不使用指针
```



```
1 char *RetMemory(void)
2 {
3     char p[] = "hellow world";
4     return p;
5 }
6 void Test(void)
7 {
8     char *str = NULL;
9     str = RetMemory();
10    printf(str);
11 }
12 由于p[]是自动变量，所以RetMemory执行完毕时，资源被回收，p指向未知地址。因此str的内容应是
    不可预测的，打印的应该是str的地址
13
```

```
14
15 void getmemery(char *p)
16 {
17     p = (char *)malloc(100);
18 }
19
20 main()
21 {
22     char *str = NULL;
23     getmemery(str);
24     strcpy(str,"hello world!");
25     printf("%s\n",str);
26 }    段错误
27
```

28 问题出在这，上述代码传入getmemery(char *p)函数的字符串指针是形参，在函数内部修改形参并不能真正的改变传入形参的值，

29 执行完char *str = NULL; gememory(str);后的str仍为NULL；

30 一般函数的传递都是值传递，不会改变函数外的变量值。简单地说，就是形参不能够改变实参，实参只是复制了一份给形参！其自身并没有被改变

```
31
32 void getmemery(char **p)
33 {
34     *p = (char *)malloc(100);
35 }
36
37 main()
38 {
39     char *str = NULL;
40     getmemery(&str);
41     strcpy(*str,"hello world!");
42     printf("%s\n",*str);
43 }
```

44 这就是我们常说的“地址传递”，将str的地址传给getmemery()函数，getmemery()函数就会通过地址修改str里面的值，这样就会得到正确的结果。

45 所以，我们要记住函数传参的两种方式：1) 值传递 2) 地址传递。



1 下面两种if语句判断方式。请问哪种写法更好？为什么？

```
2 int n;
3 if (n == 10) // 第一种判断方式
4 if (10 == n) // 第二种判断方式
```

5 【答案】这是一个风格问题，第二种方式如果少了个=号，编译时就会报错，减少了出错的可能行，可以检测出是否少了=。



```
1 #include "stdafx.h"
2 #include <iostream>
3 using namespace std;
4
5 class human {
6 public:
7     ~human() {
```



```

8 //析构函数,该类对象在生存期结束的时候会自动调用的一个函数
9     cout << "human over....." << endl;
10 }
11 void Disp() {
12     cout << "human disp....." << endl;
13 }
14 };
15
16 class man : public human
17 {
18 public:
19     ~man() {
20         cout << "man over....." << endl;
21     }
22     void Disp() {
23         cout << "man disp....." << endl;
24     }
25 };
26
27 int main()
28 {
29     human* p = new man;    //定义和初始化指向一个类的指针变量
30     p->Disp();             //指向成员运算符,得到一个指针指向的元素
31     delete p;
32     system("pause");
33     return 0;
34 }
35 答案】
36 human disp...
37 human over...

```



```

1 以下三条输出语句分别输出什么？
2 char str1[] = "abc";
3 char str2[] = "abc";
4 const char str3[] = "abc";
5 const char str4[] = "abc";
6 const char* str5 = "abc";
7 const char* str6 = "abc";
8 cout << boolalpha << ( str1 == str2 ) << endl; // 输出什么？
9 cout << boolalpha << ( str3 == str4 ) << endl; // 输出什么？
10 cout << boolalpha << ( str5 == str6 ) << endl; // 输出什么？
11 【答案】分别输出false,false,true。str1和str2都是字符数组，每个都有其自己的存储区，它们的值则是各存储区首地址，不等；str3和str4同上，只是按const语义，它们所指向的数据区不能修改。str5和str6并非数组而是字符指针，并不分配存储区，其后的“abc”以常量形式存于静态数据区，而它们自己仅是指向该区首地址的指针，相等。

```



```
1 int a=5, b=7, c;  
2 c = a++b;  
3 【答案】a=6,b=7,c=12
```



```
1 一个栈的入栈序列是A, B, C, D, E, 则栈的不可能的输出序列是 ( )  
2 A、EDCBA;  
3 B、DECBA;  
4 C、DCEAB;  
5 D、ABCDE  
6 【答案】C
```



原文作者：[xiaogudexuexibiji](https://www.cnblogs.com/xiaogudexuexibiji)

原文链接：<https://www.cnblogs.com/xiaogudexuexibiji/p/16398905.html>

491.一个C++源文件从文本到可执行文件经历的过程

对于C/C++编写的程序，从源代码到可执行文件，一般经过下面四个步骤：

- 1).预处理，产生.i文件
- 2).编译，产生汇编文件(.s文件)
- 3).汇编，产生目标文件(.o或.obj文件)
- 4).链接,产生可执行文件(.out或.exe文件)

492.#include 的顺序以及尖括号和双引号的区别

1. #include的顺序的区别：

头文件的引用顺序对于程序的编译还是有一定影响的。如果要在文件a.h中声明一个在文件b.h中定义的变量，而不引用b.h。那么要在a.c文件中引用b.h文件，并且要先引用b.h，后引用a.h,否则汇报变量类型未声明错误，也就是常见的某行少个“；”符号。

2. #include尖括号和双引号的区别：

- 1) #include<>，认为该头文件是标准头文件。编译器将会在预定义的位置集查找该头文件，这些预定义的位置可以通过设置查找路径环境变量或者通过命令行选项来修改。使用的查找方式因编译器的不同而差别迥异。
- 2) #include""，认为它是非系统头文件，非系统头文件的查找通常开始于源文件所在的路径。查找范围大于<>。

493.进程和线程，为什么要有线程

- 1、和进程相比，它是一种非常“节俭”的多任务操作方式。在linux系统下，启动一个新的进程必须分配给它独立的地址空间，建立众多的数据表来维护它的代码段、堆栈段和数据段，这是一种“昂贵”的多任务工作方式。（资源）
- 2、运行于一个进程中的多个线程，它们之间使用相同的地址空间，而且线程间彼此切换所需时间也远远小于进程间切换所需要的时间。据统计，一个进程的开销大约是一个线程开销的30倍左右。（切换效率）

3、线程间方便的通信机制。对不同进程来说，它们具有独立的数据空间，要进行数据的传递只能通过进程间通信的方式进行，这种方式不仅费时，而且很不方便。线程则不然，由于同一进程下的线程之间贡献数据空间，所以一个线程的数据可以直接为其他线程所用，这不仅快捷，而且方便。

（通信）除以上优点外，多线程程序作为一种多任务、并发的工作方式，还有如下优点：

- 1、使多CPU系统更加有效。操作系统会保证当线程数不大于CPU数目时，不同的线程运行于不同的CPU上。（CPU设计保证）
- 2、改善程序结构。一个既长又复杂的进程可以考虑分为多个线程，成为几个独立或半独立的运行部分，这样的程序才会利于理解和修改。（代码易维护）

494.C++11有哪些新特性

- 1) 关键字及新语法：auto、nullptr、for
- 2) STL容器：std::array、std::forward_list、std::unordered_map、std::unordered_set
- 3) 多线程：std::thread、std::atomic、std::condition_variable
- 4) 智能指针内存管理：std::shared_ptr、std::weak_ptr
- 5) 其他：std::function、std::bind和lambda表达式

495.malloc的原理，brk系统调用干什么的，mmap呢

malloc的实现方案：

- 1) malloc函数的实质是它有一个将可用的内存块连接为一个长长的列表的所谓空闲链表。
- 2) 调用malloc（）函数时，它沿着连接表寻找一个大到足以满足用户请求所需要的内存块。然后，将该内存块一分为二（一块的大小与用户申请的大小相等，另一块的大小就是剩下的字节）。接下来，将分配给用户的那块内存存储区域传给用户，并将剩下的那块（如果有的话）返回到连接表上。
- 3) 调用free函数时，它将用户释放的内存块连接到空闲链表上。
- 4) 到最后，空闲链会被切成很多的小内存片段，如果这时用户申请一个大的内存片段，那么空闲链表上可能没有可以满足用户要求的片段了。于是，malloc（）函数请求延时，并开始在空闲链表上检查各内存片段，对它们进行内存整理，将相邻的小空闲块合并成较大的内存块。

brk和mmap：

从操作系统角度来看，进程分配内存有两种方式，分别由两个系统调用完成：brk和mmap（不考虑共享内存）。

- 1、brk是将数据段(.data)的最高地址指针_edata往高地址推；
- 2、mmap是在进程的虚拟地址空间中（堆和栈中间，称为文件映射区域的地方）找一块空闲的虚拟内存。

这两种方式分配的都是虚拟内存，没有分配物理内存。在第一次访问已分配的虚拟地址空间的时候，发生缺页中断，操作系统负责分配物理内存，然后建立虚拟内存和物理内存之间的映射关系。

在标准C库中，提供了malloc/free函数分配释放内存，这两个函数底层是由brk，mmap，munmap这些系统调用实现的。

496.C++的内存管理方式，STL的allocator，最新版本默认使用的分配器

C++的内存管理方式：

在c++中内存主要分为5个存储区：

栈（Stack）：局部变量，函数参数等存储在该区，由编译器自动分配和释放。栈属于计算机系统的数据结构，进栈出栈有相应的计算机指令支持，而且分配专门的寄存器存储栈的地址，效率分高，内存空间是连续的，但栈的内存空间有限。

堆(Heap)：需要程序员手动分配和释放（new,delete），属于动态分配方式。内存空间几乎没有限制，内存空间不连续，因此会产生内存碎片。操作系统有一个记录空闲内存的链表，当收到内存申请时遍历链表，找到第一个空间大于申请空间的堆节点，将该节点分配给程序，并将该节点从链表中删除。一般，系统会在该内存空间的首地址处记录本次分配的内存大小，用于delete释放该内存空间。

全局/静态存储区：全局变量，静态变量分配到该区，到程序结束时自动释放，包括DATA段（全局初始化区）与BSS段（全局未初始化段）。其中，初始化的全局变量和静态变量存放在DATA段，未初始化的全局变量和静态变量存放在BSS段。BSS段特点：在程序执行前BSS段自动清零，所以未初始化的全局变量和静态变量在程序执行前已经成为0。

文字常量区：存放常量，而且不允许修改。程序结束后由系统释放。程序代码区：存放程序的二进制代码

SGI 版本STL的默认配置器std::alloc参见：

《STL源码剖析》

1) 考虑到小型区块所可能造成的内存碎片问题，SGI设计了双层配置器。第一级配置器直接使用malloc()和free()；第二级则视情况采取不同的策略：当配置区块超过128bytes时，视为“足够大”，便调用第一级配置器；当配置区块小于128bytes时，视之为“过小”，为了降低额外负担，便采用memory pool（内存池）整理方式，而不在求助于第一级配置器。

2) 内存池的核心：内存池和16个自由链表（各自管理8,16, ..., 128bytes的小额区块）。在分配一个小区块时，首先在所属自由链表中寻找，如果找到，直接抽出分配；若所属自由链表为空，则请求内存池为所属自由链表分配空间；默认情况下，为该自由链表分配20个区块，若内存池剩余容量不足，则分配可分配的最大容量；若内存池连一个区块都无法分配，则调用chunk_alloc为内存池分配一大块区块；若内存不足，则尝试调用malloc分配，否则返回bad_alloc异常。

497.hash表的实现，包括STL中的哈希桶长度常数。

hash表的实现主要涉及两个问题：散列函数和碰撞处理。

1) hash function（散列函数）。最常见的散列函数： $f(x) = x \% \text{TableSize}$ 。

2) 碰撞问题（不同元素的散列值相同）。解决碰撞问题的方法有许多种，包括线性探测、二次探测、开链等做法。SGI版本使用开链法，使用一个链表保持相同散列值的元素。虽然开链法并不要求表格大小必须为质数，但SGI STL仍然以质数来设计表格大小，并且将28个质数（逐渐呈现大约两倍的关系）计算好，以备随时访问，同时提供一个函数，用来查询在这28个质数之中，“最接近某数并大于某数”的质数。

498.hash表如何rehash，怎么处理其中保存的资源

先想想为什么需要rehash：

因为，当loadFactor（负载因子） ≤ 1 时，hash表查找的期望复杂度为 $O(1)$ 。因此，每次往hash表中添加元素时，我们必须保证是在loadFactor < 1 的情况下，才能够添加。

模仿C++的vector扩容方式，Hash表中每次发现loadFactor==1时，就开辟一个原来桶数组的两倍空间（称为新桶数组），然后把原来的桶数组中元素全部转移过来到新的桶数组中。注意这里转移是需要元素一个个重新哈希到新桶中的。

499.Redis的rehash怎么做的，为什么要渐进rehash，渐进rehash怎么实现的

为了避免rehash对服务器造成影响，服务器不是一次将ht[0]里面的所有键值对全部rehash到ht[1]，而是分多次、渐进式地将ht[0]里面的键值对慢慢地rehash到ht[1]。

以下是哈希表渐进式rehash的详细步骤：

为 ht[1] 分配空间，让字典同时持有 ht[0] 和 ht[1] 两个哈希表。

在字典中维持一个索引计数器变量 rehashidx，并将它的值设置为 0，表示rehash工作正式开始。

在rehash进行期间，每次对字典执行添加、删除、查找或者更新操作时，程序除了执行指定的操作以外，还会顺带将 ht[0] 哈希表在 rehashidx 索引上的所有键值对rehash到 ht[1]，当rehash工作完成之后，程序将 rehashidx 属性的值增一。

随着字典操作的不断执行，最终在某个时间点上，ht[0] 的所有键值对都会被rehash至 ht[1]，这时程序将 rehashidx 属性的值设为 -1，表示rehash操作已完成。

渐进式rehash的好处在于它采取分而治之的方式，将rehash键值对所需的计算工作均摊到对字典的每个添加、删除、查找和更新操作上，从而避免了集中式rehash而带来的庞大计算量。

500.Redis的定时机制怎么实现的，有哪些弊端，你将如何改进这个弊端

Redis服务器是一个事件驱动程序，服务器需要处理以下两类事件：文件事件（服务器对套接字操作的抽象）和时间事件（服务器对定时操作的抽象）。Redis的定时机制就是借助时间事件实现的。

一个时间事件主要由以下三个属性组成：id：时间事件标识号；when：记录时间事件的到达时间；timeProc：时间事件处理器，当时间事件到达时，服务器就会调用相应的处理器来处理时间。一个时间事件根据时间事件处理器的返回值来判断是定时事件还是周期性事件。

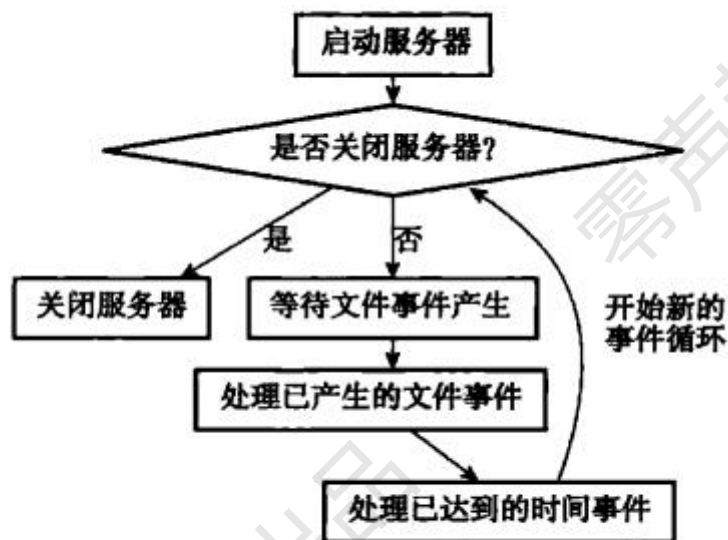


图 12-10 事件处理角度下的服务器运行流程

弊端：Redis对时间事件的实际处理时间并不准时，通常会比时间事件设定的到达事件稍晚一些。

改进：多线程？一个处理文件事件，一个处理时间事件？（不确定）。

501.Redis是单线程的，为什么这么高效

虽然Redis文件事件处理器以单线程方式运行，但是通过使用I/O多路复用程序来监听多个套接字，文件事件处理器既实现了高性能的网络通信模型，又可以很好地与Redis服务器中其他同样以单线程运行的模块进行对接，这保持了Redis内部单线程设计的简单性。

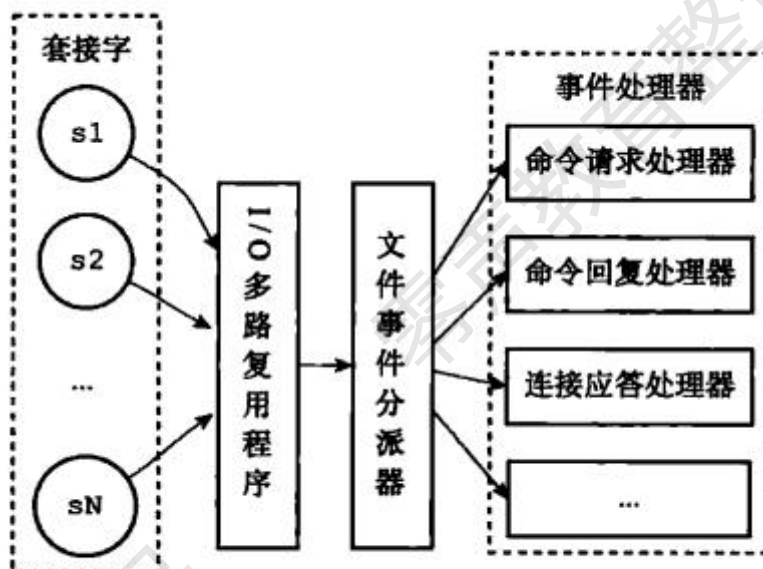


图 12-1 文件事件处理器的四个组成部分

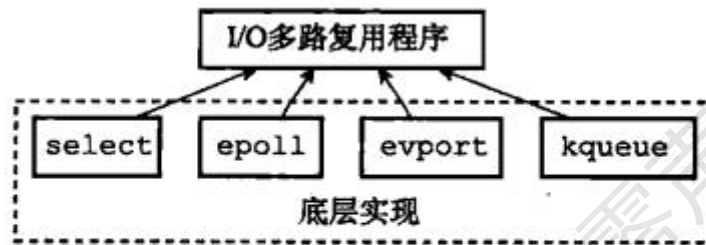


图 12-3 Redis 的 I/O 多路复用程序有多个 I/O 多路复用库实现可选

502.Redis的数据类型有哪些，底层怎么实现

- 1) 字符串：整数值、embstr编码的简单动态字符串、简单动态字符串（SDS）
- 2) 列表：压缩列表、双端链表
- 3) 哈希：压缩列表、字典
- 4) 集合：整数集合、字典
- 5) 有序集合：压缩列表、跳跃表和字典

503.Redis和memcached的区别

Redis和memcached的区别：

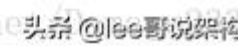
- 1) 数据类型：redis数据类型丰富，支持set list等类型；memcache支持简单数据类型，需要客户端自己处理复杂对象
- 2) 持久性：redis支持数据落地持久化存储；memcache不支持数据持久存储。
- 3) 分布式存储：redis支持master-slave复制模式；memcache可以使用一致性hash做分布式。
- 4) value大小不同：memcache是一个内存缓存，key的长度小于250字符，单个item存储要小于1M，不适合虚拟机使用
- 5) 数据一致性不同：redis使用的是单线程模型，保证了数据按顺序提交；memcache需要使用cas保证数据一致性。CAS（Check and Set）是一个确保并发一致性的机制，属于“乐观锁”范畴；原理很简单：拿版本号，操作，对比版本号，如果一致就操作，不一致就放弃任何操作
- 6) cpu利用：redis单线程模型只能使用一个cpu，可以开启多个redis进程

504.TCP的模型，状态转移

TCP四层模型：

头条 @166哥说网络

熟悉三次握手和四次释放的TCP状态转移。



505. 用过吗？ 全问题

设计模式

1) TemplateMethod模式：《effectivec++》条款35：借助Non-virtualInterface手法实现TemplateMethod模式

2) Strategy模式：《effectivec++》条款35：借助FunctionPointers实现Strategy模式、借助std::function完成Strategy模式、古典Strategy模式

506.用过多线程吗，以前的多线程代码还能怎么优化，线程池的实现

线程的创建

```
#include <pthread.h>

int pthread_create(pthread_t *restrict thread,
                  const pthread_attr_t *restrict attr,
                  void (start_routine)(void*),
                  void *restrict arg);
```

线程终止：

从线程函数**return**。这种方法对主线程不适用，从main函数return相当于调用exit。

一个线程可以调用**pthread_cancel**终止同一进程中的另一个线程。

线程可以调用**pthread_exit**终止自己。

507.epoll怎么实现的，reactor模型组成

epoll实现：

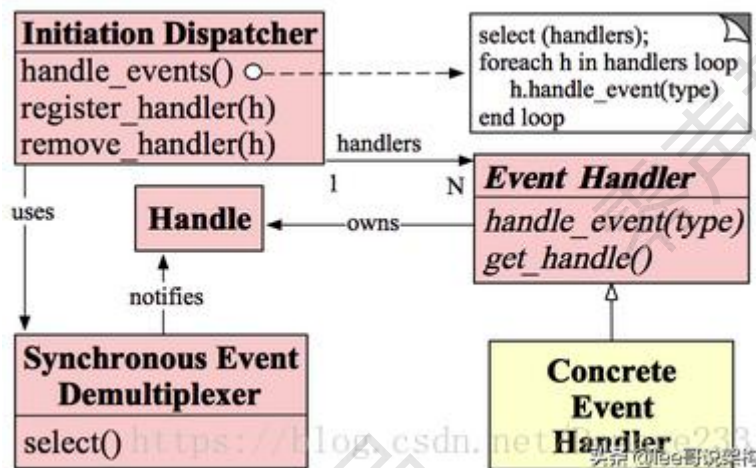
第一步：epoll_create()系统调用。此调用返回一个句柄，之后所有的使用都依靠这个句柄（eventpoll的对象）来标识。

```
struct eventpoll{
    ....
    /红黑树的根节点，这颗树中存储着所有添加到epoll中的需要监控的事件/
    struct rb_rootrb;
    /双链表中则存放着将要通过epoll_wait返回给用户的满足条件的事件/
    struct list_headrdlist;
    ....
};
```

第二步：epoll_ctl()系统调用。通过此调用向epoll对象中添加、删除、修改感兴趣的事件，返回0标识成功，返回-1表示失败。

第三步：epoll_wait()系统调用。通过此调用收集收集在epoll监控中已经发生的事件。

Reactor模型：



1) **Handle**：即操作系统中的句柄，是对资源在操作系统层面上的一种抽象，它可以是打开的文件、一个连接(Socket)、Timer等。由于Reactor模式一般使用在网络编程中，因而这里一般指Socket Handle，即一个网络连接。

2) **Synchronous Event Demultiplexer**（同步事件复用器）：阻塞等待一系列的Handle中的事件到来，如果阻塞等待返回，即表示在返回的Handle中可以不阻塞的执行返回的事件类型。这个模块一般使用操作系统的select来实现。

3) **Initiation Dispatcher**：用于管理EventHandler，即EventHandler的容器，用以注册、移除EventHandler等；另外，它还作为Reactor模式的入口调用SynchronousEventDemultiplexer的select方法以阻塞等待事件返回，当阻塞等待返回时，根据事件发生的Handle将其分发给对应的EventHandler处理，即回调EventHandler中的handle_event()方法。

4) **EventHandler**：定义事件处理方法：handle_event()，以供InitiationDispatcher回调使用。

5) **ConcreteEventHandler**：事件EventHandler接口，实现特定事件处理逻辑。

508.线程间的同步方式，最好说出具体的系统调用

1) 互斥量 (mutex)

```
#include <pthread.h>

int pthread_mutex_destroy(pthread_mutex_t *mutex); //销毁

int pthread_mutex_init(pthread_mutex_t *restrict mutex,
const pthread_mutexattr_t *restrict attr); //初始化

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int pthread_mutex_lock(pthread_mutex_t *mutex); //上锁

int pthread_mutex_trylock(pthread_mutex_t *mutex); //尝试上锁

int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

2) 条件变量 (Condition Variable)

```
#include <pthread.h>

int pthread_cond_destroy(pthread_cond_t *cond);    //销毁

int pthread_cond_init(pthread_cond_t *restrict cond,
const pthread_condattr_t *restrict attr);    //初始化

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_timedwait(pthread_cond_t *restrict cond,
pthread_mutex_t *restrict mutex,
const struct timespec *restrict abstime);

int pthread_cond_wait(pthread_cond_t *restrict cond,
pthread_mutex_t *restrict mutex);

int pthread_cond_broadcast(pthread_cond_t *cond);

int pthread_cond_signal(pthread_cond_t *cond);
```

3) 信号量 (Semaphore)

```
#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);

int sem_wait(sem_t *sem);

int sem_trywait(sem_t *sem);

int sem_post(sem_t * sem);

int sem_destroy(sem_t * sem);

_____
```

调用sem_wait()可以获得资源，使semaphore的值减1，如果调用sem_wait()时semaphore的值已经是0，则挂起等待。如果不希望挂起等待，可以调用sem_trywait()。调用sem_post()可以释放资源，使semaphore的值加1，同时唤醒挂起等待的线程。

509. 哈希表的桶个数为什么是质数，合数有何不妥？

质数比合数更容易避免冲撞，也就是说使用质数时，哈希效果更好，原始数据经哈希后分布更均匀。

原文作者：Yt_cc

原文链接：<https://www.cnblogs.com/albert32/p/13428021.html>

510. C/C++内存有哪几种类型？

C中，内存分为5个区：堆(malloc)、栈(如局部变量、函数参数)、程序代码区（存放二进制代码）、全局/静态存储区（全局变量、static变量）和常量存储区（常量）。此外，C++中有自由存储区（new）一说。

全局变量、static变量会初始化为缺省值，而堆和栈上的变量是随机的，不确定的。

511.堆和栈的区别？

- 1).堆存放动态分配的对象——即那些在程序运行时动态分配的对象，比如 new 出来的对象，其生存期由程序控制；
- 2).栈用来保存定义在函数内的非static对象，如局部变量，仅在其定义的程序块运行时才存在；
- 3).静态内存用来保存static对象，类static数据成员以及定义在任何函数外部的变量，static对象在使用之前分配，程序结束时销毁；
- 4).栈和静态内存的对象由编译器自动创建和销毁。

512.堆和自由存储区的区别？

总的来说，堆是C语言和操作系统的术语，是操作系统维护的一块动态分配内存；自由存储是C++中通过new与delete动态分配和释放对象的抽象概念。他们并不是完全一样。

从技术上来说，堆（heap）是C语言和操作系统的术语。堆是操作系统所维护的一块特殊内存，它提供了动态分配的功能，当运行程序调用malloc()时就会从中分配，稍后调用free可把内存交还。而自由存储是C++中通过new和delete动态分配和释放对象的抽象概念，通过new来申请的内存区域可称为自由存储区。基本上，所有的C++编译器默认使用堆来实现自由存储，也即是缺省的全局运算符new和delete也许会按照malloc和free的方式来被实现，这时藉由new运算符分配的对象，说它在堆上也对，说它在自由存储区上也正确。

513.程序编译的过程？

程序编译的过程中就是将用户的文本形式的源代码(c/c++)转化成计算机可以直接执行的机器代码的过程。主要经过四个过程：预处理、编译、汇编和链接。具体示例如下。

一个hello.c的c语言程序如下。

```
#include <stdio.h>
int main()
{
    printf("happy new year!\n");
    return 0;
}
```

其编译过程如下：



514.计算机内部如何存储负数和浮点数？

负数比较容易，就是通过一个标志位和补码来表示。

拓展问题：

什么是补码？

负数补码为反码加1

正数补码为原码

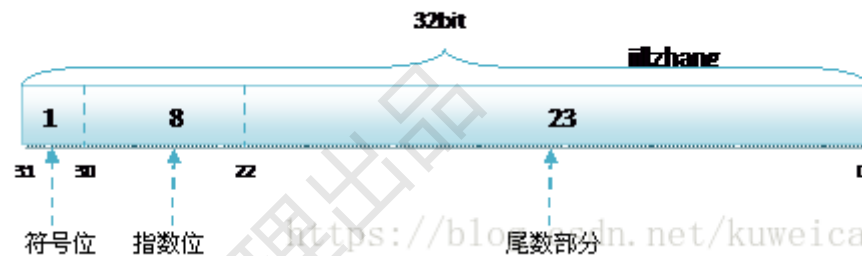
负数为什么用补码？

统一加减法，正负零问题

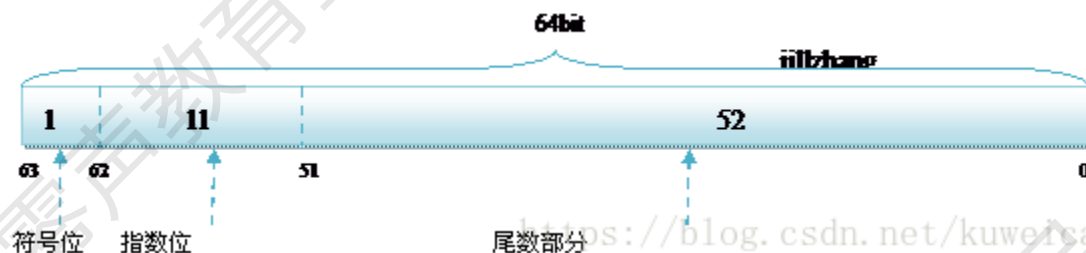
对于浮点类型的数据采用单精度类型 (float) 和双精度类型(double)来存储, float数据占用32bit,double数据占用64bit,我们在声明一个变量float f= 2.25f的时候,是如何分配内存的呢?如果胡乱分配,那世界岂不是乱套了么,其实不论是float还是double在存储方式上都是遵从IEEE的规范的, float遵从的是IEEE R32.24 ,而double 遵从的是R64.53。更多可以参考浮点数表示。无论是单精度还是双精度在存储中都分为三个部分：

- 1). 符号位(Sign) : 0代表正, 1代表为负
- 2). 指数位 (Exponent) :用于存储科学计数法中的指数数据, 并且采用移位存储
- 3). 尾数部分 (Mantissa) : 尾数部分

其中float的存储方式如下图所示：



而双精度的存储方式如下图:



515.函数调用的过程？

如下结构的代码，

```
int main(void)
{
    ...
    d = fun(a, b, c);
    cout<<d<<endl;
    ...
    return 0;
}
```

调用fun()的过程大致如下：

main()=====

- 1).参数拷贝（压栈），注意顺序是从右到左，即c-b-a；
- 2).保存 d = fun(a, b, c)的下一条指令，即cout<<d<<endl（实际上是这条语句对应的汇编指令的起始位置）；

- 3).跳转到fun()函数，注意，到目前为止，这些都是在main()中进行的；

fun()=====

- 4).移动ebp、esp形成新的栈帧结构;
- 5).压栈（push）形成临时变量并执行相关操作;
- 6).return一个值;

7).出栈 (pop) ;
8).恢复main函数的栈帧结构;
9).返回main函数;
main()=====

...

516.左值和右值

不是很严谨的来说，左值指的是既能够出现在等号左边也能出现在等号右边的变量(或表达式)，右值指的则是只能出现在等号右边的变量(或表达式)。举例来说我们定义的变量 `a` 就是一个左值，而 `malloc` 返回的就是一个右值。或者左值就是在程序中能够寻址的东西，右值就是一个具体的真实的值或者对象，没法取到它的地址的东西(不完全准确)，因此没法对右值进行赋值，但是右值并非是不可修改的，比如自己定义的 `class`，可以通过它的成员函数来修改右值。

归纳一下就是：

- 可以取地址的，有名字的，非临时的就是左值
- 不能取地址的，没有名字的，临时的，通常生命周期就在某个表达式之内的是右值

但是到了 C++11 之后概念变的略微复杂，引入了 `lvalue`, `glvalue`, `rvalue`, `xvalue` 和 `prvalue`。具体可以参考 [What are rvalues, lvalues, xvalues, glvalues, and prvalues?](#)

517.什么是内存泄漏？面对内存泄漏和指针越界，你有哪些方法？你通常采用哪些方法来避免和减少这类错误？

用动态存储分配函数动态开辟的空间，在使用完毕后未释放，结果导致一直占据该内存单元即为内存泄露。

- 1). 使用的时候要记得指针的长度.
- 2). `malloc`的时候得确定在那里`free`.
- 3). 对指针赋值的时候应该注意被赋值指针需要不需要释放.
- 4). 动态分配内存的指针最好不要再次赋值.
- 5). 在C++中应该优先考虑使用智能指针.

518.C++11 中有哪些智能指针？`shared_ptr` 的引用计数是如何实现的？`unique_ptr` 的`unique`是如何实现的？`make_shared` 和 `make_unique` 的作用？智能指针使用注意事项？

C++ 11 中的智能指针有：`shared_ptr`, `unique_ptr` 和 `weak_ptr`。

`shared_ptr` 的引用计数是存放在堆上的，多个 `shared_ptr` 的对象的引用计数都指向同一个堆地址。

`unique_ptr` 中拷贝构造函数和赋值操作符都声明为`delete`或`private`。

优先使用 `make_shared` 和 `make_unique` 的原因是为了避免内存泄露。参考 C++11 中的 `Smart Pointer` (`shared_ptr`/`weak_ptr`/`unique_ptr`) 总结

智能指针使用注意事项：

- 不使用相同的内置指针值初始化，或`reset`多个智能指针
- 不`delete` `get()`返回的指针
- 不使用`get()`初始化或`reset`另一个智能指针
- `get()`返回的智能指针可能变成`dangling pointer`

- 如果智能指针管理的内存不是new出来的，需要提供删除器

拓展问题

- shared_ptr 是否线程安全？
- 侵入式智能指针？

519.C和C++的区别？

- 1). C++是C的超集;
- 2). C是一个结构化语言，它的重点在于算法和数据结构。C程序的设计首要考虑的是如何通过一个过程，对输入（或环境条件）进行运算处理得到输出（或实现过程（事务）控制），而对于C++，首要考虑的是如何构造一个对象模型，让这个模型能够契合与之对应的问题域，这样就可以通过获取对象的状态信息得到输出或实现过程（事务）控制。

520.int fun() 和 int fun(void)的区别？

这里考察的是c 中的默认类型机制。

- 在c中，int fun() 会解读为返回值为int(即使前面没有int，也是如此，但是在c++中如果没有返回类型将报错)，输入类型和个数没有限制，而int fun(void)则限制输入类型为一个void。
- 在c++下，这两种情况都会解读为返回int类型，输入void类型。

521.const 有什么用途

主要有三点：

- 1).定义只读变量，或者常量（只读变量和常量的区别参考下面一条）；
- 2).修饰函数的参数和函数的返回值;
- 3).修饰函数的定义体，这里的函数为类的成员函数，被const修饰的成员函数代表不能修改成员变量的值，因此const成员函数只能调用const成员函数，可以访问非const成员，但是不能修改；
- 4).只读对象。只读对象只能调用const成员函数。

```
class Screen {
public:
    const char cha; //const成员变量
    char get() const; //const成员函数
};

const Screen screen; //只读对象
```

522.在C中用const 能定义真正意义上的常量吗？C++中的const呢？

不能。c中的const仅仅是从编译层来限定，不允许对const 变量进行赋值操作，在运行期是无效的，所以并非是真的常量（比如通过指针对const变量是可以修改值的），但是c++中是有区别的，c++在编译时会把const常量加入符号表，以后（仍然在编译期）遇到这个变量会从符号表中查找，所以在C++中是不可能修改到const变量的。

补充：

- 1) .c中的局部const常量存储在栈空间，全局const常量存在只读存储区，所以全局const常量也是无法修改的，它是一个只读变量。
- 2) .这里需要说明的是，常量并非仅仅是不可修改，而是相对于变量，它的值在编译期已经决定，而不是在运行时决定。
- 3) .c++中的const 和宏定义是有区别的，宏是在预编译期直接进行文本替换，而const发生在编译期，是可以进行类型检查和作用域检查的。
- 4) .c语言中只有enum可以实现真正的常量。
- 5) .c++中只有用字面量初始化的const常量会被加入符号表，而变量初始化的const常量依然只是只读变量。
- 6) .c++中const成员为只读变量，可以通过指针修改const成员的值，另外const成员变量只能在初始化列表中进行初始化。

下面我们通过代码来看看区别。

同样一段代码，在c编译器下，打印结果为*pa = 4，a = 4

在c++编译下打印的结果为 *pa = 4，a = 8

```
int main(void)
{
    const int a = 8;
    int *pa = (int *)&a;
    *pa = 4;
    printf("*pa = %d, a = %d", *pa, a);
    return 0;
}
```

另外值得一说的是，由于c++中const常量的值在编译期就已经决定，下面的做法是OK的，但是c中是编译通不过的。

```
int main(void)
{
    const int a = 8;
    const int b = 2;
    int array[a+b] = {0};
    return 0;
}
```

另外 C++ 11 中引入了 constexpr，专门用来表示常量（在此之前 const 即表示只读，也表示常量）。所以在 C++11 以后，建议凡是「常量」语义的场景都使用 constexpr，只对「只读」语义使用 const。对于 constexpr 修饰的函数表示其结果在编译期就可以算出来（前提是为了算出它所依赖的东西也是在编译期可以算出来的）。更多可以参考：C++ const 和 constexpr 的区别。

```
constexpr int foo(int i)
{
    return i + 5;
}

std::array<int, foo(5)> arr; // OK
```

523.宏和内联（inline）函数的比较？

- 1). 首先宏是C中引入的一种预处理功能；
- 2). 内联（inline）函数是C++中引入的一个新的关键字；C++中推荐使用内联函数来替代宏代码片段；
- 3). 内联函数将函数体直接扩展到调用内联函数的地方，这样减少了参数压栈，跳转，返回等过程；
- 4). 由于内联发生在编译阶段，所以内联相较于宏，是有参数检查和返回值检查的，因此使用起来更为安全；
- 5). 需要注意的是，inline会向编译器提出内联请求，但是是否内联由编译器决定（当然可以通过设置编译器，强制使用内联）；
- 6). 由于内联是一种优化方式，在某些情况下，即使没有显示的声明内联，比如定义在class内部的方法，编译器也可能将其作为内联函数。
- 7). 内联函数不能过于复杂，最初C++限定不能有任何形式的循环，不能有过多的条件判断，不能对函数进行取地址操作等，但是现在的编译器几乎没有什么限制，基本都可以实现内联。

更多请参考inline关键字

524.C++中有了malloc / free，为什么还需要 new / delete？

- 1). malloc与free是C++/C语言的标准库函数，new/delete是C++的运算符。它们都可用于申请动态内存和释放内存。
- 2). 对于非内部数据类型（自定义类型）的对象而言，光用malloc/free无法满足动态对象的要求。对象在创建的同时要自动执行构造函数，对象在消亡之前要自动执行析构函数。

由于malloc/free是库函数而不是运算符，不在编译器控制权限之内，不能够把执行构造函数和析构函数的任务强加于malloc/free。因此C++语言需要一个能完成动态内存分配和初始化工作的运算符new，以一个能完成清理与释放内存工作的运算符delete。注意new/delete不是库函数。

最后补充一点题外话，new 在申请内存的时候就可以初始化（如下代码），而malloc是不允许的。另外，由于malloc是库函数，需要相应的库支持，因此某些简易的平台可能不支持，但是new就没有这个问题了，因为new是C++语言所自带的运算符。

```
int *p = new int(1);
```

特别的，在C++中，如下的代码，用new创建一个对象(new 会触发构造函数，delete会触发析构函数)，但是malloc仅仅申请了一个空间，所以在C++中引入new和delete来支持面向对象。

```
#include <cstdlib>
class Test
{
    ...
}

Test* pn = new Test;
Test* pm = (Test*)malloc(sizeof(Test));
```

525.C和C++中的强制类型转换？

C中是在变量或者表达式前面加上（小括号括起来的）目标类型来进行转换，一招走天下，操作简单，但是由于太过直接，缺少检查，因此容易发生编译检查不到错误，而人工检查又及其难以发现的情况；而C++中引入了下面四种转换：

- 1). static_cast
 - a. 用于基本类型间的转换
 - b. 不能用于基本类型指针间的转换
 - c. 用于有继承关系类对象间的转换和类指针间的转换
- 2). dynamic_cast
 - a. 用于有继承关系的类指针间的转换
 - b. 用于有交叉关系的类指针间的转换
 - c. 具有类型检查的功能
 - d. 需要虚函数的支持
- 3). reinterpret_cast
 - a. 用于指针间的类型转换
 - b. 用于整数和指针间的类型转换
- 4). const_cast
 - a. 用于去掉变量的const属性
 - b. 转换的目标类型必须是指针或者引用

拓展

在C++中，普通类型可以通过类型转换构造函数转换为类类型，那么类可以转换为普通类型吗？答案是肯定的。但是在工程应用中一般不用类型转换函数，因为无法抑制隐式的调用类型转换函数（类型转换构造函数可以通过explicit来抑制其被隐式的调用），而隐式调用经常是bug的来源。实际工程中替代的方式是定义一个普通函数，通过显式的调用来达到类型转换的目的。

```
class test{
    int m_value;
    ...
public:
    operator int() //类型转换函数
    {
        return m_value;
    }

    int toInt() //显示调用普通函数来实现类型转换
    {
        return m_value
    }

};

int main()
{
    ...
    test a(5);
    int i = a; // 相当于 int i = test::operator int(&a)
    ...

    return 0;
}
```

526.static 有什么用途

- 1). 静态 (局部/全局) 变量
- 2). 静态函数
- 3). 类的静态数据成员
- 4). 类的静态成员函数

527.类的静态成员变量和静态成员函数各有哪些特性？

静态成员变量

- 1). 静态成员变量需要在类内声明 (加static) , 在类外初始化 (不能加static) , 如下例所示 ;
- 2). 静态成员变量在类外单独分配存储空间 , 位于全局数据区 , 因此静态成员变量的生命周期不依赖于类的某个对象 , 而是所有类的对象共享静态成员变量 ;
- 3). 可以通过对象名直接访问公有静态成员变量 ;
- 4). 可以通过类名直接调用公有静态成员变量 , 即不需要通过对象 , 这一点是普通成员变量所不具备的。

```
class example{
public:
    static int m_int; //static成员变量
};

int example::m_int = 0; //没有static

cout<<example::m_int; //可以直接通过类名调用静态成员变量
```

静态成员函数

- 1). 静态成员函数是类所共享的 ;
- 2). 静态成员函数可以访问静态成员变量 , 但是不能直接访问普通成员变量 (需要通过对象来访问) ; 需要注意的是普通成员函数既可以访问普通成员变量 , 也可以访问静态成员变量 ;
- 3). 可以通过对象名直接访问公有静态成员函数 ;
- 4). 可以通过类名直接调用公有静态成员函数 , 即不需要通过对象 , 这一点是普通成员函数所不具备的。

```
class example{
private:
    static int m_int_s; //static成员变量
    int m_int;
    static int getI() //静态成员函数在普通成员函数前加static即可
    {
        return m_int_s; //如果返回m_int则报错, 但是可以return d.m_int是合法的
    }
};

cout<<example::getI(); //可以直接通过类名调用静态成员变量
```

528.在C++程序中调用被C编译器编译后的函数, 为什么要加extern"C"?

C++语言支持函数重载, C语言不支持函数重载, 函数被C++编译器编译后在库中的名字与C语言的不同, 假设某个函数原型为 :

```
void foo(int x, int y);
```

该函数被C编译器编译后在库中的名字为 `_foo`，而C++编译器则会产生像 `_foo_int_int` 之类的名字。为了解决此类名字匹配的问题，C++提供了C链接交换指定符号 `extern "C"`。

529.头文件中的 `ifndef/define/endif` 是干什么用的？该用法和 `program once` 的区别？

相同点：

它们的作用是防止头文件被重复包含。

不同点

- 1). `ifndef` 由语言本身提供支持，但是 `program once` 一般由编译器提供支持，也就是说，有可能出现编译器不支持的情况(主要是比较老的编译器)。
- 2). 通常运行速度上 `ifndef` 一般慢于 `program once`，特别是在大型项目上，区别会比较明显，所以越来越多的编译器开始支持 `program once`。
- 3). `ifndef` 作用于某一段被包含（`define` 和 `endif` 之间）的代码，而 `program once` 则是针对包含该语句的文件，这也是为什么 `program once` 速度更快的原因。
- 4). 如果用 `ifndef` 包含某一段宏定义，当这个宏名字出现“撞车”时，可能会出现这个宏在程序中提示宏未定义的情况（在编写大型程序时特别需要注意，因为有很多程序员在同时写代码）。相反由于 `program once` 针对整个文件，因此它不存在宏名字“撞车”的情况，但是如果某个头文件被多次拷贝，`program once` 无法保证不被多次包含，因为 `program once` 是从物理上判断是不是同一个头文件，而不是从内容上。

530.当*i*是一个整数的时候++i和i++那个更快一点？i++和++i的区别是什么？

答：理论上++i更快，实际与编译器优化有关，通常几乎无差别。

```
//i++实现代码为：
int operator++(int)
{
    int temp = *this;
    ++*this;
    return temp;
} //返回一个int型的对象本身

// ++i实现代码为：
int& operator++()
{
    *this += 1;
    return *this;
} //返回一个int型的对象引用
```

`i++`和`++i`的考点比较多，简单来说，就是`i++`返回的是*i*的值，而`++i`返回的是*i*+1的值。也就是`++i`是一个确定的值，是一个可修改的左值，如下使用：

```
cout << ++(++(++i)) << endl;
cout << ++ ++i << endl;
```


可以不停的嵌套++i。

这里有很多的经典笔试题，一起来观摩下：

```
int main()
{
    int i = 1;
    printf("%d,%d\n", ++i, ++i);    //3,3
    printf("%d,%d\n", ++i, i++);    //5,3
    printf("%d,%d\n", i++, i++);    //6,5
    printf("%d,%d\n", i++, ++i);    //8,9
    system("pause");
    return 0;
}
```

首先是函数的参数入栈顺序从右向左入栈的，计算顺序也是从右往左计算的，不过都是计算完以后再进行的压栈操作：

对于第1个printf，首先执行++i，返回值是i，这时i的值是2，再次执行++i，返回值是i，得到i=3，将i压入栈中，此时i为3，也就是压入3，3；

对于第2个printf，首先执行i++，返回值是原来的i，也就是3，再执行++i，返回值是i，依次将3，5压入栈中得到输出结果

对于第3个printf，首先执行i++，返回值是5，再执行i++返回值是6，依次将5，6压入栈中得到输出结果

对于第4个printf，首先执行++i，返回i，此时i为8，再执行i++，返回值是8，此时i为9，依次将i，8也就是9，8压入栈中，得到输出结果。

上面的分析也是基于VS搞的，不过准确来说函数多个参数的计算顺序是未定义的(the order of evaluation of function arguments are undefined)。笔试题目的运行结果随不同的编译器而异。

这里还有一个 i++ 的典型应用案例。

```
map<char, int> b = {{'a', 1}, {'b', 2}};

for(auto iter = b.begin(); iter != b.end();){
    if(iter->first == 'a'){
        b.erase(iter++);    // 等价于 auto t = iter; iter = iter + 1; b.erase(t);
    }
    else{
        iter++;
    }
}
```

531.指针和引用的区别？

相同点：

- 1). 都是地址的概念；
- 2). 都是“指向”一块内存。指针指向一块内存，它的内容是所指内存的地址；而引用则是某块内存的别名；
- 3). 引用在内部实现其实是借助指针来实现的，一些场合下引用可以替代指针，比如作为函数形参。

不同点：

- 1). 指针是一个实体，而引用(看起来，这点很重要)仅是个别名；
- 2). 引用只能在定义时被初始化一次，之后不可变；指针可变；引用“从一而终”，指针可以“见异思迁”；

- 3). 引用不能为空，指针可以为空；
- 4). “sizeof 引用”得到的是所指向的变量(对象)的大小，而“sizeof 指针”得到的是指针本身的大小；
- 5). 指针和引用的自增(++)运算意义不一样；
- 6). 引用是类型安全的，而指针不是(引用比指针多了类型检查)
- 7). 引用具有更好的可读性和实用性。

532. 引用占用内存空间吗？

如下代码中对引用取地址，其实是取的引用所对应的内存空间的地址。这个现象让人觉得引用好像并非一个实体。但是引用是占用内存空间的，而且其占用的内存和指针一样，因为引用的内部实现就是通过指针来完成的。

比如 `Type& name ; <==> Type* const name.`

```
int main(void)
{
    int a = 8;
    int &b = a;
    int *p = &b;    // 等价于 int *p = &a;
    *p = 0;
    cout<<a; //output 0
    return 0;
}
```

533. 三目运算符

在C中三目运算符(?:)的结果仅仅可以作为右值，比如如下的做法在C编译器下是会报错的，但是C++中却是可以通过的。这个进步就是通过引用来实现的，因为下面的三目运算符的返回结果是一个引用，然后对引用进行赋值是允许的。

```
int main(void)
{
    int a = 8;
    int b = 6;
    (a>b ? a : b) = 88;
    cout<<a; //output 88
    return 0;
}
```

534. 指针数组和数组指针的区别

数组指针，是指向数组的指针，而指针数组则是指该数组的元素均为指针。

数组指针，是指向数组的指针，其本质为指针，形式如下。如 `int (p)[n]`，`p`即为指向数组的指针，`()`优先级高，首先说明`p`是一个指针，指向一个整型的一维数组，这个一维数组的长度是`n`，也可以说是`p`的步长。也就是说执行`p+1`时，`p`要跨过`n`个整型数据的长度。数组指针是指向数组首元素的地址的指针，其本质为指针，可以看成是二级指针。

类型名 (数组标识符)[数组长度]

指针数组，在C语言和C++中，数组元素全为指针的数组称为指针数组，其中一维指针数组的定义形式如下。指针数组中每一个元素均为指针，其本质为数组。如 `int p[n]`，[]优先级高，先与p结合成为一个数组，再由int说明这是一个整型指针数组，它有n个指针类型的数组元素。这里执行p+1时，则p指向下一个数组元素，这样赋值是错误的：`p=a`；因为p是个不可知的表示，只存在p[0]、p[1]、p[2]...p[n-1]，而且它们分别是指针变量可以用来存放变量地址。但可以这样 `p=a`；这里p表示指针数组第一个元素的值，a的首地址的值。

类型名 数组标识符[数组长度]

535.左值引用与右值引用

该部分主要摘自：c++ 学习笔记

左值引用就是我们通常所说的引用，如下所示。左值引用通常可以看作是变量的别名。

```
type-id & cast-expression

// demo
int a = 10
int &b = a

int &c = 10 // 错误，无法对一个立即数做引用

const int &d = 10 // 正确，常引用引用常数量是ok的，其等价于 const int temp = 10;
const int &d = temp
```

右值引用是 C++11 新增的特性，其形式如下所示。右值引用用来绑定到右值，绑定到右值以后本来会被销毁的右值的生存期会延长至与绑定到它的右值引用的生存期。

```
type-id && cast-expression

// demo
int &&var = 10; // ok

int a = 10
int &&b = a // 错误，a 为左值

int &&c = var // 错误，var 为左值

int &&d = move(a) // ok，通过move得到左值的右值引用
```

在汇编层面右值引用做的事情和常引用是相同的，即产生临时量来存储常量。但是，唯一一点的区别是，右值引用可以进行读写操作，而常引用只能进行读操作。

536.右值引用的意义

右值引用支持移动语义的实现，可以减少拷贝，提升程序的执行效率。

下面的代码是没有采用右值引用时的实现。

```
class Stack
```

```
{
public:
    // 构造
    Stack(int size = 1000)
        :msize(size), mtop(0)
    {
        cout << "Stack(int)" << endl;
        mpstack = new int[size];
    }

    // 析构
    ~Stack()
    {
        cout << "~Stack()" << endl;
        delete[]mpstack;
        mpstack = nullptr;
    }

    // 拷贝构造
    Stack(const Stack &src)
        :msize(src.msize), mtop(src.mtop)
    {
        cout << "Stack(const Stack&)" << endl;
        mpstack = new int[src.msize];
        for (int i = 0; i < mtop; ++i) {
            mpstack[i] = src.mpstack[i];
        }
    }

    // 赋值重载
    Stack& operator=(const Stack &src)
    {
        cout << "operator=" << endl;
        if (this == &src)
            return *this;

        • delete[]mpstack;

        • msize = src.msize;
        • mtop = src.mtop;
        • mpstack = new int[src.msize];
        • for (int i = 0; i < mtop; ++i) {
            mpstack[i] = src.mpstack[i];
        • }
        • return *this;
    }

    int getSize()
    {
        return msize;
    }

private:
    int *mpstack;
```

```
int mtop;
int msize;
};

Stack GetStack(Stack &stack)
{
    Stack tmp(stack.getSize());
    return tmp;
}

int main()
{
    Stack s;
    s = GetStack(s);
    return 0;
}
```

运行结果如下。

```
Stack(int)           // 构造s
Stack(int)           // 构造tmp
Stack(const Stack&)   // tmp拷贝构造main函数栈帧上的临时对象
~Stack()             // tmp析构
operator=            // 临时对象赋值给s
~Stack()             // 临时对象析构
~Stack()             // s析构
```

执行代码的过程中调用拷贝构造，将内存中的内容逐个拷贝，在 C++ 11 中可以借助右值引用实现移动拷贝构造和移动赋值来解决这个问题。

```
Stack(Stack &&src)
    :msize(src.msize), mtop(src.mtop)
{
    cout << "Stack(Stack&&)" << endl;

    /*此处没有重新开辟内存拷贝数据，把src的资源直接给当前对象，再把src置空*/
    mpstack = src.mpstack;
    src.mpstack = nullptr;
}

// 带右值引用参数的赋值运算符重载函数
Stack& operator=(Stack &&src)
{
    cout << "operator=(Stack&&)" << endl;

    if(this == &src)
        return *this;

    delete[]mpstack;

    msize = src.msize;
    mtop = src.mtop;
```

```
/*此处没有重新开辟内存拷贝数据，把src的资源直接给当前对象，再把src置空*/
mpstack = src.mpstack;
src.mpstack = nullptr;

return *this;

}
```

执行结果如下。可以看到，在有拷贝构造和移动拷贝构造函数的時候，优先调用了移动拷贝构造和移动赋值。在移动拷贝构造和移动赋值中直接把资源所有权进行了转移，而非拷贝，这就大大提高了执行效率。

```
Stack(int)           // 构造s
Stack(int)           // 构造tmp
Stack(Stack&&)        // 调用带右值引用的拷贝构造函数，直接将tmp的资源给临时对象
~Stack()             // tmp析构
operator=(Stack&&)    // 调用带右值引用的赋值运算符重载函数，直接将临时对象资源给s
~Stack()             // 临时对象析构
~Stack()             // s析构
```

右值引用可以使重载函数变得更加简洁。右值引用可以适用 `const T&` 和 `T&` 形式的参数。

```
struct W
{
    W(int&, int&) {}
};

struct X
{
    X(const int&, int&) {}
};

struct Y
{
    Y(int&, const int&) {}
};

struct Z
{
    Z(const int&, const int&) {}
};

template <typename T, typename A1, typename A2>
T* factory(A1& a1, A2& a2)
{
    return new T(a1, a2);
}

template <typename T, typename A1, typename A2>
T* factory_new(A1&& a1, A2&& a2)
{
    return new T(std::forward<A1>(a1), std::forward<A2>(a2));
}
```

```
}

// demo
int a = 2;
int b = 2;

W* c = factory<W>(a, b);    // ok
Z* d = factory<Z>(2, 2);    // 错误, 2 是右值

W* pw = factory_new<W>(a, b);    // ok
X* px = factory_new<X>(2, b);    // ok
Y* py = factory_new<Y>(a, 2);    // ok
Z* e = factory_new<Z>(2, 2);    // ok
W* f = factory_new<W>(2, 2);    // 错误,
```

更多相关内容可以参考：c++——左值、右值、左值引用、右值引用

537.什么是面向对象（ OOP ）？面向对象的意义？

Object Oriented Programming, 面向对象是一种对现实世界理解和抽象的方法、思想，通过将需求要素转化为对象进行问题处理的一种思想。其核心思想是数据抽象、继承和动态绑定（多态）。

面向对象的意义在于：将日常生活中习惯的思维方式引入程序设计中；将需求中的概念直观的映射到解决方案中；以模块为中心构建可复用的软件系统；提高软件产品的可维护性和可扩展性。

538.解释下封装、继承和多态？

1). 封装：

封装是实现面向对象程序设计的第一步，封装就是将数据或函数等集合在一个个的单元中（我们称之为类）。

封装的意义在于保护或者防止代码（数据）被我们无意中破坏。

从封装的角度看，public，private 和 protected 属性的特点如下。

不管哪种属性，内类都是可以访问的

public 是一种暴露的手段，比如暴露接口，类的对象可以访问

private 是一种隐藏的手段，类的对象不能访问

protected 成员：

和 public 一样可以被子类继承

和 private 一样不能在类外被直接调用

特例：在衍生类中可以通过衍生类对象访问，如下代码所示

```
class Base
{
public:
    Base();
    virtual ~Base();
protected:
    int int_pro;
};
class A : public Base
{
public:
    A();
```

```
A(int da){int_pro = da;}
// 通过 obj 对象直接访问 protected 成员
void Set(A &obj){obj.int_pro = 24;}
void PrintPro(){cout << "The proteted data is " << int_pro << endl;}
};
```

2). 继承：

继承主要实现重用代码，节省开发时间。

子类可以继承父类的一些东西。

a.公有继承(public) 公有继承的特点是基类的公有成员和保护成员作为派生类的成员时，它们都保持原有的状态（基类的私有成员仍然是私有的，不能被这个派生类的子类所访问）。

b.私有继承(private) 私有继承的特点是基类的公有成员和保护成员都作为派生类的私有成员（并且不能被这个派生类的子类所访问）。

c.保护继承(protected) 保护继承的特点是基类的所有公有成员和保护成员都成为派生类的保护成员（并且只能被它的派生类成员函数或友元访问，基类的私有成员仍然是私有的）。

这里特别提一下虚继承。虚继承是解决C++多重继承问题（其一，浪费存储空间；第二，存在二义性问题）的一种手段。比如菱形继承，典型的应用就是 iostream, 其继承于 istream 和 ostream，而 istream 和 ostream 又继承于 ios。

3).多态：

多态是指通过基类的指针或者引用，在运行时动态调用实际绑定对象函数的行为。与之相对应的编译时绑定函数称为静态绑定。多态是设计模式的基础，多态是框架的基础。

539.什么时候生成默认构造函数（无参构造函数）？什么时候生成默认拷贝构造函数？什么是深拷贝？什么是浅拷贝？默认拷贝构造函数是哪种拷贝？什么时候用深拷贝？

1). 没有任何构造函数时，编译器会自动生成默认构造函数，也就是无参构造函数；当类没有拷贝构造函数时，会生成默认拷贝构造函数。

2). 深拷贝是指拷贝后对象的逻辑状态相同，而浅拷贝是指拷贝后对象的物理状态相同；默认拷贝构造函数属于浅拷贝。

3). 当系统中有成员指代了系统中的资源时，需要深拷贝。比如指向了动态内存空间，打开了外存中的文件或者使用了系统中的网络接口等。如果不进行深拷贝，比如动态内存空间，可能会出现多次被释放的问题。是否需要定义拷贝构造函数的原则是，类是否有成员调用了系统资源，如果定义拷贝构造函数，一定是定义深拷贝，否则没有意义。

更多可以参考下面的代码，比较容易混淆的是赋值操作符，其实区分很简单，在出现等号的时候，如果有构造新的对象时调用的就是构造，不然就是调用赋值操作符。特别注意下面的 b 和 f，一个是拷贝构造，一个是构造。

```
class A {
public:
    A() {
        m = new int[4]{ 1,2,3,4 };
        std::cout << "constructor" << std::endl;
    }
    ~A() {
        if (m != nullptr) {
            delete[] m;
        }
    }
    A(const A& a) {
        this->m = new int[4];
    }
};
```



```

memcpy(a.m, this->m, this->len * sizeof(int));
std::cout << "copy constructor" << std::endl;
}
// 移动构造
A(A&& a) : m(a.m) {
    a.m = nullptr;
    std::cout << "move constructor" << std::endl;
}
// 赋值操作符重载
A& operator= (const A& a) {
    memcpy(a.m, this->m, this->len * sizeof(int));
    std::cout << "operator" << std::endl;
    return *this;
}

private:
    int len = 4;
    int* m = nullptr;
};

A getA(A a) {
    return a;
}

int main(void)
{
    A a;    // construct

    A b = a;    // copy construct
    A c(a); // copy construct

    A d;    // construct
    d = a;  // operate

    A e = getA(a); // construct, move construct

    A f = A(); // construct
    return 0;
}

```

540.构造函数和析构函数的执行顺序？

构造函数

- 1). 首先调用父类的构造函数；
- 2). 调用成员变量的构造函数；
- 3). 调用类自身的构造函数。

析构函数

对于栈对象或者全局对象，调用顺序与构造函数的调用顺序刚好相反，也即后构造的先析构。对于堆对象，析构顺序与delete的顺序相关。

541.虚析构函数的作用？

基类采用虚析构函数可以防止内存泄漏。比如下面的代码中，如果基类 A 中不是虚析构函数，则 B 的析构函数不会被调用，因此会造成内存泄漏。

```
class A{
public:
    A(){}
    //~A(){}
    virtual ~A(){cout << "A destruct" << endl;} // 虚析构
    // ~A(){cout << "A destruct" << endl;} // 析构

};

class B : public A{
public:
    B(){
        // new memory
        // ...
        cout << "B construct" << endl;
    }
    ~B(){
        // delete memory
        // ...
        cout << "B destruct" << endl;
    }
};

int main(int argc, char **argv)
{
    A *p = new B;

    // some operations
    // ...

    delete p; // 由于基类中是虚析构，这里会先调用B的析构函数，然后调用A的析构函数

    return 0;
}
```

但并不是要把所有类的析构函数都写成虚函数。因为当类里面有虚函数的时候，编译器会给类添加一个虚函数表，里面来存放虚函数指针，这样就会增加类的存储空间。所以，只有当一个类被用来作为基类的时候，才把析构函数写成虚函数。

542.细看拷贝构造函数

对于 class A，它的拷贝构造函数如下：

```
A::A(const A &a){}
```

1. 为什么必须是当前类的引用呢？

循环调用。如果拷贝构造函数的参数不是当前类的引用，而是当前类的对象，那么在调用拷贝构造函数时，会将另外一个对象直接传递给形参，这本身就是一次拷贝，会再次调用拷贝构造函数，然

后又将一个对象直接传递给了形参，将继续调用拷贝构造函数.....这个过程会一直持续下去，没有尽头，陷入死循环。

只有当参数是当前类的引用时，才不会导致再次调用拷贝构造函数，这不仅是逻辑上的要求，也是 C++ 语法的要求。

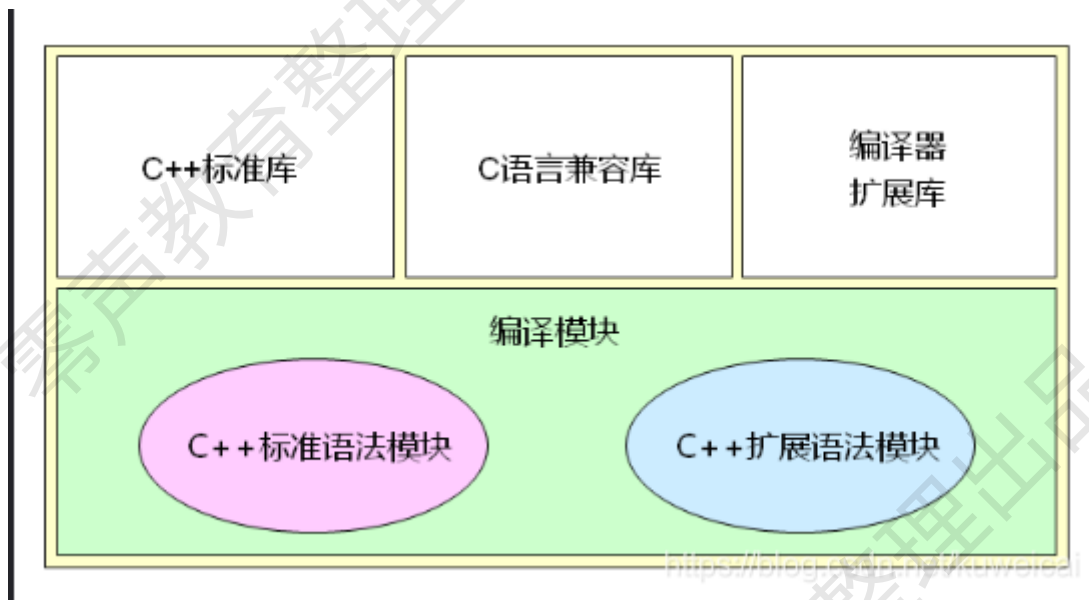
2. 为什么是 const 引用呢？

拷贝构造函数的目的是用其它对象的数据来初始化当前对象，并没有期望更改其它对象的数据，添加 const 限制后，这个含义更加明确了。

另外一个原因是，添加 const 限制后，可以将 const 对象和非 const 对象传递给形参了，因为非 const 类型可以转换为 const 类型。如果没有 const 限制，就不能将 const 对象传递给形参，因为 const 类型不能直接转换为非 const 类型，这就意味着，不能使用 const 对象来初始化当前对象了。

543.C++的编译环境

如下图所示，C++的编译环境由如下几部分构成：C++标准库、C语言兼容库、编译器扩展库及编译模块。



```
#include<iostream> //C++标准库，不带".h"
#include<string.h> //C语言兼容库，由编译器厂商提供
```

值得注意的是，C语言兼容库功能上跟C++标准库中的C语言子库相同，它的存在主要为了兼容C语言编译器，也就是说如果一个文件只包含C语言兼容库（不包含C++标准库），那么它在C语言编译器中依然可以编译通过。

544.Most vexing parse

直接上代码吧。下面 f 和 g 是有问题的，这种情况就称为 Most vexing parse。

```
class A {
public:

    A() { cout << "const without param" << endl; }
    A(int a) { cout << "const with param" << endl; }
    A(const A& b) { cout << "copy construct" << endl; }

};
```

```
int main(void)
{

    A a;    // const(construct) without param
    A b(10); // const with param
    A c = A(); // const without param
    A d = A(10); // const with param
    A e(d); // copy construct
    A f();
    A g(A());
    A h{}; // const without param
    A i{A{}}; // const without param

    return 0;

}
```

问题在哪？

A f(); // 这个是不是可以看做声明了一个返回值为A的函数，函数名为 f，参数无
A g(A()); // 这个是不是可以看做声明了一个返回值为A的函数，函数名为 g，参数类型为函数指针，这个函数指针的返回值类型为A，参数无

解决办法参考上面的 h，j。

545.STL 六大组件

STL 六大组件：容器（Container）、算法（Algorithm）、迭代器（Iterator）、仿函数（Function object）、适配器（Adaptor）和空间配置器（allocator）。

546.stack 中有 pop() 和 top() 方法，为什么不直接用 pop() 实现弹出和取值的功能？

如果 stack 中存放的是较大是内容时，比如 vector 类型，取值的时候就会发生拷贝，如果拷贝失败，这是，

假设有一个stack，vector是一个动态容器，当你拷贝一个vector时，标准库会从堆上分配很多内存来完成这次拷贝。当这个系统处在重度负荷，或有严重的资源限制的情况下，这种内存分配就会失败，所以vector的拷贝构造函数可能会抛出一个std::bad_alloc异常。当vector中存有大量元素时，这种情况发生的可能性更大。当pop()函数返回“弹出值”时(也就是从栈中将这个值移除)，会有一个潜在的问题：这个值被返回到调用函数的时候，栈才被改变；但当拷贝数据的时候，调用函数抛出一个异常会怎么样？如果事情真的发生了，要弹出的数据将会丢失；它的确从栈上移出了，但是拷贝失败了！std::stack的设计人员将这个操作分为两个部分：先获取顶部元素(top())，然后从栈中移除元素(pop())。这样，在不能安全的将元素拷贝出去的情况下，栈中的这个数据还依旧存在，没有丢失。当问题是堆空间不足时，应用可能会释放一些内存，然后再进行尝试。

参考：为什么适配器stack中成员函数top()和pop()需要分离实现

547.map 和 unordered_map 的区别？各自的优缺点？

map 的内部实现是一个红黑树（红黑树是非严格平衡二叉搜索树，而AVL是严格平衡二叉搜索树），它具有如下性质：

- 红黑树具有自动排序的功能，因此map内部的所有元素都是有序的
- 查找、插入、删除的时间复杂度为 $\log(n)$
- map中的元素是按照二叉搜索树（又名二叉查找树、二叉排序树，特点就是左子树上所有节点的键值都小于根节点的键值，右子树所有节点的键值都大于根节点的键值）存储的，使用中序遍历可将键值按照从小到大遍历出来。

unordered_map 的内部实现是 hash 表。它具有如下性质：

- 查找、插入、删除的平均时间复杂度可达到 $O(1)$
- 哈希表的建立比较耗费时间，占用内存相比红黑树要高

一般情况下会使用 map，因为 unordered_map 的构建费时。对于查找问题，unordered_map 会更加高效一些，因此遇到查找问题，常会考虑优先用 unordered_map。

问题拓展：

- 什么是红黑数？红黑树是一种二叉查找树，但在每个节点增加一个存储位表示节点的颜色，可以是红或黑（非红即黑）。通过对任何一条从根到叶子的路径上各个节点着色的方式的限制，红黑树确保没有一条路径会比其它路径长出两倍，因此，红黑树是一种弱平衡二叉树，相对于要求严格的AVL树来说，它的旋转次数少，所以对于搜索，插入，删除操作较多的情况下，通常使用红黑树。
- 什么是AVL？红黑树是在AVL树的基础上提出来的。平衡二叉树又称为AVL树，是一种特殊的二叉排序树。其左右子树都是平衡二叉树，且左右子树高度之差的绝对值不超过1。AVL树中所有结点为根的树的左右子树高度之差的绝对值不超过1。将二叉树上结点的左子树深度减去右子树深度的值称为平衡因子BF，那么平衡二叉树上的所有结点的平衡因子只可能是-1、0和1。只要二叉树上有一个结点的平衡因子的绝对值大于1，则该二叉树就是不平衡的。
- map 为什么用红黑树，而不是AVL？AVL树是高度平衡的，频繁的插入和删除，会引起频繁的rebalance，导致效率下降；红黑树不是高度平衡的，算是一种折中，查找，插入删除的性能都是 $O(\log n)$ ，且性能稳定（插入最多两次旋转，删除最多三次旋转）。

版权声明：本文为CSDN博主「kuweicai」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/kuweicai/article/details/82779648>

548.如何初始化一个指针数组

答案：

c++中的指针是一个很经典的用法，但是也是最容易出错的，比如定义了一个指针，必须对其进行初始化，不然这个指针指向的是一个未知的内存地址，后续对其操作的时候，会报错。这只是其次，最让人头疼的就是指针错误问题，往往编译的时候可以通过，在程序运行的时候，就会出现异常，如果对程序不是很熟悉，则不是很容易找到问题所在，我最近就遇到过很多这样的问题，定义了一个结构体指针，使用的时候忘记初始化，导致在后边使用的时候程序报异常。下面就总结一下c++指针初始化的一些方法，

以及我自己遇到的一些问题以及心得体会。

一.c++指针初始化的一般方法：

1.将一个已经在内存中存在变量的地址传递给定义的指针，这个指针就指向这个变量的内存地址（相同的数据类型），完成初始化。

比如：

```
int a=2;
int *b=&a;
```

2.利用new开辟一块地址空间

```
struct test{
    int a;
    int b;
}*t;

void main()
{
    int c=0;
    test *t=new test();
    c=t->a;
}
```

使用 new开辟的空间，记得使用delete释放，因为new出来的是返回的堆的空间，堆的空间是不会自动释放的，存放变量的栈才会自动释放。delete释放其实只是释放了申请的这块内存空间，但是指针并没有撤销，指针还是指向这块地址，但是不可用（靠人品吃饭的有可能可以用），是非法的。所以用delete释放掉一块堆内存时，应该自己手动将指针设置为NULL。

3.把指针设置为NULL或者0

这样做一般只是为了没有具体初始化的时候做的，这样避免了野指针，后面可以使用if（指针==NULL）来判断，然后再进行操作。

错题解析：首先明确一个概念，就是指向数组的指针，和存放指针的数组。指向数组的指针：char (*array)[5];含义是一个指向存放5个字符的数组的指针。存放指针的数组：char *array[5];含义是一个数组中存放了5个指向字符型数据的指针。按照题意，我理解为初始化一个存放指针的数组，char *array[2]={"China","Beijing"}；其含义是初始化了一个有两个指向字符型数据的指针的数组，这两个指针分别指向字符串"China"和"Beijing"。

549.关键字const是什么含意？

正确答案：

在标准C++中，这样定义的是一个常量，用来修饰内置类型变量，自定义对象，成员函数，返回值，函数参数。

const使用（类型）：

1、用于指针的两种情况:const是一个左结合的类型修饰符.

int constA;//A可变,A不可变

intconst A;//A不可变,A可变

2、限定函数的传递值参数:

void function(const int Var);//传递过来的参数在函数内不可以改变.

3、限定函数返回值型.

const int function();//此时const无意义

const myclassname function();//函数返回自定义类型myclassname.

4、限定函数类型.

void function()const;//常成员函数,常成员函数是不能改变成员变量值的函数.

错题解析：我只要一听到被面试官说：“const意味着常数”，我就知道我正在和一个业余者打交道。去年Dan Saks已经在他的文章里完全概括了const的所有用法，因此ESP(译者：Embedded Systems Programming)的每一位读者应该非常熟悉const能做什么和不能做什么。如果你从没有读到那篇文章，只要能说出const意味着“只读”就可以了。尽管这个答案不是完全的答案，但我接受它作为一个正确的答案。（如果你想知道更详细的答案，仔细读一下Saks的文章吧。）如果应试者能正确回答这个问题，我将问他一个附加的问题：下面的声明都是什么意思？

```
const int a;
int const a;
const int *a;
int * const a;
int const * a const;
```

前两个的作用是一样，a是一个常整型数。第三个意味着a是一个指向常整型数的指针（也就是，整型数是不可修改的，但指针可以）。第四个意思a是一个指向整型数的常指针（也就是说，指针指向的整型数是可以修改的，但指针是不可修改的）。最后一个意味着a是一个指向常整型数的常指针（也就是说，指针指向的整型数是不可修改的，同时指针也是不可修改的）。如果应试者能正确回答这些问题，那么他就给我留下了一个好印象。顺带提一句，也许你可能会问，即使不用关键字const，也还是能很容易写出功能正确的程序，那么我为什么还要如此看重关键字const呢？我也如下的几下理由：1). 关键字const的作用是为给读你代码的人传达非常有用的信息，实际上，声明一个参数为常量是为了告诉了用户这个参数的应用目的。如果你曾花很多时间清理其它人留下的垃圾，你就会很快学会感谢这多余的信
息。（当然，懂得用const的程序员很少会留下的垃圾让别人来清理的。）2). 通过给优化器一些附加的信息，使用关键字const也许能产生更紧凑的代码。3). 合理地使用关键字const可以使编译器很自然地保护那些不希望被改变的参数，防止其被无意的代码修改。简而言之，这样可以减少bug的出现。

550.什么是动态特性？

正确答案：

在绝大多数情况下，程序的功能是在编译的时候就确定下来的，我们称为静态特性。反之，如果程序的功能是在运行时刻才确定下来的，则称为动态特性。

动态特性是面向对象语言最强大的功能之一，因为它在语言层面上支持程序的可扩展性，而可扩展性是软件设计追求的重要目标之一。

c++虚函数、抽象基类、动态绑定、多态构成了出色的动态特性。

1.虚函数

假定几何形状的基类为Shape，其派生类有Circle、Rectangle、Ellipse等，每个派生类都能够绘制自己所代表的形状。不管派生类的形状如何，我们希望用统一的方式来调用绘制函数，最好是使用Shape定义的接口函数Draw（），并让程序在运行是动态地确定应该使用哪一个派生类的Draw（）函数。

为了使这种行为可行，我们把基类Shape中的函数Draw（）声明为虚函数，然后在派生类中重新定义Draw（）使之绘制正确的形状，这种方法叫覆盖，虚函数的声明方法是在基类的函数原型之前加上关键字virtual。

一旦类的一个函数被声明为虚函数，那么其派生类的对应函数也自动成为虚函数，这样一级一级传递下去。

2.抽象基类

当我们把类看成是一种数据类型时，通常会认为该类肯定是要被实例为一个或多个对象的。但是在很多情况下，定义那些不能实例化出对象的类也是很有用的，这种类就称为抽象类。抽象类的唯一目的就是让其派生类继承并实现它的接口方法，因此它通常也被称为抽象基类。

如果将基类的虚函数声明为虚函数，那么该类就被定义为了抽象基类。纯虚函数是在声明时将其“初始化”为0的函数，例如：


```
class Shape {    //Shape是抽象基类
public:
    virtual void Draw(void)=0; //Draw()为纯虚函数
}
```

抽象基类Shape的纯虚函数Draw()根本不知道自己应该怎么绘制出一个“形状”来，具体功能必须有代表具体形状的派生类对应的Draw()函数来实现。

我们知道，函数名就是函数的地址，将一个函数初始化为0意味着函数的地址将为0，这就是在告诉编译器，不要为该函数编址，从而阻止该类的实例化行为。

抽象基类的主要用途是“接口与实现分离”；不仅要把数据成员（信息）隐藏起来，而且还要把实现完全隐藏起来，只留一些接口给外部调用。

错题解析：在绝大多数情况下，程序的功能是在编译的时候就确定下来的，我们称之为静态特性。反之，如果程序的功能是在运行时刻才能确定下来的，则称之为动态特性。C++中，虚函数，抽象基类，动态绑定和多态构成了出色的动态特性。

551.基类的有1个虚函数，子类还需要申明为virtual吗？为什么。

正确答案：建议加上virtual，代码是写给人看的

错题解析：不申明没有关系的。不过，我总是喜欢显式申明，使得代码更加清晰。

552.在C++ 程序中调用被 C 编译器编译后的函数，为什么要加 extern “C”声明？

正确答案：

c语言程序和c++程序在编译时，是相互独立的编译；在.c文件中，编译器将Add函数重命名为Add；而在.cpp文件中，编译器将用extern声明的函数重命名为（?Add@@YAHHH@Z）；那么在链接阶段，在main函数中该Add函数被调用；

此时编译器带着（?Add@@YAHHH@Z）函数名进入test.c文件的编译文件中去寻找（?

Add@@YAHHH@Z）函数，但是在test.c文件中这个Add函数被编译器重命名为_Add，由于在.cpp文件中和.c文件中同一个函数名被编译器重命名后的名字不一样；所以.cpp文件声明的外部函数找不到真正定义的地方；程序错误；

（2）使用extern “C” int Add(int,int); 外部函数——正确

```
#include<iostream>
using namespace std;
extern "C" int Add(int,int);
int main()
{
    cout<<Add(50,50)<<endl;
    return 0;
}
```

解释：

c++语言为了支持重载，对编译时函数的重命名规则进行更改，

使用extern “C”的方式在c++程序中声明c语言文件中的函数，可以在编译时，告诉编译器使用C语言的规则对该函数的函数名进行重命名，这样在链接的时候，就可以顺利在.c文件中找到该函数；

错题解析：函数和变量被C++编译后在符号库中的名字与C语言的不同，被extern “C”修饰的变量和函数

是按照C语言方式编译和连接的。由于编译后的名字不同，C++程序不能直接调用C函数。C++提供了一个C连接交换指定符号extern"C"来解决这个问题。

553.如何定义Bool变量的TRUE和FALSE的值。

正确答案：

对于C语言，并不存在系统自带的bool类型和False和True的定义，如果要使用则必须自定义类型。

有两种常见的方法。

1、使用宏定义。

通过define来进行定义False和True。可以采用如下方式：

(1) 直接定义数值

```
#define False 0
#define True 1
```

(2) 通过逻辑判断定义

```
#define False (1!=1)
#define True (1==1)
```

2、使用枚举定义。

这种方法可以同时定义类型BOOL

```
typedef enum
{
    False,
    True
}BOOL;
```

错题解析：不知道这个题有什么陷阱，写到现在神经已经大了，一般来说先要把TURE和FALSE给定义了，使用#define就可以：#define TURE 1 #define FALSE 0 如果有一个变量需要定义成bool型的，举个例子：bool a=TURE;就可以了。

554.内联函数Inline和宏定义一起使用的区别。

正确答案：

- 1、宏定义不会进行类型的判断，只是单存的替换文本；inline会对参数类型进行判断。
- 2、宏不是函数，inline修饰的是函数。
- 3、宏返回值不能强制转换成合适的类型，inline返回值可以。

错题解析：内联函数是在编译的时候已经做好将对应的函数代码替换嵌入到对应的位置，适用于代码较少的函数。宏定义是简单的替换变量，如果定义的是有参数的函数形式，参数不做类型校验。

555.编写my_strcpy函数，实现与库函数strcpy类似的功能，不能使用任何库函数；

正确答案：

```
char *strcpy(char *strDest, const char *strSrc)
{
    if ( strDest == NULL || strSrc == NULL)
        return NULL ;
    if ( strDest == strSrc)
        return strDest ;
    char *tempPtr = strDest ;
    while( (*strDest++ = *strSrc++) != '\0')
        return tempPtr ;
}
```

556.完成程序，实现对数组的降序排序

```
#include
void sort(int array[] );
int main()
{
    int array[]={45, 56, 76, 234, 1, 34, 23, 2, 3}; //数字任意给出
    sort( array );
    return 0;
}
void sort( int array[] )
{
    _____
    int i,j,k;
    for(i=1;i<=7;i++) { if(array[i]>array[i-1])
    {
        k=array[i];
        j=i-1;
        do
        {
            array[j+1]=array[j];
            j- ;
        }
        while(k>array[j]&&j>=0);
        array[j+1]=k;
    }
    _____
}
```

557.ICMP是什么协议,处于哪一层?

正确答案：

Internet控制报文协议，处于网络层（IP层）

558.C中static有什么作用

正确答案：

(1)隐藏。当我们同时编译多个文件时，所有未加static前缀的全局变量和函数都具有全局可见性，故使用static在不同的文件中定义同名函数和同名变量，而不必担心命名冲突。

(2)static的第二个作用是保持变量内容的持久。存储在静态数据区的变量会在程序刚开始运行时就完成初始化，也是唯一的一次初始化。共有两种变量存储在静态存储区：全局变量和static变量。

(3)static的第三个作用是默认初始化为0.其实全局变量也具备这一属性，因为全局变量也存储在静态数据区。在静态数据区，内存中所有的字节默认值都是0×00,某些时候这一特点可以减少程序员的工作量。

559.请问运行Test函数会有什么样的结果？

```
Void GetMemory2(char **p, int num)
{
    *p = (char *)malloc(num);
}
void Test(void)
{
    char *str = NULL;
    GetMemory(&str, 100);
    strcpy(str, "hello");
    printf(str);
}
```

正确答案：

可以运行

560.C++特点是什么，如何实现多态？画出基类和子类在内存中的相互关系。

正确答案：

多态的基础是继承，需要虚函数的支持，简单的多态是很简单的。子类继承父类大部分的资源，不能继承的有构造函数，析构函数，拷贝构造函数，operator=函数，友元函数等等

561.C++中的什么是多态性？是如何实现的？

正确答案：

多态性是面向对象程序设计语言继数据抽象和继承之后的第三个基本特征。它是在运行时出现的多态性通过派生类和虚函数实现。基类和派生类中使用同样的函数名，完成不同的操作具体实现相隔离的另一类接口，即把"what"从"how"分离开来。多态性提高了代码的组织性和可读性，虚函数则根据类型的不同来进行不同的隔离。

562.关键字static的作用是什么？

正确答案：

这个问题很少有人能回答完全。在C语言中，关键字static有三个明显的作用：

- 1). 在函数体，一个被声明为静态的变量在这一函数被调用过程中维持其值不变。
- 2). 在模块内（但在函数体外），一个被声明为静态的变量可以被模块内所用函数访问，但不能被模块外其它函数访问。它是一个本地的全局变量。
- 3). 在模块内，一个被声明为静态的函数只可被这一模块内的其它函数调用。那就是，这个函数被限制在声明它的模块的本地范围内使用。大多数应试者能正确回答第一部分，一部分能正确回答第二部分，同是很少的人能懂得第三部分。这是一个应试者的严重的缺点，因为他显然不懂得本地化数据和代码范围的好处和重要性。

563.#define MAX_LEN 500 char arry[MAX_LEN]; cin>>arry; 这段代码有问题吗？若有，请指出并修改；

正确答案：

有问题。头文件缺少。 #include

564.delete []array 和 delete array 一样吗？不一样请说明；

正确答案：

delete []array 释放的是多个同一类型的地址空间 Delete[]array 释放的是一个某种类型的地址空间

565.多态的作用？

正确答案：

主要是两个：

- 1) 隐藏实现细节，使得代码能够模块化；扩展代码模块，实现代码重用；
- 2) 接口重用，为了类在继承和派生的时候，保证使用家族中任一类的实例的某一属性时的正确调用。

566.C语言的volatile的含义是什么。使用时会对编译器有什么暗示。

正确答案：

从词面上讲，volatile的意思是易变的，也就是说，在程序运行过程中，有一些变量可能会被莫名其妙的改变，而优化器为了节约时间，有时候不会重读这个变量的真实值，而是去读在寄存器的备份，这样的话，这个变量的真实值反而被优化器给“优化”掉了，用时髦的词说就是被“和谐”了。如果使用了这个修饰词，就是通知编译器别犯懒，老老实实去重新读一遍！可能我说的太“通俗”了，那么我引用一下“大师”的标准解释：volatile的本意是“易变的”。由于访问寄存器的速度要快过RAM,所以编译器一般都会作减少存取外部RAM的优化，但也有可能会读脏数据。当要求使用volatile 声明的变量的值的时候，系统总是重新从它所在的内存读取数据，即使它前面的指令刚刚从该处读取过数据。而且读取的数据立刻被保存。精确地说就是，优化器在用到这个变量时必须每次都小心地重新读取这个变量的值，而不是使用保存在寄存器里的备份。下面是volatile变量的几个例子：

- 1). 并行设备的硬件寄存器（如：状态寄存器）
- 2). 一个中断服务子程序中会访问到的非自动变量(Non-automatic variables)
- 3). 多线程应用中被几个任务共享的变量 嵌入式系统程序员经常同硬件、中断、RTOS等等打交道，所用这些都要求volatile变量。不懂得volatile内容将会带来灾难。

567.请简述以下两个for循环的优缺点

1) for (i=0; i<n; i++)

```
{  
if (condition)  
DoSomething();  
else  
DoOtherthing();  
}
```

2) if (condition)

```
{  
for (i=0; i<n; i++)  
DoSomething();  
}  
else  
{  
for (i=0; i<n; i++) dootherthing(); } }
```

正确答案：

- 1) 优点：程序简洁。="" 缺点：多执行了n-1次逻辑判断，并且打断了循环“流水线”作业，使得编译器不能对循环进行优化处理，降低了效率。=""
- 2) 优点：循环的效率。缺点：程序不简洁。=""

568.预处理器标识#error的目的是什么？

正确答案：

如果你不知道答案，请看参考文献1。

569.C语言的volatile的含义是什么。使用时会对编译器有什么暗示。

正确答案：

从词面上讲，volatile的意思是易变的，也就是说，在程序运行过程中，有一些变量可能会被莫名其妙的改变，而优化器为了节约时间，有时候不会重读这个变量的真实值，而是去读在寄存器的备份，这样的话，这个变量的真实值反而被优化器给“优化”掉了，用时髦的词说就是被“和谐”了。如果使用了这个修饰词，就是通知编译器别犯懒，老老实实去重新读一遍！可能我说的太“通俗”了，那么我引用一下“大师”的标准解释：volatile的本意是“易变的”。由于访问寄存器的速度要快过RAM,所以编译器一般都会作减少存取外部RAM的优化，但也有可能会读脏数据。当要求使用volatile 声明的变量的值的时候，系统总是重新从它所在的内存读取数据，即使它前面的指令刚刚从该处读取过数据。而且读取的数据立刻被保存。精确地说就是，优化器在用到这个变量时必须每次都小心地重新读取这个变量的值，而不是使用保存在寄存器里的备份。下面是volatile变量的几个例子：1). 并行设备的硬件寄存器（如：状态寄存器）2). 一个中断服务子程序中会访问到的非自动变量(Non-automatic variables) 3). 多线程应用中被几个任务共享的变量 嵌入式系统程序员经常同硬件、中断、RTOS等等打交道，所用这些都要求volatile变量。不懂得volatile内容将会带来灾难。

570.MFC中CString是类型安全类么？

正确答案：

不是，其它数据类型转换到CString可以使用CString的成员函数Format来转换

571.内联函数Inline和宏定义一起使用的区别。

正确答案：

内联函数是在编译的时候已经做好将对应的函数代码替换嵌入到对应的位置，适用于代码较少的函数。宏定义是简单的替换变量，如果定义的是有参数的函数形式，参数不做类型校验。

572.C++中什么数据分配在栈或堆中，New分配数据是在近堆还是远堆中？

正确答案：

栈: 存放局部变量，函数调用参数,函数返回值，函数返回地址。由系统管理 堆: 程序运行时动态申请，new 和 malloc申请的内存就在堆上

573.DB事务处理的四个特性：

正确答案：

原子性，一致性，隔离性，持久性 就答对一个：一致性，

574.如何初始化一个指针数组。

正确答案：

首先明确一个概念，就是指向数组的指针，和存放指针的数组。指向数组的指针：char (*array)[5];含义是一个指向存放5个字符的数组的指针。存放指针的数组：char *array[5];含义是一个数组中存放了5个指向字符型数据的指针。按照题意，我理解为初始化一个存放指针的数组，char *array[2]={“China”,“Beijing”};其含义是初始化了一个有两个指向字符型数据的指针的数组，这两个指针分别指向字符串“China”和“Beijing”。

575.int i=(j=4,k=8,l=16,m=32); printf(“%d”, i); 输出是多少？

正确答案：

相当于 i=j=4 ; i=k=8 ; i=l=16 ; i=m=32; 故最后i=32 ;

576.如何在C中初始化一个字符数组。

正确答案：

这个问题看似很简单，但是我们要将最简单的问题用最严谨的态度来对待。关键的地方：初始化、字符型、数组。最简单的方法是char array[];。这个问题看似解决了，但是在初始化上好像还欠缺点什么，个人认为：char array[5]={‘1’,‘2’,‘3’,‘4’,‘5’};或者char array[5]={“12345”};或者char array[2][10]={“China”,“Beijing”};也许更符合“初始化”的意思。

577.参数传递有几种方式；实现多态参数传递采用什么方式，如果没有使用某种方式原因是什么

正确答案：

传值，传指针或者引用

578.请填写BOOL，float，指针变量与“零值”比较的if语句。

正确答案：

这里“零值”可以是0, 0.0, FALSE或者“空指针”。例如 int 变量 n 与“零值”比较的 if 语句为：

```
if ( n == 0 )
if ( n != 0 )
```

579.C++特点是什么，如何实现多态？画出基类和子类在内存中的相互关系。

正确答案：

多态的基础是继承，需要虚函数的支持，简单的多态是很简单的。子类继承父类大部分的资源，不能继承的有构造函数，析构函数，拷贝构造函数，operator=函数，友元函数等等

580.什么是“引用”？申明和使用“引用”要注意哪些问题？

正确答案：

引用就是某个目标变量的“别名”(alias)，对应用的操作与对变量直接操作效果完全相同。申明一个引用的时候，切记要对其进行初始化。引用声明完毕后，相当于目标变量名有两个名称，即该目标原名称和引用名，不能再把该引用名作为其他变量名的别名。声明一个引用，不是新定义了一个变量，它只表示该

引用名是目标变量名的一个别名，它本身不是一种数据类型，因此引用本身不占存储单元，系统也不给引用分配存储单元。不能建立数组的引用。

581.触发器怎么工作的？

正确答案：

触发器主要是通过事件进行触发而被执行的，当对某一表进行诸如UPDATE、INSERT、DELETE 这些操作时，数据库就会自动执行触发器所定义的SQL 语句，从而确保对数据的处理必须符合由这些SQL 语句所定义的规则。

582.C也可以通过精心封装某些函数功能实现重用，那C++的类有什么优点吗，难道仅仅是为实现重用。

正确答案：

并不仅仅是这样的。OOD，OOP从根本上改变了程序设计模式 and 设计思想，具备重大和深远的意义。类的三大最基本的特征：封装，继承，多态。

583.CSingleLock是干什么的。

正确答案：

同步多个线程对一个数据类的同时访问

584.C++中引用和指针的区别？

正确答案：

引用是对象的别名，操作引用就是操作这个对象，必须在创建的同时有效得初始化(引用一个有效的对象，不可为NULL)，初始化完毕就再也不可改变，引用具有指针的效率，又具有变量使用的方便性和直观性，在语言层面上引用和对象的用法一样，在二进制层面上引用一般都是通过指针来实现的，只是编译器帮我们完成了转换。之所以使用引用是为了用适当的工具做恰如其分的事，体现了最小特权原则。

585.C与C++各自是如何定义常量的？有什么不同？

正确答案：

C中是使用宏#define定义，C++使用更好的const来定义。区别：1)const是有数据类型的常量，而宏常量没有，编译器可以对前者进行静态类型安全检查，对后者仅是字符替换，没有类型安全检查，而且在字符替换时可能会产生意料不到的错误(边际效应)。2)有些编译器可以对const常量进行调试，不能对宏调试。

586.C++函数中值的传递方式有哪几种？

正确答案：

C++函数的三种传递方式为：值传递、指针传递和引用传递。

587.一般数据库若出现日志满了，会出现什么情况，是否还能使用？

正确答案：

只能执行查询等读操作，不能执行更改，备份等写操作，原因是任何写操作都要记录日志。也就是说基本上处于不能使用的状态。

588.C++里面如何声明const void f(void)函数为C程序中的库函数？

正确答案：

在该函数前添加extern “C”声明。由于编译后的名字不同，C++程序不能直接调用C函数。

589.c++中类和c语言中struct的区别（至少两点）

正确答案：

- (1) c++中的类默认的成员是私有的，struct默认的是共有的。
- (2) c++中的类可以定义成员函数，struct只能定义成员变量。
- (3) C++中的类有继承、多态的特性，struct没有。

590.IP组播有那些好处？

正确答案：

Internet上产生的许多新的应用，特别是高带宽的多媒体应用，带来了带宽的急剧消耗和网络拥挤问题。组播是一种允许一个或多个发送者（组播源）发送单一的数据包到多个接收者（一次的，同时的）的网络技术。组播可以大大的节省网络带宽，因为无论有多少个目标地址，在整个网络的任何一条链路上只传送单一的数据包。所以说组播技术的核心就是针对如何节约网络资源的前提下保证服务质量。

591.变量的声明和定义有什么区别？

正确答案：

声明变量不分配空间，定义变量要分配空间。声明主要是告诉编译器,后面的引用都按声明的格式。定义其实包含了声明的意思,同时要分配内存空间。

592.程序什么时候应该使用线程，什么时候单线程效率高。

正确答案：

- 1 耗时的操作使用线程，提高应用程序响应
- 2 并行操作时使用线程，如C/S架构的服务器端并发线程响应用户的请求。
- 3 多CPU系统中，使用线程提高CPU利用率
- 4 改善程序结构。一个既长又复杂的进程可以考虑分为多个线程，成为几个独立或半独立的运行部分，这样的程序会利于理解和修改。

其他情况都使用单线程。

593.介绍一下模板和容器。如何实现？（也许会让你当场举例实现）

正确答案：

模板可以说比较古老了，但是当前的泛型编程实质上就是模板编程。它体现了一种通用和泛化的思想。STL有7种主要容器：vector,list,deque,map,multimap,set,multiset.

594.以下为WindowsNT下的32位C++程序，请计算sizeof的值

```
char str[]="Hello";
```

```
charp=str;
intn=10;
//请计算
sizeof(str)=?
sizeofs=?
sizeof(n)=?
voidFunc(charstr[100])
{
//请计算
sizeof(str)=?
}
voidp=malloc(100);
//请计算
sizeofs=?
```

正确答案:

```
sizeof (str ) = 6
sizeof ( p ) = 4
sizeof ( n ) =4
void Func ( char str[100])
{
sizeof( str ) = 4
}
void *p = malloc( 100 );
sizeof ( p ) =4
```

595.C语言同意一些令人震惊的结构,下面的结构是合法的 吗，如果是它做些什么？inta=5,b=7,c;c=a+++b;

正确答案：

这个问题将做为这个测验的一个愉快的结尾。不管你相不相信，上面的例子是完全合乎语法的。问题是编译器如何处理它？水平不高的编译作者实际上会争论这个问题，根据最处理原则，编译器应当能处理尽可能所有合法的用法。因此，上面的代码被处理成：c = a++ + b; 因此, 这段代码执行后a = 6, b = 7, c = 12。如果你知道答案，或猜出正确答案，做得好。如果你不知道答案，我也不把这个当作问题。我发现这个问题的最大好处是:这是一个关于代码编写风格，代码的可读性，代码的可修改性的好的话题

596.#include与#include“file.h”的区别？

正确答案：

前者是从Standard Library的路径寻找和引用file.h，而后者是从当前工作路径搜寻并引用file.h。

597.如何在C中初始化一个字符数组。

正确答案：

这个问题看似很简单，但是我们要将最简单的问题用最严谨的态度来对待。关键的地方：初始化、字符型、数组。最简单的方法是char array[];。这个问题看似解决了，但是在初始化上好像还欠缺点什么，个人认为：char array[5]={‘1’,‘2’,‘3’,‘4’,‘5’};或者char array[5]={“12345”};或者char array[2][10]={“China”,“Beijing”};也许更符合“初始化”的意思。

598.在C++程序中调用被C编译器编译后的函数，为什么要加extern“C”？

正确答案：

extern是C/C++语言中表明函数和全局变量作用范围（可见性）的关键字，其声明的函数和变量可以在本模块或其它模块中使用。通常，在模块的头文件中对本模块提供给其它模块引用的函数和全局变量以关键字extern声明。

599.内存的分配方式的分配方式有几种？

正确答案：

- 1) 从静态存储区域分配。内存存在程序编译的时候就已经分配好，这块内存存在程序的整个运行期间都存在。例如全局变量。
- 2) 在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。
- 3) 从堆上分配，亦称动态内存分配。程序在运行的时候用malloc或new申请任意多少的内存，程序员自己负责在何时用free或delete释放内存。动态内存的生存期由我们决定，使用非常灵活，但问题也最多。

600.在C++程序中调用被C编译器编译后的函数，为什么要加extern"C"？

正确答案：

C++语言支持函数重载，C语言不支持函数重载。函数被C++编译后在库中的名字与C语言的不同。假设某个函数的原型为：void foo(int x, int y);该函数被C编译器编译后在库中的名字为foo,而C++编译器则会生成像foo_int_int之类的名字。C++提供了C连接交换指定符号extern"C"来解决名字匹配问题。

601.如何让局部变量具有全局生命期。

正确答案：

具体的生命期的概念我觉得我还要好好深入的学习一下，但是这个题目还算比较简单，即用static修饰就可以了，但是只是生命期延长，范围并没有扩大，除非把这个变量定义在函数体外的静态区，不过那样就变成全局变量了，仿佛不符合题目要求。

602.解释堆和栈的区别。

正确答案：

具体的生命期的概念我觉得我还要好好深入的学习一下，但是这个题目还算比较简单，即用static修饰就可以了，但是只是生命期延长，范围并没有扩大，除非把这个变量定义在函数体外的静态区，不过那样就变成全局变量了，仿佛不符合题目要求。

603.在C++程序中调用被C编译器编译后的函数，为什么要加extern"C"声明？

正确答案：

函数和变量被C++编译后在符号库中的名字与C语言的不同，被extern "C"修饰的变量和函数是按照C语言方式编译和连接的。由于编译后的名字不同，C++程序不能直接调用C函数。C++提供了一个C连接交换指定符号extern"C"来解决这个问题。

604.strtok函数在使用上要注意什么问题。

正确答案：

这个问题我不知道能不能回答全面，因为实在是用的很少。这个函数的作用是分割字符串，但是要分割的字符串不能是常量，这是要注意的。比如先定义一个字符串：char array[]="part1,part2";，strtok的原形是char *strtok(char *string, char *delim);，我们将","作为分隔符，先用pt=strtok(array,",");，得到的结果print出来就是"part1"，那后面的呢，要写成pt=strtok(NULL,",");，注意，要用NULL，如果被分割的字符串会被分成N段，那从第二次开始就一直要用NULL。总结起来，需要注意的是：被分割的字符串和分隔符都要使用变量；除第一次使用指向字符串的指针外，之后的都要使用NULL；注意使用这个函数的时候千万别把指针跟丢了，不然就全乱了。

605.用预处理指令#define声明一个常数，用以表明1年中有多少秒（忽略闰年问题）

正确答案：

#define SECONDS_PER_YEAR (60 * 60 * 24 * 365)UL 我在这想看到几件事情：

- 1). #define 语法的基本知识（例如：不能以分号结束，括号的使用，等等）
- 2). 懂得预处理器将为你计算常数表达式的值，因此，直接写出你是如何计算一年中有多少秒而不是计算出实际的值，是更清晰而没有代价的。3). 意识到这个表达式将使一个16位机的整型数溢出-因此要用到长整型符号L,告诉编译器这个常数是长整型数。
- 4). 如果你在你的表达式中用到UL（表示无符号长整型），那么你有了一个好的起点。记住，第一印象很重要。

606.说一说C与C++的内存分配方式？

正确答案：

- 1)从静态存储区域分配。内存存在程序编译的时候就已经分配好，这块内存存在程序的整个运行期间都存在，如全局变量，static变量。
- 2)在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。
- 3)从堆上分配(动态内存分配)程序在运行的时候用malloc或new申请任意多少的内存，程序员负责在何时用free或删除释放内存。动态内存的生存期自己决定，使用非常灵活。

607.你如何理解MVC。简单举例来说明其应用。

正确答案：

MVC模式是observer 模式的一个特例,典型的有MFC里面的文档视图架构。

608.在C++程序中调用被C编译器编译后的函数，为什么要加extern"C"声明？

正确答案：

二者的编译器不同

609.int i=(j=4,k=8,l=16,m=32);printf("%d",i);输出是多少？

正确答案：

相当于i=j=4；i=k=8；i=l=16；i=m=32;故最后i=32；

610. #include与#include"file.h"的区别？

正确答案：

相当于i=j=4；i=k=8；i=l=16；i=m=32;故最后i=32；

611.既然C++中有更好的const为什么还要使用宏？

正确答案：

相当于i=j=4；i=k=8；i=l=16；i=m=32;故最后i=32；

612.重载（overload）和重写（overried，有的书也叫做“覆盖”）的区别？

正确答案：

从定义上来说：重载：是指允许存在多个同名函数，而这些函数的参数表不同（或许参数个数不同，或许参数类型不同，或许两者都不同）。重写：是指子类重新定义复类虚函数的方法。从实现原理上来说：重载：编译器根据函数不同的参数表，对同名函数的名称做修饰，然后这些同名函数就成了不同的函数。重写：当子类重新定义了父类的虚函数后，父类指针根据赋给它的不同的子类指针，动态的调用属于子类的该函数，这样的函数调用在编译期间是无法确定的（调用的子类的虚函数的地址无法给出）。

613.C++和C定义结构的分别是什么。

正确答案：

Clanguage的结构仅仅是数据的结合Cplusplus的struct和class其实具备几乎一样的功能，只是默认的属性不一样而已。

614.#include和#include"a.h"有什么区别？

正确答案：

对于#include,编译器从标准库路径开始搜索a.h对于#include"a.h",编译器从用户的工作路径开始搜索a.h

615. #include和#include"filename.h"有什么区别？

正确答案：

前者用来包含开发环境提供的库头文件，后者用来包含自己编写的头文件。

616.C函数可否单独编译？

正确答案：

外部函数，可以在开始引进来

617.请简述以下两个for循环的优缺点

```
1) for(i=0;i<n;i++)
{
    if(condition)
    DoSomething();
    else
    DoOtherthing();
}
```

```
2) if(condition)
{
for(i=0;i<n;i++)
DoSomething();
}
else
{
for(i=0;i<n;i++)dootherthing();}
}
```

正确答案：

1) 优点：程序简洁。缺点：多执行了n-1次逻辑判断，并且打断了循环“流水线”作业，使得编译器不能对循环进行优化处理，降低了效率。

2) 优点：循环的效率。缺点：程序不简洁。

618.完成程序，实现对数组的降序排序

```
#include
voidsort(intarray[]);
intmain()
{
intarray[]={45, 56, 76, 234, 1, 34, 23, 2, 3}; //数字任//意给出
sort(array);
return0;
}
voidsort(intarray[])
{
_____
inti,j,k;
for(i=1;i<=7;i++){if(array[i]>array[i-1])
{
k=ARRAY[i];
j=i-1;
do
{
array[j+1]=array[j];
j-;
}
while(k>array[j]&&j>=0);
array[j+1]=k;
}
}
_____
}
```

619.delete[]arry和deletearry一样吗？不一样请说明；

正确答案：

delete[]arry释放的是多个同一类型的地址空间Delete[]arry释放的是一个某种类型的地址空间

620.结合1个你认为比较能体现OOP思想的项目，用UML来描述。

正确答案：

(最好这个项目继承，多态，虚函数都有体现) 这个问题大概会占面试时间的一半，并且会问很多问题，一不小心可能会被问住)

621.C与C++各自是如何定义常量的?有什么不同?

正确答案：

C中是使用宏#define定义，C++使用更好的const来定义。区别：1)const是有数据类型的常量，而宏常量没有，编译器可以对前者进行静态类型安全检查，对后者仅是字符替换，没有类型安全检查，而且在字符替换时可能会产生意料不到的错误(边际效应)。2)有些编译器可以对const常量进行调试，不能对宏调试。

622.头文件中的ifndef/define/endif干什么用?

正确答案：

防止重复定义

623.C++中为什么用模板类。

正确答案：

- (1) 可用来创建动态增长和减小的数据结构
- (2) 它是类型无关的，因此具有很高的可复用性。
- (3) 它在编译时而不是运行时检查数据类型，保证了类型安全
- (4) 它是平台无关的，可移植性
- (5) 可用于基本数据类型

624.动态连接库的两种方式?

正确答案：

调用一个DLL中的函数有两种方法：1载入时动态链接(load-timedynamiclinking)，模块非常明确调用某个导出函数，使得他们就像本地函数一样。这需要链接时链接那些函数所在DLL的导入库，导入库向系统提供了载入DLL时所需的信息及DLL函数定位。2运行时动态链接(run-timedynamiclinking)，运行时可以通过LoadLibrary或LoadLibraryEx函数载入DLL。DLL载入后，模块可以通过调用GetProcAddress获取DLL函数的出口地址，然后就可以通过返回的函数指针调用DLL函数了。如此即可避免导入库文件了。

625.在什么时候需要使用“常引用”?

正确答案：

如果既要利用引用提高程序的效率，又要保护传递给函数的数据不在函数中被改变，就应使用常引用。
常引用声明方式：const类型标识符&引用名=目标变量名；

626.预处理器标识#error的目的是什么?

正确答案：

如果你不知道答案，请看参考文献1。这问题对区分一个正常的伙计和一个书呆子是很有用的。只有书呆子才会读C语言课本的附录去找出现这种问题的答案。当然如果你不是在找一个书呆子，那么应试者最好希望自己不要知道答案。

627.GCC3.2.2版本中支持哪几种编程语言。

正确答案：

这个问题实在变态，就像问你#error的作用是什么一样。不可否认，gcc是linux下一个亮点，是一个备受无数程序员推崇的编译器，其优点省略1000字，有兴趣可以自己查，我翻了翻书，书上曰：支持C,C++,Java,Obj-C,Ada,Fortran,Pascal,Modula-3等语言，这个“等”比较要命，不过我认为已经很全了，如果认为还是不全，干脆把ASM也加上算了，不过那已经不算是编译了。

628.已知strcpy的函数原型： charstrcpy(charstrDest,constchar*strSrc)其中 strDest是目的字符串，strSrc是源字符串。不调用C++/C 的字符串库函数，请编写函数strcpy。

正确答案：

```
charstrcpy(charstrDest,constcharstrSrc)
{
    if(strDestNULL||strSrcNULL)
        returnNULL;
    if(strDest==strSrc)
        returnstrDest;
    chartempPtr=strDest;
    while((*strDest++=*strSrc++)!='\0')
        ;
    returntempPtr;
}
```

2. 请问运行Test函数会有什么样的结果？

```
charGetMemory(void)
{
    charp[]="helloworld";
    returnp;
}

voidTest(void)
{
    charstr=NULL;
    str=GetMemory();
    printf(str);
}
```

正确答案：乱码

629.重载 (overload)和重写(overried , 有的书也叫做“覆盖”) 的区别？

正确答案：

从定义上来说：重载：是指允许存在多个同名函数，而这些函数的参数表不同（或许参数个数不同，或许参数类型不同，或许两者都不同）。重写：是指子类重新定义复类虚函数的方法。从实现原理上来说：重载：编译器根据函数不同的参数表，对同名函数的名称做修饰，然后这些同名函数就成了不同的函数。重写：当子类重新定义了父类的虚函数后，父类指针根据赋给它的不同的子类指针，动态的调用属于子类的该函数，这样的函数调用在编译期间是无法确定的（调用的子类的虚函数的地址无法给出）。

630.多重继承如何消除向上继承的二义性。

正确答案：

使用虚拟继承即可。

631.#include与#include“file.h”的区别？

正确答案：

前者是从StandardLibrary的路径寻找和引用file.h，而后者是从当前工作路径搜寻并引用file.h。

632.对数据库的一张表进行操作,同时要对另一张表进行操作,如何实现？

正确答案：

将操作多个表的操作放入到事务中进行处理

633.#include<filename.h>和#include“filename.h”有什么区别？

正确答案：

查找范围不同，后者先查找工作路径，再查找VC标准路径；前者只查工作路径。

634.预处理器标识#error的目的是什么？

正确答案：

如果你不知道答案，请看参考文献1。这问题对区分一个正常的伙计和一个书呆子是很有用的。只有书呆子才会读C语言课本的附录去找出现这种问题的答案。当然如果你不是在找一个书呆子，那么应试者最好希望自己不要知道答案。

635.头文件的作用是什么？

正确答案：

1) 通过头文件来调用库功能。

2) 头文件能加强类型安全检查。

636.请问运行Test函数会有什么样的结果？

```
void GetMemory(charp)
{
    p=(char)malloc(100);
}
void Test(void)
{
    char*str=NULL;
    GetMemory(str);
    strcpy(str,"helloworld");
    printf(str);
}
```

正确答案：

程序崩溃了

637.delete[]arry和deletearry一样吗？不一样请说明；

正确答案：

delete[]arry释放的是多个同一类型的地址空间Delete[]arry释放的是一个某种类型的地址空间

638.请问运行Test函数会有什么样的结果？

```
void GetMemory(char**p, int num){
    p=(char*)malloc(num);
}
void Test(void){
    char*str=NULL;
    GetMemory(&str,100);
    strcpy(str, "hello");
    printf(str);
}
```

正确答案：

输出"hello"

639.请简述以下两个for循环的优缺点

```
//第一个
for(i=0;i<N;i++)
{
    if(condition)
        DoSomething();
    else
        DoOtherthing();
}
//第二个
if(condition)
{
    for(i=0;i<N;i++)
        DoSomething();
}
else
{
    for(i=0;i<N;i++)
        DoOtherthing();
}
```

正确答案：

先循环再判断，先判断再循环
第一个优点：每个循环都先判断，再执行
第二个优点：条件判断，只发生其中一个行为

640.构造函数可否是虚函数，为什么？析构函数呢，可否是纯虚的呢？

正确答案：

错题解析：构造函数不能为虚函数，要构造一个对象，必须清楚地知道要构造什么，否则无法构造一个对象。析构函数可以为纯虚函数。

641.在C++程序中调用被C编译器编译后的函数，为什么要加extern"C"?

正确答案：

C++语言支持函数重载，C语言不支持函数重载。函数被C++编译后在库中的名字与C语言的不同。假设某个函数的原型为：void foo(intx,inty);该函数被C编译器编译后在库中的名字为foo,而C++编译器则会产生像foo_int_int之类的名字。C++提供了C连接交换指定符号extern"C"来解决名字匹配问题。

642.请写出下面代码在32位平台上的运行结果，并说明sizeof的性质：

```
#include
#include
int main(void)
{
    char a[30];
    char b=(char)malloc(20*sizeof(char));
    printf("%d\n",sizeof(a));
    printf("%d\n",sizeof(b));
    printf("%d\n",sizeof(a[3]));
    printf("%d\n",sizeof(b+3));
    printf("%d\n",sizeof((b+4)));
    return 0;
}
```

正确答案：

在32位系统下（如WIN32），指针长度为32位。a是一个有30个元素的字符型数组；b是一个字符串指针；a[3]是字符型；b+3是指针；*(b+4)是字符型。因此输出：30、4、1、4、1

643.高级通信包括信号量，——，——

正确答案：

通常把信号、信号量通信称为低级通信,而把管道、消息队列、共享存储区通信称为高级通信，这个题目我也不知道怎么填了，。。。。。

644.关联、聚合(Aggregation)以及组合(Composition)的区别？

正确答案：

涉及到UML中的一些概念：关联是表示两个类的一般性联系，比如“学生”和“老师”就是一种关联关系；聚合表示has-a的关系，是一种相对松散的关系，聚合类不需要对被聚合类负责，如下图所示，用空的菱形表示聚合关系：从实现的角度讲，聚合可以表示为: class A{...} class B{A*a;.....}而组合表示contains-a的关系，关联性强于聚合：组合类与被组合类有相同的生命周期，组合类要对被组合类负责，采用实心的菱形表示组合关系：实现的形式是: class A{...} class B{Aa;...}

645.尽管不像非嵌入式计算机那么常见，嵌入式系统还是有从堆（heap）中动态分配内存的过程的。那么嵌入式系统中，动态分配内存可能发生的问题是什么？

正确答案：

这里，我期望应试者能提到内存碎片，碎片收集的问题，变量的持行时间等等。这个主题已经在ESP杂志中被广泛地讨论过了（主要是选项：J.Plauger,他的解释远远超过我这里能提到的任何解释），所有回过头看一下这些杂志吧！让应试者进入一种虚假的安全感觉后，我拿出这么一个小节目：下面的代码片段的输出是什么，为什么？

```
charptr;  
if((ptr=(char)malloc(0))!=NULL)  
puts("Gotanullpointer");  
else  
puts("Gotavalidpointer");
```

这是一个有趣的问题。最近在我的一个同事不经意把0值传给了函数malloc，得到了一个合法的指针之后，我才想到这个问题。这就是上面的代码，该代码的输出是“Gotavalidpointer”。

646.请问运行Test函数会有什么样的结果？

```
voidTest(void){  
charstr=(char)malloc(100);  
strcpy(str,"hello");  
free(str);  
if(str!=NULL){  
strcpy(str,"world");  
printf(str);  
}  
}
```

正确答案：

错题解析：输出“world”

版权声明：本文为CSDN博主「Lotay.」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_43233219/article/details/124457640

647.多态的实现

在基类的函数前加上virtual关键字，在派生类中重写该函数，运行时将会根据对象的实际类型来调用相应的函数。

如果对象类型是派生类，就调用派生类的函数；如果对象类型是基类，就调用基类的函数，此为多态的表现；

648.Cpp四种强制类型转换

const_cast：从字面意思上就可以理解，去除变量的const属性。

static_cast：静态类型转换，一般用于基本类型间的转换，如char->int

dynamic_cast：动态转换，同于多态之间的类型转换

reinterpret_cast：用于不同类型的指针类型的转换。

649.类的static成员的特点

static成员只有一份拷贝，被该类的所有对象所共享；

static成员只能在类外初始化，并存放在全局（静态）存储区，不计入类的大小中；

static可以通过类名直接访问，也可以通过对象访问；

static成员函数只能访问static成员变量，因为其他的数据成员与生成的对象是绑定的，static成员函数不属于任何对象，没有this指针；

650.指针和引用的区别

引用是被引用对象的一个别名，其只能在定义的时候初始化，并且其值不能改变不能为空

指针可以在任何时候给其赋值，并且其可以为nullptr

sizeof引用为其引用对象的大小，sizeof指针为指针本身的大小

对引用取地址为其引用对象的地址

651.谈谈对C++内存的理解

1、栈区（stack）— 由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。

2、堆区（heap）— 一般由程序员分配释放，若程序员不释放，程序结束时可能由OS回收。注意它与数据结构中的堆是两回事，分配方式倒是类似于链表。

3、全局区（静态区）（static）— 全局变量和静态变量的存储是放在一块的，初始化的全局变量和静态变量在一块区域，未初始化的全局变量和未初始化的静态变量在相邻的另一块区域。- 程序结束后有系统释放

4、文字常量区 — 常量字符串就是放在这里的。程序结束后由系统释放

5、程序代码区 — 存放函数体的二进制代码。

652.谈谈new、delete、malloc、free

1.malloc与free是C++/C语言的标准库函数，new/delete是C++的运算符。它们都可用于申请动态内存和释放内存。

2.对于非内部数据类型的对象而言，光用malloc/free无法满足动态对象的要求。对象在创建的同时要自动执行构造函数，对象在消亡之前要自动执行析构函数。由于malloc/free是库函数而不是运算符，不在编译器控制权限之内，不能够把执行构造函数和析构函数的任务强加于malloc/free。

3.因此C++语言需要一个能完成动态内存分配和初始化工作的运算符new，以一个能完成清理与释放内存工作的运算符delete。注意new/delete不是库函数

653.const关键字

1.const 修饰类的成员变量，表示成员常量，不能被修改。

2.const修饰函数承诺在本函数内部不会修改类内的数据成员，不会调用其它非 const 成员函数。

3.如果 const 构成函数重载，const 对象只能调用 const 函数，非 const 对象优先调用非 const 函数。

4.const 函数只能调用 const 函数。非 const 函数可以调用 const 函数。

5.类体外定义的 const 成员函数，在定义和声明处都需要 const 修饰符。

int const *p / const int *p; //value是常数

int * const p; //常指针

int *const p const; //常指针、value值也是常数

654.知道STL吗，挑两个你最常用的容器说一说

vector：动态扩容数组

map：key-value数据，自动排序去重。有以下几种不同的map（map、multimap、unordered_map、unordered_multimap），其中map用的是红黑树，unordered_map用的是hash表。

655.怎么确定一个程序是C编译的还是C++编译的

如果编译器在编译cpp文件，那么_cplusplus就会被定义，如果是一个C文件被编译，那么_STDC就会被定义，STDC是预定义宏，当它被定义后，编译器将按照ANSI C标准来编译C语言程序。

656.一个文件从源码到可执行文件所经历的过程

- 1.预处理，产生.i文件
- 2.编译，产生汇编文件(.s文件)
- 3.汇编，产生目标文件(.o或.obj文件)
- 4.链接，产生可执行文件(.out或.exe文件)

657.了解C++新特性吗

- 1.关键字及新语法：auto、nullptr、for
- 2.STL容器：std::array、std::forward_list、std::unordered_map、std::unordered_set
- 3.多线程：std::thread、std::atomic、std::condition_variable
- 4.智能指针内存管理：std::shared_ptr、std::weak_ptr
- 5.其他：std::function、std::bind和lambda表达式

658.什么是纯虚函数

相当于一个函数接口，只声明不定义。在其派生类里会重写。有纯虚函数的类为抽象类，不能实例化出对象。

659.构造函数和析构函数可以为虚函数吗

构造函数不可以，析构函数可以甚至有时候必须声明为虚函数。

660.栈和堆的区别，什么时候必须使用堆

栈是由程序分配的，而堆是由程序员手动去分配释放的。当需要的空间特别大的时候，就必须使用堆，因为栈的大小是有限制的，一般为5MB左右，所以当需要一个大块空间是，必须在堆上开辟空间。

661.用宏定义实现swap

```
#define F(a, b) (a = a ^ b);(b = a ^ b);(a = a ^ b);
```

662.头文件<>和""的区别

遇到#include<math.h>时，系统先从系统默认的头文件目录中查找头文件

遇到#include"math.h"时，系统先从当前的目录中搜索，若没有找到，再从系统默认的头文件中找

故包含系统提供的库函数用#include<math.h>更快

当包含用户自定义的.h文件时，使用#include"math.h"更快

663.编写string的构造函数、拷贝构造函数、赋值操作符重载和析构函数

```
#include<cstring>
#include<iostream>

using namespace std;

class MyString {
public:
    MyString(const char* pcData = nullptr) {
        if(pcData == nullptr) {
            m_pdata = new char[1];
            *m_pdata = '\0';
        }
        else {
            int len = strlen(pcData);
            m_pdata = new char[len+1];
            strcpy(m_pdata, pcData);
        }
    }

    MyString(const MyString& other) {
        int len = strlen(other.m_pdata);
        m_pdata = new char[len+1];
        strcpy(m_pdata, other.m_pdata);
    }

    MyString& operator =(const MyString &str) {
        if(this == &str)
            return *this;
        delete [] m_pdata;
        m_pdata = nullptr;
        m_pdata = new char[strlen(str.m_pdata)+1];
        strcpy(m_pdata, str.m_pdata);
        return *this;
    }

    void Print() {
        cout << this->m_pdata << endl;
    }

    ~MyString() {
        delete [] m_pdata;
    }

private:
    char* m_pdata;
};

int main() {
    MyString mstr;
    MyString mstr2("hello world!");
    mstr = mstr2;
    mstr.Print();
    mstr2.Print();

    return 0;
}
```

```
}
```

664.进程和线程间的通信方式

Linux进程：管道、有名管道、信号、信号量、共享内存、消息队列、套接字

Linux线程：互斥体、信号量、条件变量

Windows进程：管道、共享内存、消息队列、信号量、套接字

Windows线程：临界区、互斥量、信号量、事件

665.死锁产生的原因和死锁的条件

原因：系统资源的竞争、进程推进顺序非法

条件：互斥条件、不剥夺条件、请求和保持条件、循环等待条件

666.如何采用单线程处理高并发

采取I/O复用来提高单线程处理多请求的能力（epoll和select）

采用事件驱动模型，基于异步回调来处理事件

667.线程的状态

新建(NEW)、可运行(RUNNABLE)、运行(RUNNING)、阻塞(BLOCKED)、死亡(DEAD)

668.进程的状态

运行状态：进程正在处理器上运行，在单处理器环境下，每一时刻最多只有一个进程处于运行状态。

就绪状态：进程已处于准备运行的状态，即进程获得了除处理器之外的一切所需要的资源，一旦得到处理器即可运行。

阻塞状态：又称为等待状态，进程正在等待某一事件而暂停运行。如等待某资源为可用（不包括处理器）或等待输入/输出完成。即使处理器空闲，该进程也不能运行。

创建状态：进程正在被创建，尚未转到就绪状态。

结束状态：进程正从系统中消失。可能是进程正常结束或其它原因中断退出运行。

669.系统调用brk和mmap

从操作系统角度来看，进程分配内存有两种方式，分别由两个系统调用完成：brk和mmap（不考虑共享内存）。

1、brk是将数据段(.data)的最高地址指针_edata往高地址推；

2、mmap是在进程的虚拟地址空间中（堆和栈中间，称为文件映射区域的地方）找一块空闲的虚拟内存。

这两种方式分配的都是虚拟内存，没有分配物理内存。在第一次访问已分配的虚拟地址空间的时候，发生缺页中断，操作系统负责分配物理内存，然后建立虚拟内存和物理内存之间的映射关系。

在标准C库中，提供了malloc/free函数分配释放内存，这两个函数底层是由brk，mmap，munmap这些系统调用实现的。

670.说说三种内存管理机制

页式管理：

段式管理：

分段分页管理：

671.大端和小端，用C++代码怎么确定

大端低地址存放高位，高地址存放地位。小端相反。

```
union un {
    char a;
    int b;
}
un tmp.b = 1;
if(tmp.a) {
    cout << "小端" << endl;
}
else {
    cout << "大端" << endl;
}
```

672.TCP和UDP的区别

用户数据报协议 UDP(User Datagram Protocol) 是无连接的，尽最大可能交付，没有拥塞控制，面向报文（对于应用程序传下来的报文不合并也不拆分，只是添加UDP首部），支持一对一、一对多、多对一和多对多的交互通信。

传输控制协议 TCP(Transmission Control Protocol) 是面向连接的，提供可靠交付，有流量控制，拥塞控制，提供全双工通信，面向字节流（把应用层传下来的报文看成字节流，把字节流组织成大小不等的块），每一条 TCP 连接只能是点对点的（一对一）。

- 1.TCP面向连接，UDP面向无连接的
- 2.TCP有保障的，UDP传输无保障的
- 3.TCP是效率低的，UDP效率高的
- 4.TCP是基于流的，UDP基于数据报文
- 5.TCP传输重要数据，UDP传输不重要的数据

673.TCP三次握手

首先 B 处于 LISTEN（监听）状态，等待客户的连接请求。A 向 B 发送连接请求报文，SYN=1，ACK=0，选择一个初始的序号 x。

B 收到连接请求报文，如果同意建立连接，则向 A 发送连接确认报文，SYN=1，ACK=1，确认号为 x+1，同时也选择一个初始的序号 y。

A 收到 B 的连接确认报文后，还要向 B 发出确认，确认号为 y+1，序号为 x+1。

B 收到 A 的确认后，连接建立。

674.三次握手的原因

第三次握手是为了防止失效的连接请求到达服务器，让服务器错误打开连接。

客户端发送的连接请求如果在网络中滞留，那么就会隔很长一段时间才能收到服务器端发回的连接确认。客户端等待一个超时重传时间之后，就会重新请求连接。但是这个滞留的连接请求最后还是会到达服务器，如果不进行三次握手，那么服务器就会打开两个连接。如果有第三次握手，客户端会忽略服务器之后发送的对滞留连接请求的连接确认，不进行第三次握手，因此就不会再次打开连接。

675.TCP四次挥手

以下描述不讨论序号和确认号，因为序号和确认号的规则比较简单。并且不讨论 ACK，因为 ACK 在连接建立之后都为 1。

A 发送连接释放报文，FIN=1。

B 收到之后发出确认，此时 TCP 属于半关闭状态，B 能向 A 发送数据但是 A 不能向 B 发送数据。

当 B 不再需要连接时，发送连接释放报文，FIN=1。

A 收到后发出确认，进入 TIME-WAIT 状态，等待 2 MSL（最大报文存活时间）后释放连接。

B 收到 A 的确认后释放连接。

676.四次挥手的原因

客户端发送了 FIN 连接释放报文之后，服务器收到了这个报文，就进入了 CLOSE-WAIT 状态。这个状态是为了让服务器端发送还未传送完毕的数据，传送完毕之后，服务器会发送 FIN 连接释放报文。

677.TIME_WAIT

客户端接收到服务器端的 FIN 报文后进入此状态，此时并不是直接进入 CLOSED 状态，还需要等待一个时间计时器设置的时间 2MSL。这么做有两个理由：

确保最后一个确认报文能够到达。如果 B 没收到 A 发送来的确认报文，那么就会重新发送连接释放请求报文，A 等待一段时间就是为了处理这种情况的发生。

等待一段时间是为了让本连接持续时间内所产生的所有报文都从网络中消失，使得下一个新的连接不会出现旧的连接请求报文。

678.Http协议

http（超文本传输协议）是一个基于请求与响应模式的、无状态的、应用层的协议，常基于TCP的连接方式，HTTP1.1版本中给出一种持续连接的机制，绝大多数的Web开发，都是构建在HTTP协议之上的Web应用。

679.几种常见的排序算法

快排、插入、选择，这是最基本的三个，建议背下来，最有可能让你手撕的。

680.链表的一些性质和操作

链表是最常用也是比较简单的一个基本数据结构，几乎是构成所有高级结构的基础，面试常拿链表做文章。建议了解链表的一些常见问题：

如何判断链表有环

使用快慢指针，当两个指针重合证明有环

转置链表O(n)时间、O(1)空间

三个指针，改变next指针指向

681.常见的查找算法

二分法

682.动态规划

版权声明：本文为CSDN博主「长安某~」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_44816732/article/details/105646613

683.关键字 static 的作用是什么？

这个问题很少有人能回答完全。在 C 语言中，关键字 static 有三个明显的作用：

- 1). 在函数体，一个被声明为静态的变量在这一函数被调用过程中维持其值不变。
- 2). 在模块内（但在函数体外），一个被声明为静态的变量可以被模块内所用函数访问，但不能被模块外其它函数访问。它是一个本地的全局变量。
- 3). 在模块内，一个被声明为静态的函数只可被这一模块内的其它函数调用。那就是，这个函数被限制在声明它的模块的本地范围内使用。、大多数应试者能正确回答第一部分，一部分能正确回答第二部分，同是很少的人能懂得第三部分。这是一个应试者的严重的缺点，因为他显然不懂得本地化数据和代码范围的好处和重要性。

684.“引用”与指针的区别是什么？

- (1) 引用必须被初始化，指针不必。
- (2) 引用初始化以后不能被改变，指针可以改变所指的对象。
- (3) 不存在指向空值的引用，但是存在指向空值的指针。

指针通过某个指针变量指向一个对象后，对它所指向的变量间接操作。程序中使用指针，程序的可读性差；而引用本身就是目标变量的别名，对引用的操作就是对目标变量的操作。流操作符<<和>>、赋值操作符=的返回值、拷贝构造函数的参数、赋值操作符=的参数、其它情况都推荐使用引用。

685..h 头文件中的 ifndef/define/endif 的作用？

防止该头文件被重复引用。

686.#include 与 #include "file.h"的区别？

前者是从 Standard Library 的路径寻找和引用 file.h，而后者是从当前工作路径搜寻并引用 file.h。

687.描述实时系统的基本特性

在特定时间内完成特定的任务，实时性与可靠性。

688.全局变量和局部变量在内存中是否有区别？如果有，是什么区别？

全局变量储存在静态数据区，局部变量在堆栈中。

689.什么是平衡二叉树？

左右子树都是平衡二叉树 且左右子树的深度差值的绝对值不大于1

690.堆栈溢出一般是由什么原因导致的？

- (1)没有回收垃圾资源
- (2)层次太深的递归调用

691.冒泡排序算法的时间复杂度是什么？

$O(n^2)$

692.什么函数不能声明为虚函数？

constructor

693.队列和栈有什么区别？

队列先进先出，栈后进先出

694.不能做 switch()的参数类型

switch 的参数不能为实型。

695.局部变量能否和全局变量重名？

能，局部会屏蔽全局。要用全局变量，需要使用“::” 局部变量可以与全局变量同名，在函数内引用这个变量时，会用到同名的局部变量，而不会用到全局变量。对于有些编译器而言，在同一个函数内可以定义多个同名的局部变量，比如 在两个循环体内都定义一个同名的局部变量，而那个局部变量的作用域就在那个循环体内

696.如何引用一个已经定义过全局变量？

可以用引用头文件的方式，也可以用 extern 关键字，如果用引用头文件方式来引用某个在头文件中声明的全局变量，假定你将那个变量写错了，那么在编译期间会报错，如果你用 extern 方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在连接期间报错。

697.全局变量可不可以定义在可被多个.C 文件包含的头文件中？为什么？

可以，在不同的 C 文件中以 static 形式来声明同名全局变量。可以在不同的 C 文件中声明同名的全局变量，前提是其中只能有一个 C 文件中对此变量赋初值，此时连接不会出错。

698.语句 for(; 1 ;)有什么问题？它是什么意思？

和 while(1)相同，无限循环。

699.do.....while 和 while.....do 有什么区别？

前一个循环一遍再判断，后一个判断以后再循环。

700.static 全局变量、局部变量、函数与普通全局变量、局部变量、函数 static 全局变量与普通的全局变量有什么区别？static局部变量和普通局部变量有什么区别？static函数与普通函数有什么区别？

全局变量(外部变量)的说明之前再冠以 static 就构成了静态的全局变量。全局变量本身 就是静态存储方式，静态全局变量当然也是静态存储方式。这两者在存储方式上并无不同。

这两者的区别虽在于非静态全局变量的作用域是整个源程序，当一个源程序由多个源文件 组成时，非静态的全局变量在各个源文件中都是有效的。而静态全局变量则限制了其作用 域，即只在定义该变量的源文件内有效，在同一源程序的其它源文件中不能使用它。由于静态全局变量的作用域局限于一个源文件内，只能为该源文件内的函数公用，因此可以避免在其它源文件中引起错误。从以上分析可以看出，把局部变量改变为静态变量后是改变了它的存储方式即改变了它的 生存期。把全局变量改变为静态变量后是改变了它的作用域，限制了它的使用范围。

static 函数与普通函数作用域不同。仅在本文件。只在当前源文件中使用的函数应该说明为 内部函数 (static)，内部函数应该在当前源文件中说明和定义。对于可在当前源文件以外使用 的函数，应该在一个头文件中说明，要使用这些函数的源文件要包含这个头文件

static 全局变量与普通的全局变量有什么区别：static 全局变量只初使化一次，防止在其他文 件单元中被引用；

static 局部变量和普通局部变量有什么区别：static 局部变量只被初始化一次，下一次依据上一次结果值；

static 函数与普通函数有什么区别：static 函数在内存中只有一份，普通函数在每个被调用中 维持一份拷贝

701.程序的内存分配

一个由 C/C++编译的程序占用的内存分为以下几个部分

1、栈区 (stack)——由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。

2、堆区 (heap)——一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。注意它与数据结构中的堆是两回事，分配方式倒是类似于链表，呵呵。

3、全局区 (静态区) (static)——全局变量和静态变量的存储是放在一块的，初始化的全局变量和静态变量在一块区域，未初始化的全局变量和未初始化的静态变量在相邻的另一块区域。程序结束后由系统释放。

4、文字常量区——常量字符串就是放在这里的。程序结束后由系统释放。

5、程序代码区——存放函数体的二进制代码

例子程序

这是一个前辈写的，非常详细

```
//main.cpp
int a=0; //全局初始化区
char *p1; //全局未初始化区
main()
{
    int b; //栈
    char s[]="abc"; //栈
    char *p2; //栈
    char *p3="123456"; //123456\0在常量区，p3在栈上。
    static int c=0; //全局（静态）初始化区
    p1 = (char*)malloc(10); p2 = (char*)malloc(20); //分配得来得10和20字节的区域就在堆区。
    strcpy(p1,"123456"); //123456\0放在常量区，编译器可能会将它与 p3所向"123456"优化成一个地方。
}
```

702.解释堆和栈的区别

堆 (heap) 和栈(stack)的区别

(1) 申请方式

stack:由系统自动分配。例如,声明在函数中一个局部变量 int b;系统自动在栈中为 b 开辟空间 heap:需要程序员自己申请,并指明大小,在 c 中 malloc 函数,如 p1=(char)malloc(10);在 C++中用 new 运算符;如p2=(char)malloc(10);但是注意 p1、p2本身是在栈中的。

(2) 申请后系统的响应

栈:只要栈的剩余空间大于所申请空间,系统将程序提供内存,否则将报异常提示栈溢出。

堆:首先应该知道操作系统有一个记录空闲内存地址的链表,当系统收到程序的申请时,会遍历该链表,寻找第一个空间大于所申请空间的堆结点,然后将该结点从空闲结点链表中删除,并将该结点的空间分配给程序,另外,对于大多数系统,会在这块内存空间中的首地址处记录本次分配的大小,这样,代码中的 delete 语句才能正确的释放本内存空间。另外,由于找到的堆结点的大小不一定正好等于申请的大小,系统会自动的将多余的那部分重新放入空闲链表中。

(3) 申请大小的限制

栈:在 Windows 下,栈是向低地址扩展的数据结构,是一块连续的内存的区域。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的,在 WINDOWS 下,栈的大小是2M(也有的说是1M,总之是一个编译时就确定的常数),如果申请的空间超过栈的剩余空间时,将提示 overflow。因此,能从栈获得的空间较小。

堆:堆是向高地址扩展的数据结构,是不连续的内存区域。这是由于系统是用链表来存储的空闲内存地址的,自然是不连续的,而链表的遍历方向是由低地址向高地址。堆的大小受限于计算机系统中有效的虚拟内存。由此可见,堆获得的空间比较灵活,也比较大。

(4) 申请效率的比较:

栈:由系统自动分配,速度较快。但程序员是无法控制的。

堆:是由 new 分配的内存,一般速度比较慢,而且容易产生内存碎片,不过用起来最方便。

另外,在 WINDOWS 下,最好的方式是用 Virtual Alloc 分配内存,他不是堆,也不是在栈,而是直接在进程的地址空间中保留一块内存,虽然用起来最不方便。但是速度快,也最灵活。

(5) 堆和栈中的存储内容
栈:在函数调用时,第一个进栈的是主函数中后的下一条指令(函数调用语句的下一条可执行语句)的地址,然后是函数的各个参数,在大多数的 C 编译器中,参数是由右往左入栈的,然后是函数中的局部变量。注意静态变量是不入栈的。当本次函数调用结束后,局部变量先出栈,然后是参数,最后栈顶指针指向最开始存的地址,也就是主函数中的下一条指令,程序由该点继续运行。

堆:一般是在堆的头部用一个字节存放堆的大小。堆中的具体内容由程序员安排。

(6) 存取效率的比较

char s1[]="aaaaaaaaaaaaaa";

char *s2="bbbbbbbbbbbbbbbbbb";

aaaaaaaaaaaa 是在运行时刻赋值的;

而 bbbbbbbbbbbb 是在编译时就确定的;

但是,在以后的存取中,在栈上的数组比指针所指向的字符串(例如堆)快。

比如:

```
#include
void main()
{
    char a=1;
    char c[]="1234567890";
    char *p="1234567890";
    a = c[1];
    a = p[1];
    return;
}
```

对应的汇编代码

```
10:a=c[1];
004010678A4DF1movcl,byteptr[ebp-0Fh]
0040106A884DFCmovbyteptr[ebp-4],cl
11:a=p[1];
0040106D8B55ECmovedx,dwordptr[ebp-14h]
004010708A4201mova1,byteptr[edx+1]
004010738845FCmovbyteptr[ebp-4],a1
```

第一种在读取时直接就把字符串中的元素读到寄存器 cl 中，而第二种则要先把指针值读到 edx 中，在根据 edx 读取字符，显然慢了。

703.什么是预编译,何时需要预编译?

预编译又称为预处理,是做些代码文本的替换工作。处理#开头的指令,比如拷贝#include 包含的文件代码, #define 宏定义的替换,条件编译等, 就是为编译做的预备工作的阶段, 主要处理#开始的预编译指令, 预编译指令指示了在程序正式编译前就由编译器进行的操作, 可以放在程序中的任何位置。c 编译系统在对程序进行通常的编译之前, 先进行预处理。c 提供的预处理功能主要有以下三种: (1) 宏定义 (2) 文件包含 (3) 条件编译

1、总是使用不经常改动的大型代码体。

2、程序由多个模块组成, 所有模块都使用一组标准的包含文件和相同的编译选项。在这种情况下, 可以将所有包含文件预编译为一个预编译头。

704.关键字 const 是什么含意?

我只要一听到被面试者说: “const 意味着常数”, 我就知道我正在和一个业余者打交道。去年 Dan Saks 已经在他的文章里完全概括了 const 的所有用法, 因此 ESP(译者: Embedded Systems Programming) 的每一位读者应该非常熟悉 const 能做什么和不能做什么.如果你从没有读到那篇文章, 只要能说出 const 意味着“只读”就可以了。尽管这个答案不是完全的答案, 但我接受它作为一个正确的答案。(如果你想知道更详细的答案, 仔细读一下 Saks 的文章吧。)如果应试者能正确回答这个问题, 我将问他一个附加的问题: 下面的声明都是什么意思?

```
const int a;
int const a;
const int *a;
int * const a;
int const * a const;
```

前两个的作用是一样, a 是一个常整型数。第三个意味着 a 是一个指向常整型数的指针(也就是, 整型数是不可修改的, 但指针可以)。

第四个意思 a 是一个指向整型数的常指针(也就是说, 指针指向的整型数是可以修改的, 但指针是不可修改的)。

最后一个意味着 a 是一个指向常整型数的常指针(也就是说, 指针指向的整型数是不可修改的, 同时指针也是不可修改的)。如果应试者能正确回答这些问题, 那么他就给我留下了一个好印象。顺带提一句, 也许你可能会问, 即使不用关键字 const, 也还是能很容易写出功能正确的程序, 那么我为什么还要如此看重关键字 const 呢? 我也如下的几下理由:

1). 关键字 const 的作用是为给读你代码的人传达非常有用的信息, 实际上, 声明一个参数为常量是为了告诉了用户这个参数的应用目的。如果你曾花很多时间清理其它人留下的垃圾, 你就会很快学会感谢这有点多余的信息。(当然, 懂得用 const 的程序猿很少会留下的垃圾让别人来清理的。)

2). 通过给优化器一些附加的信息, 使用关键字 const 也许能产生更紧凑的代码。

3). 合理地使用关键字 const 可以使编译器很自然地保护那些不希望被改变的参数, 防止其被无意的代码修改。简而言之, 这样可以减少 bug 的出现

705.关键字 volatile 有什么含意 并给出三个不同的例子。

一个定义为 volatile 的变量是说这变量可能会被意想不到地改变，这样，编译器就不会去假设这个变量的值了。精确地说就是，优化器在用到这个变量时必须每次都小心地重新读取这个变量的值，而不是使用保存在寄存器里的备份。下面是 volatile 变量的几个例子：

- (1). 并行设备的硬件寄存器（如：状态寄存器）
- (2). 一个中断服务子程序中会访问到的非自动变量(Non-automatic variables)
- (3). 多线程应用中被几个任务共享的变量

回答不出这个问题的人是不会被雇佣的。我认为这是区分 C 程序员和嵌入式系统程序员的最基本的问题。嵌入式系统程序员经常同硬件、中断、RTOS 等等打交道，所用这些都要求 volatile 变量。不懂得 volatile 内容将会带来灾难。假设被面试者正确地回答了这是问题（嗯，怀疑这否会是这样），我将稍微深究一下，看一下这家伙是不是真正懂得 volatile 完全的重要性。

- 1). 一个参数既可以是 const 还可以是 volatile 吗？解释为什么。
- 2). 一个指针可以是 volatile 吗？解释为什么。
- 3). 下面的函数有什么错误：

```
int square(volatile int *ptr)
{
    return *ptr * ptr;
}
```

下面是答案：

- 1). 是的。一个例子是只读的状态寄存器。它是 volatile 因为它可能被意想不到地改变。它是 const 因为程序不应该试图去修改它。
- 2). 是的。尽管这并不很常见。一个例子是当一个中服务子程序修该一个指向一个 buffer 的指针时。
- 3). 这段代码的有个恶作剧。这段代码的目的是用来返回指针 ptr 指向值的平方，但是，由于 *ptr 指向一个 volatile 型参数，编译器将产生类似下面的代码：

```
int square(volatile int *ptr)
{
    int a,b;
    a = *ptr;
    b = ptr;
    return a * b;
}
```

由于 ptr 的值可能被意想不到地该变，因此 a 和 b 可能是不同的。结果，这段代码可能返回不是你所期望的平方值！正确的代码如下：

```
long square(volatile int *ptr)
{
    int a;
    a = *ptr;
    return a * a;
}
```

706.三种基本的数据模型

按照数据结构类型的不同，将数据模型划分为层次模型、网状模型和关系模型。

707.结构与联合有和区别？

- (1). 结构和联合都是由多个不同的数据类型成员组成,但在任何同一时刻,联合中只存放了一个被选中的成员（所有成员共用一块地址空间），而结构的所有成员都存在（不同成员的存放地址不同）。
- (2). 对于联合的不同成员赋值,将会对其它成员重写,原来成员的值就不存在了,而对于结构的不同成员赋值是互不影响的

708.描述内存分配方式以及它们的区别？

(1) 从静态存储区域分配。内存存在程序编译的时候就已经分配好，这块内存存在程序的整个运行期间都存在。例如全局变量，static 变量。

(2) 在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集。

(3) 从堆上分配，亦称动态内存分配。程序在运行的时候用 malloc 或 new 申请任意多少的内存，程序员自己负责在何时用 free 或 delete 释放内存。动态内存的生存期由程序员决定，使用非常灵活，但问题也最多

709.请说出 const 与#define 相比，有何优点？

Const 作用：定义常量、修饰函数参数、修饰函数返回值三个作用。被 Const 修饰的东西都受到强制保护，可以预防意外的变动，能提高程序的健壮性。

(1) const 常量有数据类型，而宏常量没有数据类型。编译器可以对前者进行类型安全检查。

而对后者只进行字符替换，没有类型安全检查，并且在字符替换可能会产生意料不到的错误。

(2) 有些集成化的调试工具可以对 const 常量进行调试，但是不能对宏常量进行调试。

710.简述数组与指针的区别？

数组要么在静态存储区被创建（如全局数组），要么在栈上被创建。指针可以随时指向任意类型的内存块。

(1)修改内容上的差别

```
char a[] = "hello";
a[0] = 'x';
char *p = "world"; // 注意 p 指向常量字符串
p[0] = 'x'; // 编译器不能发现该错误，运行时错误
```

(2) 用运算符 sizeof 可以计算出数组的容量（字节数）。sizeof \$p 为指针得到的是一个指针变量的字节数，而不是 p 所指的内存容量。C++/C 语言没有办法知道指针所指的内存容量，除非在申请内存时记住它。注意当数组作为函数的参数进行传递时，该数组自动退化为同类型的指针。

```
char a[] = "hello world";
char *p = a;
cout<< sizeof(a) << endl; // 12 字节 cout<< sizeof $ << endl; // 4 字节
```

计算数组和指针的内存容量

```
void Func(char a[100])
{
    cout<< sizeof(a) << endl; // 4 字节而不是100 字节
}
```

711.分别写出 BOOL,int,float,指针类型的变量 a 与“零”的比较语句。

```
BOOL : if ( !a ) or if(a)
int : if ( a == 0)
float : const EXPRESSION EXP = 0.000001
if ( a < EXP && a >-EXP)
pointer : if ( a != NULL) or if(a == NULL)
```

712.如何判断一段程序是由 C 编译程序还是由 C++编译程序编译的？

```
#ifdef __cplusplus
cout<<"c++";
#else
cout<<"c";
#endif
```

713.用两个栈实现一个队列的功能？要求给出算法和思路！

设2个栈为 A,B, 一开始均为空.

入队:

将新元素 push 入栈 A;

出队:

(1)判断栈 B 是否为空；

(2)如果不为空，则将栈 A 中所有元素依次 pop 出并 push 到栈 B；

(3)将栈 B 的栈顶元素 pop 出；

这样实现的队列入队和出队的平摊复杂度都还是 $O(1)$, 比上面的几种方法要好

714.嵌入式系统中经常要用到无限循环，你怎么样用 C 编写死循环呢？

这个问题用几个解决方案。我首选的方案是：

```
while(1){
}
```

一些程序员更喜欢如下方案：

```
for(;;)
{
}
```

这个实现方式让我为难，因为这个语法没有确切表达到底怎么回事。如果一个应试者给出这个作为方案，我将用这个作为一个机会去探究他们这样做的 基本原理。如果他们的基本答案是：“我被教着这样做，但从没有想到过为什么。”这会给我 留下一个坏印象。第三个方案是用 goto

Loop:

...

goto Loop;

应试者如给出上面的方案，这说明或者他是一个汇编语言程序员（这也许是好事）或者他是一个想进入新领域的 BASIC/FORTRAN 程序员。

715.位操作（Bit manipulation）

嵌入式系统总是要用户对变量或寄存器进行位操作。给定一个整型变量 *a*，写两段代码，第一个设置 *a* 的 bit 3，第二个清除 *a* 的 bit 3。在以上两个操作中，要保持其它位不变。对这个问题有三种基本的反应

(1) 不知道如何下手。该被面者从没做过任何嵌入式系统的工作。

(2) 用 bit fields。Bit fields 是被扔到 C 语言死角的东西，它保证你的代码在不同编译器之间是不可移植的，同时也保证了你的代码是不可重用的。我最近不幸看到 Infineon 为其较复杂的通信芯片写的驱动程序，它用到了 bit fields 因此完全对我无用，因为我的编译器用其它的方式来实现 bit fields 的。从道德讲：永远不要让一个非嵌入式的家伙粘实际硬件的边。

(3) 用 #defines 和 bit masks 操作。这是一个有极高可移植性的方法，是应该被用到的方法。

最佳的解决方案如下：

```
#define BIT3 (0x1 << 3)
static int a;
void set_bit3(void)
{
    a |= BIT3;
}
void clear_bit3(void)
{
    a &= ~BIT3;
}
```

一些人喜欢为设置和清除值而定义一个掩码同时定义一些说明常数，这也是可以接受的。我希望看到几个要点：说明常数、|=和&=~操作。

716.访问固定的内存位置 (Accessing fixed memory locations)

嵌入式系统经常具有要求程序员去访问某特定的内存位置的特点。在某工程中，要求设置一绝对地址为 0x67a9 的整型变量的值为 0xaa66。编译器是一个纯粹的 ANSI 编译器。写代码去完成这一任务。这一问题测试你是否知道为了访问一绝对地址把一个整型数强制转换 (typecast) 为一指针是合法的。这一问题的实现方式随着个人风格不同而不同。典型的类似代码如下：

```
int *ptr;
ptr = (int *)0x67a9;
*ptr = 0xaa66;
A more obscure approach is:
一个较晦涩的方法是：
*(int * const)(0x67a9) = 0xaa55;
```

即使你的品味更接近第二种方案，但我建议你在面试时使用第一种方案。

717.中断 (Interrupts)

中断是嵌入式系统中重要的组成部分，这导致了很多编译开发商提供一种扩展—让标准 C 支持中断。具代表事实是，产生了一个新的关键字 **interrupt**。下面的代码就使用了 interrupt 关键字去定义了一个中断服务子程序 (ISR)，请评论一下这段代码的。


```
__interrupt double compute_area (double radius)
{
    double area = PI * radius * radius;
    printf("\nArea = %f", area);
    return area;
}
```

这个函数有太多的错误了，以至让人不知从何说起了：

- (1) ISR 不能返回一个值。如果你不懂这个，那么你不会被雇用的。
- (2) ISR 不能传递参数。如果你没有看到这一点，你被雇用的机会等同第一项。
- (3) 在许多的处理器/编译器中，浮点一般都是不可重入的。有些处理器/编译器需要让额处的寄存器入栈，有些处理器/编译器就是不允许在 ISR 中做浮点运算。此外，ISR 应该是短而有效率的，在 ISR 中做浮点运算是不明智的。
- (4) 与第三点一脉相承，printf()经常有重入和性能上的问题。如果你丢掉了第三和第四点，我不会太为难你的。不用说，如果你能得到后两点，那么你的被雇用前景越来越光明了。

718.动态内存分配 (Dynamic memory allocation)

尽管不像非嵌入式计算机那么常见，嵌入式系统还是有从堆 (heap) 中动态分配内存的过程的。那么嵌入式系统中，动态分配内存可能发生的问题是什么？这里，我期望应试者能提到内存碎片，碎片收集的问题，变量的持行时间等等。这个主题已经在 ESP 杂志中被广泛地讨论过了（主要是 P.J. Plauger, 他的解释远远超过我这里能提到的任何解释），所有回过头看一下这些杂志吧！让应试者进入一种虚假的安全感觉后，我拿出这么一个小节目：下面的代码片段的输出是什么，为什么？

```
char *ptr;
if ((ptr = (char *)malloc(0)) == NULL)
    puts("Got a null pointer");
else
    puts("Got a valid pointer");
```

这是一个有趣的问题。最近在我的一个同事不经意把0值传给了函数 malloc，得到了一个合法的指针之后，我才想到这个问题。这就是上面的代码，该代码的输出是"Got a valid pointer"。我用这个来开始讨论这样的一问题，看看被面试者是否想到库例程这样做是正确。得到正确的答案固然重要，但解决问题的方法和你做决定的基本原理更重要些。

719.Typedef

Typedef 在 C 语言中频繁用以声明一个已经存在的数据类型的同义字。也可以用预处理器做类似的事。例如，思考一下下面的例子：

```
#define dps struct s *
typedef struct s * tPS;
```

以上两种情况的意图都是要定义 dps 和 tPS 作为一个指向结构 s 指针。哪种方法更好呢？（如果有的话）为什么？这是一个非常微妙的问题，任何人答对这个问题（正当的原因）是应当被恭喜的。答案是：

typedef 更好。思考下面的例子：

```
dPS p1,p2;  
tPS p3,p4;  
第一个扩展为  
struct s * p1, p2;
```

上面的代码定义 p1 为一个指向结构的指针，p2 为一个实际的结构，这也许不是你想要的。第二个例子正确地定义了 p3 和 p4 两个指针。

720.用变量 a 给出下面的定义

- a) 一个整型数 (An integer)
- b) 一个指向整型数的指针 (A pointer to an integer)
- c) 一个指向指针的指针，它指向的指针是指向一个整型数 (A pointer to a pointer to an integer)
- d) 一个有10个整型数的数组 (An array of 10 integers)
- e) 一个有10个指针的数组，该指针是指向一个整型数的 (An array of 10 pointers to integers)
- f) 一个指向有10个整型数数组的指针 (A pointer to an array of 10 integers)
- g) 一个指向函数的指针，该函数有一个整型参数并返回一个整型数 (A pointer to a function that takes an integer as an argument and returns an integer)
- h) 一个有10个指针的数组，该指针指向一个函数，该函数有一个整型参数并返回一个整型数 (An array of ten pointers to functions that take an integer argument and return an integer)

答案是：

```
a) int a; // An integer  
b) int *a; // A pointer to an integer  
c) int **a; // A pointer to a pointer to an integer  
d) int a[10]; // An array of 10 integers  
e) int *a[10]; // An array of 10 pointers to integers  
f) int (*a)[10]; // A pointer to an array of 10 integers  
g) int (*a)(int); // A pointer to a function a that takes an integer argument and  
returns an integer  
h) int (*a[10])(int); // An array of 10 pointers to functions that take an  
integer argument and return  
an integer
```

721.写一个“标准”宏

交换两个参数值的宏定义为：

```
#define SWAP(a,b)\  
(a)=(a)+(b);\n(b)=(a)-(b);\n(a)=(a)-(b);
```

输入两个参数，输出较小的一个：

```
#define MIN(A,B) ((A) < (B)) ? (A) : (B)
```

表明1年中有多少秒（忽略闰年问题）：

```
#define SECONDS_PER_YEAR (60 * 60 * 24 * 365)UL  
#define DOUBLE(x) x+x 与 #define DOUBLE(x) ((x) + (x))  
i = 5DOUBLE(5); i 为30 i = 5DOUBLE(5); i 为50
```

```
#define NTBL
#define NTBL (sizeof(table)/sizeof(table[0]))
```

722.A.c 和 B.c 两个 c 文件中使用了两个相同名字的 static 变量,编译的时候会不会有问题? 这两个 static 变量会保存到哪里 (栈还是堆或者其他的)?

static 的全局变量，表明这个变量仅在本模块中有意义，不会影响其他模块。他们都放在数据区，但是编译器对他们的命名是不同的。如果要使变量在其他模块也有意义的话，需要使用 extern 关键字。

723.一个单向链表，不知道头节点,一个指针指向其中的一个节点，问如何删除这个指针指向的节点？

将这个指针指向的 next 节点值 copy 到本节点，将 next 指向 next->next,并随后删除原 next 指向的节点。

版权声明：本文为CSDN博主「CoutCodes」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/CoutCodes/article/details/109254740>

724.比较字符串，输出它们第一个不同字母的位置，大小写不敏感

```
int str_cmp_diff(const char *str1,const char *str2)
{
while(str1[i] != '\0' && str2[i] != '\0')
{
if((str1[i] | 0x20) != (str2[i] | 0x20))
return i + 1;
i++;
}
if(str1[i] != str2[i]) return i + 1;
return 0;
}
```

725.判断一个数是不是回文数，数字 1234321。

```
int isrevert(int data)
{
data-->temp

if(data == temp)
return 1;
return 0;
}
```

726.比较两字符串长短，并返回结果。

```
int str_cmp_len(const char *str1,const char *str2)
{
    len1 =
    len2 =

    return (len1 - len2);
}
```

727.给一个字符串，编程取其中一个特定的字符并输出。

```
int find_ch(const char *str,char ch)
{
    while(str[i] != '\0')
    {
        if(str[i] == ch) return i;
        i = i + 1;
    }
    return -1;
}

char *find_ch(const char *str,char ch)
{
}
```

728.是比较两个英文字符串的不相同的字符的位置（忽略字母大小写）

```
int find_diff(const char *str1,const char *str2,int array[])
{
    j = 0;
    while(str1[i] != '\0' && str2[i] != '\0')
    {
        if(str1[i] != str2[i])
        {
            array[j] = i;
            j++;
        }
        i = i + 1;
    }

    return j;
}
```

729.主函数调用一函数

如：检索出字符串中出现次数最多的那个字符，不考虑大小写，然后返回该字符。

```
char find_max(const char *str)
{
    max = 0;
    ch = str[0];
```

```
while(str[i] != '\0')
{
    if(isupper(str[i]))
    {
        i = i + 1;
        continue;
    }
    统计str[i]个数
        j = i;

    ct = 0;
    while(str[j] != '\0')
    if(str[i] == str[j]) {ct++;j++;}
        比较记录个数最多的字符
    if(max < ct) {max = ct;ch = str[i]};
}

    return 字符
}
```

730.输入一个整数n，计算不大于n的数中和7相关的数的个数，包括能被7整出的数和含有字符7的数。

```
int count_sever(int data)
{
    i = 7;
    while(i < n)
    {
        i 是否和7 相关

        i = i + 1;
    }
}
```

731.输入一个整数将每一位上的奇数放在一个新整数中，高位放在高位，地位在低位。

```
int fun(int data)
{
    temp = 0
    bit = 0;

    while(data > 0)
    {
        bit = data % 10;
        if(bit % 2 != 0)
            temp = temp * 10 + bit;
        data = data / 10
    }

    while(temp > 0)
```

```
{  
    bit = temp % 10;  
    tep = tem * 10 + bit;  
    temp = temp / 10  
}  
  
return tep;  
}
```

12345--->135

```
int fun(int data)  
{  
    temp = 0  
    bit = 0;  
    n = 1;  
    while(data > 0)  
    {  
        bit = data % 10;  
        if(bit % 2 != 0)  
        {  
            temp = temp + bit*n  
            n = n * 10;  
        }  
        data = data / 10  
    }  
  
}
```

732.输入一串数，将其最小的放在第一位，次小的放在最后一位，再小的放在第二位，再再小的放在倒数第二位，以此类推。

```
void sort(int array[],int count)  
{  
    循环count/2  
        每一次寻找两个数  
}
```

733.写一个函数，传入参数为应付钱数。返回值为买家最少付出的钱的张数`int get MoneyNum(int iInputMoney)`
例如：买家应付351元，最少张数为5.备注：可支付的钱币只有100、50、10、5、1不考虑2、20以及小数部分。

```
int get MoneyNum(int iInputMoney)  
{  
    int ct1 = 0,ct2 = 0,ct3 = 0,ct4 = 0,ct5 = 0;
```

```
int money = 0;

ct1 = iInputMoney / 100;
money = iInputMoney % 100;
ct2 = money / 50;
money = money % 50;
ct3 = money / 10;
money = money % 10;
ct4 = money / 5;

return ct1 + ct2 + ...

}
```

734.对姓氏进行排名

```
Char str[ ]="zhang wang li zhaohao"
Char str_new[ ]="li wang zhang zhao"
接口：

void sort (const char *str,int len,char *str_new)
{

char **strs = (char **)malloc(sizeof(char *) * len);
memset()

for(int i = 0;i < len;i++)
{
strs[i] = (char *)malloc(sizeof(char) * N)
memset()

}

while(str[i] != '\0')
{
while(str[i]是字符) str[i]-->strs[j][k]
while(str[i]不是字符)

}

排序

for(i = 0;i < len;i++)
{
strncpy
str_new[] = ' '
}

}
```

735.将一组整数中为奇数的数提取出来，高低位顺序不变。 如：8 3 7 9 5 2 1 4-----》 3 7 5 1


```
int find_data(int array[],int count,int dest[])
{

}
```

736.一组 $2n+1$ 个元素的正整数数组，按升序排序，然后将小于中间数值的成员替换为中间的值。

（貌似还有：“位置不变”，不过实在不理解其含义，看了例子就不用关心它的意思了），例如：

1,2,3,4,5，输出为：3,3,3,4,5，原型：int fun(int arry[],int n,char*

```
int fun(int array[],int n,char*output)
{
if(array == NULL || output == NULL) return -1;

return 0;
}
```

737.输入一个四位的十进制整数，编程实现将这四位整数转化为十六进制的字符串，并输出十六进制的字符串（注意负数的处理）

```
char *int_to_hex(int data,char *str)
{
if(data < 0)
data = ~(-1 * data) + 1

while(data > 0)
{
bit = data % 16;
if(bit < 10)
str[i] = bit + '0'
else
str[i] = bit - 10 + 'a'
data = data / 16;
}

str逆序

return str;
}
```

原文作者：[黄土堰](#)

原文链接：<https://www.cnblogs.com/huangtuyuan/p/10637326.html>

738.介绍一下STL，详细说明STL如何实现vector。

STL (标准模版库, Standard Template Library) 它由容器、算法、迭代器组成。

STL有以下的一些优点：

可以方便容易地实现搜索数据或对数据排序等一系列的算法；

调试程序时更加安全和方便；

即使是人们用STL在UNIX平台下写的代码你也可以很容易地理解（因为STL是跨平台的）。

vector实质上就是一个动态数组，会根据数据的增加,动态的增加数组空间。

739.如果用VC开发程序，常见这么几个错误，C2001,c2005,c2011,这些错误的原因是什么。

在学习VC++的过程中，遇到的LNK2001错误的错误消息主要为：

unresolved external symbol "symbol" (不确定的外部"符号")。

如果连接程序不能在所有的库和目标文件内找到所引用的函数、变量或标签，将产生此错误消息。

一般来说，发生错误的原因有两个：一是所引用的函数、变量不存在、拼写不正确或者使用错误；其次可能使用了不同版本的连接库。

编程中经常能遇到LNK2005错误——重复定义错误，其实LNK2005错误并不是一个很难解决的错误。

740.继承和委派有什么分别，在决定使用继承或者委派的时候需要考虑什么。

在OOD,OOP中，组合优于继承。

当然多态的基础是继承，没有继承多态无从谈起。

当对象的类型不影响类中函数的行为时，就要使用模板来生成这样一组类。

当对象的类型影响类中函数的行为时，就要使用继承来得到这样一组类。

741.指针和引用有什么分别；如果传引用比传指针安全，为什么？如果我使用常量指针难道不行吗？

(1) 引用在创建的同时必须初始化，即引用到一个有效的对象；而指针在定义的时候不必初始化，可以在定义后面的任何地方重新赋值。

(2) 不存在NULL引用，引用必须与合法的存储单元关联；而指针则可以是NULL。

(3) 引用一旦被初始化为指向一个对象，它就不能被改变为另一个对象的引用；而指针在任何时候都可以改变为指向另一个对象。给引用赋值并不是改变它和原始对象的绑定关系。

(4) 引用的创建和销毁并不会调用类的拷贝构造函数

(5) 语言层面，引用的用法和对象一样；在二进制层面，引用一般都是通过指针来实现的，只不过编译器帮我们完成了转换。

不存在空引用，并且引用一旦被初始化为指向一个对象，它就不能被改变为另一个对象的引用，显得很安全。

const 指针仍然存在空指针，并且有可能产生野指针。

总的来说：引用既具有指针的效率，又具有变量使用的方便性和直观性。

742.参数传递有几种方式；实现多态参数传递采用什么方式，如果没有使用某种方式原因是什么；

传值，传指针或者引用

743.结合一个项目说明你怎样应用设计模式的理念。

设计模式更多考虑是扩展和重用，而这两方面很多情况下，往往会被忽略。

不过，我不建议滥用设计模式，以为它有可能使得简单问题复杂化。

744.介绍一下你对设计模式的理解。（这个过程中有很多很细节的问题随机问的）

设计模式概念是由建筑设计师Christopher Alexander提出:"每一个模式描述了一个在我们周围不断重复发生的问题,以及该问题的解决方案的核心.这样,你就能一次又一次地使用该方案而不必做重复劳动."上述定义是对设计模式的广义定义.将其应用到面向对象软件的领域内,就形成了对设计模式的狭义定义.

可以简单的认为:设计模式就是解决某个特定的面向对象软件问题的特定方法,并且已经上升到理论程度.

框架与设计模式的区别:

- 1,设计模式和框架针对的问题域不同.设计模式针对面向对象的问题域;框架针对特定业务的问题域
- 2,设计模式比框架更为抽象.设计模式在碰到具体问题后,才能产生代码;框架已经可以用代码表示
- 3,设计模式是比框架更小的体系结构元素.框架中可以包括多个设计模式

设计模式就像武术中基本的招式.将这些招式合理地组合起来,就形成套路(框架),框架是一种半成品.

745.C++和C定义结构的分别是什么。

C language 的结构仅仅是数据的结合

C plus plus的struct 和 class 其实具备几乎一样的功能,只是默认访问属性不一样而已。

746.构造函数可否是虚函数,为什么?析构函数呢,可否是纯虚的呢?

构造函数不能为虚函数,要构造一个对象,必须清楚地知道要构造什么,否则无法构造一个对象。

析构函数可以为纯虚函数。

747.拷贝构造函数相关问题,深拷贝,浅拷贝,临时对象等。

深拷贝意味着拷贝了资源和指针,而浅拷贝只是拷贝了指针,没有拷贝资源

这样使得两个指针指向同一份资源,造成对同一份析构两次,程序崩溃。

临时对象的开销比局部对象小些。

748.结合1个你认为比较能体现OOP思想的项目,用UML来描述。

(最好这个项目继承,多态,虚函数都有体现)这个问题大概会占面试时间的一半,并且会问很多问题,一不小心可能会被问住)。

...

749.基类的有1个虚函数,子类还需要申明为virtual吗?为什么。

不申明没有关系的。

不过,我总是喜欢显式申明,使得代码更加清晰。

750.C也可以通过精心封装某些函数功能实现重用,那C++的类有什么优点吗,难道仅仅是为实现重用。

并不仅仅是这样的。

OOD，OOP从根本上改变了程序设计模式和设计思想，具备重大和深远的意义。

类的三大最基本的特征：封装，继承，多态。

751.C++特点是什么，如何实现多态？画出基类和子类在内存中的相互关系。

多态的基础是继承，需要虚函数的支持，简单的多态是很简单的。

子类继承父类大部分的资源，不能继承的有构造函数，析构函数，拷贝构造函数，operator=函数，友元函数等等

752.为什么要引入抽象基类和纯虚函数？

主要目的是为了实现一种接口的效果。

753.介绍一下模板和容器。如何实现？（也许会让你当场举例实现）

模板可以说比较古老了，但是当前的泛型编程实质上就是模板编程。

它体现了一种通用和泛化的思想。

STL有7种主要容器：vector,list,deque,map,multimap,set,multiset.

754.你如何理解MVC。简单举例来说明其应用。

MVC模式是observer 模式的一个特例,典型的有MFC里面的文档视图架构。

755.多重继承如何消除向上继承的二义性。

使用虚拟继承即可。

原文作者：[Iron](#)

原文链接：<https://www.cnblogs.com/hxf829/archive/2009/02/24/1659825.html>

756.什么是并行计算？

并行计算（Parallel Computing）是指同时使用多种计算资源解决计算问题的过程，是提高计算机系统计算速度和处理能力的一种有效手段。它的基本思想是用多个处理器来协同求解同一问题，即将被求解的问题分解成若干个部分，各部分均由一个独立的处理机来并行计算。并行计算系统既可以是专门设计的、含有多个处理器的超级计算机，也可以是以某种方式互连的若干台的独立计算机构成的集群。通过并行计算集群完成数据的处理，再将处理的结果返回给用户。

并行计算可分为时间上的并行和空间上的并行。

时间上的并行：是指流水线技术，比如说工厂生产食品的时候步骤分为：

1. 清洗：将食品冲洗干净。
2. 消毒：将食品进行消毒处理。
3. 切割：将食品切成小块。
4. 包装：将食品装入包装袋。

如果不采用流水线，一个食品完成上述四个步骤后，下一个食品才进行处理，耗时且影响效率。但是采用流水线技术，就可以同时处理四个食品。这就是并行算法中的时间并行，在同一时间启动两个或两个以上的操作，大大提高计算性能。

空间上的并行：是指多个处理机并发的执行计算，即通过网络将两个以上的处理机连接起来，达到同时计算同一个任务的不同部分，或者单个处理机无法解决的大型问题。

757.与10.110.12.29mask 255.255.255.224属于同一网段的主机IP地址有哪些？

根据你提供的ip地址和掩码计算出来的ip地址段为10.110.12.0/27.

也就是从10.110.12.0到10.110.12.31.

758.讲一讲Makefile的内容.

target - 目标文件, 可以是 Object File, 也可以是可执行文件

prerequisites - 生成 target 所需要的文件或者目标

command - make需要执行的命令 (任意的shell命令), Makefile中的命令必须以 [tab] 开头

显示规则 :: 说明如何生成一个或多个目标文件(包括 生成的文件, 文件的依赖文件, 生成的命令)

隐晦规则 :: make的自动推导功能所执行的规则

变量定义 :: Makefile中定义的变量

文件指示 :: Makefile中引用其他Makefile; 指定Makefile中有效部分; 定义一个多行命令

注释 :: Makefile只有行注释 "#", 如果要使用或者输出"#"字符, 需要进行转义, "\\"

最后, 还值得一提的是, 在Makefile中的命令, 必须要以[Tab]键开始。

759.讲一讲C++的内联函数

内联函数inline：引入内联函数的目的是为了解决程序中函数调用的效率问题，这么说吧，程序在编译器编译的时候，编译器将程序中出现的内联函数的调用表达式用内联函数的函数体进行替换，而对于其他的函数，都是在运行时候才被替代。这其实就是个空间代价换时间的节省。所以内联函数一般都是1-5行的小函数。在使用内联函数时要留神：

- 1.在内联函数内不允许使用循环语句和开关语句；
- 2.内联函数的定义必须出现在内联函数第一次调用之前；
- 3.类结构中所在的类说明内部定义的函数是内联函数。

760.vector, deque, list, set, map底层数据结构 vector (向量) ——STL中标准而安全的数组。只能在vector 的“前面”增加数据。

deque (双端队列double-ended queue) ——在功能上和vector相似，但是可以在前后 两端向其中添加数据。

list (列表) ——游标一次只可以移动一步。如果你对链表已经很熟悉，那么STL中的list 则是一个双向链表 (每个节点有指向前驱和指向后继的两个指针) 。

set (集合) ——包含了经过排序了的数据，这些数据的值(value)必须是唯一的。

map（映射）——经过排序了的二元组的集合，map中的每个元素都是由两个值组成，其中的key（键值，一个map中的键值必须是唯一的）是在排序或搜索时使用，它的值可以在容器中重新获取；而另一个值是该元素关联的数值。比如，除了可以 `ar[43] = "overripe"` 这样找到一个数据，map还可以通过 `ar["banana"] = "overripe"` 这样的方法找到一个数据。如果你想获得其中的元素信息，通过输入元素的全名就可以轻松实现。

761.宏定义的优缺点

优点：

提高了程序的可读性，同时也方便进行修改；

提高程序的运行效率：使用带参的宏定义既可完成函数调用的功能，又能避免函数的出栈与入栈操作，减少系统开销，提高运行效率；

3.宏是由预处理器处理的，通过字符串操作可以完成很多编译器无法实现的功能。比如##连接符。

缺点：

由于是直接嵌入的，所以代码可能相对多一点；

嵌套定义过多可能会影响程序的可读性，而且很容易出错；

对带参的宏而言，由于是直接替换，并不会检查参数是否合法，存在安全隐患。

762.bfs和dfs如何遍历

1.深度优先搜索（DFS）

原文里的深度优先搜索代码是有问题的，那是中序遍历的推广，而深度优先搜索是先序遍历的推广，我这里把两种代码都给出来，深度优先搜索的非递归实现使用了一个栈。

深度优先遍历图的方法是，从图中某顶点v出发：

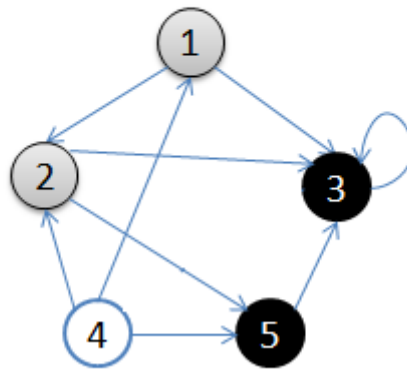
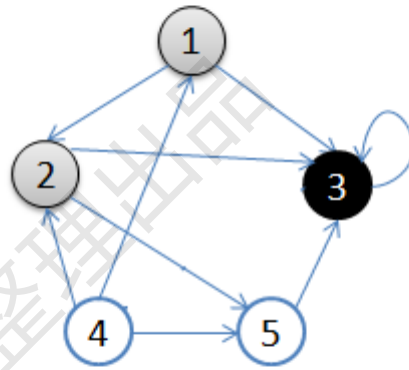
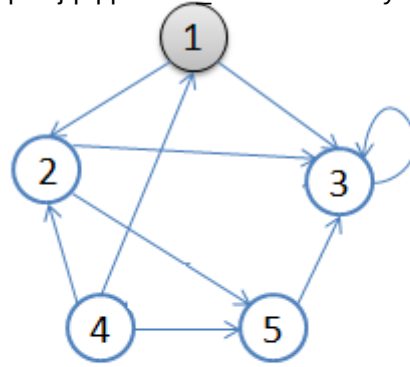
a.访问顶点v；

b.依次从v的未被访问的邻接点出发，对图进行深度优先遍历；直至图中和v有路径相通的顶点都被访问；

c.若此时图中尚有顶点未被访问，则从一个未被访问的顶点出发，重新进行深度优先遍历，直到图中所有顶点均被访问过为止。

用一副图来表达这个流程如下：

1).从v = 顶点1开始出发，先访问顶点1



2).按深度优先搜索递归访问v的某个未被访问的邻接点2，顶点2结束后，应该访问3或5中的某一个，这里为顶点3，此时顶点3不再有出度，因此回溯到顶点2，再访问顶点2的另一个邻接点5，由于顶点5的唯一一条边的弧头为3，已经访问了，所以此时继续回溯到顶点1，找顶点1的其他邻接点。

上图可以用邻接矩阵来表示为：

```
int maze[][] = {  
{ 0, 1, 1, 0, 0 },  
{ 0, 0, 1, 0, 1 },  
{ 0, 0, 1, 0, 0 },  
{ 1, 1, 0, 0, 1 },  
{ 0, 0, 1, 0, 0 }  
};
```

具体的代码如下：

```
import java.util.LinkedList;  
import classEnhance.EnhanceModual;
```



```
public class DepthFirst extends EnhanceModual {

    @Override
    public void internalEntrance() {
        // TODO Auto-generated method stub
        int maze[][] = {
            { 0, 1, 1, 0, 0 },
            { 0, 0, 1, 0, 1 },
            { 0, 0, 1, 0, 0 },
            { 1, 1, 0, 0, 1 },
            { 0, 0, 1, 0, 0 }
        };
        dfs(maze, 1);
    }

    public void dfs(int[][] adjacentArr, int start) {
        int nodeNum = adjacentArr.length;
        if (start <= 0 || start > nodeNum || (nodeNum == 1 && start != 1)) {
            System.out.println("Wrong input !");
            return;
        } else if (nodeNum == 1 && start == 1) {
            System.out.println(adjacentArr[0][0]);
            return;
        }

        int[] visited = new int[nodeNum + 1]; // 0表示结点尚未入栈，也未访问
        LinkedList<Integer> stack = new LinkedList<Integer>();
        stack.push(start);
        visited[start] = 1; // 1表示入栈

        while (!stack.isEmpty()) {
            int nodeIndex = stack.peek();
            boolean flag = false;
            if (visited[nodeIndex] != 2) {
                System.out.println(nodeIndex);
                visited[nodeIndex] = 2; // 2表示结点被访问
            }

            // 沿某一条路径走到无邻接点的顶点
            for (int i = 0; i < nodeNum; i++) {
                if (adjacentArr[nodeIndex - 1][i] == 1 &&
                    visited[i + 1] == 0) {
                    flag = true;
                    stack.push(i + 1);
                    visited[i + 1] = 1;
                    break; // 这里的break不能掉!!!
                }
            }

            // 回溯
            if (!flag) {
                int visitedNodeIndex = stack.pop();
            }
        }
    }
}
```

```
}  
}  
}
```

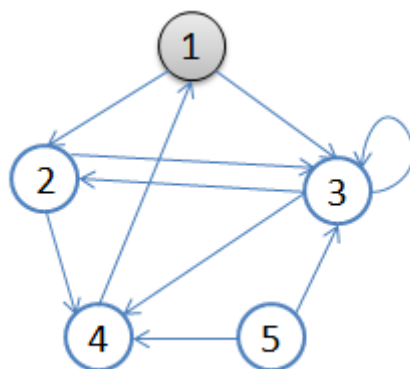
广度优先搜索 (BFS)

广度优先搜索是按层来处理顶点，距离开始点最近的那些顶点首先被访问，而最远的那些顶点则最后被访问，这个和树的层序变量很像，BFS的代码使用了一个队列。搜索步骤：

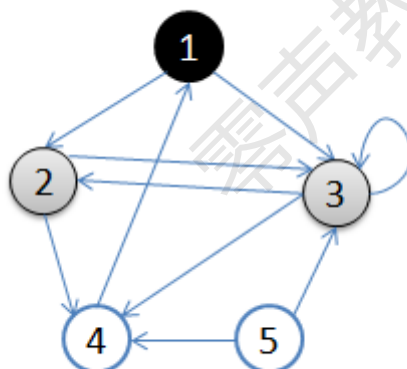
- 首先选择一个顶点作为起始顶点，并将其染成灰色，其余顶点为白色。
- 将起始顶点放入队列中。
- 从队列首部选出一个顶点，并找出所有与之邻接的顶点，将找到的邻接顶点放入队列尾部，将已访问过顶点涂成黑色，没访问过的顶点是白色。如果顶点的颜色是灰色，表示已经发现并且放入了队列，如果顶点的颜色是白色，表示还没有发现
- 按照同样的方法处理队列中的下一个顶点。

基本就是出队的顶点变成黑色，在队列里的是灰色，还没入队的是白色。

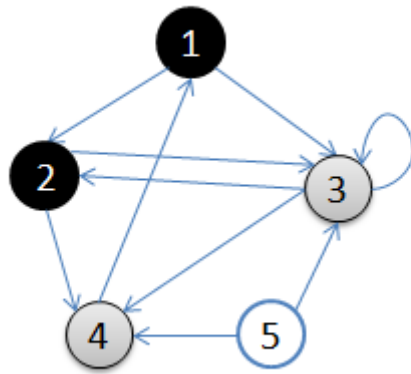
用一副图来表达这个流程如下：



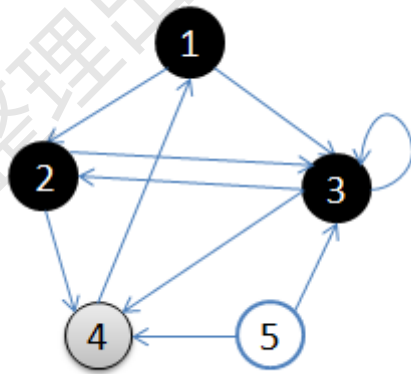
1.初始状态，从顶点1开始，队列={1}



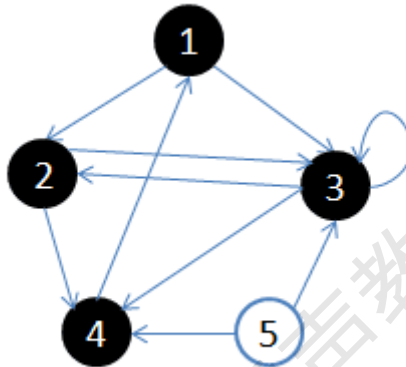
2.访问1的邻接顶点，1出队变黑，2,3入队，队列={2,3}



3.访问2的邻接顶点，2出队，4入队，队列={3,4}



4.访问3的邻接顶点，3出队，队列={4}



5.访问4的邻接顶点，4出队，队列={空}

分析：

从顶点1开始进行广度优先搜索：

初始状态，从顶点1开始，队列={1}

访问1的邻接顶点，1出队变黑，2,3入队，队列={2,3,}

访问2的邻接顶点，2出队，4入队，队列={3,4}

访问3的邻接顶点，3出队，队列={4}

访问4的邻接顶点，4出队，队列={空}

顶点5对于1来说不可达。

上面图可以用如下邻接矩阵来表示：

```
int maze[][] = {
{ 0, 1, 1, 0, 0 },
{ 0, 0, 1, 1, 0 },
{ 0, 1, 1, 1, 0 },
{ 1, 0, 0, 0, 0 },
{ 0, 0, 1, 1, 0 }
};
```

具体的代码如下，这段代码有两个功能，bfs（）函数求出从某顶点出发的搜索结果，minPath（）函数求从某一顶点出发到另一顶点的最短距离：

```
import java.util.LinkedList;
import classEnhance.EnhanceModule;

public class BreadthFirst extends EnhanceModule {

    @Override
    public void internalEntrance() {
        // TODO Auto-generated method stub
        int maze[][] = {
            { 0, 1, 1, 0, 0 },
            { 0, 0, 1, 1, 0 },
            { 0, 1, 1, 1, 0 },
            { 1, 0, 0, 0, 0 },
            { 0, 0, 1, 1, 0 }
        };

        bfs(maze, 5); //从顶点5开始搜索图

        int start = 5;
        int[] result = minPath(maze, start);
        for(int i = 1; i < result.length; i++){
            if(result[i] != 5 ){
                System.out.println("从顶点" + start + "到顶点" +
                    i + "的最短距离为: " + result[i]);
            }else{
                System.out.println("从顶点" + start + "到顶点" +
                    i + "不可达");
            }
        }
    }

    public void bfs(int[][] adjacentArr, int start) {
        int nodeNum = adjacentArr.length;
        if (start <= 0 || start > nodeNum || (nodeNum == 1 && start != 1)) {
            System.out.println("Wrong input !");
            return;
        } else if (nodeNum == 1 && start == 1) {
            System.out.println(adjacentArr[0][0]);
            return;
        }
    }
}
```

```
//0表示顶点尚未入队，也未访问，注意这里位置0生出来了
int[] visited = new int[nodeNum + 1];
LinkedList<Integer> queue = new LinkedList<Integer>();
queue.offer(start);
visited[start] = 1;//1表示入队

while (!queue.isEmpty()) {
    int nodeIndex = queue.poll();
    System.out.println(nodeIndex);
    visited[nodeIndex] = 2;//2表示顶点被访问

    for (int i = 0; i < nodeNum; i++) {
        if (adjacentArr[nodeIndex - 1][i] == 1 &&
visited[i + 1] == 0) {
            queue.offer(i + 1);
            visited[i + 1] = 1;
        }
    }
}

/*
 * 从start顶点出发，到图里各个顶点的最短路径
 */
public int[] minPath(int[][] adjacentArr, int start) {

    int nodeNum = adjacentArr.length;

    LinkedList<Integer> queue = new LinkedList<Integer>();
    queue.offer(start);
    int path = 0;
    int[] nodePath = new int[nodeNum + 1];
    for (int i = 0; i < nodePath.length; i++) {
        nodePath[i] = nodeNum;
    }
    nodePath[start] = 0;

    int incount = 1;
    int outcount = 0;
    int tempcount = 0;

    while (path < nodeNum) {
        path++;
        while (incount > outcount) {
            int nodeIndex = queue.poll();
            outcount++;

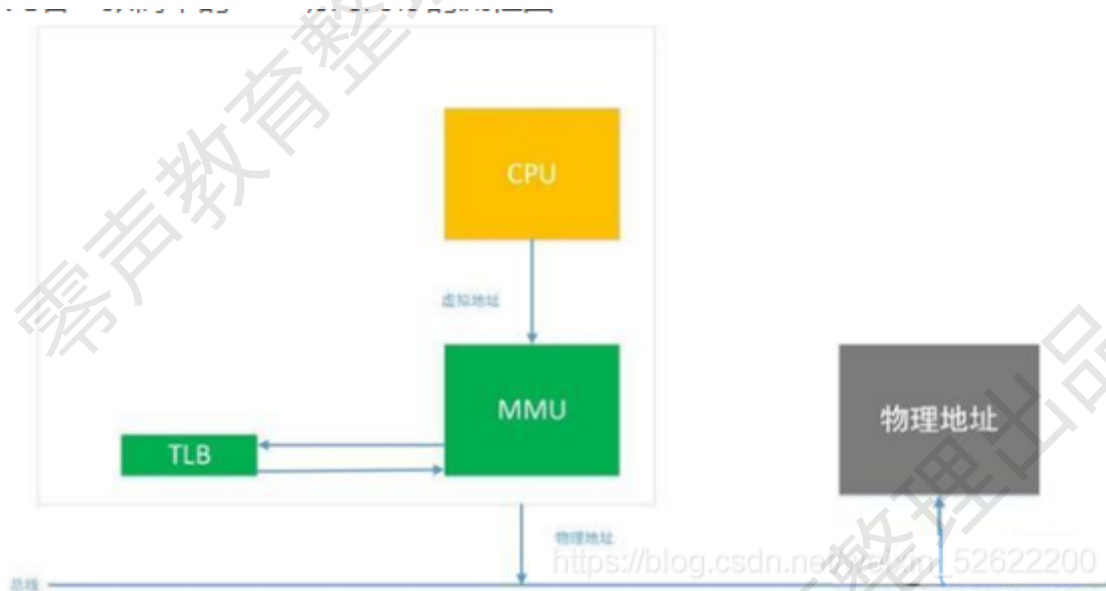
            for (int i = 0; i < nodeNum; i++) {
                if (adjacentArr[nodeIndex - 1][i] == 1 &&
nodePath[i + 1] == nodeNum) {
                    queue.offer(i + 1);
                    tempcount++;
                    nodePath[i + 1] = path;
                }
            }
        }
    }
}
```

```
}  
}  
  
incount = tempcount;  
tempcount = 0;  
outcount = 0;  
}  
  
return nodePath;  
}  
}
```

763.CPU如果访问内存?

通过内存管理单元(MMU)

先看一张简单的CPU访问内存的流程图：



TLB：转换lookaside 缓存，有了它可以使虚拟地址到物理地址转换速度大增。

从上图中可以清楚的知道了,CPU,DDR,MMU它们三者之间的关系。CPU在MMU开启的情况下，访问的都是虚拟地址。

首先通过MMU将虚拟地址转换为物理地址，

然后再通过总线上去访问内存(我们都知道内存是挂在总线上的)。

那MMU是怎么将虚拟地址转换为物理地址呢?当然是通过页表的方式。MMU从页表中查出虚拟地址对应的物理地址是什么，然后就去访问物理内存了。

764.找出在A数组中，B数组中没有的数字，在B数组中，A数组中没有的数字

```
public static void find(int arr[],int[]b){  
    HashMap<Integer,Integer> map = new HashMap<Integer, Integer>();  
    for (int i = 0; i < arr.length; i++) {  
        map.put(arr[i],0);  
    }  
}
```

```
}  
for (int i=0;i<b.length;i++) {  
    if(map.containsKey(b[i])){  
        map.put(b[i],1);  
    }else{  
        System.out.println("在B数组中A不存在的数字");  
        System.out.println(b[i]);  
    }  
}  
for (int i = 0; i <arr.length ; i++) {  
    if(map.get(arr[i])==0){  
        System.out.println("在A数组中存在的，在B数组不存在的数字");  
        System.out.println(arr[i]);  
    }  
}  
}
```

版权声明：本文为CSDN博主「攻城狮百里」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_52622200/article/details/110563825

765.在C++ 程序中调用被C 编译器编译后的函数，为什么要加extern "C" ？

答：首先，extern是C/C++语言中表明函数和全局变量作用范围的关键字，该关键字告诉编译器，其声明的函数和变量可以在本模块或其它模块中使用。

通常，在模块的头文件中对本模块提供给其它模块引用的函数和全局变量以关键字extern声明。extern "C"是连接申明(linkage

declaration),被extern

"C"修饰的变量和函数是按照C语言方式编译和连接的。作为一种面向对象的语言，C++支持函数重载，而过程式语言C则不支持。函数被C++编译后在符号库中的名字与C语言的不同。例如，假设某个函数的原型为：void

foo(int x, int y

);该函数被C编译器编译后在符号库中的名字为foo，而C++编译器则会产生像foo_int_int之类的名字。这样的名字包含了函数名、函数参数数量及类型信息，C++就是靠这种机制来实现函数重载的。

所以，可以用一句话概括extern "C"这个声明的真实目的:解决名字匹配问题，实现C++与C的混合编程。

766.头文件中的ifndef/define/endif有什么作用？

答：这是C++预编译头文件保护符，保证即使文件被多次包含，头文件也只定义一次。

767. # include<file.h> 与 # include "file.h"的区别？

答：前者是从标准库路径寻找和引用file.h，而后者是从当前工作路径搜寻并引用file.h。

768.评价一下C/C++各自的特点

答：C语言是一种结构化语言，面向过程；基于算法和数据结构，所考虑的是如何通过一个过程或者函数从输入得到输出；

C++是面向对象，基于类、对象和继承，所考虑的是如何构造一个对象模型，让这个模型能够契合与之对应的问题，通过获取对象的状态信息得到输出或实现过程控制。

769.const 有什么用途？

答：在C/C++中，（1）可以定义const常量，（2）修饰函数的返回值和形参；

在C++中，还可以修饰函数的定义体，定义类的const成员函数。被const修饰的东西受到强制保护，可以预防意外的变动，提高了程序的健壮性。

770.const和#define有什么区别？

答：（1）const和#define都可以定义常量，但是const用途更广。

（2）const 常量有数据类型，而宏常量没有数据类型。编译器可以对前者进行类型安全检查。而对后者只进行字符替换，没有类型安全检查，并且在字符替换可能会产生意料不到的错误。

（3）有些集成化的调试工具可以对const 常量进行调试，但是不能对宏常量进行调试。

771.关于sizeof小结的。

答：sizeof计算的是在栈中分配的内存大小。

（1）sizeof不计算static变量占得内存；

（2）32位系统的指针的大小是4个字节，64位系统的指针是8字节，而不用管指针类型；

（3）char型占1个字节，int占4个字节，short int占2个字节

long int占4个字节，float占4字节，double占8字节，string占4字节

一个空类占1个字节，单一继承的空类占1个字节，虚继承涉及到虚指针所以占4个字节

（4）数组的长度：

若指定了数组长度，则不看元素个数，总字节数=数组长度*sizeof（元素类型）

若没有指定长度，则按实际元素个数类确定

Ps：若是字符数组，则应考虑末尾的空字符。

（5）结构体对象的长度

在默认情况下，为方便对结构体内元素的访问和管理，当结构体内元素长度小于处理器位数的时候，便以结构体内最长的数据元素的长度为对齐单位，即为其整数倍。若结构体内元素长度大于处理器位数则以处理器位数为单位对齐。

（6）unsigned影响的只是最高位的意义，数据长度不会改变，所以sizeof（unsigned int）=4

（7）自定义类型的sizeof取值等于它的类型原型取sizeof

（8）对函数使用sizeof，在编译阶段会被函数的返回值的类型代替

（9）sizeof后如果是类型名则必须加括号，如果是变量名可以不加括号，这是因为sizeof是运算符

（10）当使用结构类型或者变量时，sizeof返回实际的大小。当使用静态数组时返回数组的全部大小，sizeof不能返回动态数组或者外部数组的尺寸

772.sizeof与strlen的区别？

答：（1）sizeof的返回值类型为size_t（unsigned int）；

（2）sizeof是运算符，而strlen是函数；

（3）sizeof可以用类型做参数，其参数可以是任意类型的或者是变量、函数，而strlen只能用char*做参数，且必须是以'\0'结尾；

（4）数组作sizeof的参数时不会退化为指针，而传递给strlen是就退化为指针；

（5）sizeof是编译时的常量，而strlen要到运行时才会计算出来，且是字符串中字符的个数而不是内存大小；

773.指针和引用的区别？

答：指针和引用都提供了间接操作对象的功能。

（1）指针定义时可以不初始化，而引用在定义时就要初始化，和一个对象绑定，而且一经绑定，只要引用存在，就会一直保持和该对象的绑定；

（2）赋值行为的差异：指针赋值是将指针重新指向另外一个对象，而引用赋值则是修改对象本身；

（3）指针之间存在类型转换，而引用分const引用和非const应用，非const引用只能和同类型的对象绑定，const引用可以绑定到不同但相关类型的对象或者右值

774.数组和指针的区别？

答：（1）数组要么在全局数据区被创建，要么在栈上被创建；指针可以随时指向任意类型的内存块；

（2）修改内容上的差别：

```
char a[] = "hello";
```

```
a[0] = 'X';
```

```
char *p = "world"; // 注意p 指向常量字符串
```

```
p[0] = 'X'; // 编译器不能发现该错误，运行时错误
```

（3）用运算符sizeof 可以计算出数组的容量（字节数）。sizeof(p),p 为指针得到的是一个指针变量的字节数，而不是p

所指的内存容量。C++/C

语言没有办法知道指针所指的内存容量，除非在申请内存时记住它。注意当数组作为函数的参数进行传递时，该数组自动退化为同类型的指针。

775.空指针和悬垂指针的区别？

答：空指针是指被赋值为NULL的指针；delete指向动态分配对象的指针将会产生悬垂指针。

（1）空指针可以被多次delete，而悬垂指针再次删除时程序会变得非常不稳定；

（2）使用空指针和悬垂指针都是非法的，而且有可能造成程序崩溃，如果指针是空指针，尽管同样是崩溃，但和悬垂指针相比是一种可预料的崩溃。

776.C++中有malloc/free，为什么还有new/delete？

答：malloc/free是C/C++标准库函数，new/delete是C++运算符。他们都可以用于动态申请和释放内存。

对于内置类型数据而言，二者没有多大区别。malloc申请内存的时候要制定分配内存的字节数，而且不会做初始化；new申请的时候有默认的初始化，同时可以指定初始化；

对于类类型的对象而言，用malloc/free无法满足要求的。对象在创建的时候要自动执行构造函数，消亡之前要调用析构函数。由于malloc/free是库函数而不是运算符，不在编译器控制之内，不能把执行构造函数和析构函数的任务强加给它，因此，C++还需要new/delete。

777.什么是智能指针？

答：当类中有指针成员时，一般有两种方式来管理指针成员：一是采用值型的方式管理，每个类对象都保留一份指针指向的对象的拷贝；另一种更优雅的方式是使用智能指针，从而实现指针指向的对象的共享。

智能指针的一种通用实现技术是使用引用计数。智能指针类将一个计数器与类指向的对象相关联，引用计数跟踪该类有多少个对象共享同一指针。

每次创建类的新对象时，初始化指针并将引用计数置为1；当对象作为另一对象的副本而创建时，拷贝构造函数拷贝指针并增加与之相应的引用计数；对一个对象进行赋值时，赋值操作符减少左操作数所指对象的引用计数（如果引用计数为减至0，则删除对象），并增加右操作数所指对象的引用计数；调用析构函数时，构造函数减少引用计数（如果引用计数减至0，则删除基础对象）。

778.面向对象技术的基本概念是什么，三个基本特征是什么？

答：基本概念：类、对象、继承；基本特征：封装、继承、多态。

封装：将低层次的元素组合起来形成新的、更高实体的技术；

继承：广义的继承有三种实现形式：实现继承、可视继承、接口继承。

多态：允许将子类类型的指针赋值给父类类型的指针

779.C++空类默认有哪些成员函数？

答：默认构造函数、析构函数、复制构造函数、赋值函数

780.哪一种成员变量可以在一个类的实例之间共享？

答：static静态成员变量

781.继承层次中，为什么基类析构函数是虚函数？

答：编译器总是根据类型来调用类成员函数。但是一个派生类的指针可以安全地转化为一个基类的指针。这样删除一个基类的指针的时候，C++不管这个指针指向一个基类对象还是一个派生类的对象，调用的都是基类的析构函数而不是派生类的。如果你依赖于派生类的析构函数的代码来释放资源，而没有重载析构函数，那么会有资源泄漏。

782.为什么构造函数不能为虚函数？

答：虚函数采用一种虚调用的方法。需调用是一种可以在只有部分信息的情况下工作的机制。如果创建一个对象，则需要知道对象的准确类型，因此构造函数不能为虚函数。

783.如果虚函数是有效的，那为什么不把所有函数设为虚函数？

答：不行。首先，虚函数是有代价的，由于每个虚函数的对象都要维护一个虚函数表，因此在使用虚函数的时候都会产生一定的系统开销，这是没有必要的。

784.什么是多态？多态有什么作用？

答：多态就是将基类类型的指针或者引用指向派生类型的对象。多态通过虚函数机制实现。

多态的作用是接口重用。

785.重载和覆盖有什么区别？

答：虚函数是基类希望派生类重新定义的函数，派生类重新定义基类虚函数的做法叫做覆盖；

重载就在允许在相同作用域中存在多个同名的函数，这些函数的参数表不同。重载的概念不属于面向对象编程，编译器根据函数不同的形参表对同名函数的名称做修饰，然后这些同名函数就成了不同的函数。

重载的确定是在编译时确定，是静态的；虚函数则是在运行时动态确定。

786.公有继承、受保护继承、私有继承

答：（1）公有继承时，派生类对象可以访问基类中的公有成员，派生类的成员函数可以访问基类中的公有和受保护成员；

（2）私有继承时，基类的成员只能被直接派生类的成员访问，无法再往下继承；

（3）受保护继承时，基类的成员也只被直接派生类的成员访问，无法再往下继承。

24.公有继承时基类受保护的成员，可以通过派生类对象访问但不能修改。

787.有哪几种情况只能用构造函数初始化列表而不能用赋值初始化？

答：const成员，引用成员

788.什么是虚指针？

答：虚指针或虚函数指针是虚函数的实现细节。带有虚函数的每一个对象都有一个虚指针指向该类的虚函数表。

789.C++如何阻止一个类被实例化？一般在什么时候将构造函数声明为private？

答：（1）将类定义为抽象基类或者将构造函数声明为private；

（2）不允许类外部创建类对象，只能在类内部创建对象

790.main函数执行之前会执行什么？执行之后还能执行代码吗？

答：（1）全局对象的构造函数会在main函数之前执行；

(2) 可以, 可以用 `_onexit` 注册一个函数, 它会在 `main` 之后执行;

如果你需要加入一段在 `main` 退出后执行的代码, 可以使用 `atexit()` 函数, 注册一个函数。

语法:

```
#include <stdlib.h>

#include <stdio.h>

int atexit(void (*function))(void));

void fn1( void ), fn2( void ), fn3( void );

int main( void )
{
    atexit(fn1);

    atexit( fn2 );

    printf( "This is executed first.\n" );
}

void fn1()
{
    printf( " This is\n" );
}

void fn2()
{
    printf( " executed next." );
}
```

结果:

This is executed first.

This is executed next.

791.请描述进程和线程的区别？

答: (1) 进程是程序的一次执行, 线程是进程中的执行单元;

(2) 进程间是独立的, 这表现在内存空间、上下文环境上, 线程运行在进程中;

(3) 一般来讲, 进程无法突破进程边界存取其他进程内的存储空间; 而同一进程所产生的线程共享内存空间;

(4) 同一进程中的两段代码不能同时执行，除非引入多线程。

792.进程间如何通信？

答：信号、信号量、消息队列、共享内存

793.在网络编程中涉及并发服务器，使用多进程与多线程的区别？

答：(1) 线程执行开销小，但不利于资源管理和保护；进程则相反，进程可跨越机器迁移。

(2) 多进程时每个进程都有自己的内存空间，而多线程间共享内存空间；

(3) 线程产生的速度快，线程间通信快、切换快；

(4) 线程的资源利用率比较好；

(5) 线程使用公共变量或者资源时需要同步机制。

794.TCP和UDP有什么区别。

答：

TCP——传输控制协议,提供的是面向连接、可靠的字节流服务。

当客户和服务器彼此交换数据前，必须先在双方之间建立一个TCP连接，之后才能传输数据。TCP提供超时重发，丢弃重复数据，检验数据，流量控制等功能，保证数据能从一端传到另一端。

UDP——用户数据报协议，是一个简单的面向数据报的传输层协议。UDP不提供可靠性，它只是把应用程序传给IP层的数据报发送出去，但是并不能保证它们能到达目的地。由于UDP在传输数据报前不用在客户和服务器之间建立一个连接，且没有超时重发等机制，故而传输速度很快。

TCP协议和UDP协议的一些特性区别如下：

1.TCP协议在传送数据段的时候要给段标号；UDP 协议不需要。

2.TCP协议可靠；UDP协议不可靠。

3.TCP协议是面向连接；UDP协议采用无连接。

4.TCP协议负载较高,采用虚电路；UDP协议低负载。

5.TCP协议的发送方要确认接受方是否收到数据段(3次握手协议)。

6.TCP协议采用窗口技术和流控制。

795.调用函数时要进行参数压栈，一般情况下顺序是从最右边参数往左压栈。

796.经常要操作的内存分为那几个类别？

答：(1) 栈区：由编译器自动分配和释放，存放函数的参数值、局部变量的值等；

(2) 堆：一般由程序员分配和释放，存放动态分配的变量；

(3) 全局区（静态区）：全局变量和静态变量存放在这一块，初始化的和未初始化的分开放；

(4) 文字常量区：常量字符串就放在这里，程序结束自动释放；

(5) 程序代码区：参访函数体的二进制代码。

797.请讲述堆和栈的区别。

- 答：（1）申请方式不同。栈上有系统自动分配和释放；堆上有程序员自己申请并指明大小；
- （2）栈是向低地址扩展的数据结构，大小很有限；堆是向高地址扩展，是不连续的内存区域，空间相对大且灵活；
- （3）栈由系统分配和释放速度快；堆由程序员控制，一般较慢，且容易产生碎片；

798.全局变量放在数据段，内部变量static int count；

放在数据段，内部变量char *p="AAA"，p的位置在堆栈上，指向的空间的位置数据段，内部变量char *p=new char；p的位置堆，指向的空间的位置数据段

799.字符数组与字符串的比较：最明显的区别是字符串会在末尾自动添加空字符。

800.函数指针相关概念（C++学习笔记）

801.类使用static成员的优点，如何访问？

答：优点：

- （1）static 成员的名字是在类的作用域中，因此可以避免与其他类的成员或全局对象名字冲突；
- （2）可以实施封装。static 成员可以是私有成员，而全局对象不可以；
- （3）static 成员是与特定类关联的，可清晰地显示程序员的意图。

static 数据成员必须在类定义体的外部定义(正好一次)，static 关键字只能用于类定义体内部的声明中，定义不能标示为static。

不像普通数据成员，static成员不是通过类构造函数进行初始化，也不能在类的声明中初始化，而是应该在定义时进行初始化.保证对象正好定义一次的最好办法，就是将static

数据成员的定义放在包含类非内联成员函数定义的文件中。

静态数据成员初始化的格式为：

<数据类型> <类名>::<静态数据成员名>=<值>

类的静态数据成员有两种访问形式：

<类对象名>.<静态数据成员名> 或 <类类型名>::<静态数据成员名>

802.static数据成员和static成员函数

答：（1）static数据成员：

static数据成员独立于该类的任意对象而存在；每个static数据成员是与类关联的对象，并不与该类的对象相关联。Static数据成员（const

static数据成员除外）必须在类定义体的外部定义。不像普通数据成员，static成员不是通过类的构造函数进行初始化，而是应该在定义时进行初始化。

- （2）static成员函数：

Static成员函数没有this形参，它可以直接访问所属类的static成员，不能直接使用非static成员。因为static成员不是任何对象的组成部分，所以static成员不能被声明为const。同时，static成员函数也不能被声明为虚函数。

803.static成员变量定义放在cpp文件中，不能放在初始化列表中。Const static成员可就地初始化。

804.如何引用一个已经定义过的全局变量？

答：可以用引用头文件的方式，也可以用extern关键字，如果用引用头文件方式来引用某个在头文件中声明的全局变量，假定你将那个变量写错了，那么在编译期间会报错，如果你用extern方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在连接期间报错。

805.static关键字的作用。

答：static总是使得变量或对象的存储形式变成静态存储，连接方式变成内部连接，对于局部变量（已经是内部连接了），它仅改变其存储方式；对于全局变量（已经是静态存储了），它仅改变其连接类型。

806.奈奎斯特定理

807.香农定理

808.多态类中的虚函数表是 Compile-Time，还是 Run-Time时建立的？

答案：虚拟函数表是在编译期就建立了,各个虚拟函数这时被组织成了一个虚拟函数的入口地址的数组。而对象的隐藏成员--虚拟函数表指针是在运行期--也就是构造函数被调用时进行初始化的，这是实现多态的关键。

809.一个父类写了一个 virtual 函数，如果子类覆盖它的函数不加 virtual ,也能实现多态？

在子类的空间里，有没有父类的这个函数，或者父类的私有变量? (华为笔试题)

答案：只要基类在定义成员函数时已经声明了 virtual关键字，在派生类实现的时候覆盖该函数时，virtual关键字可加可不加，不影响多态的实现。子类的空间里有父类的所有变量(static除外)。

810.完成字符串拷贝可以使用 sprintf、strcpy 及 memcpy 函数，请问这些函数有什么区别，你喜欢使用哪个，为什么？

答案：这些函数的区别在于 实现功能以及操作对象不同。

(1) strcpy 函数操作的对象是字符串，完成从源字符串到目的字符串的拷贝功能。

(2) sprintf 函数操作的对象不限于字符串：虽然目的对象是字符串，但是源对象可以是字符串、也可以是任意基本类型的数据。这个函数主要用来实现（字符串或基本数据类型）向字符串的转换功能。如果源对象是字符串，并且指定 %s 格式符，也可实现字符串拷贝功能。

(3) memcpy

函数顾名思义就是内存拷贝，实现将一个内存块的内容复制到另一个内存块这一功能。内存块由其首地址以及长度确定。程序中出现的实体对象，不论是什么类型，其最终表现就是在内存中占据一席之地（一个内存区间或块）。因此，memcpy

的操作对象不局限于某一类数据类型，或者说可适用于任意数据类型，只要能给出对象的起始地址和内存长度信息、并且对象具有可操作性即可。鉴于memcpy

函数等长拷贝的特点以及数据类型代表的物理意义，memcpy

函数通常限于同种类型数据或对象之间的拷贝，其中当然也包括字符串拷贝以及基本数据类型的拷贝。

对于字符串拷贝来说，用上述三个函数都可以实现，但是其实现的效率和使用的方便程度不同：

- strcpy 无疑是最合适的选择：效率高且调用方便。
- sprintf 要额外指定格式符并且进行格式转化，麻烦且效率不高。
- memcpy 虽然高效，但是需要额外提供拷贝的内存长度这一参数，易错且使用不便；并且如果长度指定过大的话（最优长度是源字符串长度 +

1），还会带来性能的下降。其实 strcpy 函数一般是在内部调用 memcpy 函数或者用汇编直接实现的，以达到高效的目的。因此，使用

memcpy 和 strcpy 拷贝字符串在性能上应该没有什么大的差别。

对于非字符串类型的数据的复制来说，strcpy 和 sprintf 一般就无能为力了，可是对 memcpy

却没有什么影响。但是，对于基本数据类型来说，尽管可以用 memcpy

进行拷贝，由于有赋值运算符可以方便且高效地进行同种或兼容类型的数据之间的拷贝，所以这种情况下 memcpy 几乎不被使用。memcpy

的长处是用来实现（通常是内部实现居多）对结构或者数组的拷贝，其目的是或者高效，或者使用方便，甚或两者兼有。

811.应用程序在运行时的内存包括代码区和数据区，其中数据区又包括哪些部分？

答：对于一个进程的内存空间而言，可以在逻辑上分成 3 个部份：代码区，静态数据区和动态数据区。

动态数据区一般就是“堆栈”。栈是一种线性结构，堆是一种链式结构。进程的每个线程都有私有的“栈”。

全局变量和静态变量分配在静态数据区，本地变量分配在动态数据区，即堆栈中。程序通过堆栈的基地址和偏移量来访问本地变量。

812.C++函数中值的传递方式有哪几种？

答：三种传递方式为：值传递、指针传递和引用传递。

813.C++里面是不是所有的动作都是main()引起的？如果不是，请举例。

比如全局变量的初始化，就不是由main函数引起的

举例：class A{;

A a; //a的构造函数限执行

```
int main() {}
```

814.下列哪两个是等同的

int b;

A const int* a = &b;

B const* int a = &b;

C const int* const a = &b;

D int const* const a = &b;

815.内联函数在编译时是否做参数类型检查？

答：内联函数要做参数类型检查, 这是内联函数跟宏相比的优势。

816.全局变量和局部变量有什么区别？实怎么实现的？操作系统和编译器是怎么知道的？

(1) 生命周期不同：

全局变量随主程序创建和创建，随主程序销毁而销毁

局部变量在局部函数内部，甚至局部循环体等内部存在，退出就不存在；内存中

分配在全局数据区

(2) 使用方式不同：通过声明后全局变量程序的各个部分都可以用到；局部变量只能在局部使用，分配在栈区

操作系统和编译器通过内存分配的位置来知道的，全局变量分配在全局数据段并且在程序开始运行的时候被加载。局部变量则分配在堆栈里面。

817.有 A、B、C、D 四个人，要在夜里过一座桥。他们通过这座桥分别需要耗时 1、2、5、10 分钟，只有一支手电，并且同时最多只能两个人一起过桥。请问，如何安排，能够在 17 分钟内这四个人都过桥？

Solution:关键是时间最长的两个人必须同时过桥

The First Time : A(1) 和 B(2) 过桥，A(1) 返回 Cost : 1+2

The Second Time : C(5) 和 D(10) 过桥，B(2) 返回 Cost : 10+2

The Third Time A(1) 和 B(2) 过桥 Cost : 2

Total Time Cost : (1+2)+(10+2)+2=17 minutes

818.static全局变量与普通的全局变量有什么区别？static局部变量和普通局部变量有什么区别？static函数与普通函数有什么区别？

答：static全局变量与普通全局变量区别：static全局变量只初使化一次，防止在其他文件单元中被引用;

static局部变量和普通局部变量区别：static局部变量只被初始化一次，下一次依据上一次结果值；

static函数与普通函数区别：static函数在内存中只有一份，普通函数在每个被调用中维持一份拷贝。

819.程序的局部变量存在于（堆栈）中，全局变量存在于（静态区）中，动态申请数据存在于（堆）中。

820.对于一个频繁使用的短小函数,在C语言中应用什么实现,在C++中应用什么实现?

c用宏定义，c++用inline

821.有1,2,...一直到n的无序数组,求排序算法,并且要求时间复杂度为 $O(n)$,空间复杂度 $O(1)$,使用交换,而且一次只能交换两个数。

```
#include<iostream.h>

using namespace std;

int main(){

int a[] = {10,6,9,5,2,8,4,7,1,3};

int len = sizeof(a) / sizeof(int);

int temp;

for(int i = 0; i < len; )

{

temp = a[a[i] - 1];

a[a[i] - 1] = a[i];

a[i] = temp;

if ( a[i] == i + 1)

i++;

}

for (int j = 0; j < len; j++)

cout<<a[j]<<" ";

return 0;

}
```

版权声明：本文为CSDN博主「自由如风呼呼呼」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：https://blog.csdn.net/weixin_44363885/article/details/99567746

822.sizeof相关系列问题, const相关系列问题

- a、对于 struct s{char a;int b} sizeof(s) = 8 因为内存对齐
- b、对于 int a[200] sizeof(a) = 200* sizeof(int) = 800 对整个数组评测，int* a = new int[200]，sizeof(a) = 4 对指针评测
- c、这种使用位域的也有，从上到下最多相加不大于8便占1个位置，bits = 1 + 1(4+2 < 8) + 1(3) = 3. 其中元素最大为1个char 大小 8 位

```
struct bits
{
    char a:8;
    char b:4;
    char c:2;
    char d:3;
};
```

写出运行结果:

```
union v {
    struct x {
        unsigned char s1:2;
        unsigned char s2:3;
        unsigned char s3:3;
    } x;

    unsigned char c;
} v;

v.c = 100;
printf("%d", v.x.s3);

}
```

100 的2进制是1100100 去掉后面的5位余11放入x.s3中

结果：3

- d、对于空的类进行评测 class A {}; sizeof(A) = 1,默认空类是有一个占位符的
- e、对于虚函数 class A{ virtual test()}; class B:public A{ sizeof(B) =4
- f、设有以下说明和定义：

```
typedef union {long i; int k[5]; char c;} DATE; // sizeof(int)*5 = 20
struct data { int cat; DATE cow; double dog;} too; //4+20+8 = 32
DATE max;
```

则语句 `printf("%d",sizeof(struct date)+sizeof(max));` 的执行结果是：52
https://jq.qq.com/?_wv=1027&k=Fy1cyWkz

g、使用malloc或者new 分配内存，`void *pp = malloc(10); sizeof(p) = 4`；跟指针一样,sizeof 只能测出静态数组的长度，无法检测动态分配的或外部数组大小

h、下面函数输出结果：4 外部数组

```
void Func(char str[100])
{
    printf("%d/n", sizeof(str));
}
char str[100];
Func(str);
```

823.写出二分查找的代码.

```
int bfind(int* a,int len,int val)
{
    int m = len/2;
    int l = 0;
    int r = len;
    while(l!=m && r!= m)
    {
        if(a[m] > val)
        {
            r = m;
            m = (m+l)/2;
        }
        else if(a[m] < val)
        {
            l = m;
            m = (m+r)/2;
        }
        else
            return m;
    }
    return -1; //没有找到
}
```

824.写出在母串中查找子串出现次数的代码.

```
int count1(char* str,char* s)
{
    char* s1;
    char* s2;
    int count = 0;
    while(*str!='\0')
    {
        s1 = str;
        s2 = s;
        while(*s2 == *s1&&(*s2!='\0')&&(*s1!='\0'))
```

```
{
    s2++;
    s1++;
}
if(*s2 == '/')
    count++;
    str++;
}
return count;
}
```

825.查找第一个匹配子串位置,如果返回的是s1长度len1表示没有找到

```
size_t find(char* s1, char* s2)
{
    size_t i=0;
    size_t len1 = strlen(s1);
    size_t len2 = strlen(s2);
    if(len1-len2<0) return len1;
    for(;i<len1-len2;i++)
    {
        size_t m = i;
        for(size_t j=0;j<len2;j++)
        {
            if(s1[m]!=s2[j])
                break;
            m++;
        }
        if(j==len2)
            break;
    }
    return i<len1-len2?i:len1;
}
```

826.实现strcpy函数

```
char *strcpy(char *destination, const char *source)
{
    assert(destination!=NULL&&source!=NULL);
    char* target = destination;
    while(*destination++=*source++);
    return target ;
}
```

出现次数相当频繁

827.实现strcmp函数


```
int strcmp11(char* l, char* r)
{
    assert(l!=0&&r!=0);
    while(*l == *r &&*l != '/0') l++, r++;
    if(*l > *r)
        return 1;
    else if(*l == *r)
        return 0;
    return -1;
}
```

828.实现字符串翻转

```
void reserve(char* str)
{
    assert(str != NULL);
    char * p1 = str;
    char * p2 = str-1;
    while(++p2); //一般要求不能使用strlen
    p2 -= 1;
    while(p1<p2)
    {
        char c = *p1;
        *p1++ = *p2;
        *p2-- = c;
    }
}
```

829.用指针的方法，将字符串“ABCD1234efgh”前后对调显示

//不要用strlen求字符串长度，这样就没分了
代码如下：

```
char str123[] = "ABCD1234efgh";
char * p1 = str123;
char * p2 = str123-1;
while(++p2);
p2 -= 1;
while(p1<p2)
{
    char c = *p1;
    *p1++ = *p2;
    *p2-- = c;
}
```

830.给定字符串A和B,输出A和B中的最大公共子串。比如A="aocdfe" B="pmcdfa" 则输出"cdf"

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>

char *commanstring(char shortstring[], char longstring[])
{
    int i, j;
    char *substring=malloc(256);
    if(strstr(longstring, shortstring)!=NULL)           //如果....., 那么返回shortstring
        return shortstring;

    for(i=strlen(shortstring)-1;i>0; i--)              //否则, 开始循环计算
    {
        for(j=0; j<=strlen(shortstring)-i; j++)
        {
            memcpy(substring, &shortstring[j], i);
            substring[i]='\0';
            if(strstr(longstring, substring)!=NULL)
                return substring;
        }
    }
    return NULL;
}

main()
{
    char *str1=malloc(256);
    char *str2=malloc(256);
    char *comman=NULL;

    gets(str1);
    gets(str2);

    if(strlen(str1)>strlen(str2))                       //将短的字符串放前面
        comman=commanstring(str2, str1);
    else
        comman=commanstring(str1, str2);

    printf("the longest comman string is: %s/n", comman);
}

```

831.判断一个字符串是不是回文

```

int IsReverseStr(char *str)
{
    int i,j;
    int found=1;
    if(str==NULL)
        return -1;
    char* p = str-1;
    while(*++p!= '\0');
    --p;
    while(*str==*p&&str<p) str++,p--;
}

```

```
if(str < p)
    found = 0;
return found;
}
```

832.写函数完成内存的拷贝

```
void* memcpy( void *dst, const void *src, unsigned int len )
{
    register char *d;
    register char *s;
    if (len == 0)
        return dst;
    if ( dst > src ) //考虑覆盖情况
    {
        d = (char *)dst + len - 1;
        s = (char *)src + len - 1;
        while ( len >= 4 ) //循环展开，提高执行效率
        {
            *d-- = *s--;
            *d-- = *s--;
            *d-- = *s--;
            *d-- = *s--;
            len -= 4;
        }
        while ( len-- )
        {
            *d-- = *s--;
        }
    }
    else if ( dst < src )
    {
        d = (char *)dst;
        s = (char *)src;
        while ( len >= 4 )
        {
            *d++ = *s++;
            *d++ = *s++;
            *d++ = *s++;
            *d++ = *s++;
            len -= 4;
        }
        while ( len-- )
        {
            *d++ = *s++;
        }
    }
    return dst;
}
```

出现次数相当频繁

833.写一个函数,它的原形是int continumax(char *outputstr,char *inputstr)

功能：

在字符串中找出连续最长的数字串，并把这个串的长度返回，并把这个最长数字串付给其中一个函数参数outputstr所指内存。例如："abcd12345ed125ss123456789"的首地址传给inputstr后，函数将返回9，outputstr所指的值为123456789

```
int continumax(char *outputstr, char *inputstr)
{
    char *in = inputstr, *out = outputstr, *temp, *final;
    int count = 0, maxlen = 0;

    while( *in != '\0' )
    {
        if( *in > 47 && *in < 58 )
        {
            for(temp = in; *in > 47 && *in < 58 ; in++ )
                count++;
        }
        else
            in++;

        if( maxlen < count )
        {
            maxlen = count;
            count = 0;
            final = temp;
        }
    }
    for(int i = 0; i < maxlen; i++)
    {
        *out = *final;
        out++;
        final++;
    }
    *out = '\0';
    return maxlen;
}
```

834.编写一个 C 函数，该函数在一个字符串中找到可能的最长的子字符串，且该字符串是由同一字符组成的。

```
char * search(char *cpSource, char ch)
{
    char *cpTemp=NULL, *cpDest=NULL;
    int iTemp, iCount=0;
    while(*cpSource)
    {
        if(*cpSource == ch)
        {
            iTemp = 0;
            cpTemp = cpSource;
        }
    }
}
```

```
while(*cpSource == ch)
    ++iTemp, ++cpSource;
if(iTemp > iCount)
    iCount = iTemp, cpDest = cpTemp;
if(!*cpSource)
    break;
}
++cpSource;
}
return cpDest;
}
```

835.写出快速排序或者某种排序算法代码

快速排序:

```
int partition(int* a, int l, int r)
{
    int i=l-1, j=r, v=a[r];
    while(1)
    {
        while(a[++i]<v);
        while(a[--j]>v) if(j<=i) break;
        if(i>=j)
            break;
        swap(a[i], a[j]);
    }
    swap(a[i], a[r]);
    return i;
}

void qsort(int* a, int l, int r)
{
    if(r>l)
    {
        int i = partition(a, l, r);
        qsort(a, l, i-1);
        qsort(a, i+1, r);
    }
}
```

有兴趣可以看看下面2个。一般面试不会要求的

改进1:

```
void qsort(int* a, int l, int r)
{
    while(l<r) //防止过多递归
    {
        int i = partition(a, l, r);
        qsort(a, l, i-1);
        l = i+1;
    }
}
```

```
}
改进2:
void qsort(int* a,int l,int r)
{
    while(l<r)
    {
        if(r-l<32)    //防止分割恶化
        {
            insertsort(a+l,r-l+1); //后面的插入排序
            return;
        }
        int i = partition(a,l,r);
        qsort(a,l,i-1);
        l = i+1;
    }
}
```

冒泡排序: 出现次数相当频繁

```
void bubble(int *a,int n)
{
    for(int i=0;i<n;i++)
    {
        for(int j=1;j<n-i;j++)
        {
            if(a[j]<a[j-1])
            {
                int temp=a[j];
                a[j] = a[j-1];
                a[j-1] = temp;
            }
        }
    }
}
```

插入排序：

```
void insertsort(int* a,int n)
{
    int key;
    for(int j=1;j<n;j++)
    {
        key = a[j];
        for(int i=j-1;i>=0&& a[i]>key;i--)
        {
            a[i+1] = a[i];
        }
        a[i+1] = key;
    }
}
```

836. 将一个单链表逆序

```

struct list_node
{
    list_node(int a, list_node* b):data(a),next(b) //这个为了测试方便
    {}
    int data;
    list_node* next;
};

// 认为头节点存在，如果list类内函数不用判断头结点是否为空。不是类内部函数得判断头结点
void reserve(list_node* phead)
{
    list_node* p = phead->next;
    if(p == NULL || p->next == NULL) return; //只有头节点或一个节点
    list_node* p1=p->next;
    p->next=NULL;
    while(p1!=NULL)
    {
        p = p1->next;
        p1->next = phead->next;
        phead->next = p1;
        p1 = p;
    }
}

```

测试程序：

```

list lt;
lt.phead = new list_node(0,0);
lt.phead->next = new list_node(1,0);
lt.phead->next->next = new list_node(2,0);
lt.phead->next->next->next = new list_node(3,0);
lt.reserve();
list_node * p = lt.phead;
while(p)
{
    cout<<p->data<<endl;
    p = p->next;
}

```

837. 循环链表的节点对换和删除。

```

//双向循环
list_node* earse(list_node* node)
{
    // if(node == rear) return node->next; //对于头节点可判断也可不判断。最好加上
    list_node* next = node->next;
    next->prev = node->prev;
    node->prev->next = next;
    delete node;
    return next;
}

```



```
// 单项循环
list_node* earse(list_node* node)
{
    // if(node == rear) return node->next; //对于头节点可判断也可不判断。最好加上
    list_node* p = rear;
    while(p->next != node) p=p->next;
    p->next = node->next;
    delete node;
    return p->next;
}
```

838.有双向循环链表结点定义为：

```
struct node {
    int data;
    struct node *front, *next;
};
```

有两个双向循环链表A, B, 知道其头指针为: pHeadA, pHeadB, 请写一函数将两链表中data值相同的结点删除

```
BOOL DeleteNode(Node *pHeader, DataType Value)
{
```

```
    if (pHeader == NULL) return;
```

```
    BOOL bRet = FALSE;
```

```
    Node *pNode = pHeader;
```

```
    while (pNode != NULL)
```

```
    {
```

```
        if (pNode->data == Value)
```

```
        {
```

```
            if (pNode->front == NULL)
```

```
            {
```

```
                pHeader = pNode->next;
```

```
                pHeader->front = NULL;
```

```
            }
```

```
        else
```

```
        {
```

```
            if (pNode->next != NULL)
```

```
            {
```

```
                pNode->next->front = pNode->front;
```

```
            }
```

```
            pNode->front->next = pNode->next;
```

```
        }
```

- Node *pNextNode = pNode->next;
- delete pNode;
- pNode = pNextNode;

- bRet = TRUE;
- //不要break或return, 删除所有
- }
- else
- {
- pNode = pNode->next;

```
.    }  
    }  
  
    return bRet;  
}  
  
void DE(Node *pHeadA, Node *pHeadB)  
{  
    if (pHeadA == NULL || pHeadB == NULL)  
    {  
        return;  
    }  
  
    Node *pNode = pHeadA;  
    while (pNode != NULL)  
    {  
        if (DeleteNode(pHeadB, pNode->data))  
        {  
            if (pNode->front == NULL)  
            {  
                pHeadA = pNode->next;  
                pHeadA->front = NULL;  
            }  
            else  
            {  
                pNode->front->next = pNode->next;  
                if (pNode->next != NULL)  
                {  
                    pNode->next->front = pNode->front;  
                }  
            }  
            Node *pNextNode = pNode->next;  
            delete pNode;  
            pNode = pNextNode;  
        }  
        else  
        {  
            pNode = pNode->next;  
        }  
    }  
}
```

839.写出程序删除链表中的所有结点

```
void del_all(node *head)
{
    node *p;
    while(head!=NULL)
    {
        p=head->next;
        free(head);
        head=p;
    }
    cout<<"释放空间成功!"<<endl;
}
```

840.线形表a、b为两个有序升序的线形表，编写一程序，使两个有序线形表合并成一个有序升序线形表h；

可以参考 stl 函数 merge()

```
Linklist *unio(Linklist *p,Linklist *q)
{
    linklist *R,*pa,*qa,*ra;
    pa=p;
    qa=q;
    R=ra=p;
    while(pa->next!=NULL&&qa->next!=NULL)
    {
        if(pa->data>qa->data)
        {
            ra->next=qa;
            qa=qa->next;
        }
        else
        {
            ra->next=pa;
            pa=pa->next;
        }
    }
    if(pa->next!=NULL)
        ra->next=pa;
    if(qa->next!=NULL)
        ra->next==qa;
    return R;
}
```

841.怎么判断链表中是否有环？

```
bool CircleInList(Link* pHead)
{
    if(pHead == NULL || pHead->next == NULL)//无节点或只有一个节点并且无自环
        return (false);
    if(pHead->next == pHead)//自环
        return (true);
    Link *pTemp1 = pHead;//step 1
    Link *pTemp = pHead->next;//step 2
```

```
while(pTemp != pTemp1 && pTemp != NULL && pTemp->next != NULL)
{
    pTemp1 = pTemp1->next;
    pTemp = pTemp->next->next;
}
if(pTemp == pTemp1)
    return (true);
return (false);
}
```

842.static有什么用途？（请至少说明两种）

- 1). 在函数体，一个被声明为静态的变量在这一函数被调用过程中维持其值不变。
- 2). 在模块内（但在函数体外），一个被声明为静态的变量可以被模块内所用函数访问，但不能被模块外其它函数访问。它是一个本地的全局变量。
- 3). 在模块内，一个被声明为静态的函数只可被这一模块内的其它函数调用。那就是，这个函数被限制在声明它的模块的本地范围内使用。

843.引用与指针有什么区别？

1. 引用必须被初始化，指针不必。
2. 引用初始化以后不能被改变，指针可以改变所指的对象。
3. 不存在指向空值的引用，但是存在指向空值的指针。
4. 重载操作符使用引用可以完成串试操作

844.全局变量和局部变量在内存中是否有区别？如果有，是什么区别？

全局变量储存在全局静态存储区，局部变量在堆栈

845.static变量和static 函数各有什么特点？

答:static变量：在程序运行期内一直有效，如果定义在函数外，则在编译单元内可见，如果在函数内，在在定义的block内可见；static函数：在编译单元内可见；

846.static全局变量与普通的全局变量有什么区别？

static全局变量只初使化一次，防止在其他文件单元中被引用;

847.static函数与普通函数有什么区别：

static函数在内存中只有一份，普通函数在每个被调用中维持一份拷贝
程序的局部变量存在于（堆栈）中，全局变量存在于（静态区）中，动态申请数据存在于（堆）中。

848.什么是平衡二叉树？

左右子树都是平衡二叉树 且左右子树的深度差值的绝对值不大于1

849.什么函数不能声明为虚函数？

constructor

850.冒泡排序算法的时间复杂度是什么 $O(n^2)$ 快速排序 $O(n \lg n)$

851.写出float x 与“零值”比较的if语句。

```
if(x>0.000001&& x<-0.000001)
```

这个都够古董的, 恐怕是8086以前的事情吧. 汇编早都可以用一条指令比较了. 既然想考精度, 就换个不是0的, 比如0.00002, `if(x-0.00002>0.000001&& x-0.00002<-0.000001)`

852.进程间通信的方式有?

进程间通信的方式有 共享内存, 管道, Socket, 消息队列, DDE等

853.const 符号常量;

(1)const char *p

(2)char const *p

(3)char * const p

说明上面三种描述的区别;

如果const位于星号的左侧, 则const就是用来修饰指针所指向的变量, 即指针指向为常量;

如果const位于星号的右侧, const就是修饰指针本身, 即指针本身是常量。

854.c和c++中的struct有什么不同？

c和c++中struct的主要区别是c中的struct不可以含有成员函数, 而c++中的struct可以。c++中struct和class的主要区别在于默认的存取权限不同, struct默认为public, 而class默认为private

855.纯虚函数如何定义？使用时应注意什么？

```
virtual void f()=0;
```

是接口, 子类必须要实现

856.数组和链表的区别

数组: 数据顺序存储, 固定大小

链表: 数据可以随机存储, 大小可动态改变

857.线程与进程的区别和联系? 线程是否具有相同的堆栈? dll是否有独立的堆栈?

进程是死的, 只是一些资源的集合, 真正的程序执行都是线程来完成的, 程序启动的时候操作系统就帮你创建了一个主线程。

每个线程有自己的堆栈。

DLL中有没有独立的堆栈, 这个问题不好回答, 或者说这个问题本身是否有问题。因为DLL中的代码是被某些线程所执行, 只有线程拥有堆栈, 如果DLL中的代码是EXE中的线程所调用, 那么这个时候是不是说这个DLL没有自己独立的堆栈? 如果DLL中的代码是由DLL自己创建的线程所执行, 那么是不是说DLL有独立的堆栈?

以上讲的是堆栈，如果对于堆来说，每个DLL有自己的堆，所以如果是从DLL中动态分配的内存，最好是从DLL中删除，如果你从DLL中分配内存，然后在EXE中，或者另外一个DLL中删除，很有可能导致程序崩溃

858.一语句实现x是否为2的若干次幂的判断

```
int i = 512;
cout << boolalpha << ((i & (i - 1)) ? false : true) << endl;
```

859.计算结果题目：

```
class A
{
    virtual void func1();
    void func2();
}
class B: class A
{
    void func1(){cout << "fun1 in class B" << endl;}
    virtual void func2(){cout << "fun2 in class B" << endl;}
}
```

- A, A中的func1和B中的func2都是虚函数.
 B, A中的func1和B中的func2都不是虚函数.
 C, A中的func2是虚函数. , B中的func1不是虚函数.
 D, A中的func2不是虚函数, B中的func1是虚函数.

答:A

860.输出下面程序结果。

```
class A
{
public:
    virtual void print(void)
    {
        cout<<"A::print()"<<endl;
    }
};
class B:public A
{
public:
    virtual void print(void)
    {
        cout<<"B::print()"<<endl;
    };
};
class C:public B
{
public:
    virtual void print(void)
    {
        cout<<"C::print()"<<endl;
    }
};
```

```
}  
};  
void print(A a)  
{  
    a.print();  
}  
void main(void)  
{  
    A a, *pa,*pb,*pc;  
    B b;  
    C c;  
    pa=&a;  
    pb=&b;  
    pc=&c;  
    a.print();  
    b.print();  
    c.print();  
    pa->print(); //多态  
    pb->print();  
    pc->print();  
    print(a);  
    print(b);  
    print(c);  
}
```

答案:

```
A::print()  
B::print()  
C::print()  
A::print()  
B::print()  
C::print()  
A::print()  
A::print()  
A::print()
```

861.写出程序运行结果

```
int sum(int a)  
{  
    auto int c=0;  
    static int b=3;  
    c+=1;  
    b+=2;  
    return(a+b+c);  
}  
  
void main()  
{  
    int I;  
    int a=2;  
    for(I=0;I<5;I++)  
    {  
        printf("%d,", sum(a));  
    }
```



```
}  
}  
// static会保存上次结果，记住这一点，剩下的自己写
```

输出：8,10,12,14,16,

862.求函数返回值，输入x=9999;

```
int func ( x )  
{  
    int countx = 0;  
    while ( x )  
    {  
        countx ++;  
        x = x&(x-1);  
    }  
    return countx;  
}
```

结果呢？

知道了这是统计9999的二进制数值中有多少个1的函数，且有

$9999 = 9 \times 1024 + 512 + 256 + 15$

9×1024中含有1的个数为2；

512中含有1的个数为1；

256中含有1的个数为1；

15中含有1的个数为4；

故共有1的个数为8，结果为8。

$1000 - 1 = 0111$ ，正好是原数取反。这就是原理。

用这种方法来求1的个数是很效率很高的。

不必去一个一个地移位。循环次数最少。

863.用户输入M,N值，从1至N开始顺序循环数数，每数到M输出该数值，直至全部输出。写出C程序。

循环链表，用取余操作做

//这样写感觉不是太好，置1表示被访问过。

```
void joe(int n,int m)  
{  
    int *a = new int[n];  
    int i=0;  
    int pos=0;  
    while(i<n)  
    {  
        int c=m;  
        pos %= n;  
        while(c)  
        {  
            c--;  
            while(a[pos]==1)  
            {  
                pos++;  
            }  
        }  
    }  
}
```

```

        pos %= n;
    }
    pos++;
    pos %= n;
}
a[pos-1] = 1;
cout<<pos<<" ";
i++;
}
delete[] a;
}

```

方法2：

```

int Josephu(int n, int m)
{
    int flag, i, j = 0;
    int *arr = (int *)malloc(n * sizeof(int));
    for (i = 0; i < n; ++i)
        arr[i] = 1;
    for (i = 1; i < n; ++i)
    {
        flag = 0;
        while (flag < m)
        {
            if (j == n)
                j = 0;
            if (arr[j])
                ++flag;
            ++j;
        }
        arr[j - 1] = 0;
        printf("第%d个出局的人是: %4d号/n", i, j);
    }
    free(arr);
    return j;
}

int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    printf("最后胜利的是%d号! /n", Josephu(n, m));
    system("pause");
    return 0;
}

```

链表实现：

```

#include <stdio.h>
#include <malloc.h>
typedef struct Node
{
    int index;
    struct Node *next;
}

```

```

}JosephuNode;
int Josephu(int n, int m)
{
    int i, j;
    JosephuNode *head, *tail;
    head = tail = (JosephuNode *)malloc(sizeof(JosephuNode));
    for (i = 1; i < n; ++i)
    {
        tail->index = i;
        tail->next = (JosephuNode *)malloc(sizeof(JosephuNode));
        tail = tail->next;
    }
    tail->index = i;
    tail->next = head;

    for (i = 1; tail != head; ++i)
    {
        for (j = 1; j < m; ++j)
        {
            tail = head;
            head = head->next;
        }
        tail->next = head->next;
        printf("第%d个出局的人是: %4d号/n", i, head->index);
        free(head);
        head = tail->next;
    }
    i = head->index;
    free(head);
    return i;
}
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    printf("最后胜利的是%d号! /n", Josephu(n, m));
    system("pause");
    return 0;
}

```

864.有10亿个浮点数，求出其中最大的10000个，用了标准库的，不让用的话，只能自己写堆函数

```

vector<float> bigs(10000,0);
vector<float>::iterator it;
for(it=bigs.begin();it!=bigs.end();it++)
{
    *it = (float)rand()/7; //数据都是用随机数模拟的
}
cout<<bigs.size()<<endl;
make_heap(bigs.begin(),bigs.end(),greater<float>() );
float ff;

time_t t1,t2;

```

```
time(&t1);
for(int i=0;i<1000000000;i++)
{
    ff = (float) rand()/7;
    if(ff>big5[0])
    {
        pop_heap(big5.begin(),big5.end(),greater<float>());
        big5.pop_back();
        big5.push_back(ff);
        push_heap(big5.begin(),big5.end(),greater<float>());
    }
}
time(&t2);
cout<<(long)(t2-t1)<<endl;
```

如果要写堆排序可以用:

swap是std内部函数可以交换数组中两个数, 这里省点事

```
void fixdown(int*a,int k,int n)
{
    while(2*k<=n)
    {
        int j=2*k;
        if(j<n&&a[j]<a[j+1]) ++j;
        if(a[j] < a[k] )break;
        swap(a[j],a[k]);
        k = j;
    }
}

void heapsort(int* a,int n)
{
    int k = n/2;
    int* p = a-1;
    for(int i=k;i>0;i--)
    {
        fixdown(p,k,n);
    }
    while(n>0)
    {
        swap(p[n],p[1]);
        fixdown(p,1,--n);
    }
}
```

865.在不用第三方参数的情况下，交换两个参数的值 感觉比较:(，bt 而且还是基础题。

```
\#include <stdio.h>
void main()
{
    int i=60;
    int j=50;
    i=i+j;
    j=i-j;
    i=i-j;
    printf("i=%d/n",i);
    printf("j=%d/n",j);
}
```

方法二：

```
i^=j;
j^=i;
i^=j;
```

方法三：

// 用加减实现，而且不会溢出

a = a+b-(b=a)

4) 输出和为一个给定整数的所有组合

例如n=5

5=1+4 ; 5=2+3 (相加的数不能重复)

则输出

1 , 4 ; 2 , 3。

```
#include <stdio.h>

int main(void)
{
    unsigned long int i,j,k;
    printf("please input the number/n");
    scanf("%d",&i);
    if( i % 2 == 0)
        j = i / 2;
    else
        j = i / 2 + 1;

    printf("The result is /n");
    for(k = 0; k < j; k++)
        printf("%d = %d + %d/n",i,k,i - k);
    return 0;
}
```

866.写一段程序，找出数组中第k大小的数，输出数所在的位置。

例如{2, 4, 3, 4, 7}中, 第一大的数是7, 位置在4。第二大、第三大的数都是4, 位置在1、3随便输出哪一个均可。函数接口为: int find_orderk(const int* narry, const int n, const int k)
要求算法复杂度不能是 $O(n^2)$, 应该 $O(n \lg n)$ 吧 n^2 也太容易了冒泡排序都可以
谢谢!

可以先用快速排序进行排序, 其中用另外一个进行地址查找

代码如下, 在VC++6.0运行通过。给分吧^^, 鄙视明明是个 partial_sort, 全排sort 效率差多了
贴一份标准库代码, 以后把堆排序所有函数不上。

```
template<class _RanIt,
        class _Ty> inline
void _Partial_sort(_RanIt _First, _RanIt _Mid, _RanIt _Last, _Ty *)
{ // order [First, _Last) up to _Mid, using operator<
  std::make_heap(_First, _Mid);

  for (_RanIt _Next = _Mid; _Next < _Last; ++_Next)
    if (*_Next < *_First)
      _Pop_heap(_First, _Mid, _Next, _Ty(*_Next),
        _Dist_type(_First)); // replace top with new largest
  std::sort_heap(_First, _Mid);
}
```

867.求1000! 的末尾有几个0 (用素数相乘的方法来做 , 如 72=22233) ;

求出1->1000里,能被5整除的数的个数n1,能被25整除的数的个数n2,能被125整除的数的个数n3,能被625整除的数的个数n4.

1000!末尾的零的个数= $n1+n2+n3+n4$;

```
#include<stdio.h>
#define NUM 1000

int find5(int num)
{
  int ret=0;
  while(num%5==0)
  {
    num/=5;
    ret++;
  }
  return ret;
}

int main()
{
  int result=0;
  int i;
  for(i=5;i<=NUM;i+=5)
  {
    result+=find5(i);
  }
  printf(" the total zero number is %d/n",result);
  return 0;
}
```

868.编程实现：把十进制数(long型)分别以二进制和十六进制形式输出，不能使用printf系列库函数

```
char* test3(long num)
{
    char* buffer = (char*)malloc(11);
    buffer[0] = '0';
    buffer[1] = 'x';
    buffer[10] = '/0';

    char* temp = buffer + 2;
    for (int i=0; i < 8; i++)
    {
        temp[i] = (char)(num<<4*i>>28);
        temp[i] = temp[i] >= 0 ? temp[i] : temp[i] + 16;
        temp[i] = temp[i] < 10 ? temp[i] + 48 : temp[i] + 55;
    }
    return buffer;
}
```

869.输入N, 打印 N*N 矩阵

比如 N = 3，打印：螺旋矩阵

1 2 3
8 9 4
7 6 5

N = 4，打印：

1 2 3 4
12 13 14 5
11 16 15 6
10 9 8 7

解答：

```
#define N 15
int s[N][N];
void main()
{
    int k = 0, i = 0, j = 0;
    int a = 1;
    for( ; k < (N+1)/2; k++ )
    {
        while( j < N-k ) s[i][j++] = a++; i++; j--;
        while( i < N-k ) s[i++][j] = a++; i--; j--;
        while( j > k-1 ) s[i][j--] = a++; i--; j++;
        while( i > k ) s[i--][j] = a++; i++; j++;
    }
    for( i = 0; i < N; i++ )
    {
        for( j = 0; j < N; j++ )
```



```

        cout << s[i][j] << " ";
        cout << endl;
    }
}

```

870.斐波拉契数列递归实现的方法如下：

```

int Funct( int n )
{
    if(n==0) return 1;
    if(n==1) return 1;
    retrurn Funct(n-1) + Funct(n-2);
}

```

如何不使用递归，来实现上述函数？

解答：

```

int Funct( int n ) // n 为非负整数
{
    int a=1;
    int b=1;
    int c;
    if(n==0 || n == 1)
        return 1;
    for(int i=1;i<n;i++)
    {
        c=a+b;
        a=b;
        b=c;
    }
    return b;
}

```

871.将一个数字字符串转换为数字."1234" -->1234

```

int atoi(char* s)
{
    assert(s!=NULL);
    int num = 0;
    int temp;
    while(*s>'0' && *s<'9')
    {
        num *= 10;
        num += *s-'0';
        s++;
    }
    return num;
}

```

出现次数相当频繁

872.编程实现：把十进制数(long型)分别以二进制和十六进制形式输出，不能使用printf系列库函数

```
char* test3(long num)
{
    char* buffer = (char*)malloc(11);
    buffer[0] = '0';
    buffer[1] = 'x';
    buffer[10] = '/0';

    char* temp = buffer + 2;
    for (int i=0; i < 8; i++)
    {
        temp[i] = (char)(num<<4*i>>28);
        temp[i] = temp[i] >= 0 ? temp[i] : temp[i] + 16;
        temp[i] = temp[i] < 10 ? temp[i] + 48 : temp[i] + 55;
    }
    return buffer;
}
```

873.实现任意长度的整数相加或者相乘功能。

```
void bigadd(char* num, char* str, int len)
{
    for(int i=len; i>0; i--)
    {
        num[i] += str[i];
        int j = i;
        while(num[j]>=10)
        {
            num[j--] -= 10;
            num[j] += 1;
        }
    }
}
```

874.用递归算法判断数组a[N]是否为一个递增数组。

递归的方法，记录当前最大的，并且判断当前的是否比这个还大，大则继续，否则返回false结束：

```
bool fun( int a[], int n )
{
    if( n==1 )
        return true;
    if( n==2 )
        return a[n-1] >= a[n-2];
    return fun( a,n-1 ) && ( a[n-1] >= a[n-2] );
}
```

875.运用四色定理，为N个局域举行配色，颜色为1、2、3、4四种，另有数组adj[][N]，如adj[i][j]=1则表示i区域与j区域相邻，数组color[N]，如color[i]=1,表示i区域的颜色为1号颜色。四色填充

正在看图的程序，以后补一个

876.给两个数组和他们的大小，还有一动态开辟的内存，求交集，把交集放到动态内存dongtai，并且返回交集个数

long jiaoji(long* a[],long b[],long* alength,long blength,long* dongtai[])

如果让用库，放入两个set中，然后调用set_difference函数。

不让的话就先排序，然后依次比较了。

877.象搜索的输入信息是一个字符串，统计300万输入信息中的最热门的前十条，我们每次输入的一个字符串为不超过255byte,内存使用只有1G,

请描述思想，写出算发（c语言），空间和时间复杂度。

跟10亿浮点数的相同，使用堆做部分排序时间复杂度 $N\log 10$ 。空间？？用10的信息空间

878.国内的一些贴吧，如baidu,有几十万个主题，假设每一个主题都有上亿的跟帖子，怎么样设计这个系统速度最好，请描述思想，写出算发（c语言），空间和时间复杂度，

每一个主题都有上亿的跟帖子，那baidu就崩溃了。天知道想问什么？？

879.用两个栈实现一个队列的功能？要求给出算法和思路！

设2个栈为A,B，一开始均为空。

入队：

将新元素push入栈A；

出队：

(1)判断栈B是否为空；

(2)如果不为空，则将栈A中所有元素依次pop出并push到栈B；

(3)将栈B的栈顶元素pop出；

880.求组合数：求n个数（1...n）中k个数的组合....如：combination(5,3)

要求输出：543，542，541，532，531，521，432，431，421，321，

```
#include<stdio.h>
```

```
int pop(int *);
```

```
int push(int );
```

```
void combination(int ,int );

int stack[3]={0};
top=-1;

int main()
{
    int n,m;
    printf("Input two numbers:/n");
    while( (2!=scanf("%d*c%d",&n,&m)) )
    {
        fflush(stdin);
        printf("Input error! Again:/n");
    }
    combination(n,m);
    printf("/n");
}

void combination(int m,int n)
{
    int temp=m;
    push(temp);
    while(1)
    {
        if(1==temp)
        {
            if(pop(&temp)&&stack[0]==n) //当栈底元素弹出&&为可能取的最小值，循环退出
                break;
        }
        else if( push(--temp))
        {
            printf("%d%d%d ",stack[0],stack[1],stack[2]);
            pop(&temp);
        }
    }
}

int push(int i)
{
    stack[++top]=i;
    if(top<2)
        return 0;
    else
        return 1;
}

int pop(int *i)
{
    *i=stack[top--];
    if(top>=0)
        return 0;
    else
        return 1;
}
```

881.下面是C语言中两种if语句判断方式。请问哪种写法更好？为什么？

```
int n;  
if (n == 10) // 第一种判断方式  
if (10 == n) // 第二种判断方式
```

如果少了个=号,编译时就会报错,减少了出错的可能行,可以检测出是否少了=

882.下面的代码有什么问题？

```
void DoSomething(...)  
{  
    char* p;  
    ...  
    p = malloc(1024); // 分配1K的空间  
    if (NULL == p)  
        return;  
    ...  
    p = realloc(p, 2048); // 空间不够, 重新分配到2K  
    if (NULL == p)  
        return;  
    ...  
}
```

A:
p = malloc(1024); 应该写成: p = (char *) malloc(1024*sizeof(char));
没有释放p的空间, 造成内存泄漏。

883.下面的代码有什么问题？并请给出正确的写法。

```
void DoSomething(char* p)  
{  
    char str[16];  
    int n;  
    assert(NULL != p);  
    sscanf(p, "%s%d", str, n);  
    if (0 == strcmp(str, "something"))  
    {  
        ...  
    }  
}
```

A:
sscanf(p, "%s%d", str, n); 这句该写成: scanf(p, "%s%d", str, &n);
如果 %s 在前必须指定长度, 不然sscanf 不知何时取字符串结束 scanf(p)

884.下面代码有什么错误？

```
void test1()
{
    char string[10];
    char *str1="0123456789";
    strcpy(string, str1);
}
```

数组越界

885.下面代码有什么问题?

```
void test2()
{
    char string[10], str1[10];
    for(i=0; i<10;i++)
    {
        str1[i] = 'a';
    }
    strcpy(string, str1);
}
```

str1没有置字符串结束符'/0', 数组越界

886.下面代码有什么问题?

```
void test3(char* str1)
{
    char string[10];
    if(strlen(str1)<=10)
    {
        strcpy(string, str1);
    }
}
```

==数组越界

==strcpy拷贝的结束标志是查找字符串中的/0 因此如果字符串中没有遇到/0的话 会一直复制，直到遇到/0,上面的123都因此产生越界的情况

建议使用 strncpy 和 memcpy

887.下面代码有什么问题?

```
#define MAX_SRM 256

DSN get_SRM_no()
{
    static int SRM_no; //是不是这里没赋初值?
    int I;
```

```

for(I=0;I<MAX_SRM;I++,SRM_no++)
{
    SRM_no %= MAX_SRM;
    if(MY_SRM.state==IDLE)
    {
        break;
    }
}
if(I>=MAX_SRM)
    return (NULL_SRM);
else
    return SRM_no;
}

```

// 网上有写：系统会初始化static int变量为0,但该值会一直保存,所谓的不可重入.. 扯淡.

答案: 函数永远返回NULL_SRM , 因为最后i是等于MAX_SRM的, SRM_no 最大会是256 , 而不是255. 连个都是后++产生的问题。

888.下面代码有什么问题?

```

void GetMemory(char *p){
    p=(char *)malloc(100);
}
void Test(void){
    char *str=NULL;
    GetMemory=(str);
    strcpy(str,"hello world");
    printf(str);
}

```

A:错误--参数的值改变后, 不会传回,GetMemory并不能传递动态内存, Test函数中的 str一直都是NULL。

strcpy(str, "hello world");将使程序崩溃。

889.下面这个程序执行后会有什么错误或者效果:

```

#define MAX 255
int main()
{
    unsigned char A[MAX]; //i被定义为unsigned char
    for (unsigned char i=0;i<=MAX;i++)
        A[i]=i;
}

```

解答：死循环加数组越界访问（C/C++不进行数组越界检查）

MAX=255 数组A的下标范围为:0..MAX-1,这是其一..

其二.当i循环到255时,循环内执行: A[255]=255;

这句本身没有问题..但是返回for (i=0;i<=MAX;i++)语句时,

由于unsigned char的取值范围在(0..255),i++以后i又为0了..无限循环下去.

890.请找出下面代码中的所以错误

说明：以下代码是把一个字符串倒序，如“abcd”倒序后变为“dcba”


```
1、 #include "string.h"
2、 main()
3、 {
4、   char*src="hello,world";
5、   char* dest=NULL;
6、   int len=strlen(src);
7、   dest=(char*)malloc(len);
8、   char* d=dest;
9、   char* s=src[len];
10、  while(len--!=0)
11、    d++=s--;
12、  printf("%s",dest);
13、  return 0;
14、 }
```

答：

方法1：

```
int main(){
char* src = "hello,world";
int len = strlen(src);
char* dest = (char*)malloc(len+1); //要为/0分配一个空间
char* d = dest;
char* s = &src[len-1]; //指向最后一个字符
while( len-- != 0 )
*d++=*s--;
*d = 0; //尾部要加/0
printf("%s/n",dest);
free(dest); // 使用完，应当释放空间，以免造成内存泄露
return 0;
}
```

891.请问下面程序有什么错误？

```
int a[60][250][1000],i,j,k;
for(k=0;k<=1000;k++)
for(j=0;j<250;j++)
for(i=0;i<60;i++)
a[i][j][k]=0;
```

把循环语句内外换一下,造成大量的内存页失效

原文作者：[smarts](#)

原文链接：<https://blog.csdn.net/u014313623/article/details/22588079>

892.32位，64位系统中，各种常用内置数据类型占用的字节数？

char：1个字节(固定)

(即指针变量)：4个字节(32位机的寻址空间是4个字节。同理64位编译器)(变化)

short int : 2个字节(固定)

int : 4个字节(固定)

unsigned int : 4个字节(固定)

float: 4个字节(固定)

double: 8个字节(固定)

long: 4个字节

unsigned long: 4个字节(变化*,其实就是寻址控件的地址长度数值)

long long: 8个字节(固定)

64位操作系统

char : 1个字节(固定)

*(即指针变量): 8个字节

short int : 2个字节(固定)

int : 4个字节(固定)

unsigned int : 4个字节(固定)

float: 4个字节(固定)

double: 8个字节(固定)

long: 8个字节

unsigned long: 8个字节(变化*其实就是寻址控件的地址长度数值)

long long: 8个字节(固定)

除*与long 不同其余均相同。

893.悬空指针与野指针区别

- 悬空指针：当所指向的对象被释放或者收回，但是没有让指针指向NULL；

```
{  
  
    char *dp = NULL;  
  
    {  
  
        char c;  
  
        dp = &c;  
  
    }  
  
    //变量c释放，dp变成空悬指针  
  
}
```

```
void func()

{

char *dp = (char *)malloc(A_CONST);    *

*free(dp);          //dp变成一个空悬指针    *

*dp = NULL;          //dp不再是空悬指针    /* ... */

}
```

野指针：那些未初始化的指针；

```
int func()

{

char *dp;//野指针，没有初始化

static char *sdp;//非野指针，因为静态变量会默认初始化为0

}
```

894.vector、map、multimap底层数据结构

底层数据结构：

- vector基于**数组**，map、multimap基于**红黑树**，unordered_map、unordered_multimap基于**哈希表**。

根据应用场景进行选择：

- map/unordered_map **不允许重复元素**
- multimap/unordered_multimap **允许重复元素**
- map/multimap **底层基于红黑树，元素自动有序，且插入、删除效率高**
- unordered_map/unordered_multimap **底层基于哈希表，故元素无序，查找效率高。**

895.C++的内存分区

- **栈区 (stack)**：主要存放函数参数以及局部变量，由系统自动分配释放。
- **堆区 (heap)**：由用户通过 malloc/new 手动申请，手动释放。注意它与数据结构中的堆是两回事，分配方式倒是类似于链表。
- **全局/静态区**：存放全局变量、静态变量；程序结束后由系统释放。
- **字符串常量区**：字符串常量就放在这里，程序结束后由系统释放。
- **代码区**：存放程序的二进制代码。

896.结构与联合有和区别？

(1). 结构和联合都是由多个不同的数据类型成员组成,但在任何同一时刻,联合中只存放了一个被选中的成员(所有成员共用一块地址空间),而结构的所有成员都存在(不同成员的存放地址不同)。

(2). 对于联合的不同成员赋值, 将会对其它成员重写, 原来成员的值就不存在了, 而对于结构的不同成员赋值是互不影响的。

897.将“引用”作为函数参数有哪些特点？

(1) 传递引用给函数与传递指针的效果是一样的。这时, 被调函数的形参就成为原来主调函数中的实参变量或对象的一个别名来使用, 所以在被调函数中对形参变量的操作就是对其相应的目标对象(在主调函数中)的操作。

(2) 使用引用传递函数的参数, 在内存中并没有产生实参的副本, 它是直接对实参操作; 而使用一般变量传递函数的参数, 当发生函数调用时, 需要给形参分配存储单元, 形参变量是实参变量的副本; 如果传递的是对象, 还将调用拷贝构造函数。因此, 当参数传递的数据较大时, 用引用比用一般变量传递参数的效率和所占空间都好。

(3) 使用指针作为函数的参数虽然也能达到与使用引用的效果, 但是, 在被调函数中同样要给形参分配存储单元, 且需要重复使用“*指针变量名”的形式进行运算, 这很容易产生错误且程序的阅读性较差; 另一方面, 在主调函数的调用点处, 必须用变量的地址作为实参。而引用更容易使用, 更清晰。

898.多态, 虚函数, 纯虚函数

多态: 是对于不同对象接收相同消息时产生不同的动作。C++的多态性具体体现在运行和编译两个方面: 在程序运行时的多态性通过继承和虚函数来体现;

在程序编译时多态性体现在函数和运算符的重载上;

虚函数: 在基类中冠以关键字 virtual 的成员函数。它提供了一种接口界面。允许在派生类中对基类的虚函数重新定义。

纯虚函数的作用: 在基类中为其派生类保留一个函数的名字, 以便派生类根据需要对它进行定义。作为接口而存在 纯虚函数不具备函数的功能, 一般不能直接被调用。

从基类继承来的纯虚函数, 在派生类中仍是虚函数。如果一个类中至少有一个纯虚函数, 那么这个类被称为抽象类 (abstract class)。

抽象类中不仅包括纯虚函数, 也可包括虚函数。抽象类必须用作派生其他类的基类, 而不能用于直接创建对象实例。但仍可使用指向抽象类的指针支持运行时多态性。

899.delete与 delete []区别

delete只会调用一次析构函数, 而delete[]会调用每一个成员的析构函数。在More Effective C++中有更为详细的解释: “当delete操作符用于数组时, 它为每个数组元素调用析构函数, 然后调用operator delete来释放内存。”delete与new配套, delete []与new []配套

```
MemTest *mTest1=new MemTest[10];
```

```
MemTest *mTest2=new MemTest;
```

```
Int *pInt1=new int [10];
```

```
Int *pInt2=new int;
```

```
delete[]pInt1; //-1-
```

```
delete[]pInt2; //-2-
```

```
delete[]mTest1; //-3-
```

```
delete[]mTest2; //-4-
```

在-4-处报错。

这就说明：对于内建简单数据类型，delete和delete[]功能是相同的。对于自定义的复杂数据类型，delete和delete[]不能互用。delete[]删除一个数组，delete删除一个指针。简单来说，用new分配的内存用delete删除；用new[]分配的内存用delete[]删除。delete[]会调用数组元素的析构函数。内部数据类型没有析构函数，所以问题不大。如果你在用delete时没用括号，delete就会认为指向的是单个对象，否则，它就会认为指向的是一个数组。

900.new、delete、malloc、free关系

delete会调用对象的析构函数,和new对应free只会释放内存，new调用构造函数。malloc与free是C++/C语言的标准库函数，new/delete是C++的运算符。它们都可用于申请动态内存和释放内存。对于非内部数据类型的对象而言，光用maloc/free无法满足动态对象的要求。对象在创建的同时要自动执行构造函数，对象在消亡之前要自动执行析构函数。由于malloc/free是库函数而不是运算符，不在编译器控制权限之内，不能够把执行构造函数和析构函数的任务强加于malloc/free。因此C++语言需要一个能完成动态内存分配和初始化工作的运算符new，以及一个能完成清理与释放内存工作的运算符delete。注意new/delete不是库函数。

901.链表和数组存储线性表的比较

数组的优点：

数组中的元素在内存中是连续存放的，能根据数组的首地址计算出各数组元素的内存地址，所以可以直接用下标访问到数组元素；而链表中的元素在内存中通常是不连续存放的，因此不能被立即访问到。

链表的优点：

1、可伸缩性：数组一旦在内存分配空间之后，大小就不能改变；而链表是动态的，在需要的时候可以增加或删除结点；数组的空间可能很快就用完，而链表只有在系统没有足够的内存满足动态分配存储空间的请求时 才会达到全满的状态；

2、插入和删除操作：数组的插入和删除涉及到移动元素的操作，因此比较费时；而链表的插入和删除比较简单；

902.C语言中链表的特点

链表是一种存储结构，用于存放线性表；

链表的结点是根据需要调用动态内存分配函数进行分配的，因此链表可随需要伸长缩短，在要存储的数据个数未知的情况下节省内存；

链表的结点在逻辑上是连续的，但是各结点的内存通常是不连续的，因此不能立即被访问到，只能从头结点开始逐结点访问。

903.C语言中链表定义及结构

链表是用链节指针链在一起的自引用结构变量（称为结点）的线性集合，是线性表的一种存储结构。

（1）headPtr——指向链表首结点的指针变量。

（2）每个结点由2个域组成：

数据域——存储结点本身的信息。

指针域——存储指向后继结点的指针。

尾结点的指针域置为NULL(用反斜杠表示)，作为链表结束的标志。

904.C++中的临时对象

- (1)构造函数是一个特殊的函数,调用构造函数将产生一个临时对象
- (2)临时对象的生命期只有一条语句的时间
- (3)临时对象的作用域只在一条语句中
- (4)临时对象是 C++中值得警惕的灰色地带

905.C++中的析构函数

- (1)C++的类中可以定义一个特殊的清理函数,叫析构函数
- (2)析构函数的功能与构造函数相反
- (3)定义:~ClassName();//注意,无参无返回值;对象销毁时会被自动调用

析构函数的定义准则：当类中自定义了构造函数,并且构造函数中使用了系统资源(如:内存申请、文件打开等),则需要自定义析构函数

906.C++中对象的构造的顺序

- (1) 对于局部对象:当程序执行流到达对象的定义语句时进行构造
- (2)对于堆对象
 - ①当程序执行流到达 new 语句时创建对象
 - ②使用 new 创建对象将自动触发构造函数的调用
- (3)对于全局对象
 - ①对象的构造顺序是不确定的
 - ②不同的编译器使用不同的规则确定构造顺序；

907.C++中赋值和初始化的区别

- ①初始化:对正在创建的对象进行初值设置(如 int a = 1;或初始化列表的形式)
- ②赋值:对已经存在的对象进行值设置(如 a = 1;)

类中可以使用初始化列表对成员进行初始化,初始化列表先于构造函数体执行, const 成员变量必须在初始化列表中指定初值, const 成员变量为只读变量。

908.C++类成员的初始化

- (1)C++中提供了初始化列表,可以对成员变量进行初始化
- (2)语法规则:

```
ClassName::ClassName():m1(v1), m2(v2,v3),m3(v3)

{

    //其它初始化操作

}
```

(3)注意事项

- ①成员的初始化顺序与成员的声明顺序相同。而与初始化列表中的位置无关
- ②初始化列表先于构造函数的函数体执行

909.C++什么时候需要进行深拷贝

- ①对象中有成员指代了系统中的资源
- ②如成员指向了动态内存空间、打开了外存中的文件或使用了系统中的网络端口等
- ③自定义拷贝构造函数时,必然需要实现深拷贝

910.拷贝构造函数的意义

①兼容 C 语言的初始化方式,即利用已经存在的对象去创建新的对象。(因为 C++中初始化会涉及到拷贝构造函数的调用。注意初始化与赋值是不同的,赋值时“=”运算符会被调用)

如: `int a = b;` //C 中,用一个变量来初始化另一个变量;

`Student s2 = s1;` //利用已经存在的 s1 对象来初始化,很像 C 的初始化方式

- ②初始化行为能够符合预期的逻辑

911.无参构造函数和拷贝构造函数的比较

	无参构造函数	拷贝构造函数
参数形式	没有参数的构造函数	参数为 <code>const class_name&</code> 的构造函数
默认情况	当类中没有定义构造函数时,编译器默认提供一个无参构造函数,并且其函数体为空	当类中没有定义拷贝构造函数时,编译器默认提供一个拷贝构造函数,简单的进行成员变量的值复制
注意:	当类中定义了构造函数(含带参、不带参、或拷贝构造函数),则系统就不再提供默认无参构造函数。而拷贝构造函数只有在我们定义时,系统才不提供。	

912.C++中对象的声明和定义

- ①对象定义:申请对象的空间并调用构造函数(如 `Test t;` //定义并调用构造函数)
- ②对象声明:告诉编译器已经存在一个对象,并不调用构造函数(如 `extern Test t;`)

913.C++中带参数的构造函数

- (1)构造函数可以根据需要定义参数
- (2)一个类中可以存在多个重载的构造函数

914.C++中的构造函数

C++中可以定义与类名相同的特殊成员函数,这个函数叫构造函数

①构造函数是没有任何返回类型(连 void 都没有,因为这个函数是编译器在创建对象时插入二进制代码用的,即由编译器来调用的)

②构造函数在对象定义时自动被调用

915.C++对象初始化

(1)从程序设计的角度看,对象只是变量,因此:

①在栈上创建对象时,成员变量初始化为随机值

②在堆上创建对象时,成员变量初始化为随机值

③在静态存储区创建对象时,成员变量初始化为 0 值

(2)生活中的对象都是初始化了的

(3)初始状态是对象普遍存在一个状态

916.C++中类和对象两个基本概念

	类	对象
概念	指的是一类事物，是一个抽象的概念	指的是属于某个类的具体实例
关系	类是一种模型，这种模型可以创建出不同的对象。一个类可以有很多对象。	对象实体是类模型的一个具体实例。一个对象必然属于某个类
意义	类用于抽象的描述一类事物所特有的属性和行为。如电脑拥有CPU、内存和硬盘，并县城可以开机和运行程序	对象是具体的事物，拥用所属类中描述的一切属性和行为。如每一只老虎都有不同的体重、不同的食量和不同的性情。

917.C++面向对象的意义

①将日常生活中习惯的思维方式引入程序设计中

②将需求中的概念直观的映射到解决方案中

③以模块为中心构建可复用的软件系统

④提高软件产品的可维护性和可扩展性

918.C++中类之间的基本关系

继承

- ①从已存在类细分出来的类和原类之间具有继承关系(is-a)
- ②继承的类(子类)拥有原类(父类)的所有属性和行为

组合

- ①一些类的存在必须依赖于其它的类,这种关系叫组合
- ②组合的类在某一个局部上由其它的类组成

919.C++中类成员的作用域

- (1)类成员的作用域只在类的内部,外部无法直接访问
- (2)成员函数可以直接访问成员变量和调用成员函数
- (3)类的外部可以通过类变量访问 public 成员
- (4)类成员的作用域与访问级别没有关系。(注意 C++中用 struct 定义的类中所有成员默认为 public,而 class 定义的类成员的默认属性为 private)

920.C++中类的关键字

- (1)struct 在 C 语言中已经有了自己的含义,必须继续兼容
- (2)在 C++中提供了新的关键字 class 用于类定义
- (3)class 和 struct 的用法完全相同。但 C++中用 struct 定义的类中所有成员默认访问级别为 public,而 class 定义的类成员的默认访问级别为 private

921.C++中类声明和实现的分离

- (1).h 头文件只用类的声明:成员变量和成员函数
- (2).cpp 源文件中完成类的其它实现(如成员函数的实现)

922.C++中的命名空间

C++中命名空间概念用于解决名称冲突问题

- (1)在 C 语言中只有一个全局作用域
 - ①C 语言中所有的全局标识符共享同一个作用域
 - ②标识符之间可能发生冲突
- (2)C++中提出了命名空间的概念
 - ①命名空间将全局作用域分成不同的部分
 - ②不同命名空间中的标识符可以同名而不会发生冲突
 - ③命名空间可以相互嵌套
 - ④全局作用域也叫默认命名空间
- (3)C++命名空间的定义:namespace Name {}

- ①使用整个命名空间:using namespace name;
- ②使用命名空间中的变量:using name::variable;
- ③使用默认命名空间中的变量: ::variable

923.C和C++相互调用

- (1)C++编译器能够兼容 C 语言的编译方式
- (2)C++编译器会优先使用 C++编译的方式
- (3)extern 关键字能强制 C++编译器进行 C 方式的编译

924.函数重载的定义、条件、注意事项

1.函数重载 (overload) 的概念

- (1)用同一个函数名定义不同的函数
- (2)当函数名和不同的参数搭配时,函数的含义不同

2.重载的条件:必须至少满足下面的一个条件

- ①参数个数不同
- ②参数类型不同
- ③参数顺序不同

3.函数重载的注意事项

- ①重载函数在本质上是相互独立的不同函数。
- ②重载函数的函数类型不同
- ③函数的返回值不能作为函数重载的依据
- ④函数重载是由函数名和参数列表共同决定的。

925.C++中 inline 内联编译的限制

- (1)含有递归调用的函数不能设置为 inline
- (2)使用了复杂流程控制语句:循环语句和 switch 语句,无法设置为 inline(说明:如上述实例,在 VS2013 下,循环语句是可以被内联的)
- (3)函数体不能过于庞大
- (4)不能对函数进行取址操作
- (5)函数内联声明必须在调用语句之前.

926.内联函数的定义和特点

内联函数的定义

- (1)C++编译器可以将一个函数进行内联编译,被 C++编译器内联编译的函数叫内联函数。
- (2)C++中使用 inline 关键字声明内联函数
- (3)内联函数声明时 inline 关键字必须和函数定义结合在一起,否则编译器会直接忽略内联请求。

内联函数的特点

- (1) C++编译器直接将内联函数的函数体插入到函数调用的地方
- (2) 内联函数没有普通函数调用时的额外开销(压栈、跳转、返回)
- (3) C++中推荐使用内联函数替代宏代码片段。
- (4) C++编译器也不一定满足函数的内联请求。

927.C++引用的意义

- (1) 功能性: 引用在大多数情况下代替指针, 可以满足需要使用指针的场合
- (2) 安全性: 可以避开由于指针操作不当而带来的内存错误
- (3) 操作性: 简单易用, 又不失功能强大

928.C++引用的本质

- (1) 引用在 C++ 中的内部实现是一个常量指针, 因此引用所占用的空间大小与指针相同。
- (2) 从使用的角度, 引用只是一个别名, C++ 为了实用性而隐藏了引用的存储空间这一细节。
 - ① 在编译过程中, 编译器看到 `int& a` 的声明就会转换为 `int* const a`;
 - ② 看到使用引用时, 会转为 `*a`, 如此隐藏了使用指针的事实。

929.C++中特殊的引用--const引用

- (1) `const Type& name = var;` // 让变量拥有只读属性
- (2) 当使用常量对 `const` 引用进行初始化时, C++ 编译器会为此常量值分配空间, 并将引用名作为这段空间的别名。但这样用常量对 `const` 引用初始化将生成的是一个只读变量。

930.C 到 C++ 的升级 (至少列出三点)

- (1) 变量的定义

C++ 中所有的变量都可以在需要使用时再定义, 而 C 语言中的变量都必须在作用域开始位置定义。

- (2) `register` 关键字

`register` 关键字请求编译器将局部变量存储于寄存器中

在 C++ 中依然支持 `register` 关键字, 但这只是为了兼容 C, C++ 编译器有自己的优化方式, 在 C 语言中无法获取 `register` 变量的地址, 因为 C 认为寄存器是没有地址的。但 C++ 中可以取得 `register` 变量的地址。

- ① C++ 编译器发现程序中需要取变量地址时, `register` 对变量的声明变得无效。所以 C++ 将无视这个关键字, 直接将其 `register` 的变量放入内存(注意, 不是寄存器)。
- ② 早期 C 语言编译器不会对代码进行优化。因此可以利用 `register` 变量来进行一个很好补充。但 C++ 认为寄存器非常宝贵, 这个 `register` 关键字并不被推荐使用, 它只是为了兼容 C 而存在。

- (3) 重命名全局变量名

(1) 在 C 语言中, 重复定义多个同名的全局变量是合法的。C 编译器将把多个同名的全局变量最终链接到全局数据区的同一个地址空间上。

(2)在 C++中,不允许定义多个同名的全局变量。因为这是一种有二义性的做法,会被C++直接拒绝

931.C和C++语言中的三目运算符

(1)C 语言中的三目运算符返回的是变量的值,因值不能作为左值使用

(2)C++中的三目运算符

①如果?后面的两个操作数有一个是常量,则行为与 C 语言的运算符一样,返回的是变量的值。

②如果?后面的两个操作数都是变量,则返回变量的引用(相当于变量本身),如此,既可以作为右值,也可以作为左值使用。

932.宏的局限和妙用

局限

(1)宏的效率比函数稍高,但是其副作用巨大

(2)宏是文本替换,参数无法进行类型检查

(3)可以用函数完成的功能,绝对不用宏

(4)宏的定义中不能出现递归定义

妙用

(1)用于生成一些常规性的代码

(2)封装函数,加上类型信息

933.C 语言中的顺序点

(1)每个完整表达式结束时,即分号处

(2)&&、||、?:、逗号表达式的每个参数计算之后

(3)函数调用时所有实参求值完成后(进入函数体之前)

934.C/C++语言中的函数参数

(1)函数参数在本质上与局部变量相同,都在栈上分配空间

(2)函数参数的初始值是函数调用时的实参值

(3)函数参数的求值顺序依赖于编译器的实现

935.声明和定义

(1)声明的意义在于告诉编译器程序单元(以下均指变量或函数)的存在

(2)定义则明确指示程序单元的意义

(3)C 语言中通过 extern 进行程序单元的声明。

(4)一些程序单元在声明时可以省略 extern(如结构体)

(5)严格意义上的声明和定义并不相同!

936.C/C++语言中内存操作的交通规则

(1) 动态内存申请之后,应该立即检查指针的值是否为 NULL,防止使用 NULL 指针

(2) free 指针之后必须立即赋值为 NULL。(如 free(p);p=NULL;)

(3) 任何与内存操作相关的函数都必须带长度信息

(4) malloc 操作和 free 操作必须匹配,防止内存泄露和多次释放。并且遵循谁申请谁释放的原则。

内存错误的本质源于指针保存的地址为非法值(如指针未初始化或指针运算导致的越界)。内存泄漏源于 malloc 和 free 不匹配。当 malloc 次数多于 free 里产生内存泄漏。反之程序可能崩溃。

937.C/C++语言中常见的内存错误

(1) 结构体成员指针未初始化

(2) 结构体成员指针未分配足够的内存

(3) 内存分配成功,但并未初始化

(4) 内存操作越界

938.内存操作的基本原则

(1) 绝不返回局部变量和局部数组的地址

(2) 任何变量在定义后必须用 0 初始化

(3) 字符数组必须确认 0 结束符后才能成为字符串

(4) 任何使用与内存操作相关的函数必须指定长度信息

939.C/C++语言中野指针的含义

(1) 指针变量中的值是非法的内存地址,进而形成野指针

(2) 野指针不是 NULL 指针,是指向不可用内存地址的指针

(3) NULL 指针并无危害,很好判断,也很好调试

(4) C 语言中无法判断一个指针所保存的地址是否合法

940.C/C++语言中文件布局在内存中的映射

1) 各个段的作用

① 堆栈段在程序运行后才正式存在,是程序运行的基础

② .bss 段存放的是未初始化的全局变量和静态变量

③ .text 段存放的是程序中的可执行代码

④ .data 段保存的是已经初始化了的全局变量和静态变量

⑤ .rodata 段存放程序中的常量值,如字符串常量

(2) 程序术语的对应关系

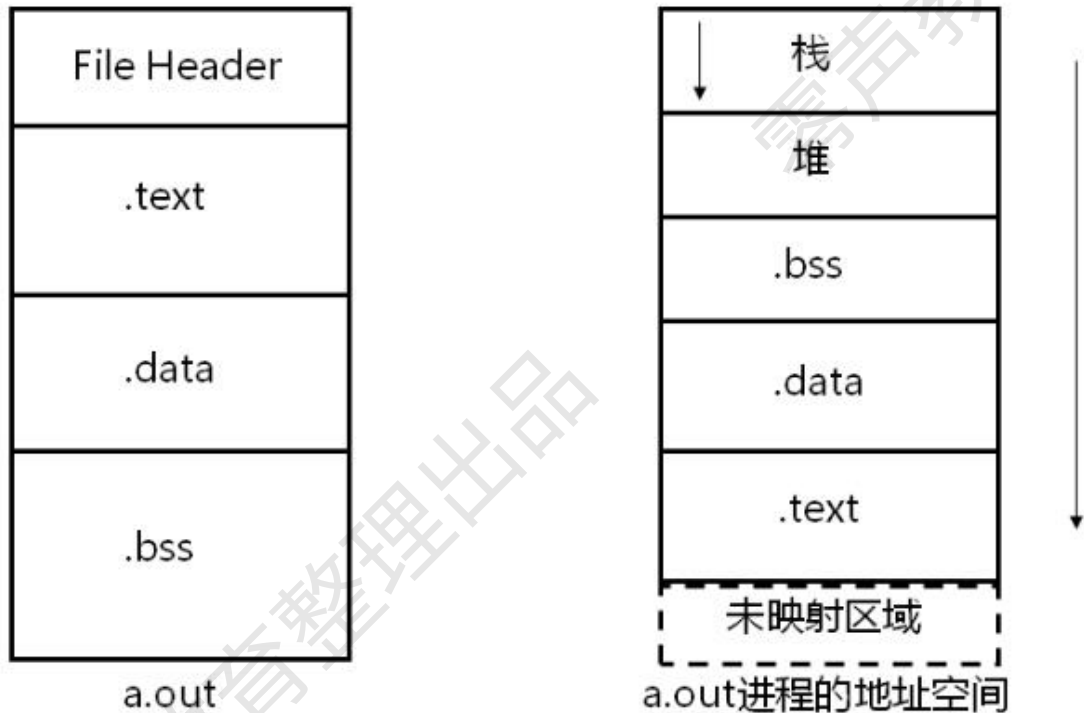
① 静态存储区通常指程序中的.bss 和.data 段

② 只读存储区通常指程序中的.rodata 段

③ 局部变量所占空间为栈上的空间

④动态空间为堆中的空间

⑤程序可执行代码存放于.text 段



941.C/C++语言中程序与进程

- (1)程序是静态的概念,表现形式为一个可执行文件
- (2)进程是动态的概念,程序由操作系统加载运行后得到进程
- (3)每个程序可以对应多个进程,但每个进程只能对应一个程序
- (4)包含脚本代码的文本文件也可以看作是一个可执行程序,其对应进程的功能与脚本的内容有关。

每种脚本文件对应着一些脚解释程序,双击脚本文件,操作系统加载相应的解释程序,变成一个进程,这个进程会读取并解释执行脚本代码。

这个进程与普通进程不同,它要完成的功能并不是固定的,而是与脚本文本的内容有关。

942.C/C++程序中的静态存储区

- (1)静态存储区随着程序的运行而分配空间
- (2)静态存储区的生命周期直到程序运行结束
- (3)在程序的编译期静态存储区的大小就已经确定。
- (4)静态存储区主要用于保存全局变量和静态局部变量
- (5)静态存储区的信息最终会保存到可执行程序中

943.C/C++程序中的堆

- (1)堆是程序中一块预留的内存空间,可由程序自由使用
- (2)堆中被程序申请使用的内存在被主动释放前将一直有效
- (3)为什么有了栈还需要堆?(栈上的数据在函数返回后就会被释放掉,无法传递到函数外部,如局部数组)

(4)C 语言中通过库函数的调用获得堆空间:malloc 和 free

(5)系统对堆空间的管理方式(空闲链表法、位图法、对象池法等等)

944.C/C++程序中的栈

栈的简介

(1)栈中现代计算机程序里最为重要的概念之一

(2)栈在程序中用于维护函数调用上下文

(3)函数中的参数和局部变量存储在栈上

(4)栈保存了一个函数调用所需的维护信息

函数调用过程：每次函数调用都对应着一个栈上的活动记录,调用函数的活动记录位于栈的中部,被调用函数的活动记录位于栈的顶部

函数调用栈上的数据

(1)函数调用时,对应的栈空间在函数返回前是专用的

(2)函数调用结束后,栈空间将被释放,数据不再有效

945.C语言中calloc 和 realloc 函数

(1)malloc 的同胞兄弟:

```
void* calloc(size_t num, size_t size);
```

```
void* realloc(void* pointer,size_t new_size);
```

(2)calloc 参数表示要返回 num 个某种类型(如 sizeof(int))大小的内存空间。calloc 能以类型大小为单位申请内存并初始化为 0。

(3)realloc 用于修改一个原先已经分配的内存块大小。当第一个参数 pointer 为 NUL 时,等价于 malloc。

946.malloc和free函数及使用过程需要注意的地方

malloc 和 free 函数

(1)malloc 和 free 用于执行动态内存分配的释放

(2)malloc 所分配的是一块连续的内存

(3)malloc 以字节为单位,并且返回值不带任何的类型信息:void* malloc(size_t size);

(4)free 用于将动态内存归还系统:void free(void* pointer);

(5)_msize(void* pointer)可以获取 malloc 出来的内存空间大小

使用 malloc 和 free 需要注意的地方

(1)malloc 和 free 是库函数,而不是系统调用

(2)malloc 实际分配的内存可能有会比请求的多,但不能依赖于不同平台下的 malloc 行为。

(3)当请求的动态内存无法满足时,malloc 返回 NULL

(4)当 free 的参数为 NULL 时,函数直接返回

947.C语言中动态内存分配

(1)C 语言中的一切操作都是基于内存的。

(2)变量和数组都是内存的别名。

①内存分配由编译器在编译期间决定

②定义数组的时候必须指定数组长度

③数组长度是在编译期就必须确定的

(3)但是程序运行的过程中,可能需要使用一些额外的内存空间

948.C语言中的指针阅读技巧

右左法则

(1)从最里层的圆括号中未定义的标示符看起

(2)首先往右看,再往左看

(3)遇到圆括号或方括号时可以确定部分类型,并调转方向

(4)重复 2、3 步骤,直到阅读结束

注意:

当读出是数组时,须读出元素个数、元素类型

当读出是函数是,须读出参数及类型,返回值类型

当读出是指针是,须读出指针所指向的类型,有时也须读出指针的类型。

949.C语言中的函数指针

(1)函数指针用于指向一个函数,函数名是执行函数体的入口地址。

(2)定义函数指针的两种方法

①通过函数类型定义:FuncType* pointer;

②直接定义:type(*pointer)(parameter list);

其中 type 为返回值类型,pointer 为函数指针变量名,parameter list 为参数类型列表

950.C语言中指向指针的指针

(1)指针的本质是变量,会占用一定的内存空间

(2)可以定义指针的指针来保存指针变量的地址值

(3)指针是个变量,同样也存在传值调用与传址调用

951.C语言中的数组指针和指针数组

数组指针

(1)数组指针用于指向一个数组

(2)数组名是数组首元素的起始地址,但并不是数组的起始地址。

(3)通过将&作用于数组名可以得到数组的起始地址

(4)定义数组指针的两种方式:

①可通过数组类型定义数组指针:ArrayType* pointer;

②可以直接定义:type (*pointer)[n]; //其中 pointer 为数组指针变量名,type 为数组的元素类型,n 为数组的大小

指针数组

(1)指针数组是一个普通的数组,其中的每个元素为一个指针

(2)指针数组的定义:type* pArray[n];//其中的 type为数组中元素的类型,pArray为数组名,n 为数组的大小 (如float a[3])

952.C语言中字符串相等的比较

(1)字符串之间的相等比较需要用 strcmp 完成

(2)不可直接用==进行字符串直接的比较

(3)完全相同的字符串字面量的==比较结果为 false。但一些现代编译器(如 gcc)能够将相同的字符串字面量映射同一个无名字符数组,因此==比较结果为 true。

953.C语言中的字符串和字符数组

字符串是有序字符的集合,C语言中没有字符串的概念,而是通过特殊的字符数组模拟字符串,是以'\0'结尾的字符数组。

(1)在C语言中,字双引号引用的单个或多个字符是一种特殊的字面量,被存储于程序的全局只读存储区,在本质上为字符数组,编译器自动在结尾加上'\0'字符。

(2)字符串字面量的本质是一个数组,如"Hello World!"是一个无名的字符数组

(3)字符串字面量可以看作常量指针

(4)字符串字面量中的字符不可改变

(5)字符串字面量至少包含一个字符,即'\0'

954.数组参数

数组作为函数参数时,编译器将其编译为对应的指针。因此,一般情况下,当定义的函数中有数组参数时,需要定义另一个参数来标示数组的大小。

void f(int a[])等价于 void f(int* a);

void f(int a[5])等价于 void f(int* a); //就是一个指针,丢失了数组长度的信息

955.数组的访问方式

(1)以下标的形式访问数组中的元素:如 a[i];

(2)以指针的形式访问数组中的元素:如*(a+i)

(3) 下标形式与指针形式的转换： $a[n] == (a+n) == (n+a) == n[a]$;

956.数组地址与数组名

(1) 数组名 a 代表数组首元素的地址。因此，第 2 个元素的地址为 $a+1$ ，以此类推……。注意 a 或 $a+i$ 表示元素的地址。可以用 $*(a+i)$ 取出元素的值，也可以用 $a[i]$ 来取出元素的

值，因为当编译中遇到 $a[i]$ 会自动转为 $*(a+i)$ 。反过来也可知，第 1 个元素的地址为 a 或 $\&a[0]$ ，第 2 个元素的地址为 $a+1$ 或 $\&a[1]$ ，第 i 个元素的地址为 $(a+i)$ 或 $\&a[i]$ ……

(2) 数组的地址需要用取地址符 $\&$ 才能得到。即形如 $\&a$ 取的是整个数组的地址，所以 $\&a+1$ 表示指向整个数组的最后面的位置。

(3) 数组的首元素的地址值与数组的地址值相同，但是两个不同的概念。

957.C++中类封装的基本概念

(1) 类通常分为两个部分：类的实现细节、类的使用方式。当使用类时，不需要关心其实现细节。当创建类时，才需要考虑其内部实现细节

我们在创建类的时候，要将复杂的细节封装在内部，不让使用者知道，给使用者一些非常简单的接口就可以。

(2) 根据经验，并不是类的每个属性都是对外公开的，但一些属性是对外公开的。

(3) 必须在类的表示法中定义属性和行为的公开级别

958.C++中的引用基本点

(1) 引用可以看作一个已定义变量的别名

(2) 引用的语法： $Type\& name = var$; // $Type$ 为类型名， $name$ 为引用的名字， var 为已定义的变量名

(3) 普通引用在定义时必须用同类型的变量进行初始化，函数参数引用的初始化发生在函数被调用时。

(4) 引用作为变量别名而存在，因此在一些场合可以代替指针

(5) 引用相对于指针来说，具有更好的可读性和实用性

959.函数设计原则

(1) 函数从意义上应该是一个独立的功能模块

(2) 函数名要在一定程度上反映函数的功能

(3) 函数参数名要能够体现参数的意义

(4) 尽量避免在函数中使用全局变量

```
void sc(char s1, char s1);×
```

```
void str_copy(char* dest, char* src);√
```

(5) 当函数参数不应该在函数体内部被修改时，应加上 `const` 声明

(6) 如果参数是指针，且仅作输入参数，则应加上 `const` 声明

```
void str_copy(char* dest, const char* src);
```

(7) 不能省略返回值的类型。如果没有返回值，应声明为 `void`。

(8)对参数进行有效性检查,特别是指针参数的检查尤为重要

(9)不要返回指向“栈内存”的指针,因为栈内存存在函数体结束时被自动释放

(10)函数体的规模要小,尽量控制在 80 行代码之内

(11)相同的输入对应相同的输出,避免函数带有“记忆”功能

(12)避免函数有过多的参数,参数个数尽量控制在 4 个以内

(13)有时候函数不需要返回值,但为了增加灵活性,如支持链式表达,可以附加返回值

```
char s[64];
```

```
int len = strlen(strcpy(s, "Hello")); //当中的 strcpy 返回缓冲区 s 的地址。
```

(14)函数名和返回值类型在语义上不可冲突

```
char c = getchar(); //getchar 的返回值实际上是 int 类型,而不是 char。与函数名不符。
```

960.C语言中的回调函数

(1)回调函数是利用函数指针实现的一种调用机制

(2)回调机制原理

①调用者不知道具体事件发生时需要调用的具体函数

②被调函数不知道何时被调用,只知道需要完成的任务

③当具体事件发生时,调用者通过函数指针调用具体函数。

(3)回调机制中的调用者和被调用者互不依赖。

961.C语言中二维数组参数

(1)二维数组参数同样存在退化的问题:

二维数组可以看做是一维数组,其中的每个元素又是一个一维数组

(2)二维数组参数中第一维的参数可以省略

①void f(int a[5]) $\longleftrightarrow$ void f(int a[]) $\longleftrightarrow$ void f(int* a)

②void g(int a[5][3]) $\longleftrightarrow$ void g(int a[][3]) $\longleftrightarrow$ void g(int (*a)[3]);

(3)等价关系

数组参数	等效的指针参数	备注
一维数组: float a[5]	指针: float* a	相当于数组去掉第1维,然后把数组名前加上*。
指针数组: int* a[5]	指针的指针: int** a;	
二维数组: char a[3][4]	数组的指针: char(*a)[4]	

(4)被忽视的知识点

①C 语言中无法向一个传递任意多维数组。换一句话讲,形参 int a[][3]是合法的,但 int a[][]是非法的。

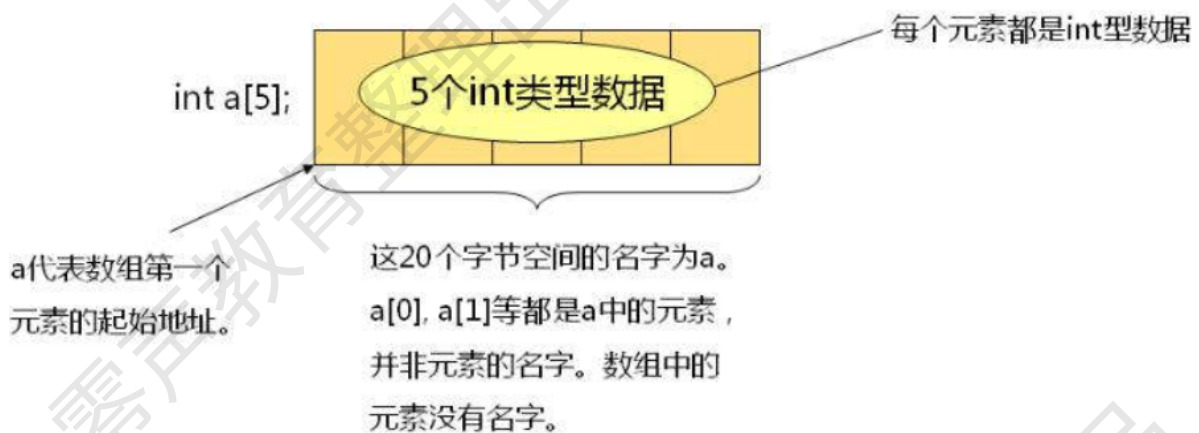
①因此,必须提供除第 1 维之外的所有其他维长度。第一维之外的维度信息用于完成指针运算。

962.数组的本质

- (1) 数组是一段连续的内存空间
- (2) 数组的空间大小：`sizeof(array_type)*array_size;`
- (3) 数组名可看做指向数组第一个元素的常量指针
- (4) 数组声明时编译器自动分配一片连续的内存空间，而指针声明时只分配了用于容纳地址值的 4 字节空间

963.数组的含义

- (1) 数组是相同类型的变量的有序集合



- (2) 数组在一片连续的内存空间中存储元素
- (3) 数组元素的个数可以显示或隐式指定

964.C语言中#pragma 的使用

- (1) #pragma 用于指示编译器完成一些特定的动作
- (2) #pragma 所定义的很多指示字是编译器特有的，在不同的编译器间是不可移植的
 - ①预处理器将忽略它不认识的#pragma 指令
 - ②不同编译器可能以不同的方式解释同一条#pragma 指令
- (3) 一般用法：`#pragma parameter //注意，不同的 parameter 参数语法和意义不同`

965.C语言中#line的用法

- (1) #line 用于强制指定新的行号和编译文件名，并对源程序的代码重新编号
- (2) 用法：`#line number newFilename //newFilename 可省略`
- (3) #line 编译指示字的本质是重定义LINE和FILE

966.C语言中#error的用法

- (1) #error 是一种预编译器指示字，用于生成一个编译错误消息
- (2) 用法：`#error message //注意：message 不需要用双引号包围`

(3) #error 编译指示字用于自定义程序员特有的编译错误消息。类似的, #warning 用于生成编译警告消息

(4) #error 可用于提示编译条件是否满足。编译过程中的任何错误意味着无法生成最终的可执行程序

967.c语言中数组参数退化为指针的意义

(1)C 语言中只会以值拷贝的方式传递参数,当向函数传递数组时,将整个数组拷贝一份传入函数导致执行效率低下,C 语言以高效作是最初的设计目标,所以这种方法是不可取的。

(2)参数位于栈上,太大的数组拷贝将导致栈溢出。

(3)将数组名看做常量指针,传递的是数组的首元素地址,而不是整个数组。

968.程序中的顺序点

(1)程序中存在一定的顺序点。

(2)顺序点指的是执行过程中修改变量值的最晚时刻

(3)在程序到达顺序点的时候,之前所做的一切操作必须完成。

969.面向过程的程序设计

(1)面向过程是一种以过程为中心的编程思想

(2)首先将复杂的问题分解为一个个容易解决的问题

(3)分解过后的问题可以按照步骤一步步完成

(4)函数是面向过程在 C 语言中的体现

(5)解决问题的每个步骤可以用函数来实现

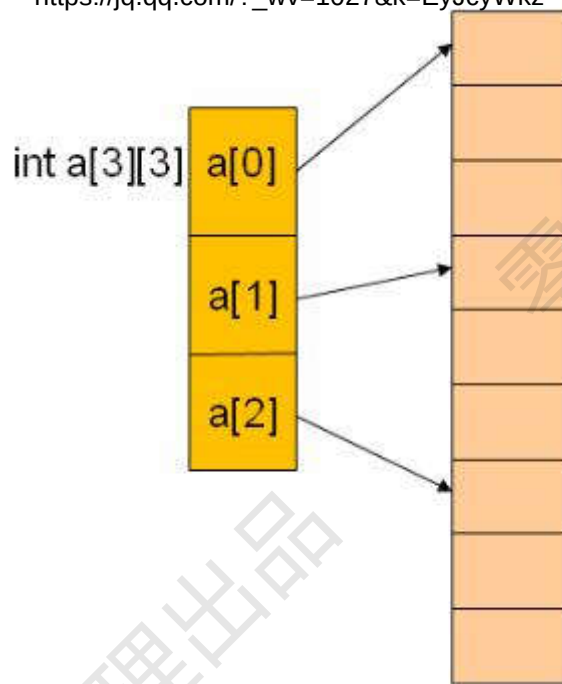
970.C语言中的函数类型

(1)C 语言中的函数有自己特定的类型,这个类型由返回值、参数类型和参数个数共同决定。如 int add(int i,int j)的类型为 int(int,int)。

(2)C 语言中通过 typedef 为函数类型重命名

```
typedef type name(parameter list);//如 typedef int f(int,int);
```

971.C语言二维数组与二级指针



- (1)二维数组在内存中以一维的方式排布
- (2)二维数组中的第一维是一维数组
- (3)二维数组中的第二维才是具体的值
- (4)二维数组的数组名可看做常量指针

972.C语言中字符串的长度

- (1)字符串的长度就是字符串所包含字符的个数
- (2)字符串长度指的是第一个'\0'字符前出现的字符个数,通过'\0'结束符来确定字符串的长
- (3)函数 `strlen` 用于返回字符串的长度(不含'\0')。

973.指针的运算

- (1) 指针是一种特殊的变量，与整数的运算规则为：

$p + n == (\text{unsigned int})p + n * \text{sizeof}(*p);$

当指针 `p` 指向一个同类型的数组的元素时，`p+1` 指向当前元素的下一个元素，`p-1` 指向上一个元素。

- (2) 指针之间只支持减法运算且参与减法运算的指针类型必须相同。

$p1 - p2 = ((\text{unsigned int})p1 - (\text{unsigned int})p2) / \text{sizeof}(\text{type})$

①只有当两个指针指向同一个数组中的元素时，指针相减才有意义，其意义为指针所指元素的下标差。

②当两个指针指向的元素不在同一个数组中时，结果未定义。

974.数组名的知识点

- (1) 数组名的内涵在于其指代实体是一种数据结构，这种数据结构就是数组。如 `int a[5]`表示 `a` 的类型为 `int[5]`，所以 `sizeof(a)`表示取整个数组的大小，`&a` 表示数组的地址。

(2) 数组名的外延：除了sizeof(a)和&a外，数组名经常可看作是一个常量指针。但要注意这里仅仅是“看作”，而不是真正的指针。不同于指针，数组名只是编译过程中的一个符号，编译器并不为其分配内存，有人称之为“伪变量”。因此，形式a++\a—或a=b（其中b是另一个数组名）这些都是错误的，因为a只是一个符号，编译器会把数组信息（如大小，地址）放入符号表中，每次遇到数组名a时，就会从符号表中取出这个数组的地址，然后用这个固定的地址代替a，所以这个符号并没有被分配内存空间，而上述操作都是针对变量而言的，故数组名只能做为右值使用。

(3) 对数组的引用，如a[i]或(a+i)，只需访问内存一次，而指针的引用如(p+i)则需要两次，首选通过&p找到p指针，然后加i，再从p+i里面取出的内容。

(4) 当数组名作为形参时，将退化为指针。即可以把数组名当成指针来用，这里的sizeof(数组名)为4，即指针的长度。

975.C语言中的条件编译

C语言中的条件编译的行为类似于C语言中的if...else...，是预编译指示命令，用于控制是否编译某段代码。

1.条件编译的本质

- (1) 预编译器根据条件编译指令有选择的删除代码
- (2) 编译器不知道代码分支的存在
- (3) if...else...语句在运行期进行分支判断，条件编译指令在预编译期进行分支判断
- (4) 可以通过命令行定义宏

①gcc -Dmarco=value file.c //针对#if 语句

②gcc -Dmarco file.c //针对#ifdef 或 ifndef 语句

2.条件编译的意义

- (1) 条件编译使得我们可以按不同的条件编译不同的代码段，因而可以产生不同的目标代码。
- (2) #if...#else...#endif 被预编译器处理，而 if...else...语句被编译器处理，必然被编译进目标代码。
- (3) 实际工程中条件编译主要用于以下两种情况：
 - ①不同的产品线共用一份代码
 - ②区分编译产品的调试版和发布版

976.C语言中函数和宏定义的对比

- (1) 宏表达式被预处理器处理，编译器不知道宏表达式的存在
- (2) 宏表达式用“实参”完全替代形参，不进行任何运算。
- (3) 宏表达式没有任何的“调用”开销
- (4) 宏表达式中不能出现递归定义。

977.c语言中动态库和静态库的使用

1.C语言中的链接器

- (1) 每个C语言源文件被编译后生成目标文件，这些目标文件最终要被链接在一起生成可执行文件。

(2) 链接器的主要作用是把各个模块之间相互引用的部分处理好,使得各个模块之间能够正确的衔接。

2.静态链接由链接器在链接时将库的内容直接加入到可执行程序中

①编译静态库源码: gcc -c lib.c -o lib.o

②生成静态库文件: ar -q lib.a lib.o //将 lib.o 与其他文件打包到 lib.a 中

③使用静态库编译: gcc main.c lib.a -o main.out

3.动态链接可执行程序在运行时才动态加载库进行链接,库的内容不会进入可执行程序当中

①编译动态库源码: gcc -shared dlib.c -o dlib.so

②使用动态库编译: gcc main.c -ldl -o main.out

978.C语言中的逗号表达式

逗号表达式: exp1,exp2,exp3,...,expN;

(1)逗号表达式是 C 语言中的“粘贴剂”

(2)逗号表达式用于将多个子表达式连接为一个表达式

(3)逗号表达式的值为最后一个子表达式的值

(4)逗号表达式中的前 N-1 个子表达式可以没有返回值

(5)逗号表达式按照从左向右的顺序计算每个子表达式的值

979.C语言中的单引号和双引号

(1)C 语言中单引号用来表示字符字面量(是个数值)被编译为对应的 ASCII 码

(2)C 语言中双引号用来表示字符串字面量(是个指针)被编译为对应的内存地址

例如: 'a'表示字符字面量(97),在内存中占用 1 个字节,'a'+1 表示'a'的 ASCII 码加 1,即'b'。"a"表示字符串字面量(是个指针),在内存中占 2 个字了,"a"+1 表示指针运算,指向"a"的结束符"\0"。

980.C语言中接续符和转义符

C语言中的接续符

(1)编译器将反斜杠剔除,跟在反斜杠后面的字符自动接续到前一行

(2)在接续单词时,反斜杠之后不能有空格,反斜杠下一行之前也不能有空格

(3)接续符适合在宏定义代码块时使用

C语言中的转义符

C 语言中的转义符()主要用于表示无回显字符,也可用于表示常规字符。当反斜杠()作为转义符使用时必须出现在单引号或双引号之间

可以看出C 语言中的反斜杠()同时具有接续符与转义符的作用,但是当接续符或转义符是有限定条件的

(1)作为接续符使用时可直接出现在程序中

(2)作为转义符使用时需出现在单引号或双引号之间

981.C语言中union关键字

(1) C 语言中的 union 在语法上与 struct 相似

(2) union 只分配最大成员的空间，所有成员共享这个空间

(3) union 的使用受系统大小端的影响

982.C语言中变量的属性关键字

变量属性关键字的使用语法：property type var_name;

1.auto 关键字

auto关键字是C语言中局部变量的默认的关键字，C编译器默认所有的局部变量都是auto的，它表明了被修饰的关键字将被存储在栈空间上。

2.register 关键字

register关键字请求将被修饰的关键字存储于寄存器中，所以register 变量必须是 CPU 寄存器可以接受的值而且不能用&运算符获取 register 变量的地址，

但是 register 关键字只是请求寄存器变量，所以不一定会成功。

3.static 关键字

(1) static 关键字指明变量的“静态”属性——局部变量存储在程序静态区

(2) static 关键字同时具有“作用域限定符”作用

①static 修饰的全局变量作用域只是声明的文件中

②static 修饰的函数作用域只是声明的文件中

4.extern 关键字

(1) extern 用于声明“外部”定义的变量和函数

①extern 变量在文件的其它地方分配空间

②extern 函数在文件的其它地方定义

(2) extern 用于“告诉”编译器用 C 方式编译

C++编译器和一些变种 C 编译器默认会按“自己”的方式编译函数和变量，通用 extern 关键字可以命令编译器“以标准 C 方式进行编译”。

983.c语言中enum关键字的作用

C语言中enum关键字用来定义枚举类型

(1) enum 是 C 语言中的一种自定义类型

(2) enum 值是可以根据需要自定义的的整型值

(3) 第一个定义的 enum 值默认为 0

(4) 默认情况下的 enum 值是在前一个定义值的基础上加 1

(5) enum 类型的变量只能取定义时的离散值

984.C语言中sizeof关键字的作用

(1) sizeof 是编译器的内置指示符

(2) sizeof 用于计算类型或变量所占的内存大小

(3) sizeof 的值在编译期就已经确定，在编译过程中所有的 sizeof 将被具体的数值所替换，程序的执行过程与 sizeof 没有任何关系。

985.c语言中extern关键字的作用

extern关键字可以用来声明变量和函数作为外部变量或者函数供其它文件使用。

引用同一个文件中的变量

引用另一个文件中的变量

引用另一个文件中的函数

extern "C"的主要作用就是为了能够正确实现C++代码调用其他C语言代码。加上extern "C"后，会指示编译器这部分代码按C语言的进行编译，而不是C++的。

986.C语言中volatile关键字的作用

volatile是一个类型修饰符（type specifier），就像我们熟悉的const一样，它被设计用来修饰被不同线程访问和修改的变量；volatile的作用是作为指令关键字，确保本条指令不会因编译器的优化而省略，且要求每次直接读值。volatile的变量是说这变量可能会意想不到地改变，这样，编译器就不会去假设这个变量的值了。

987.C语言中const关键字的作用

- (1) 定义常变量(只读变量)
- (2) 修饰函数参数，函数返回值，保护变量被意外修改，增强程序的健壮性
- (3) 让逻辑错误在编译期间被发现,而不要到了运行阶段程序异常终止

988.'#'与'##'的区别

'#'是把宏参数转化为字符串的运算符，'##'是把两个宏参数连接的运算符。

例如：

```
#define STR(arg) #arg    则宏STR(hello)展开时为"hello"
#define NAME(y) name_y    则宏NAME(1)展开时仍为name_y
#define NAME(y) name##y    则宏NAME(1)展开为name_1
#define DECLARE(name, type) typename####type##_type ,
则宏DECLARE(val, int)展开为int val_int_type
```

989.如何引用一个已经定义过的全局变量？

可以用引用头文件的方式，也可以用extern关键字，如果用引用头文件方式来引用某个在头文件中声明的全局变理，假定你将那个变量写错了，那么在编译期间会报错，如果你用extern方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在连接期间报错。

990.大小端问题

大端模式：是指数据的高字节保存在内存的低地址中，而数据的低字节保存在内存的高地址中。
arm、powerpc

小端模式：是指数据的高字节保存在内存的高地址中，而数据的低字节保存在内存的低地址中。
intelx86

991.typedef关键字

为现有类型创建一个新的名字，使用最多的地方是创建易于记忆的类型名
typedef int size;此声明定义了一个int的同义字，名字为size

992.什么是封装？C++中是如何实现的？

封装来源于信息隐藏的设计理念，是通过特性和行为的组合来创建新数据类型让接口与具体实现相隔离。C++中是通过类来实现的，为了尽量避免某个模块的行为干扰同一系统中的其它模块，应该让模块仅仅公开必须让外界知道的接口。

993.C与C++各自是如何定义常量的？有什么不同？

C中是使用宏#define定义，C++使用const来定义。

区别：

1) const是有数据类型的常量，而宏常量没有，编译器可以对前者进行静态类型安全检查，对后者仅是字符替换，没有类型安全检查，而且在字符替换时可能会产生意料不到的错误（边际效应）。

2) 有些编译器可以对const常量进行调试，不能对宏调试。

994.内存的分配方式的分配方式有几种？

1)从静态存储区域分配。内存在程序编译的时候就已经分配好，这块内存在程序的整个运行期间都存在。例如全局变量。

2)在栈上创建。在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限。

3)从堆上分配，亦称动态内存分配。程序在运行的时候用malloc或new申请任意多少的内存，程序员自己负责在何时用free或删除释放内存。动态内存的生存期由我们决定，使用非常灵活，但问题也最多。

995.头文件中的 ifndef/define/endif 干什么用？

防止头文件被重复引用

996.什么是预编译？何时需要预编译？

(1) 总是使用不经常改动的大型代码体

(2) 程序由多个模块组成，所有模块都使用一组标准的包含文件和相同的编译选项。在这种情况下，可以将所有包含文件预编译为一个预编译头

997.在C++程序中调用被C编译器编译后的函数，为什么要加extern“C”声明？

函数和变量被C++编译后在符号库中的名字与C语言的不同，被extern “C”修饰的变量和函数是按照C语言方式编译和连接的。由于编译后的名字不同，C++程序不能直接调用C函数。C++提供了一个C连接交换指定符号extern“C”来解决这个问题。

998.memset ,memcpy 的区别

memset用来对一段内存空间全部设置为某个字符，一般用在对定义的字符串进行初始化为'\0'；

memcpy用来做内存拷贝，你可以拿它拷贝任何数据类型的对象，可以指定拷贝的数据长度。

999.一下三种指针的区别？

char * const t; //常量指针，t的值不可以修改

char const *t; //指向常量的指针，指向的常量值不可以改

const char *t; //和char const *t

1000.“常量指针”和“指针常量”有什么区别？

直接上代码：

```
#include <iostream>

using namespace std;

int main() {

    int a = 10;

    int b = 20;

    // 常量指针：指针指向的值不可以改，指针的指向可以改

    const int *p1 = &a;

    // *p1 = 30; 错

    // p1 = &b; 对

    //指针常量：指针指向的值可以改，指针的指向不可以改

    int *const p2 = &a;

    // *p2 = 30; 对

    // p2 = &b; 错

    const int *const p3 = &a;

    // *p3 = 30; 错

    // p3 = &b; 错

    return 0;

}
```

记忆方式：按顺序读取

如：

```
const int *p1 = &a; //“*”代表指针，因此按顺序读取为：常量指针
```

```
int *const p2 = &a; //指针常量
```

【Linux内核/后台服务架构】技术交流Q群：762073882

https://jq.qq.com/?_wv=1027&k=EyJcyWkz

原文地址：<http://www.k6k4.com/bbs/show/bbsnqn4141586096495700>